

Introdução ao R e à Simulação Estatística

Um Laboratório de Estatística Computacional

Renato de Paula and Nuno Brites

11 de July de 2026

Índice

0.1	Porquê o R?	9
0.2	Porquê a simulação?	10
0.3	Para quem é este livro	10
0.4	Como está organizado	11
0.5	Como tirar o máximo partido	11
0.6	Uma nota final	11
1	Estatística, computação e simulação	12
1.1	A estatística tornou-se computacional	13
1.2	O que é o R	14
1.3	A ideia central: simular	15
1.4	O caminho que vamos percorrer	16
2	O R e o RStudio	18
2.1	Instalação	18
2.2	Uma visita guiada ao RStudio	20
2.3	Atalhos que poupam tempo	20
3	O R como calculadora	22
3.1	A consola e o primeiro comando	22
3.2	Comentários: escrever para humanos	23
3.3	Pedir ajuda	23
3.4	Objetos e variáveis	24
3.5	Tipos de dados	26
3.6	Operadores aritméticos	28
3.7	Objetos predefinidos e valores especiais	29
3.8	Exibir e combinar texto	31

<i>ÍNDICE</i>	3
3.9 Operadores lógicos e relacionais	33
3.10 Exercícios	34
4 Estruturas de dados	36
4.1 Vetor	37
4.2 Fatores	45
4.3 Matriz e array	49
4.4 Data frame	55
4.5 Listas	61
5 Estruturas de decisão	66
5.1 A condicional <code>if</code>	66
5.2 A estrutura <code>if...else</code>	67
5.3 A condicional <code>else if</code>	67
5.4 A função <code>ifelse()</code>	68
5.5 Erros comuns e exemplos de análise	69
5.6 Exercícios	71
6 Funções	73
6.1 Anatomia de uma função	73
6.2 Argumentos: mais flexibilidade	75
6.3 Validar entradas com <code>stop()</code>	76
6.4 Exercícios de análise	76
6.5 Exercícios	77
7 Scripts em R	80
7.1 Criar um script	80
7.2 Executar um script	81
7.3 O diretório de trabalho	81
7.4 Exercícios	82
8 Leitura de dados	84
8.1 Ler dados do teclado	84
8.2 O diretório de trabalho	85
8.3 A função <code>read.table()</code>	85

8.4	A função <code>read.csv()</code>	86
8.5	A função <code>read.csv2()</code>	86
8.6	A função <code>read_excel()</code>	87
8.7	Ler dados a partir de um URL	88
8.8	Guardar resultados em disco	88
8.9	Exercícios	88
9	O operador <i>pipe</i>	92
9.1	O <i>pipe</i> do <code>magrittr</code> (<code>%>%</code>)	92
9.2	O <i>pipe</i> nativo do R (<code> ></code>)	94
9.3	Exercícios	95
10	O ciclo <i>while</i>	97
10.1	Exemplos	97
10.2	Considerações importantes	99
10.3	Exercícios	99
11	O ciclo <i>for</i>	101
11.1	Exemplos	101
11.2	Exercícios	103
12	A família <i>apply</i>	105
12.1	A função <code>apply()</code>	106
12.2	A função <code>lapply()</code>	106
12.3	A função <code>sapply()</code>	107
12.4	A função <code>tapply()</code>	108
12.5	Exercícios	109
13	Exercícios de revisão	111
14	Gráficos com o R base	116
14.1	Gráfico de barras	116
14.2	Gráfico circular	120
14.3	Histograma	121
14.4	Caixa de bigodes (<i>boxplot</i>)	124

14.5 Gráfico de dispersão	127
14.6 Gráfico de linhas	129
14.7 Exercícios	130
15 Manipulação de dados	133
15.1 Tibbles	133
15.2 Os verbos do <code>dplyr</code>	134
15.3 Juntar tabelas	146
16 O pacote <code>ggplot2</code>	150
16.1 Um pouco de história	150
16.2 Porque é importante	150
16.3 A gramática, camada a camada	151
16.4 Exemplos	151
16.5 Exercícios	157
17 A função <code>sample()</code>	159
17.1 Extrair amostras	159
17.2 Reprodutibilidade: <code>set.seed()</code>	160
17.3 Exercícios	161
18 Probabilidade e variáveis aleatórias	163
18.1 Como se descreve uma variável aleatória	163
18.2 Exemplo: uma variável discreta	164
18.3 Exemplo: uma variável contínua	165
18.4 Exemplo: uma densidade definida por troços	167
18.5 Exercícios	169
19 Simulação	172
19.1 A simulação reproduz a teoria	172
19.2 Quando a simulação vale mesmo a pena	173
19.3 Números pseudoaleatórios	174
19.4 A função <code>set.seed()</code>	176
19.5 Exercícios	176

20 Distribuições de probabilidade	178
20.1 Primeiros exemplos	179
20.2 Função de distribuição empírica	181
20.3 Modelos discretos	182
20.4 Distribuição uniforme discreta	183
20.5 Distribuição de Bernoulli	184
20.6 Distribuição binomial	185
20.7 Distribuição geométrica	191
20.8 Distribuição de Poisson	194
20.9 Distribuição uniforme contínua	199
20.10 Distribuição exponencial	203
20.11 Distribuição normal	207
21 Exercícios extra	212
22 Método da transformada inversa	218
22.1 O resultado fundamental	218
22.2 Variáveis aleatórias discretas	219
22.3 Variáveis aleatórias contínuas	227
22.4 Quando usar o método?	231
23 Método da aceitação-rejeição	232
23.1 Contexto histórico	232
23.2 A ideia geométrica	233
23.3 Formalização do método	233
23.4 Implementação genérica em R	236
23.5 Exemplo 1 — uma densidade limitada com proposta uniforme	237
23.6 Exemplo 2 — a normal padrão a partir da exponencial	239
23.7 O caso de funções não normalizadas	241
23.8 Síntese	242
23.9 Exercícios	242

24 Teoremas limites	244
24.1 Contexto histórico	244
24.2 Modos de convergência	245
24.3 Lei fraca dos grandes números	246
24.4 Lei forte dos grandes números	246
24.5 Teorema limite central	248
24.6 Síntese	252
24.7 Exercícios	253
25 Cálculo de integrais por Monte Carlo	255
25.1 Contexto histórico	255
25.2 A ideia fundamental	256
25.3 O erro da aproximação	257
25.4 Cálculo de $\int_a^b g(x) dx$	257
25.5 Cálculo de $\int_0^\infty g(x) dx$	260
25.6 O método “acerta-ou-falha” e a estimativa de π	261
25.7 Integrais multidimensionais	262
25.8 Síntese	264
25.9 Exercícios	264
26 Resoluções dos exercícios	266
26.1 Vetores	266
26.2 Fatores	269
26.3 Listas	272
26.4 Matrizes e <i>arrays</i>	274
26.5 <i>Data frames</i>	277
26.6 Leitura de dados e tabelas de frequência	279
26.7 Gráficos com o R base	281
26.8 Estruturas condicionais (if ... else)	283
26.9 Ciclos for	285
26.10 Ciclos while	287
26.11 Funções	288
26.12 <i>Scripts</i> e a função source()	291

26.13A família <code>apply</code>	291
26.14O operador <i>pipe</i>	292
26.15Exercícios de revisão	293
26.16Amostragem com <code>sample()</code>	295
26.17A semente <code>set.seed()</code>	296
26.18Probabilidade e variáveis aleatórias	296
26.19Distribuição uniforme discreta	298
26.20Distribuição de Bernoulli	299
26.21Distribuição binomial	299
26.22Distribuição geométrica	301
26.23Distribuição de Poisson	302
26.24Distribuição uniforme contínua	303
26.25Distribuição normal	304
26.26Distribuição exponencial	305
26.27Simulação de variáveis aleatórias	306
26.28Método da transformação inversa (caso discreto)	307
26.29Método da transformação inversa (caso contínuo)	308
26.30Método da aceitação-rejeição	309
26.31Teoremas limites	313
26.32Cálculo de integrais por Monte Carlo	315

Prefácio

Descarregar PDF Descarregar EPUB

Este livro nasceu de uma convicção simples: aprende-se estatística a fazer estatística. Durante muito tempo, o ensino da disciplina viveu dividido entre, por um lado, a teoria – distribuições, teoremas, demonstrações — e, por outro, a prática, muitas vezes reduzida a exercícios de papel e lápis com números pequenos e cuidadosamente escolhidos para “dar certo”. O computador estava ausente, ou aparecia apenas no fim, como uma calculadora sofisticada.

Hoje sabemos que essa separação é artificial. A estatística moderna é, em larga medida, uma ciência computacional. Quando queremos perceber como se comporta a média de uma amostra, não basta recitar o Teorema do Limite Central: podemos também gerar milhares de amostras e observar empiricamente o fenómeno que o teorema descreve. Quando um integral não tem solução fechada, podemos estimá-lo por simulação. Quando um modelo é demasiado complexo para a análise tradicional, simulamos. A capacidade de programar deixou de ser um extra para quem estuda estatística — passou a ser uma competência central.

O que este livro se propõe fazer. Levar o leitor, sem pressupor qualquer experiência prévia de programação, desde os primeiros comandos em R até à utilização de métodos de simulação para resolver problemas de probabilidade e estatística. Fazemo-lo mantendo sempre juntas as duas metades da disciplina: a teoria que dá sentido aos métodos e a aplicação que os torna úteis.

0.1 Porquê o R?

Existem muitas linguagens de programação. Porquê estudar precisamente o R?

O R foi concebido, desde a raiz, *por* estatísticos e *para* fazer estatística. Não é uma linguagem de propósito geral à qual se acrescentaram, mais tarde, ferramentas estatísticas; é o contrário. Trabalhar com vetores, matrizes, tabelas de dados, distribuições de probabilidade e gráficos é, em R, natural e imediato porque foi para isso que a linguagem foi desenhada. Um cálculo que noutras linguagens exigiria vários passos, em R resume-se muitas vezes a uma única linha.

A isto junta-se uma comunidade científica ampla, activa e internacional. O R é gratuito, de código aberto, e conta com milhares de pacotes que estendem as suas capacidades a praticamente

todas as áreas da estatística, da ciência de dados e da computação científica. Tornou-se uma linguagem de referência na estatística acadêmica e é amplamente utilizado na investigação biomédica, nas ciências sociais, na economia, na finança, na indústria e na administração pública. Aprender R não é, por isso, aprender uma ferramenta de nicho: é adquirir uma competência técnica, transferível e valorizada em múltiplos contextos científicos e profissionais.

0.2 Porquê a simulação?

A simulação é o fio condutor deste livro. A ideia é poderosa e, ao mesmo tempo, desarmante de tão simples: em vez de calcular uma probabilidade ou um valor esperado por métodos analíticos, deixamos o computador *repetir a experiência* muitas vezes e observamos o que acontece.

Suponha que quer saber a probabilidade de, ao lançar dez moedas, obter exatamente cinco caras. Pode deduzir a fórmula binomial – e deve aprender a fazê-lo. Mas pode também pedir ao R que “lance” dez moedas cem mil vezes e conte a proporção de vezes em que saíram cinco caras. Os dois valores hão de coincidir, e essa coincidência não é um acaso: é a **Lei dos Grandes Números** em ação. A simulação transforma teoremas abstratos em fenómenos que podemos ver, medir e experimentar.

Ao longo do livro, a simulação servir-nos-á para três propósitos: **compreender** (tornando visíveis conceitos como a variabilidade amostral ou a convergência), **estimar** (aproximando probabilidades, integrais e valores esperados que não sabemos calcular de outra forma) e **validar** (confrontando resultados teóricos com evidência gerada por computador).

0.3 Para quem é este livro

Escrevemos este livro a pensar em estudantes do primeiro ano de licenciaturas em que a estatística tem um papel central – matemática aplicada, estatística, ciência de dados, economia, engenharia, biologia — e, mais geralmente, em qualquer pessoa que queira aprender a programar em R com o objetivo de fazer estatística.

Não é preciso saber programar. Começamos do princípio, com a instalação do software e os primeiros cálculos, e avançamos gradualmente. Pressupomos apenas os conhecimentos de matemática do ensino secundário e uma curiosidade genuína.

Do lado da estatística e da probabilidade, o livro é razoavelmente autossuficiente: os conceitos necessários são introduzidos à medida que fazem falta. Ainda assim, os leitores que estejam a estudar simultaneamente uma disciplina de probabilidades encontrarão aqui o complemento computacional ideal — o lugar onde a teoria ganha corpo.

0.4 Como está organizado

O livro segue uma progressão deliberada, em que cada capítulo prepara o seguinte.

Na **primeira parte** aprendemos a linguagem: instalamos o R e o RStudio, usamos o R como calculadora, conhecemos as estruturas de dados fundamentais (vetores, matrizes, tabelas e listas) e as ferramentas que dão vida à programação – estruturas de decisão, funções, ciclos e leitura de dados.

Na **segunda parte** viramo-nos para a visualização e a manipulação de dados, aprendendo a produzir gráficos informativos e a transformar conjuntos de dados reais.

Na **terceira parte** chegamos ao coração do livro: probabilidade, geração de números aleatórios e métodos de simulação – o método da transformada inversa, o método da aceitação-rejeição, a ilustração dos teoremas limite e o cálculo de integrais por simulação (Monte Carlo).

Cada capítulo alterna explicação, código comentado e exemplos aplicados, e termina com uma coleção de exercícios. As soluções encontram-se no final do livro.

0.5 Como tirar o máximo partido

Programe enquanto lê. Não há forma de aprender a programar apenas lendo sobre programação, tal como não se aprende a nadar a ver vídeos de natação. Tenha o RStudio aberto ao lado do livro, escreva o código à medida que avança, experimente variações e não tenha medo dos erros — em programação, um erro é apenas informação sobre o que ajustar.

As caixas coloridas ao longo do texto assinalam informação de tipos diferentes:

- As caixas de **nota** contêm observações e esclarecimentos úteis.
- As caixas de **dica** oferecem sugestões práticas e boas práticas.
- As caixas de **atenção** avisam para erros comuns e armadilhas frequentes.

O código aparece em caixas próprias. Quando uma linha começa por **##**, trata-se do **resultado** que o R devolve – foi assim que combinámos representar aquilo que apareceria no ecrã depois de executar o comando anterior.

0.6 Uma nota final

Um livro como este não substitui a prática paciente, nem a discussão com colegas e docentes; pretende, antes, ser um companheiro fiável nesse processo de aprendizagem. Se, ao chegar ao fim, o leitor olhar para um problema estatístico e a sua primeira reacção for “posso simular isto para perceber melhor o que acontece”, então este livro terá cumprido o seu propósito.

Bom trabalho – e bem-vindo ao laboratório.

Capítulo 1

Estatística, computação e simulação

Antes de escrevermos a primeira linha de código, vale a pena recuar um pouco. A programação, a matemática e a estatística não surgiram no vazio: fazem parte de uma longa história humana de representação, contagem, cálculo e decisão.

Desde muito cedo, os seres humanos precisaram de comunicar. A linguagem permitiu descrever o mundo, transmitir experiências, coordenar ações e preservar conhecimento. Mas comunicar não bastava. Era também necessário distinguir quantidades: saber se havia poucos ou muitos frutos numa árvore, quantos animais tinham sido avistados, quantos membros havia num grupo, quantos objectos tinham sido trocados ou quantos dias faltavam para determinado acontecimento.

Podemos imaginar uma situação elementar. Um pequeno grupo encontra uma árvore carregada de maçãs. Antes de decidir parar, colher e repartir os frutos, precisa de avaliar rapidamente se há alimento suficiente para todos – e se vale a pena correr o risco de permanecer naquele lugar, exposto ao frio, à distância ou a predadores. Mesmo sem símbolos formais, há aqui uma forma primitiva de pensamento quantitativo: comparar, estimar, contar, decidir.

Dessa necessidade de lidar com quantidades nasceram os números. Dos números nasceu a aritmética; da aritmética, formas cada vez mais sofisticadas de cálculo; e, ao longo dos séculos, a matemática tornou-se uma linguagem rigorosa para descrever relações, padrões, formas, movimentos e incertezas. Sempre que uma sociedade precisou de medir terras, construir edifícios, navegar, cobrar impostos, organizar calendários, estudar populações ou compreender fenómenos naturais, precisou também de desenvolver instrumentos matemáticos.

Mas calcular sempre foi difícil. Durante muito tempo, fazer contas exigia esforço, tempo e engenho. Por isso surgiram instrumentos auxiliares: marcas em ossos ou pedras, ábacos, tabelas numéricas, régua de cálculo, máquinas mecânicas, calculadoras electrónicas e, finalmente, computadores. Cada uma destas ferramentas ampliou a nossa capacidade de calcular; mas o computador alterou a escala do problema. Aquilo que antes demorava horas, dias ou era simplesmente impraticável passou a poder ser feito em segundos.

A estatística insere-se nesta história como a disciplina que nos ajuda a raciocinar com dados, variabilidade e incerteza. Quando observamos apenas uma parte de uma população, quando medimos fenómenos sujeitos a erro, quando comparamos tratamentos, quando estudamos riscos ou quando tentamos prever acontecimentos futuros, estamos no domínio da estatística. Como mostra David Salsburg no livro *The Lady Tasting Tea: How Statistics Revolutionized Science in the Twentieth Century*, a estatística transformou profundamente a prática científica moderna, desde o desenho de experiências até à inferência a partir de dados. A estatística transforma dados em informação e ajuda-nos a tomar decisões quando a certeza absoluta não está disponível.

Hoje, porém, a estatística é inseparável da computação. Os dados são maiores, os modelos são mais complexos e muitos dos métodos modernos dependem de algoritmos, simulações e cálculo intensivo. Por isso, alguém que estuda ou trabalha com dados, incerteza e modelos precisa de aprender a programar. Não para substituir o pensamento estatístico, mas para o tornar praticável, reprodutível e aplicável a problemas reais.

Esta é, em grande medida, a justificação para este livro. Aprender R não é apenas aprender comandos; é aprender uma forma de pensar com dados. É ganhar a capacidade de transformar uma pergunta numa sequência de operações claras, verificáveis e repetíveis. É poder explorar, simular, visualizar e modelar fenómenos aleatórios com uma precisão que seria impossível apenas com papel, lápis e calculadora.

1.1 A estatística tornou-se computacional

Durante grande parte do século XX, a estatística viveu com um travão permanente: a dificuldade de fazer contas. Os métodos que se ensinavam e usavam eram, muitas vezes, aqueles que se conseguiam calcular à mão ou com uma calculadora – tabelas de distribuições impressas no fim dos manuais, aproximações engenhosas, fórmulas fechadas cuidadosamente deduzidas para evitar cálculos impraticáveis.

O computador mudou tudo. Aquilo que antes era impossível de calcular passou a ser uma questão de segundos. E, com isso, mudou também aquilo que faz sentido ensinar e aprender. Já não precisamos de decorar valores de tabelas nem de nos limitar aos poucos modelos que admitem solução analítica simples. Podemos explorar dados reais, ajustar modelos complexos e, sobretudo, simular.

Não estamos a dizer que a teoria deixou de importar – pelo contrário. Continuamos a precisar de compreender porque é que os métodos funcionam. O que mudou é que a teoria e o computador passaram a trabalhar em conjunto: a teoria dá-nos a compreensão; o computador dá-nos o alcance.

1.2 O que é o R

O R é uma linguagem de programação e um ambiente de trabalho concebidos para a computação estatística, a análise de dados e a produção de gráficos. A sua origem está ligada à linguagem S, criada nos Bell Laboratories como uma linguagem orientada para a análise interactiva de dados. O R nasceu mais tarde como uma implementação livre dessa tradição: foi criado por Ross Ihaka e Robert Gentleman, na Universidade de Auckland, na Nova Zelândia, e o próprio nome “R” joga simultaneamente com as iniciais dos seus criadores e com a ligação à linguagem S.

Esta origem é importante porque explica a natureza do R. O R não foi pensado, em primeiro lugar, como uma linguagem genérica de programação, mas como uma linguagem para pensar estatisticamente com dados. Por isso, desde o início, operações com vetores, matrizes, tabelas, distribuições de probabilidade, modelos estatísticos e gráficos fazem parte do seu vocabulário natural. Aquilo que noutras linguagens exige bibliotecas externas ou estruturas mais pesadas surge, em R, como parte da própria gramática da linguagem.

O R é também gratuito e de código aberto. Qualquer pessoa o pode descarregar, usar, inspecionar e modificar. Esta característica teve uma consequência decisiva: reduziu a barreira de entrada ao software estatístico e permitiu que estudantes, docentes, investigadores, instituições públicas e empresas tivessem acesso à mesma ferramenta, independentemente do orçamento disponível. Nesse sentido, o R tornou-se uma ferramenta científica profundamente democrática.

Mas talvez a característica mais transformadora do R seja o seu ecossistema de pacotes. Para além das funções que vêm instaladas com o próprio R, existe o CRAN – Comprehensive R Archive Network –, uma rede de servidores que reúne, verifica e disponibiliza pacotes desenvolvidos pela comunidade. Um pacote é uma extensão do R: pode acrescentar novas funções, novos métodos estatísticos, novos conjuntos de dados, novas formas de visualização ou novas interfaces com outros sistemas.

Na prática, o CRAN funciona como uma grande biblioteca pública de métodos computacionais. Quando alguém desenvolve uma nova técnica estatística, um novo algoritmo ou uma ferramenta útil para análise de dados, pode disponibilizá-la sob a forma de pacote. Outros utilizadores podem então instalar esse pacote directamente no R e aplicar essas ferramentas aos seus próprios problemas. É esta arquitectura aberta que explica por que razão o R acompanha tão rapidamente a evolução da estatística moderna: muitos métodos recentes chegam ao R através de pacotes antes de aparecerem em software comercial.

Em termos práticos, o R reúne num só ambiente a manipulação de dados, o cálculo numérico, a simulação, a inferência estatística, a construção de modelos, a programação e a produção de gráficos de qualidade. Esta combinação torna-o particularmente adequado para aprender estatística de forma activa: não apenas lendo fórmulas, mas experimentando, simulando, visualizando e verificando propriedades nos próprios dados.

O facto de o R ser uma linguagem de programação completa distingue-o de uma folha de cálculo. Numa folha de cálculo, tendemos a organizar os dados em células e a escrever operações ligadas

à sua posição. Em R, escrevemos instruções que descrevem o que queremos fazer: transformar uma variável, aplicar uma função, repetir uma simulação, ajustar um modelo ou produzir um gráfico. Esta mudança de perspectiva é uma das fontes da flexibilidade do R.

A isto junta-se uma comunidade científica ampla, activa e internacional. O R tornou-se uma das linguagens de referência na estatística académica e é amplamente utilizado na investigação biomédica, nas ciências sociais, na economia, na finança, na indústria e na administração pública. Aprender R, portanto, não é aprender uma ferramenta de nicho: é adquirir uma competência técnica, transferível e valorizada em múltiplos contextos científicos e profissionais.

Neste livro, usaremos essa competência com um propósito central: compreender fenómenos aleatórios através da simulação. Antes de avançarmos para a sintaxe da linguagem, vale a pena introduzir esta ideia, porque ela será o fio condutor de grande parte do percurso.

1.3 A ideia central: simular

A palavra “simulação” percorre este livro de uma ponta à outra. Vale a pena introduzi-la desde já, com um exemplo minúsculo, para que o leitor perceba de imediato o poder da ideia.

Imagine que lançamos um dado equilibrado e queremos saber qual é o valor médio que sai a longo prazo. A teoria responde sem hesitar: a média é

$$\frac{(1 + 2 + 3 + 4 + 5 + 6)}{6} = 3.5.$$

Mas há uma segunda maneira de chegar à mesma resposta, sem fazer a conta: podemos *lançar o dado muitas vezes* e calcular a média dos resultados. Em R, isso escreve-se assim:

```
# Lançar um dado equilibrado 1 000 000 de vezes
lancamentos <- sample(1:6, size = 1000000, replace = TRUE)

# Média dos resultados
mean(lancamentos)
## [1] 3.499
```

O valor obtido – próximo de 3.5 – não é imposto por nenhuma fórmula: *emerge* da repetição da experiência. Esta é a essência da simulação. Em vez de deduzir o resultado, construímos uma versão computacional do fenómeno aleatório e observamos o seu comportamento.

Ainda não é preciso perceber cada detalhe do código acima – a função `sample()`, o argumento `replace`, e assim por diante. Tudo isto será explicado nos próximos capítulos. O importante, por agora, é ficar com a intuição: **o computador pode repetir uma experiência aleatória tantas vezes quantas quisermos, e a partir dessas repetições estimar aquilo que nos interessa.**

Este pequeno exemplo já contém, em pequena escala, tudo o que faremos mais tarde de forma mais ambiciosa. Quando não soubermos calcular uma probabilidade, estimá-la-emos por simulação. Quando um integral não tiver solução fechada, aproximá-lo-emos por simulação. Quando quisermos ver um teorema a funcionar, ilustrá-lo-emos por simulação. Mas nada disto será possível sem primeiro dominarmos a linguagem – e é para isso que servem os capítulos que se seguem.

1.4 O caminho que vamos percorrer

O livro está organizado como uma subida gradual. Cada capítulo introduz uma nova ferramenta, mas essa ferramenta não aparece isolada: serve para preparar o passo seguinte. A ideia é que o leitor comece por ganhar familiaridade com o R e termine capaz de escrever pequenas simulações estatísticas de forma autónoma.

Começamos por **instalar e conhecer o ambiente de trabalho** (Capítulo 2) e por usar o R como uma **calculadora programável**: uma ferramenta que calcula, guarda objectos, avalia expressões e toma decisões (Capítulo 3). Em seguida, aprendemos a organizar informação em **estruturas de dados** – vetores, matrizes, tabelas e listas (Capítulo 4) – e a controlar o comportamento dos programas com **estruturas de selecção, funções, scripts**, ciclos **for** e ciclos **while**.

A segunda parte do percurso aproxima-nos dos dados reais. Aprendemos a ler informação a partir de ficheiros, a encadear operações com o operador pipe, a manipular tabelas, a produzir gráficos básicos e a construir visualizações mais elaboradas com **ggplot2**. Nesta fase, o R deixa de ser apenas uma calculadora e passa a ser um laboratório de análise de dados.

A terceira parte introduz a linguagem da **probabilidade**. Estudamos experiências aleatórias, amostragem, distribuições de probabilidade e as funções do R associadas a essas distribuições – as famílias **d**, **p**, **q** e **r**. A partir daqui, o computador passa a ser usado não só para calcular, mas também para gerar aleatoriedade controlada.

Na parte final, todos estes elementos convergem na **simulação estatística**. Exploramos o método de Monte Carlo, a geração de variáveis aleatórias, a transformada inversa, o método de aceitação-rejeição e, por fim, a aproximação de integrais por simulação. O último capítulo, dedicado ao **cálculo de integrais por Monte Carlo**, ilustra bem o espírito do livro: usar programação, probabilidade e estatística para resolver problemas que seriam difíceis, ou mesmo impraticáveis, apenas com cálculo analítico.

Assim, o percurso vai do primeiro contacto com o R até à simulação de fenómenos aleatórios e à resolução computacional de problemas estatísticos. O objectivo não é apenas aprender comandos, mas desenvolver uma forma de pensar: transformar uma pergunta quantitativa num procedimento claro, reproduzível e verificável.

Se está a ler este livro no âmbito de uma disciplina, use os capítulos como um guia de laboratório: leia com o RStudio aberto, execute cada exemplo e resolva os exercícios no fim de cada capítulo antes de avançar. A programação aprende-se com as mãos, não apenas com os olhos.

Comecemos, então, pelo princípio: pôr as ferramentas a funcionar.

Capítulo 2

O R e o RStudio

No capítulo anterior vimos por que razão a estatística moderna é inseparável da computação e por que motivo o R se tornou uma linguagem central para trabalhar com dados, incerteza e simulação. Neste capítulo passamos da motivação à prática: vamos preparar o ambiente de trabalho.

Trabalhar com R implica, na prática, instalar dois programas complementares. O primeiro é o **R**, isto é, o motor que executa os cálculos, interpreta os comandos e contém as funções fundamentais da linguagem. O segundo é o **RStudio**, uma interface que torna a escrita, a organização e a execução do código muito mais cómodas.

O objectivo deste capítulo é simples: instalar o R, instalar o RStudio e fazer uma primeira visita guiada ao ambiente onde iremos trabalhar ao longo do livro. No fim, o leitor terá um pequeno laboratório de estatística computacional a funcionar no seu computador e estará pronto para escrever os primeiros comandos.

2.1 Instalação

2.1.1 Instalar o R

A versão base do R, que inclui o conjunto fundamental de funções, descarrega-se do sítio oficial do projecto:

<https://www.r-project.org/>

A partir dessa página, basta seguir a ligação para o **CRAN** (*Comprehensive R Archive Network*), escolher o sistema operativo – Windows, macOS ou Linux – e instalar o programa como qualquer outra aplicação. O CRAN é a rede oficial de repositórios do R: além de disponibilizar o próprio R, é também a partir dele que, mais tarde, instalaremos muitos dos pacotes usados para alargar as capacidades da linguagem.

2.1.2 Instalar o RStudio

Poderíamos trabalhar apenas com o R base, mas escrever código dessa forma é pouco confortável. Por isso, instala-se quase sempre um **ambiente de desenvolvimento integrado** – em inglês, *Integrated Development Environment* ou **IDE**. Um IDE ajuda a escrever, guardar e organizar scripts, executar comandos, visualizar objectos, consultar gráficos e acompanhar os resultados.

A escolha habitual para trabalhar com R é o **RStudio Desktop**, gratuito e disponível em:

<https://posit.co/download/rstudio-desktop/>

A ordem importa: instale **primeiro** o R e **só depois** o RStudio. O RStudio não substitui o R – precisa de o encontrar já instalado no computador para conseguir executar os comandos. Se instalar apenas o RStudio, terá a interface, mas não terá o motor da linguagem.

2.1.3 Pacotes: estender o R

Além do conjunto base de funções, o R oferece milhares de **pacotes** adicionais, desenvolvidos e partilhados pela comunidade. Um pacote é uma coleção de funções e dados que acrescenta novas capacidades ao R – desde gráficos elegantes até modelos estatísticos sofisticados.

Instalar um pacote faz-se uma única vez, com a função `install.packages()` (é preciso estar ligado à Internet):

```
install.packages("ggplot2")
```

Para ver todos os pacotes já instalados no seu sistema:

```
installed.packages()
```

Distinga bem dois passos que são frequentemente confundidos. **Instalar** um pacote (com `install.packages()`) faz-se uma vez só – descarrega o pacote para o computador. **Carregar** um pacote (com `library()`) faz-se em cada sessão em que o queremos usar – torna as suas funções disponíveis. É como comprar um livro (uma vez) e depois tirá-lo da estante sempre que precisamos dele.

```
library(ggplot2) # torna as funções do pacote disponíveis nesta sessão
```

2.2 Uma visita guiada ao RStudio

Ao abrir o RStudio pela primeira vez, o ecrã pode parecer intimidante, com vários painéis ao mesmo tempo. Na verdade, são quatro áreas, cada uma com uma função clara. Vale a pena conhecê-las, porque vamos passar muitas horas neste ambiente.

O **Editor de *scripts*** (por omissão, no canto superior esquerdo) é onde escrevemos e guardamos o nosso código. Um *script* é um ficheiro de texto com uma sequência de comandos – o “guião” da nossa análise. O editor oferece realce de sintaxe (que colore o código para o tornar legível), completamento automático e a possibilidade de executar linhas ou blocos de código diretamente. É aqui que deve escrever o código que quer conservar.

A **Consola** (canto inferior esquerdo) é onde os comandos são efetivamente executados e onde aparecem os resultados. Podemos escrever comandos diretamente na consola para testes rápidos, mas, ao contrário do editor, o que escrevemos na consola não fica guardado. Pense na consola como uma folha de rascunho e no editor como o caderno definitivo.

O **painel *Environment/History*** (canto superior direito) mostra, no separador *Environment*, todos os objetos atualmente na memória da sessão – os dados e variáveis que fomos criando. O separador *History* guarda o histórico de comandos executados, permitindo recuperá-los facilmente.

O **painel inferior direito** é multifuncional e reúne vários separadores úteis: *Files* (os ficheiros do diretório de trabalho), *Plots* (os gráficos que produzimos), *Packages* (os pacotes instalados e carregados), *Help* (o sistema de ajuda) e *Viewer* (para visualizar documentos HTML e outros conteúdos).

O separador **Help** será um dos seus melhores amigos. Todo o R está documentado, e aprender a ler essa documentação é uma competência que compensa cultivar desde cedo. Veremos, já no próximo capítulo, como pedir ajuda sobre qualquer função.

2.3 Atalhos que poupam tempo

Programar é uma atividade repetitiva, e os bons atalhos de teclado fazem uma diferença real ao fim de algumas horas. Estes são os mais úteis para começar:

Atalho	O que faz
CTRL + ENTER	Executa a(s) linha(s) selecionada(s) no <i>script</i>
ALT + -	Insere o operador de atribuição <-
CTRL + SHIFT + M	Insere o operador <i>pipe</i> (%>% ou >)
CTRL + 1	Move o cursor para o painel do <i>script</i>
CTRL + 2	Move o cursor para a consola

Atalho	O que faz
CTRL + ALT + I	Insere um novo bloco de código num documento R Markdown
CTRL + SHIFT + K	Compila um documento R Markdown
ALT + SHIFT + K	Abre a lista completa de atalhos

Num Mac, os atalhos são geralmente equivalentes aos usados em Windows ou Linux, mas a tecla **CTRL** é normalmente substituída por **Command** e a tecla **ALT** por **Option**.

Com as ferramentas instaladas e o ecrã já familiar, estamos prontos para o que realmente interessa: começar a dar ordens ao R. É o que faremos no próximo capítulo, onde o usaremos primeiro como uma calculadora e, depois, como algo bem mais poderoso.

Capítulo 3

O R como calculadora

A melhor forma de começar a programar em R é usá-lo primeiro para aquilo que ele faz de modo mais imediato: calcular. Neste capítulo, tratamos o R como uma calculadora muito poderosa. No entanto, à medida que avançarmos, veremos que ele é bastante mais do que isso.

Uma calculadora comum recebe uma expressão, devolve um resultado e, em geral, esquece-o. O R permite-nos fazer algo mais importante: guardar valores, atribuir-lhes nomes e reutilizá-los em cálculos posteriores. Este pequeno passo, passar de calcular para nomear e reutilizar, é uma das portas de entrada para a programação.

Há também uma razão matemática para começar por aqui. A estatística está profundamente ligada à álgebra e ao cálculo com estruturas numéricas. Representamos dados, variáveis e modelos através de números, vetores, matrizes e tabelas, e manipulamo-los por meio de operações aritméticas e transformações. Antes de avançarmos para comandos estatísticos mais específicos, convém compreender bem como o R lida com números, valores e operações elementares.

3.1 A consola e o primeiro comando

O R aceita comandos numa interface de linha de comando, assinalada pelo símbolo `>`, a que chamamos **prompt**. Quando escrevemos directamente na **Consola**, basta escrever o comando a seguir ao *prompt* e pressionar **Enter**: o R interpreta o comando, executa-o e mostra o resultado.

```
print("O meu primeiro comando em R!")  
## [1] "O meu primeiro comando em R!"
```

No RStudio, contudo, é comum escrevermos os comandos num **script** e enviá-los depois para a Consola. Nesse caso, colocamos o cursor na linha que queremos executar e usamos **Ctrl + Enter** no Windows ou no Linux, ou **Cmd + Enter** no macOS.

Ao longo do livro, o código que escrevemos aparece em caixas próprias. Quando uma linha começa por **##**, trata-se do **resultado** devolvido pelo R, isto é, aquilo que apareceria na Consola depois de executar o comando anterior. Assim, pode ler cada bloco como um diálogo: primeiro o que pedimos, depois o que o R respondeu.

O [1] que precede o resultado indica a posição do primeiro elemento mostrado nessa linha. Para um único valor parece supérfluo, mas quando o R devolve muitos valores, esta numeração ajuda-nos a localizá-los.

3.2 Comentários: escrever para humanos

O carácter **#** inicia um **comentário**: tudo o que vem depois dele, na mesma linha, é ignorado pelo R. Os comentários não servem para o computador – servem para nós e para quem, mais tarde, ler o nosso código (muitas vezes, esse alguém somos nós próprios, uns meses depois).

```
# Este é um comentário: o R ignora-o  
2 + 2    # o R calcula 2 mais 2  
## [1] 4
```

Comente o *porquê*, não o *quê*. Uma linha como `x <- x + 1 # incrementa x` é inútil, porque o comentário apenas repete o código. Um bom comentário explica a intenção: `x <- x + 1 # passamos ao dia seguinte`.

3.3 Pedir ajuda

Ninguém decora todas as funções do R, nem precisa de o fazer. O essencial é saber consultar a documentação. Se conhece o nome de uma função, basta precedê-lo de um ponto de interrogação para abrir a respectiva página de ajuda:

```
?sum
```

A ajuda descreve o que a função faz, quais são os seus argumentos (isto é, os valores ou opções que fornecemos à função para ela executar uma tarefa) e o que devolve. Para além disso, muitas funções trazem exemplos prontos a correr, que podem ser vistos com `example()`:

```
example(sum)
```

No comando `example(sum)`, o argumento da função `example()` é `sum`. Isto significa que estamos a pedir ao R para mostrar exemplos de utilização da função `sum()`.

Aprender a ler estas páginas é uma das competências mais rentáveis para quem programa em R.

3.4 Objetos e variáveis

Chegamos ao salto referido na abertura. Em R, tudo o que criamos ou carregamos numa sessão é um **objeto**: um número, um texto, um vetor, uma tabela ou até uma função. Um objeto é uma unidade de informação guardada na memória.

Uma **variável** é simplesmente um *nome* que associamos a um objeto. Ao criar uma variável, criamos um rótulo que nos permite aceder ao objecto guardado na memória sempre que quisermos, sem ter de o recalculá-lo ou escrever novamente.

Nesta descrição usamos a ideia de “rótulo” apenas como uma imagem intuitiva. Em linguagens de programação, é comum falar de nomes, referências e ponteiros para descrever a relação entre uma variável e o objeto armazenado na memória. No R, porém, o funcionamento interno da memória é mais subtil e não será necessário estudar ponteiros neste livro. Para os nossos objetivos, basta compreender que um nome permite guardar, recuperar e reutilizar um objeto. Leitores interessados numa descrição mais avançada podem consultar *Advanced R*, de Hadley Wickham, em particular os capítulos sobre nomes, valores e memória.

3.4.1 Atribuição

Atribuir um valor a uma variável faz-se com o operador `<-` (uma seta que se lê “recebe”). A expressão seguinte cria uma variável chamada `x` e guarda-lhe o valor 10:

```
x <- 10
```

O operador `<-` guarda o valor do lado direito na variável do lado esquerdo. Do lado esquerdo deve estar apenas um nome de variável. E, uma vez criada, a variável pode ser reutilizada:

```
a <- 10
b <- a + 1  # b recebe o valor de a mais 1
b
## [1] 11
```

Uma atribuição **não é** uma equação matemática. Escrever `a + 2 <- 10` não faz sentido: do lado esquerdo tem de estar um nome, não uma expressão a calcular. Embora o R também aceite `=` e `->` para atribuir, recomendamos o uso de `<-`, para não confundir a atribuição (`<-`) com o teste de igualdade (`==`), que veremos mais adiante.

Podemos guardar não só um número, mas uma sequência de valores. A função `c()` (de *combine*, combinar) junta vários valores num **vetor**:


```
X <- c(2, 12, 22, 32)
X
## [1]  2 12 22 32
```

O R **distingue maiúsculas de minúsculas**. `X` e `x` são dois objetos diferentes. Este é um dos erros mais comuns de quem começa, por isso convém gravá-lo desde já.

Para ver todos os objetos que existem atualmente na memória da sessão, usamos `ls()` (ou consultamos o separador *Environment* no RStudio):

```
ls()
```

3.4.2 Regras para nomear variáveis

A escolha dos nomes das variáveis não é só uma questão de gosto: bons nomes tornam o código legível, e o R impõe algumas regras.

Um nome de variável deve começar por uma letra, embora o R também aceite alguns nomes começados por ponto. Depois do primeiro carácter, o nome pode conter letras, números, pontos e sublinhados. Não pode conter espaços nem caracteres especiais como `@`, `#`, `$` ou `%`. Assim, `nome_cliente2` é válido; `2nome` ou `nome cliente` não são.

Também não podemos usar **palavras reservadas** – termos com significado especial no R, como `if`, `else`, `for`, `while`, `TRUE`, `FALSE`, `function`.

E, como já vimos, o R distingue maiúsculas de minúsculas:

```
idade <- 20
Idade <- 30
```

Aqui, `idade` e `Idade` são duas variáveis distintas.

Embora o R permita nomes de objetos que começam por ponto, como `.valor` ou `.dados`, convém evitar essa prática nesta fase. Em sistemas como macOS e Linux, ficheiros ou pastas cujo nome começa por ponto são tratados como ficheiros ocultos. Por exemplo, um ficheiro chamado `.dados.csv` pode não aparecer normalmente no Finder. Para evitar confusões, neste livro usaremos nomes simples e explícitos, sem ponto inicial.

Prefira nomes que descrevam o conteúdo. `nota_final` diz-nos muito mais do que `x`. Uma convenção comum e legível é usar minúsculas e separar palavras com sublinhados: `faculdade_de_ciencias`. Código escrito para ser lido é código que se consegue corrigir e reutilizar.

3.5 Tipos de dados

Nem todos os valores são da mesma natureza. Um número comporta-se de forma diferente de um texto, e ambos diferem de um valor lógico. Em R, os principais tipos de dados que vamos encontrar são:

- **Numeric** – números, inteiros ou decimais. Exemplos: 42, 3.14.
- **Character** – texto (cadeias de caracteres). Exemplo: "Olá".
- **Logical** – valores lógicos, TRUE ou FALSE.
- **Vectors** – coleções de elementos do mesmo tipo, como `c(1, 2, 3)`.
- **Factors** – variáveis categóricas (por exemplo, “baixo”, “médio”, “alto”).
- **Data frames** – tabelas com linhas e colunas, ao estilo de uma folha de cálculo.
- **Lists** – coleções que podem misturar elementos de tipos diferentes.

Os quatro primeiros aparecem já neste capítulo; os restantes serão o tema do Capítulo 4. Um primeiro contacto:

```
a <- 3.14           # numeric
b <- "Programação R" # character
c <- 3 < 2          # logical (resultado de uma comparação)
d <- c(1, 2, 3)     # vetor
```

3.5.1 Tipagem dinâmica

Uma característica cómoda do R é a **tipagem dinâmica**: não precisamos de declarar o tipo de uma variável, o R deduz-o a partir do valor que lhe atribuímos. A função `class()` diz-nos o tipo (a *classe*) de um objeto:

```
x <- 5
class(x)
## [1] "numeric"

y <- "Cinco"
class(y)
## [1] "character"

z <- TRUE
class(z)
## [1] "logical"
```

Existe ainda a função `typeof()`, que revela o tipo de armazenamento interno, o modo como o R guarda o objeto na memória. Nem sempre coincide com `class()`, e essa distinção é por vezes reveladora:

```
x <- 1:10
class(x)
## [1] "integer"
typeof(x)
## [1] "integer"

y <- c(1.1, 2.2, 3.3)
class(y)
## [1] "numeric"
typeof(y)
## [1] "double"

z <- data.frame(a = 1:3, b = c("A", "B", "C"))
class(z)
## [1] "data.frame"
typeof(z)
## [1] "list"
```

Repare no último caso: uma tabela (`data.frame`) tem classe `"data.frame"` mas, internamente, é armazenada como uma **lista**. Voltaremos a este facto quando estudarmos as estruturas de dados.

3.5.2 Converter entre tipos

Muitas vezes, sobretudo quando importamos dados de fontes externas, um valor chega-nos com o tipo “errado”: um número guardado como texto, por exemplo. As funções da família `as.*()` convertem entre tipos:

```
b <- as.character(15)    # de número para texto
b
## [1] "15"

y <- as.integer(1.5)     # de decimal para inteiro (trunca)
y
## [1] 1

w <- as.numeric("10")    # de texto para número
```

```
W
## [1] 10
```

3.6 Operadores aritméticos

Com valores para manipular, vejamos as operações que os combinam. Os operadores aritméticos do R são os que seria de esperar, com dois acréscimos úteis: a divisão inteira e o resto.

Operador	Descrição	Exemplo
+	Adição	5 + 2 dá 7
-	Subtração	5 - 2 dá 3
*	Multiplicação	5 * 2 dá 10
/	Divisão	5 / 2 dá 2.5
%%	Divisão inteira	5 %% 2 dá 2
%/%	Resto da divisão	5 %/% 2 dá 1
^	Potência	5 ^ 2 dá 25

Alguns exemplos, incluindo funções matemáticas comuns:

```
5 * 21
## [1] 105

sqrt(9)      # raiz quadrada
## [1] 3

3^3
## [1] 27

log(9)       # logaritmo natural
## [1] 2.197

log10(9)     # logaritmo de base 10
## [1] 0.9542

exp(1)       # número de Euler, e
## [1] 2.718
```

Os operadores %% e %/% são mais úteis do que parecem. O resto %% responde a perguntas como “este número é par?” (`n %% 2 == 0`) ou “esta hora, somadas algumas horas, dá que horas no relógio de 24?” – questões que aparecerão nos exercícios. Vale a pena habituar-se a eles.

3.6.1 Prioridade das operações

Como na matemática, as operações têm uma ordem de prioridade: primeiro potências, depois multiplicações e divisões, e só então adições e subtrações. Os parênteses alteram essa ordem e, na dúvida, tornam o código mais claro:

```
19 + 26/4 - 2*10
## [1] 5.5

((19 + 26) / (4 - 2)) * 10
## [1] 225
```

3.6.2 Controlar os dígitos apresentados

Por vezes queremos ver mais (ou menos) casas decimais. A opção global `digits` controla quantos algarismos significativos o R apresenta:

```
exp(1)
## [1] 2.718282

options(digits = 20)
exp(1)
## [1] 2.718281828459045091

options(digits = 7)    # o valor por omissão
exp(1)
## [1] 2.718282
```

Isto altera apenas a *apresentação*: internamente, o R continua a guardar o valor com toda a precisão disponível. Mudar `digits` nunca arredonda os cálculos, apenas o que vemos no ecrã.

3.7 Objetos predefinidos e valores especiais

O R traz consigo constantes matemáticas úteis, como `pi` (o número π), e representações para situações que a aritmética habitual não cobre.

```
pi
## [1] 3.141593

1/0
```

```
## [1] Inf

-1/0
## [1] -Inf

0/0
## [1] NaN

sqrt(-1)
## Warning: NaNs produced
## [1] NaN
```

Estes resultados especiais têm nomes e significados precisos, que importa distinguir:

- **Inf** (e **-Inf**) representam o infinito, resultado de operações como dividir por zero.
- **NaN** (*Not a Number*) representa um resultado numérico **indefinido**, como 0/0 ou a raiz quadrada de um número negativo. É um número que não é número.
- **NA** (*Not Available*) representa um valor **ausente**, um dado que não foi medido ou que está em falta. Ao contrário de NaN, o NA não é necessariamente numérico: pode representar qualquer tipo de dado em falta.

A distinção entre NaN e NA é subtil mas importante: o primeiro é fruto de uma conta impossível; o segundo, de informação que simplesmente não temos.

3.7.1 Lidar com valores ausentes

Os valores ausentes são frequentes em dados reais: questionários com perguntas por responder, sensores que falharam, registos incompletos ou medições que não chegaram a ser feitas. Por isso, o R trata estes casos com prudência. Por omissão, qualquer cálculo que envolva um NA devolve também NA. A razão é simples: se um dos valores é desconhecido, o resultado do cálculo pode também ser desconhecido. No caso da média, por exemplo, não podemos determinar o seu valor exacto sem saber todos os valores que entram no cálculo.

```
mean(c(1, 2, 3, NA, 5))
## [1] NA
```

Quando queremos, ainda assim, calcular ignorando os valores em falta, usamos o argumento `na.rm = TRUE` (*remove NA*), disponível na maioria das funções de resumo:

```
mean(c(1, 2, 3, NA, 5), na.rm = TRUE)
## [1] 2.75

sum(c(1, 2, 3, NA, 5), na.rm = TRUE)
## [1] 11
```

Não se pode testar a ausência com o operador de igualdade. `x == NA` **não** devolve `TRUE/FALSE` – devolve `NA`, porque comparar algo com um valor desconhecido é, ele próprio, desconhecido. Para detetar valores em falta, usamos a função `is.na()`:

```
y <- c(1, 2, NA, 4, 5)

y == NA          # não funciona como se poderia esperar
## [1] NA NA NA NA NA

is.na(y)         # a forma correta
## [1] FALSE FALSE TRUE FALSE FALSE
```

Existe ainda `is.nan()`, específica para detetar `NaN`. Note a relação entre as duas: `is.na()` deteta tanto `NA` como `NaN` (afinal, um `NaN` também não é um valor disponível), ao passo que `is.nan()` só deteta `NaN`.

3.8 Exibir e combinar texto

Para além de calcular, precisamos frequentemente de **comunicar** resultados – mostrá-los ao utilizador, combiná-los com texto explicativo, ou pedir dados a quem executa o programa. Algumas funções essenciais para isso:

- `print()` – exibe valores e resultados na consola.
- `readline()` – recebe uma entrada escrita pelo utilizador via teclado.
- `paste()` – junta (concatena) textos, com um separador (por omissão, um espaço).
- `paste0()` – como `paste()`, mas sem separador.
- `cat()` – junta e exibe textos e valores de forma direta, sem as aspas de `print()`.

Um primeiro exemplo, combinando duas palavras:

```
nome1 <- "faculdade"
nome2 <- "ciências"
print(paste(nome1, nome2))
## [1] "faculdade ciências"
```

A função `readline()` permite escrever programas interativos, que pedem informação a quem os executa. O que o utilizador escreve chega sempre como **texto**, pelo que muitas vezes é preciso convertê-lo com `as.numeric()` ou `as.integer()`:

```
n <- readline(prompt = "Digite um número: ")
n <- as.integer(n)    # converte o texto num inteiro
print(n + 1)
```

Um exemplo mais completo, que usa `cat()` para uma saudação e trata uma entrada inválida:

```
idade <- readline(prompt = "Digite a sua idade: ")
idade <- as.numeric(idade)

if (is.na(idade)) {
  cat("Entrada inválida. Insira um valor numérico.\n")
} else {
  cat("Tem", idade, "anos.\n")
}
```

A função `readline()` destina-se a uso interactivo, isto é, a situações em que existe uma consola à espera da resposta do utilizador. Por isso, estes exemplos devem ser experimentados directamente na Consola. Num documento compilado, como um ficheiro R Markdown ou bookdown, ou num script executado de forma não interactiva, não há oportunidade para responder ao pedido, e o código pode ficar bloqueado ou falhar.

3.8.1 A função `paste()` com mais detalhe

A função `paste()` é muito versátil. Aceita vários argumentos, combina vetores elemento a elemento e deixa-nos escolher o separador:

```
# Vários argumentos
paste("Data", "Science", "com", "R")
## [1] "Data Science com R"

# Separador específico
paste("2026", "04", "28", sep = "-")
## [1] "2026-04-28"

# Combinação elemento a elemento de dois vetores
nomes <- c("Ana", "Bruno", "Carlos")
```



```

apelidos <- c("Silva", "Oliveira", "Santos")
paste(nomes, apelidos)
## [1] "Ana Silva"      "Bruno Oliveira" "Carlos Santos"

# Colar texto a cada elemento de um vetor
paste("Número", 1:3)
## [1] "Número 1" "Número 2" "Número 3"

# Sem separador, com paste0()
paste0("Olá", "Mundo")
## [1] "OláMundo"

```

3.9 Operadores lógicos e relacionais

Terminamos com os operadores que permitem ao R **tomar decisões**: comparar valores e combinar condições. São a base das estruturas de decisão do Capítulo 5, por isso este é o primeiro passo para escrever programas que reagem aos dados.

3.9.1 Operadores relacionais

Os operadores relacionais comparam valores e devolvem um valor lógico (TRUE ou FALSE):

Operador	Testa se...
<code>a == b</code>	a é igual a b
<code>a != b</code>	a é diferente de b
<code>a > b</code>	a é maior que b
<code>a < b</code>	a é menor que b
<code>a >= b</code>	a é maior ou igual a b
<code>a <= b</code>	a é menor ou igual a b

Alguns exemplos:

```

2 > 1
## [1] TRUE

1 == 2/2
## [1] TRUE

```

```
1 != 2
## [1] TRUE
```

Não confunda = com ==. Um só sinal de igual (=) é uma **atribuição**; dois sinais de igual (==) formam um **teste de igualdade**. Trocar um pelo outro é uma fonte frequente de erros e, felizmente, uma das mais fáceis de corrigir quando aprendemos a reconhecê-la.

3.9.2 Operadores lógicos

Os operadores lógicos combinam ou invertem condições:

- & (**e** lógico): devolve TRUE apenas se **todas** as condições forem verdadeiras.
- | (**ou** lógico): devolve TRUE se **pelo menos uma** condição for verdadeira.
- ! (**não** lógico): inverte o valor lógico.

```
(5 > 3) & (4 > 2)    # ambas verdadeiras
## [1] TRUE

(5 < 3) | (4 > 2)    # basta uma ser verdadeira
## [1] TRUE

!(5 > 3)             # inverte TRUE em FALSE
## [1] FALSE
```

Com estas peças, valores, nomes, operações e comparações, temos já o vocabulário elementar de um programa. Nos exercícios que se seguem, vamos combiná-las para resolver pequenos problemas. No próximo capítulo, aprenderemos a organizar os valores em estruturas mais ricas, que nos permitirão trabalhar com conjuntos de dados a sério.

3.10 Exercícios

1. Escreva um programa em R que leia dois inteiros inseridos pelo utilizador e imprima: a soma dos dois números; o produto; a diferença entre o primeiro e o segundo; a divisão do primeiro pelo segundo; o resto dessa divisão; e o primeiro número elevado à potência do segundo. *Sugestão:* Use `readline()` para a entrada e `as.integer()` para a conversão.
2. Escreva um programa em R que leia dois números decimais e imprima a soma, a diferença, o produto e o primeiro número elevado à potência do segundo. *Sugestão:* Use `as.numeric()` para a conversão.

3. Escreva um programa em R que leia uma distância em milhas e a converta para quilômetros, usando a fórmula $K = M \times 1.609344$.
4. Escreva um programa em R que leia três inteiros correspondentes ao comprimento, à largura e à altura de um paralelepípedo, e imprima o seu volume.
5. Escreva um programa em R que leia três inteiros e imprima a sua média.
6. Escreva um programa em R que leia uma temperatura em graus Fahrenheit e a converta para graus Celsius, usando a fórmula $C = \frac{F - 32}{1.8}$.
7. Escreva um programa em R que leia uma hora no formato de 24 horas e imprima a hora correspondente no formato de 12 horas.
8. São exatamente 14h e definiu um alarme para tocar dentro de 51 horas. A que horas tocará o alarme? *Sugestão:* O operador %% é seu amigo.
9. Generalize o problema anterior: peça ao utilizador a hora atual e o número de horas de espera, e imprima a hora a que o alarme tocará.
10. Escreva um programa em R que leia um número inteiro e imprima TRUE se for maior que 10, ou FALSE caso contrário.
11. Escreva um programa em R que leia dois números e imprima TRUE se forem iguais, ou FALSE caso contrário.
12. Escreva um programa em R que leia dois números e imprima TRUE se o primeiro for maior ou igual ao segundo, ou FALSE caso contrário.
13. Escreva um programa em R que leia um número e imprima TRUE se ele estiver entre 0 e 100 (inclusive), ou FALSE caso contrário.
14. Escreva um programa em R que leia três números e imprima uma mensagem indicando se o primeiro é menor que o segundo e o segundo é menor que o terceiro.

Capítulo 4

Estruturas de dados

Até aqui, trabalhámos sobretudo com valores isolados: um número, um texto ou um resultado lógico. Mas a estatística raramente lida com valores isolados. Lida, quase sempre, com **conjuntos de dados**. As alturas dos estudantes de uma turma, as vendas mensais de uma empresa, as respostas de um inquérito ou as medições recolhidas por um sensor são coleções de valores que queremos guardar, organizar e analisar em conjunto. É aqui que entram as **estruturas de dados**.

Uma estrutura de dados é uma forma de organizar vários valores num único objeto, para que possamos manipulá-los como um todo. Escolher a estrutura certa para cada problema é um passo fundamental: cada estrutura tem a sua finalidade própria, e conhecê-las bem é essencial para programar de forma clara, eficiente e segura em R.

Antes de as estudarmos uma a uma, convém reter que, em R, existem dois grandes tipos de objetos:

- **Funções** – objetos que *executam* operações, como `cos()`, `print()`, `plot()` ou `integrate()`.
- **Dados** – objetos que *contêm* informação, como 23, "Olá", TRUE, ou as estruturas que vamos conhecer agora.

Neste capítulo percorremos as principais estruturas de dados do R, das mais simples às mais flexíveis: os **vectores**, que guardam vários valores do mesmo tipo; os **fatores**, usados para representar dados categóricos; as **matrizes** e os **arrays**, que organizam valores em duas ou mais dimensões; os **data frames**, que representam tabelas de dados; e as **listas**, que permitem reunir objetos de tipos diferentes. Cada uma destas estruturas responde a uma necessidade própria, e aprender a distingui-las é essencial para escolher a ferramenta certa em cada problema.

4.1 Vetor

O **vetor** é a estrutura de dados mais elementar do R e, ao mesmo tempo, uma das mais importantes. Mesmo um único número é tratado internamente como um vetor de comprimento um. Um vetor é uma sequência ordenada de elementos do mesmo tipo: apenas números, apenas texto ou apenas valores lógicos.

Há três ideias essenciais a reter desde já:

- todos os elementos de um vetor têm de pertencer ao mesmo tipo;
- os elementos são indexados a partir de **1**, ou seja, o primeiro elemento está na posição 1, e não na posição 0, como acontece em muitas outras linguagens de programação;
- os vetores são normalmente criados com a função `c()` e podem ser manipulados com uma vasta coleção de funções do R.

4.1.1 Tipos de vetores

```
# Vetor numérico
c(1.1, 2.2, 3.3)
## [1] 1.1 2.2 3.3

# Vetor de caracteres
c("a", "b", "c")
## [1] "a" "b" "c"

# Vetor lógico
c(TRUE, 1 == 2)
## [1] TRUE FALSE

# O que acontece se misturarmos tipos?
c(3, 1 == 2, "a")
## [1] "3"      "FALSE" "a"
```

Repare no último exemplo. Ao tentarmos misturar um número, um valor lógico e um texto, o R **não** dá erro: converte tudo para o tipo mais “geral”, neste caso, texto. A este fenómeno chamamos **coerção**. É cómodo, mas traiçoeiro: se esperava um vetor de números e o R o converteu silenciosamente em texto, os cálculos seguintes falharão. Convém, por isso, garantir que cada vetor contém um só tipo de dados.

4.1.2 Construir vetores

Escrever todos os elementos à mão seria impraticável para vetores longos. O R oferece atalhos poderosos para os construir:

```
# Inteiros de 1 a 10
x <- 1:10
x
## [1] 1 2 3 4 5 6 7 8 9 10

# Em ordem decrescente
10:1
## [1] 10 9 8 7 6 5 4 3 2 1

# Sequência de 0 a 50, de 10 em 10
seq(from = 0, to = 50, by = 10)
## [1] 0 10 20 30 40 50

# 15 números igualmente espaçados entre 0 e 1
seq(0, 1, length = 15)
## [1] 0.00000 0.07143 0.14286 0.21429 0.28571 0.35714 0.42857 0.50000
## [9] 0.57143 0.64286 0.71429 0.78571 0.85714 0.92857 1.00000

# Repetir um vetor inteiro várias vezes
rep(1:3, times = 4)
## [1] 1 2 3 1 2 3 1 2 3 1 2 3

# Repetir cada elemento várias vezes
rep(1:3, each = 4)
## [1] 1 1 1 1 2 2 2 2 3 3 3 3

# Combinar vetores e números num novo vetor
c(1:3, 100, 1:3)
## [1] 1 2 3 100 1 2 3
```

O operador `:` e as funções `seq()` e `rep()` vão aparecer constantemente. Note a diferença subtil entre `times` e `each` em `rep()`: `times` repete a sequência inteira; `each` repete cada elemento antes de passar ao seguinte. Confundir os dois é comum, experimente ambos até a distinção ficar natural.

4.1.3 Aceder a elementos

Guardar um vetor só é útil se conseguirmos recuperar, seleccionar ou modificar partes dele. Em R, fazemos isso com **parênteses retos** `[ ]` e índices. Esta operação chama-se **indexação** e será uma das mais usadas ao longo do livro. Por isso, vale a pena compreendê-la bem desde o início. Há três formas principais de indexar um vetor.

Por posição. Indicamos os índices dos elementos que queremos seleccionar:

```
x <- (-5):5
x
## [1] -5 -4 -3 -2 -1 0 1 2 3 4 5

x[1]          # primeiro elemento
## [1] -5

x[4:8]        # do quarto ao oitavo elemento
## [1] -2 -1 0 1 2

x[c(1, 3, 5)] # primeiro, terceiro e quinto elementos
## [1] -5 -3 -1
```

Por exclusão. Também podemos indicar as posições que queremos remover. Para isso, usamos índices negativos:

```
x[-1]          # todos os elementos exceto o primeiro
## [1] -4 -3 -2 -1 0 1 2 3 4 5

x[-length(x)]  # todos os elementos exceto o último
## [1] -5 -4 -3 -2 -1 0 1 2 3 4

x[-c(1:3, 9:11)] # remove os três primeiros e os três últimos
## [1] -2 -1 0 1 2
```

A expressão `length(x)` devolve o comprimento do vetor `x`. Em R, índices negativos removem posições, não valores. Portanto, `x[-length(x)]` significa: remover o elemento que está na última posição de `x`, e não remover um valor igual a `length(x)`.

Por condição lógica. A terceira forma de indexação é uma das mais úteis em análise de dados. Em vez de indicarmos diretamente as posições que queremos seleccionar, fornecemos ao R um vetor lógico, isto é, um vetor formado por valores `TRUE` e `FALSE`. O R devolve apenas os elementos que estão nas posições onde aparece `TRUE`.

Esta ideia é particularmente poderosa porque as comparações em R produzem precisamente vetores lógicos. Assim, podemos selecionar elementos que satisfazem uma certa condição.

```
# Selecionar os elementos com valor absoluto menor que 3
index <- abs(x) < 3
index
## [1] FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE

x[index]
## [1] -2 -1 0 1 2

# A mesma seleção, sem criar uma variável auxiliar
x[abs(x) < 3]
## [1] -2 -1 0 1 2
```

No exemplo anterior, `abs(x) < 3` cria uma condição lógica. O R verifica essa condição elemento a elemento e devolve `TRUE` nas posições em que o valor absoluto de `x` é menor que 3. Quando usamos esse vetor lógico dentro dos parênteses retos, ficamos apenas com os elementos correspondentes às posições verdadeiras.

Vejamos mais alguns exemplos:

```
y <- 1:10

y[y > 5]          # elementos maiores que 5
## [1] 6 7 8 9 10

y[y %% 2 == 0]    # elementos pares
## [1] 2 4 6 8 10

y[5] <- NA
y[!is.na(y)]      # elementos que não são NA
## [1] 1 2 3 4 6 7 8 9 10
```

Neste último exemplo, `is.na(y)` identifica os valores ausentes e devolve `TRUE` nas posições em que há `NA`. O operador `!` inverte essa condição. Por isso, `!is.na(y)` seleciona os elementos que **não** são valores ausentes.

O R inclui ainda alguns vetores predefinidos úteis, como `letters` e `LETTERS`, que contêm o alfabeto em minúsculas e em maiúsculas, respectivamente:


```
letters[1:3]
## [1] "a" "b" "c"

LETTERS[c(2, 4, 6)]
## [1] "B" "D" "F"
```

4.1.4 Funções úteis para vetores

Grande parte do valor dos vetores está nas dezenas de funções que operam sobre eles de uma só vez. Eis um catálogo das mais frequentes, aplicadas a um vetor numérico:

```
v <- c(2.2, 1.1, 3.3)

length(v)      # número de elementos
## [1] 3
max(v)         # máximo
## [1] 3.3
min(v)         # mínimo
## [1] 1.1
sum(v)         # soma
## [1] 6.6
mean(v)        # média
## [1] 2.2
median(v)      # mediana
## [1] 2.2
range(v)       # mínimo e máximo
## [1] 1.1 3.3
var(v)         # variância amostral
## [1] 1.21
sort(v)        # ordenar (crescente)
## [1] 1.1 2.2 3.3
sort(v, decreasing = TRUE) # ordenar (decrecente)
## [1] 3.3 2.2 1.1
cumsum(v)      # somas acumuladas
## [1] 2.2 3.3 6.6
```

Uma função particularmente útil é `which()`, que devolve as **posições** dos elementos que satisfazem uma condição, e não os valores em si:

```

y <- c(8, 3, 5, 7, 6, 6, 8, 9, 2, 3, 9, 4, 10, 4, 11)

which(y > 5)      # as posições dos elementos maiores que 5
## [1]  1  4  5  6  7  8 11 13 15

y[y > 5]          # os valores em si
## [1]  8  7  6  6  8  9  9 10 11

```

A distinção entre *posições* (`which(y > 5)`) e *valores* (`y[y > 5]`) é uma fonte comum de confusão. Guarde-a: uma pergunta como “em que dias o consumo excedeu a média?” pede posições; “quais foram os consumos acima da média?” pede valores.

4.1.5 Operações e reciclagem

As operações aritméticas em R são **vetorizadas**: aplicam-se a cada elemento do vetor, sem necessidade de escrever ciclos. Isto torna o código curto e legível:

```

v <- c(2.2, 1.1, 3.3)

v + 1      # soma 1 a cada elemento
## [1] 3.2 2.1 4.3

v * 2      # multiplica cada elemento por 2
## [1] 4.4 2.2 6.6

v > 2      # compara cada elemento com 2
## [1] TRUE FALSE TRUE

c(2, 3, 5, 7)^2 # eleva cada elemento ao quadrado
## [1]  4  9 25 49

```

Mas o que acontece quando operamos com dois vetores de **comprimentos diferentes**? O R aplica a chamada **regra da reciclagem**: repete o vetor mais curto até igualar o comprimento do mais longo.

```

c(2, 3, 5, 7)^c(2, 3)
## [1]  4 27 25 343

```

Internamente, o vetor `c(2, 3)` foi reciclado para `c(2, 3, 2, 3)`, de modo que a operação passou a ser `c(2, 3, 5, 7)^c(2, 3, 2, 3)`.

A reciclagem é cômoda, mas pode esconder erros. Se o comprimento do vetor maior não for múltiplo do menor, a reciclagem fica “incompleta” e o R emite um aviso:

```
c(2, 3, 5, 7)^c(2, 3, 4)
## Warning: longer object length is not a multiple of shorter object length
## [1] 4 27 625 49
```

Aqui, `c(2, 3, 4)` foi reciclado para `c(2, 3, 4, 2)`, o último elemento repetiu apenas parcialmente. **Leia sempre os avisos do R:** um aviso de reciclagem é, muitas vezes, o sintoma de um vetor com o comprimento errado.

4.1.6 Exercícios

1. Crie os vetores:

- (a) $(1, 2, 3, \dots, 19, 20)$
- (b) $(20, 19, \dots, 2, 1)$
- (c) $(1, 2, \dots, 19, 20, 19, 18, \dots, 2, 1)$
- (d) $(10, 20, 30, \dots, 90, 10)$
- (e) $(1, 1, \dots, 1, 2, 2, \dots, 2, 3, 3, \dots, 3)$, com 10 ocorrências do 1, 20 do 2 e 30 do 3.

2. Use a função `paste()` para criar o vetor de caracteres de tamanho 20: "nome 1", "nome 2", ..., "nome 20".

3. Crie um vetor x_1 igual a "A" "A" "B" "B" "C" "C" "D" "D" "E" "E".

4. Crie um vetor x_2 igual a "a" "b" "c" "d" "e" "a" "b" "c" "d" "e".

5. Crie um vetor x_3 com a palavra "uva" 10 vezes, "maçã" 9 vezes, "laranja" 6 vezes e "banana" 1 vez.

6. Crie o vetor `notas <- c(7, 8, 9)` e atribua-lhe os nomes dos alunos "Ana", "João" e "Pedro". Aceda à nota do "João" usando o nome. *Sugestão:* Use a função `names()`.

7. Crie um vetor de 15 números aleatórios entre 1 e 100 (use `sample()`). Ordene-o por ordem crescente e depois decrescente. Encontre o menor e o maior valor.

8. Crie um vetor de 20 números aleatórios entre 1 e 50 (use `sample()`). Calcule a soma, a média, o desvio padrão (`sd()`) e o produto (`prod()`) dos seus elementos.

9. Crie um vetor de 10 números aleatórios entre 1 e 100 (use `sample()`). Extraia os elementos maiores que 50. De seguida, substitua por 0 os valores menores que 30.

10. Crie um vetor de 10 números. Verifique quais elementos são maiores que 5 e quais são pares. Crie um novo vetor apenas com os que satisfazem **ambas** as condições.

11. Crie um vetor com 10 números inteiros. Multiplique por 2 os elementos nas posições 2, 4 e 6. Substitua o último elemento por 100.

12. Calcule a média dos vetores:

- (a) $x = (1, 0, NA, 5, 7)$
- (b) $y = (-\infty, 0, 1, NA, 2, 7, \infty)$

13. Crie:

- (a) um vetor com os valores $e^x \sin(x)$ nos pontos $x = 2, 2.1, 2.2, \dots, 6$;
- (b) o vetor $\left(3, \frac{3^2}{2}, \frac{3^3}{3}, \dots, \frac{3^{30}}{30}\right)$.

14. Calcule:

- (a) $\sum_{i=1}^{1000} \frac{9}{10^i}$
- (b) $\sum_{i=10}^{100} i^3 + 4i^2$
- (c) $\sum_{i=1}^{25} \frac{2^i}{i} + \frac{3^i}{i^2}$

15. Uma empresa monitorizou o consumo diário de energia (em kWh) de uma casa durante 30 dias, guardando-os num vetor `consumo`.

- (a) Defina `consumo` com 30 valores gerados aleatoriamente entre 10 e 40 kWh, usando `runif()` com semente 2026 (`set.seed(2026)`). Consulte `?runif` para os parâmetros.
- (b) Calcule o consumo médio e guarde-o em `media`.
- (c) Crie um vetor lógico `acima_da_media` indicando os dias com consumo superior à média.
- (d) Em quantos dias o consumo superou a média?
- (e) Quais foram os dias com consumo de pico (superior a 35 kWh)? Devolva os valores e as posições.
- (f) Substitua por `NA` todos os valores abaixo de 15 kWh (simulando falhas de medição).
- (g) Recalcule a média ignorando os `NA`.
- (h) Crie um vetor com os consumos dos dias pares do mês.

16. Uma cooperativa agrícola registou a produção anual de soja (em toneladas) de 12 propriedades.

- (a) Crie o vetor `producao`:

```
producao <- c(80, 95, 70, 68, 105, 120, 73, 89, 110, 66, 92, 78)
```

- (b) Calcule a produção média por propriedade e guarde-a em `media_total`.
- (c) Identifique as propriedades com produção abaixo da média e crie um vetor com os seus índices.
- (d) Uma seca afetou as propriedades 4, 10 e 12, cuja produção foi subestimada em 10%. Corrija esses valores no vetor `producao`.
- (e) Recalcule a média com os dados corrigidos e compare com a anterior.
- (f) Crie um vetor lógico `alta_produtividade` para as propriedades com produção superior a 100 toneladas.
- (g) Quantas tiveram alta produtividade? Que percentagem do total?
- (h) Ordene a produção do menor para o maior, guardando o resultado em `ordenado`.
- (i) Quais são as 3 propriedades com maior produção?

4.2 Fatores

Nem todos os dados são números ou textos livres. Muitas variáveis são **categóricas**, isto é, assumem um número limitado de valores possíveis. É o caso do género, do estado civil, do grau de satisfação ou do nível de escolaridade. Poderíamos guardar estas variáveis como vetores de texto, mas o R oferece uma estrutura própria para as representar: o **fator**.

Um fator é um vetor especial, concebido para trabalhar com dados categóricos. À primeira vista, pode parecer um vetor de texto. No entanto, internamente, o R organiza as categorias de forma mais adequada para análise estatística, gráficos e modelos. Para compreender bem um fator, precisamos de conhecer três ideias:

- **Níveis** (*levels*): os valores distintos que a categoria pode assumir. Para uma variável “tamanho”, os níveis poderiam ser “pequeno”, “médio” e “grande”.
- **Armazenamento interno**: internamente, o R guarda cada valor como um inteiro que aponta para um nível. É mais eficiente e evita erros de escrita, se “médio” é um nível, não há forma de acidentalmente escrever “medio” numa observação.
- **Ordem**: os fatores podem ser **ordenados** (quando há uma hierarquia natural, como “baixo” < “médio” < “alto”) ou **não ordenados** (quando os níveis não têm ordem intrínseca, como cores).

4.2.1 Criar fatores

Criamos um fator com a função `factor()`:

```
dados <- c("baixo", "medio", "alto", "medio", "baixo", "alto")
factor_dados <- factor(dados)
factor_dados
## [1] baixo medio alto  medio baixo alto
## Levels: alto baixo medio
```

Por omissão, o R ordena os níveis **alfabeticamente**. Repare que a ordem ficou "alto", "baixo", "medio", o que quase nunca corresponde à ordem pretendida. Para variáveis com uma hierarquia natural, isto pode ser um problema: gráficos, tabelas e resumos podem aparecer por ordem alfabética, e não pela ordem lógica das categorias. A solução é indicar explicitamente a ordem dos níveis.

```
# Especificar a ordem dos níveis
factor_dados <- factor(dados, levels = c("baixo", "medio", "alto"))
factor_dados
## [1] baixo medio alto  medio baixo alto
## Levels: baixo medio alto
```

Se, além da ordem de apresentação, quisermos que o R reconheça uma verdadeira **hierarquia** entre os níveis (para poder responder a perguntas do tipo “é este nível maior do que aquele?”), usamos `ordered = TRUE`:

```
fator_ordenado <- factor(dados,
                        levels = c("baixo", "medio", "alto"),
                        ordered = TRUE)
fator_ordenado[1] < fator_ordenado[3] # "baixo" < "alto"?
## [1] TRUE
```

4.2.2 Manipular fatores

Podemos consultar e modificar os níveis de um fator:

```
# Consultar os níveis
levels(factor_dados)
## [1] "baixo" "medio" "alto"

# Renomear os níveis
levels(factor_dados) <- c("Baixo", "Medio", "Alto")
factor_dados
## [1] Baixo Medio Alto  Medio Baixo Alto
## Levels: Baixo Medio Alto
```

4.2.3 Gerar fatores com `gl()`

Quando precisamos de criar fatores com padrões regulares, como acontece frequentemente em desenhos experimentais com grupos de igual dimensão, a função `gl()` (*generate levels*) permite fazê-lo de forma simples e rápida.

```
# gl(n, k): n níveis, cada um repetido k vezes
gl(n = 4, k = 3)
## [1] 1 1 1 2 2 2 3 3 3 4 4 4
## Levels: 1 2 3 4

# Com rótulos personalizados
gl(n = 2, k = 10, labels = c("mulher", "homem"))
## [1] mulher mulher mulher mulher mulher mulher mulher mulher mulher mulher
## [11] homem homem homem homem homem homem homem homem homem homem
## Levels: mulher homem
```

4.2.4 Exercícios

1. Crie o vetor `c("pequeno", "médio", "grande", "pequeno", "grande")` e transforme-o num fator com níveis ordenados “pequeno”, “médio”, “grande”. Exiba os níveis.
2. Crie um fator com as cores de carros `c("vermelho", "azul", "preto", "vermelho", "branco", "azul")`. *Sugestão:* Use `table()` para contar quantos carros há de cada cor.
3. Crie um fator com avaliações “bom”, “médio”, “mau” e modifique os níveis para “excelente”, “razoável” e “insatisfatório”. Exiba o fator modificado.
4. Crie o vetor `c("solteiro", "casado", "divorciado")`. Verifique se é um fator com `is.factor()` e depois converta-o em fator.
5. Crie um fator de escolaridade com “ensino superior”, “ensino secundário”, “doutoramento”, “mestrado”, “ensino secundário”, “ensino superior”. Defina uma ordem lógica para os níveis e ordene o fator.
6. Crie o fator `c("baixo", "alto", "médio", "baixo", "alto")` com níveis “baixo”, “médio”, “alto”. Converta-o num vetor numérico em que “baixo” seja 1, “médio” 2 e “alto” 3.
7. Crie um fator com as categorias “verde”, “amarelo”, “vermelho”. Substitua o nível “amarelo” por “laranja” e exiba o fator.
8. Uma empresa de Internet inquiriu 20 clientes sobre o atendimento, classificando as respostas em “Muito insatisfeito”, “Insatisfeito”, “Neutro”, “Satisfeito” e “Muito satisfeito”.

(a) Crie o vetor `respostas`:

```
respostas <- c("Satisfeito", "Muito satisfeito", "Neutro", "Insatisfeito", "Satisfeito",
  "Muito insatisfeito", "Satisfeito", "Neutro", "Insatisfeito", "Satisfeito",
  "Muito satisfeito", "Neutro", "Insatisfeito", "Satisfeito", "Neutro",
  "Satisfeito", "Muito satisfeito", "Muito insatisfeito", "Neutro", "Satisfeito")
```

- (b) Converta `respostas` num fator `respostas_fator`, com níveis ordenados do mais negativo para o mais positivo.
- (c) Verifique a estrutura com a função `str()`. O R reconhece a ordem?
- (d) Quantos clientes ficaram insatisfeitos ou muito insatisfeitos?
- (e) Qual a resposta mais frequente? *Sugestão*: Use a função `table()` ou `which.max(table(...))`.
- (f) Calcule a frequência relativa (%) de cada nível.
- (g) Substitua as respostas “Neutro” por “Indiferente”, mantendo a ordem dos níveis.

9. A Perturbação de Hiperactividade e Défice de Atenção (PHDA) é uma condição do neurodesenvolvimento que influencia a forma como o cérebro regula a atenção, a impulsividade e o nível de actividade. Estima-se que cerca de 5% dos adultos vivam com PHDA, muitos deles sem diagnóstico durante a infância. No ensino superior, estudantes com PHDA referem frequentemente dificuldades de organização, concentração e gestão do tempo, embora também possam apresentar características como hiperfoco em áreas de interesse, criatividade e pensamento divergente.

- (a) Crie o vetor `phda_desafios`:

```
phda_desafios <- c("Concentração", "Procrastinação", "Gestão do tempo", "Organização",
  "Nenhum", "Concentração", "Concentração", "Organização", "Organização", "Organização",
  "Gestão do tempo", "Gestão do tempo", "Procrastinação", "Procrastinação",
  "Procrastinação", "Procrastinação", "Concentração", "Concentração", "Concentração")
```

Converta-o num fator não ordenado `phda_fator`.

- (b) Obtenha as frequências absolutas e relativas de cada desafio.
- (c) Quantos estudantes indicaram “Concentração” ou “Procrastinação” como principal dificuldade?
- (d) Recodifique “Nenhum” para “Sem dificuldade declarada”.
- (e) Crie um gráfico de barras dos desafios, com rótulos de frequência. *Sugestão*: Use a função `barplot()`.

10. A dislexia é uma condição neurológica que afeta o processamento da linguagem escrita, com desafios na leitura fluente, escrita e ortografia que não se relacionam com o nível de inteligência. Estima-se que entre 5% e 10% da população seja disléxica. Reconhecer a dislexia como parte da neurodiversidade é essencial para ambientes de ensino inclusivos.

- (a) Crie o vetor `dificuldades` com as respostas de 10 crianças:

```
dificuldades <- c("Leitura", "Escrita", "Ortografia", "Leitura", "Compreensão",
  "Ortografia", "Memória auditiva", "Leitura", "Escrita", "Ortografia")
```

Converta-o num fator não ordenado `dislexia_fator`.

- (b) Crie uma tabela de frequências e identifique a dificuldade mais prevalente.
 (c) Recodifique “Ortografia” para “Erros ortográficos”, mantendo o fator.
 (d) Visualize as frequências com `barplot()`.

4.3 Matriz e array

Os vetores organizam dados em **uma** dimensão — uma linha de valores. Mas muitos dados são naturalmente **bidimensionais**: uma tabela de números, os pixels de uma imagem, os coeficientes de um sistema de equações. Para estes, temos a **matriz**.

Uma matriz é uma estrutura bidimensional cujos elementos são **todos do mesmo tipo**, organizados em linhas e colunas. É, no fundo, um vetor a que se deu forma retangular. O **array** generaliza a ideia a três ou mais dimensões.

4.3.1 Criar matrizes

Criamos uma matriz com `matrix()`, indicando o número de linhas (`nrow`) e de colunas (`ncol`):

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3, byrow = FALSE)
##      [,1] [,2] [,3]
## [1,]  1   3   5
## [2,]  2   4   6
```

Por omissão (`byrow = FALSE`), o R preenche a matriz **por colunas**: primeiro a coluna 1 de cima a baixo, depois a coluna 2, e assim por diante. Se quiser preencher por linhas, use `byrow = TRUE`. Esta é outra fonte frequente de surpresa para quem começa — se a matriz “sai trocada”, verifique este argumento.

As operações vetorizadas também se aplicam a matrizes:

```
matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3) > 5
##      [,1] [,2] [,3]
## [1,] FALSE FALSE FALSE
## [2,] FALSE FALSE  TRUE
```

4.3.2 Construir matrizes a partir de vetores

Frequentemente, temos vetores que queremos juntar como linhas ou colunas de uma matriz. As funções `rbind()` (*row bind*, juntar por linhas) e `cbind()` (*column bind*, juntar por colunas) fazem exatamente isso:

```
v1 <- c(1, 2, 3)
v2 <- c(4, 5, 6)

rbind(v1, v2)      # como linhas
##      [,1] [,2] [,3]
## v1     1     2     3
## v2     4     5     6

cbind(v1, v2)      # como colunas
##      v1 v2
## [1,]  1  4
## [2,]  2  5
## [3,]  3  6
```

4.3.3 Aceder a elementos de uma matriz

A indexação estende naturalmente a dos vetores, agora com **dois** índices, `[linha, coluna]`. Deixar um deles em branco significa “todas”:

```
A <- matrix((-4):5, nrow = 2, ncol = 5)
A
##      [,1] [,2] [,3] [,4] [,5]
## [1,]  -4  -2   0   2   4
## [2,]  -3  -1   1   3   5

A[1, 2]      # elemento na linha 1, coluna 2
## [1] -2

A[A < 0]      # todos os elementos negativos (indexação lógica)
## [1] -4 -3 -2 -1

A[A < 0] <- 0  # substituir os negativos por 0
A
##      [,1] [,2] [,3] [,4] [,5]
## [1,]   0   0   0   2   4
```

```
## [2,] 0 0 1 3 5

A[2, ] # toda a linha 2
## [1] 0 0 1 3 5

A[, c(2, 4)] # as colunas 2 e 4
##      [,1] [,2]
## [1,] 0    2
## [2,] 0    3
```

4.3.4 Nomear linhas e colunas

Tal como os vetores, as matrizes podem ter nomes nas linhas e colunas, o que torna os resultados muito mais legíveis:

```
x <- matrix(rnorm(12), nrow = 4)
colnames(x) <- paste0("dados", 1:3)
rownames(x) <- letters[1:4]
```

4.3.5 Álgebra de matrizes

Aqui reside uma das grandes vantagens do R para a estatística: a álgebra linear é imediata. A **multiplicação de matrizes** faz-se com o operador `%*` (repare que `*` faria a multiplicação elemento a elemento, que é outra coisa):

```
M <- matrix(rnorm(20), nrow = 4, ncol = 5)
N <- matrix(rnorm(15), nrow = 5, ncol = 3)

M %*% N # produto matricial (4x5 por 5x3 = 4x3)
```

E as operações clássicas da álgebra linear têm, cada uma, a sua função. Sendo `M` uma matriz quadrada:

Operação	Função
Dimensão	<code>dim(M)</code>
Transposta	<code>t(M)</code>
Diagonal principal	<code>diag(M)</code>
Determinante	<code>det(M)</code>
Inversa	<code>solve(M)</code>

Operação	Função
Valores e vetores próprios	<code>eigen(M)</code>
Soma de todos os elementos	<code>sum(M)</code>
Média de todos os elementos	<code>mean(M)</code>

Há ainda a poderosa função `apply()`, que aplica uma função a cada linha ou a cada coluna. O segundo argumento indica a margem: 1 para linhas, 2 para colunas:

```
apply(M, 1, sum)      # soma de cada linha
apply(M, 2, mean)     # média de cada coluna
```

4.3.6 Adicionar linhas e colunas

`rbind()` e `cbind()` também servem para acrescentar linhas e colunas a uma matriz já existente:

```
X <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)

cbind(X, c(7, 8))      # nova coluna à direita
rbind(X, c(7, 8, 9))   # nova linha em baixo
```

4.3.7 Arrays

Quando duas dimensões não bastam, generalizamos para o **array**. Um array tridimensional pode imaginar-se como um “empilhamento” de matrizes (camadas). Criamo-lo com `array()`, indicando as dimensões em `dim`:

```
# 4 linhas, 3 colunas, 2 camadas
A <- array(1:24, dim = c(4, 3, 2))
```

A indexação usa agora três índices, [linha, coluna, camada]:

```
A[1, 2, 1]      # elemento na linha 1, coluna 2, camada 1
## [1] 5

A[2, , 1]       # toda a linha 2 da camada 1
## [1] 2 6 10

A[, , 1]        # toda a primeira camada (uma matriz)
```

4.3.8 Exercícios

1. Crie uma matriz 3×4 com os números de 1 a 12, preenchida por colunas. Exiba-a e determine a soma dos elementos da segunda coluna.
2. Crie duas matrizes 2×3 , A e B , com inteiros aleatórios de 1 a 10. Some-as e multiplique-as elemento a elemento. *Sugestão:* Use a função `sample()`.
3. Crie uma matriz 3×3 , M , com números aleatórios entre 1 e 9, e um vetor v de comprimento 3. Calcule $M \times v$ e depois $v \times M$.
4. Crie uma matriz 4×2 , N , com valores de 1 a 8. Transponha-a e calcule a soma dos elementos de cada linha da transposta. Some depois a primeira e a segunda linha da transposta.
5. Crie uma matriz 3×3 , P , à sua escolha. Calcule o seu determinante e, se for não nulo, a sua inversa.
6. Crie uma matriz 5×5 , Q , com números aleatórios de 1 a 25. Extraia a submatriz das 2.^a, 3.^a e 4.^a linhas e colunas.
7. Crie uma matriz 3×3 com valores de 1 a 9. Calcule a soma da primeira linha e a soma da terceira coluna.
8. Considere a matriz

$$A = \begin{bmatrix} 1 & 1 & 3 \\ 5 & 2 & 6 \\ -2 & -1 & -3 \end{bmatrix}.$$

- (a) Verifique que $A^3 = 0$.
- (b) Substitua a terceira coluna pela soma das colunas 1 e 3.
- (c) Na matriz

$$M = \begin{bmatrix} 20 & 22 & 23 \\ 34 & 55 & 57 \\ 99 & 97 & 71 \\ 12 & 16 & 19 \\ 10 & 53 & 24 \\ 14 & 21 & 28 \end{bmatrix},$$

substitua por 0 todos os números pares.

9. Crie uma matriz 3×3 e verifique se é simétrica (igual à transposta). Teste com

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}.$$

Sugestão: Use a função `all()`.

10. Crie uma matriz 4×4 e calcule o seu traço (a soma da diagonal principal).
11. Crie uma matriz 5×5 com valores de 1 a 25 e extraia as colunas de índice par (2 e 4).
12. Uma turma usou o ChatGPT como apoio ao estudo. Para cada estudante registou-se o número de perguntas feitas numa semana, o tempo médio de resposta (segundos) e o nível de satisfação (1 a 5). Os dados de 3 alunos:

$$M = \begin{bmatrix} 12 & 8 & 5 \\ 20 & 10 & 4 \\ 15 & 9 & 5 \end{bmatrix},$$

com as linhas a corresponder aos alunos A, B e C, e as colunas a (Perguntas, Tempo, Satisfação).

- (a) Crie a matriz M e nomeie linhas e colunas.
- (b) Use `rbind()` para acrescentar o aluno D: 18 perguntas, 11 segundos, satisfação 3.
- (c) Use `cbind()` para acrescentar a coluna “Horas de uso” com os valores `c(5, 7, 6, 8)`.
- (d) Calcule a média de cada métrica com `colMeans()`.
- (e) Que aluno fez mais perguntas? *Sugestão:* Use a função `which.max()`.
13. Um ecrã em tons de cinzento pode representar-se por uma matriz, em que cada entrada é a intensidade de um pixel (0 = preto, 9 = branco). Considere a imagem 10×10 :

- (a) Crie a matriz I:

```
I <- matrix(c(
  0,0,0,0,0,0,0,0,0,0,
  0,9,9,9,0,0,9,9,9,0,
  0,9,0,9,0,0,9,0,9,0,
  0,9,9,9,0,0,9,9,9,0,
  0,9,0,0,0,0,0,0,9,0,
  0,9,0,0,0,0,0,0,9,0,
  0,9,0,0,0,0,0,0,9,0,
  0,9,0,0,0,0,0,0,9,0,
  0,9,9,9,9,9,9,9,9,0,
  0,0,0,0,0,0,0,0,0,0
), nrow = 10, byrow = TRUE)
```

- (b) Visualize-a como imagem com `image()`. *Sugestão:* Use `col = gray.colors(10)`.
- (c) Inverta as cores (substitua cada valor x por $9 - x$).
- (d) Calcule a intensidade média de todos os pixels (o brilho médio da imagem).
- (e) Acrescente uma linha de zeros no fim da matriz.

14. Numa região há Humanos (H) e Zumbis (Z). A cada hora, 20% dos humanos tornam-se zumbis e 10% dos zumbis voltam a humanos. Organizamos estas probabilidades numa **matriz de transição** P , e se $\mathbf{x}_t$ é o vetor com o número de humanos e zumbis no instante t , então $\mathbf{x}_{t+1} = P \cdot \mathbf{x}_t$. Inicialmente há 150 humanos e 150 zumbis.

- Crie a matriz P e o vetor inicial $\mathbf{x}_0 = (150, 150)^\top$.
- Calcule $\mathbf{x}_1$, $\mathbf{x}_2$ e $\mathbf{x}_{10}$. Quantos zumbis há à 10.^a hora? *Sugestão:* use `%^%` (do pacote `expm`) para potências de matrizes.

Este é, na verdade, um exemplo simples de **cadeia de Markov** — um sistema que evolui em passos sucessivos, em que o estado seguinte depende apenas do estado atual e de uma matriz de transição. Reencontraremos esta ideia mais adiante, no estudo da simulação.

15. Uma forma de cifrar mensagens é através da multiplicação por matrizes. Associando as letras a números (sem a letra K, com espaço = 0), a mensagem converte-se numa matriz M , cifra-se multiplicando por uma **matriz-chave** invertível C ($M_{\text{cod}} = M \cdot C$), e decifra-se multiplicando pela inversa. Com a chave

$$C = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 3 & 1 \\ 0 & 1 & 1 \end{bmatrix} :$$

- Recebeu-se (em blocos 3×3 , lidos por linhas): $-12, 48, 23, -2, 42, 26, 1, 42, 29$. Decifre a mensagem.
- Escolha uma matriz-chave para palavras até 16 letras e cifre e decifre à vontade.

16. Crie um array $3 \times 3 \times 3$ com valores de 1 a 27. Aceda ao elemento $[2, 3, 1]$, à segunda camada e à terceira linha da segunda camada.

17. Crie dois arrays $2 \times 2 \times 2$ com valores aleatórios de 1 a 10. Some-os, multiplique-os elemento a elemento, calcule a média da soma e multiplique a primeira matriz do primeiro array pela segunda matriz do segundo.

4.4 Data frame

Chegamos, talvez, à estrutura mais importante de todas para o trabalho estatístico do dia a dia. As matrizes obrigam a que todos os elementos sejam do mesmo tipo — mas os dados reais quase nunca são assim. Uma tabela de estudantes tem nomes (texto), idades (números inteiros), notas (números decimais) e, talvez, uma indicação de aprovação (lógica). Precisamos de uma estrutura que combine colunas de tipos diferentes. Essa estrutura é o **data frame**.

Um data frame é uma tabela bidimensional em que **cada coluna pode ter o seu próprio tipo**, embora dentro de cada coluna todos os elementos sejam do mesmo tipo. É a estrutura

mais próxima de uma folha de cálculo ou de uma tabela de base de dados — e é o formato em que a esmagadora maioria dos conjuntos de dados chega até nós.

4.4.1 Criar um data frame

Criamo-lo com `data.frame()`, indicando cada coluna pelo seu nome:

```
df <- data.frame(
  id = 1:4,
  nome = c("Ana", "Bruno", "Carlos", "Diana"),
  idade = c(23, 35, 31, 28),
  salario = c(5000, 6000, 7000, 8000)
)
df
##   id  nome idade salario
## 1  1   Ana    23    5000
## 2  2 Bruno    35    6000
## 3  3 Carlos   31    7000
## 4  4 Diana    28    8000
```

Para perceber porque não basta uma matriz, tentemos criar a “mesma” tabela com `cbind()`:

```
cbind(id = 1:4, nome = c("Ana", "Bruno", "Carlos", "Diana"),
      idade = c(23, 35, 31, 28))
##      id nome  idade
## [1,] "1" "Ana"  "23"
## [2,] "2" "Bruno" "35"
## ...
```

Repare que a matriz converteu **tudo** para texto — as idades ficaram "23", "35", entre aspas, e deixaram de ser números com os quais podemos calcular. É a coerção em ação, e é exatamente o problema que o data frame resolve: nele, cada coluna preserva o seu tipo.

4.4.2 Aceder a colunas

Um data frame oferece várias formas de aceder às suas colunas, e vale a pena conhecê-las porque devolvem coisas subtilmente diferentes. A forma mais legível é o operador `$`:

```
df$id           # a coluna 'id' como vetor
## [1] 1 2 3 4
```



```
df[["id"]]      # equivalente, útil quando o nome está numa variável
## [1] 1 2 3 4

df[, 1]         # por posição, também devolve um vetor
## [1] 1 2 3 4

df["id"]        # atenção: devolve um data frame de uma coluna, não um vetor!
##   id
## 1  1
## 2  2
## 3  3
## 4  4
```

A distinção entre “devolver um vetor” e “devolver um data frame de uma coluna” é uma subtileza que confunde muitos iniciantes, mas que tem uma regra simples: os colchetes duplos `[[ ]]` e o `$` extraem o **conteúdo** (um vetor); os colchetes simples `[ ]` preservam a **estrutura** (um data frame).

4.4.3 Aceder a linhas

As linhas acedem-se com a notação `[linha, coluna]`, tal como nas matrizes, deixando em branco a dimensão que queremos por inteiro:

```
df[1, ]         # a primeira linha (todas as colunas)
df[1:2, ]       # as duas primeiras linhas
df[c(1, 4), ]   # as linhas 1 e 4
df[-1, ]        # todas menos a primeira
df[1, "nome"]    # o valor na linha 1, coluna 'nome'
## [1] "Ana"
```

4.4.4 Adicionar e remover colunas

Acrescentar uma coluna é tão simples como atribuir-lhe um valor com `$`; muitas vezes, a nova coluna é calculada a partir das existentes:

```
# Nova coluna calculada (aumento de 10% no salário)
df$novos_salario <- df$salario * 1.1

# Remover uma coluna: atribuir-lhe NULL
df$id <- NULL
```

4.4.5 Fundir data frames

Trabalhar com dados reais implica muitas vezes juntar tabelas. Há três situações típicas, cada uma com a sua ferramenta.

Acrescentar observações (linhas) com `rbind()`, quando as tabelas têm as mesmas colunas:

```
df1 <- data.frame(id = 1:2, nome = c("Ana", "Bruno"))
df2 <- data.frame(id = 3:4, nome = c("Carlos", "Diana"))
rbind(df1, df2)
```

Acrescentar variáveis (colunas) com `cbind()`, quando as tabelas têm o mesmo número de linhas, alinhadas pela ordem.

Cruzar tabelas por uma chave comum com `merge()` — a operação mais poderosa e a mais próxima do que se faz numa base de dados. Cruzamos duas tabelas por uma variável partilhada (como `id` ou `curso`):

```
df1 <- data.frame(id = 1:3, nome = c("Ana", "Bruno", "Carlos"))
df2 <- data.frame(id = 2:4, idade = c(35, 40, 28))

merge(df1, df2, by = "id")                # só as chaves em comum (inner join)
##   id  nome idade
## 1  2 Bruno   35
## 2  3 Carlos  40

merge(df1, df2, by = "id", all.x = TRUE)  # todas as linhas de df1 (left join)
merge(df1, df2, by = "id", all = TRUE)    # todas as linhas de ambos (full join)
```

Estes cruzamentos — *inner*, *left*, *right* e *full join* — são conceitos centrais em bases de dados. O pacote `dplyr` oferece versões mais legíveis com os mesmos nomes (`inner_join()`, `left_join()`, etc.), que reencontraremos no capítulo de manipulação de dados.

4.4.6 Explorar um data frame

Perante um conjunto de dados novo, o primeiro passo é sempre *conhecê-lo*. Um punhado de funções permite fazê-lo rapidamente. Usemos como exemplo o `iris`, um famoso conjunto de dados incluído no R:

```
df <- iris

names(df)      # nomes das colunas
```

```

dim(df)      # dimensões (linhas e colunas)
## [1] 150   5
nrow(df); ncol(df)
str(df)      # a estrutura: tipos e primeiros valores de cada coluna
head(df, 3)  # as primeiras linhas
tail(df, 3)  # as últimas linhas

```

Faça de `str()` e `head()` um reflexo. Sempre que carrega um conjunto de dados, comece por aí: `str()` diz-lhe quantas observações e variáveis tem e de que tipo é cada uma; `head()` mostra-lhe como se parecem os dados. Estes dois olhares poupam horas de confusão mais à frente.

4.4.7 Filtrar dados

Uma das tarefas mais comuns é selecionar as linhas que satisfazem uma condição. A função `subset()` fá-lo de forma clara, permitindo filtrar linhas e escolher colunas ao mesmo tempo:

```

df <- data.frame(
  id = 1:4,
  nome = c("Ana", "Bruno", "Carlos", "Diana"),
  idade = c(23, 35, 31, 28),
  salario = c(5000, 6000, 7000, 8000)
)

subset(df, idade > 30)                # linhas com idade > 30
subset(df, idade > 30, select = c(nome, salario)) # essas linhas, só duas colunas
subset(df, idade > 25, select = -id)    # todas as colunas menos 'id'

```

4.4.8 A função `summary()`

Para um resumo estatístico rápido, a função `summary()` é insuperável. Adapta-se ao objeto: para uma variável numérica, dá os quartis, a média e os extremos; para um fator, conta as frequências; para um data frame inteiro, resume cada coluna à sua maneira:

```

summary(iris)
##   Sepal.Length   Sepal.Width   Petal.Length   Petal.Width   Species
##   Min.    :4.30   Min.    :2.00   Min.    :1.00   Min.    :0.1   setosa    :50
##   1st Qu.:5.10   1st Qu.:2.80   1st Qu.:1.60   1st Qu.:0.3   versicolor:50
##   Median :5.80   Median :3.00   Median :4.35   Median :1.3   virginica :50
##   Mean    :5.84   Mean    :3.06   Mean    :3.76   Mean    :1.2

```

```
## 3rd Qu.:6.40 3rd Qu.:3.30 3rd Qu.:5.10 3rd Qu.:1.8
## Max. :7.90 Max. :4.40 Max. :6.90 Max. :2.5
```

4.4.9 Valores ausentes em data frames

Como já vimos com os vetores, os valores em falta exigem cuidado. O conjunto de dados `airquality`, por exemplo, tem NA em algumas colunas, o que faz com que a sua média direta seja NA:

```
colMeans(airquality)
## Ozone Solar.R Wind Temp Month Day
## NA NA 9.958 77.882 6.993 15.804
```

A solução é a mesma: o argumento `na.rm = TRUE` ignora os valores em falta:

```
mean(airquality$Ozone, na.rm = TRUE)
## [1] 42.13

colMeans(airquality, na.rm = TRUE)
```

4.4.10 Exercícios

1. Crie um data frame `estudantes` com as colunas `Nome`, `Idade`, `Curso` e `Nota`, com dados de cinco estudantes fictícios. Acrescente a estudante Filipa, de 20 anos, de Matemática, com nota 16. Use `order()` para ordenar por `Nota`.
2. No data frame `estudantes`, aumente todas as notas em 0,5. Depois, mude o `Curso` para “Estatística” em todos os estudantes com nota superior a 8,0.
3. No data frame `mtcars`, filtre as linhas com `cyl == 6` e `hp > 100`, criando um novo data frame com o resultado.
4. Acrescente ao `mtcars` uma coluna `Peso_KG` que converta o peso (`wt`, em milhares de libras) para quilogramas (multiplicando por 453,592).
5. No `mtcars`, calcule a média, o mínimo e o máximo da potência (`hp`) para cada número de cilindros (`cyl`). Use a função `aggregate()`.
6. No `mtcars`, use `sapply()` para calcular o desvio padrão de todas as colunas numéricas.
7. Transforme a coluna `am` do `mtcars` num fator com os rótulos “Automático” (0) e “Manual” (1). Conte quantos carros há de cada tipo.
8. Ordene o `mtcars` por `mpg` decrescente e mostre os 5 carros mais e os 5 menos económicos. *Sugestão:* Use a função `order()`.

9. A partir do `mtcars`, acrescente uma coluna `car` com os nomes das linhas. Crie `carros1` com as colunas `car`, `mpg`, `cyl` e `carros2` com `car`, `hp`, `wt`. Junte-os com `merge()` pela coluna `car`.
10. Carregue o `airquality` e mostre as primeiras e as últimas 6 linhas.
11. Conte quantos NA existem em cada coluna do `airquality`. *Sugestão:* Use a função `colSums()`.
12. Calcule a média de `Solar.R` no `airquality`, ignorando os NA.
13. Crie um subconjunto do `airquality` com `Wind > 10` e `Temp > 80`. Mostre as primeiras 6 linhas.
14. Acrescente ao `airquality` uma coluna `Temp_Wind_Sum` com a soma de `Temp` e `Wind`. Mostre as primeiras 6 linhas.
15. No `iris`, filtre a espécie “setosa”. Crie a coluna `Petal.Area` (comprimento $\times$ largura da pétala), filtre `Petal.Area > 0.2` e ordene por `Petal.Length` decrescente.
16. No `airquality`, remova as linhas com NA (use `na.omit()`). Acrescente `Temp.Celsius =  $\frac{Temp-32}{1.8}$` . Agrupe por `Month` e calcule a média de `Ozone` com `aggregate()`. Faça um gráfico de dispersão de `Ozone` contra `Temp.Celsius`, colorindo por `Month`.
17. O naufrágio do RMS Titanic, em 1912, deu origem a um dos conjuntos de dados mais famosos da ciência de dados. Trabalhe com uma versão reduzida e fictícia:

(a) Crie o data frame:

```
titanic <- data.frame(
  nome = c("Ana", "Bruno", "Clara", "Daniel", "Eva", "Filipe"),
  sexo = c("F", "M", "F", "M", "F", "M"),
  idade = c(22, 35, 18, 50, 28, 40),
  classe = c("1ª", "3ª", "3ª", "1ª", "2ª", "2ª"),
  sobreviveu = c(TRUE, FALSE, TRUE, TRUE, FALSE, FALSE)
)
```

- (b) Quantas passageiras (sexo feminino) sobreviveram?
- (c) Qual a idade média das pessoas que sobreviveram?
- (d) Quantas pessoas viajavam na 3.ª classe? Qual a taxa de sobrevivência nesse grupo?

4.5 Listas

Vimos que os vetores e as matrizes são homogêneos (um só tipo) e que o data frame é uma tabela com colunas de tipos diferentes, mas ainda com uma forma retangular rígida. Falta-nos

uma estrutura verdadeiramente livre — capaz de guardar, num só objeto, coisas tão diferentes como um número, um vetor, uma matriz e até uma função. Essa estrutura é a **lista**.

A lista é a estrutura mais flexível do R. Ao contrário do vetor, os seus elementos podem ser de tipos e tamanhos completamente diferentes. É a ferramenta ideal para agrupar informação heterogénea — e, não por acaso, é a estrutura sobre a qual o próprio data frame é construído (recorde que `typeof()` de um data frame devolvia "list").

4.5.1 Criar e manipular listas

Criamos uma lista com `list()`, dando (opcionalmente) um nome a cada elemento:

```
minha_lista <- list(
  nome = "Estudante",
  idade = 21,
  notas = c(85, 90, 92),
  disciplinas = c("Matemática", "Estatística", "Computação"),
  matriz_exemplo = matrix(1:9, nrow = 3, byrow = TRUE),
  media = function(x) mean(x)
)
```

Repare na variedade: um texto, um número, dois vetores, uma matriz e uma função — tudo dentro de um único objeto. Aceder aos elementos faz-se de três formas, cuja distinção é importante:

```
# [[ ]] ou $ extraem o CONTEÚDO do elemento (o vetor, o número, etc.)
minha_lista[[1]]
## [1] "Estudante"
minha_lista$nome
## [1] "Estudante"

# [ ] devolve uma SUB-LISTA (mantém a estrutura de lista)
minha_lista[1]
## $nome
## [1] "Estudante"

# Podemos combinar: aceder a um elemento e depois indexá-lo
minha_lista$notas[2]
## [1] 90
```

A diferença entre `minha_lista[[1]]` e `minha_lista[1]` é a mesma que já encontramos nos data frames, e é uma das subtilezas mais características do R. `[[ ]]` “abre a caixa” e devolve o que

está lá dentro; `[ ]` devolve uma caixa mais pequena com o elemento ainda embrulhado. Uma imagem útil: se a lista é uma mala com compartimentos, `[[ ]]` tira o objeto do compartimento, ao passo que `[ ]` dá-nos uma mala mais pequena só com esse compartimento.

Acrescentar e remover elementos segue a lógica habitual:

```
# Acrescentar um elemento
minha_lista$nota_final <- mean(minha_lista$notas)

# Remover um elemento
minha_lista$idade <- NULL
```

4.5.2 Exercícios

1. Crie uma lista `dados_estudante` com: `nome` = “João”, `idade` = 21 e `notas` = um vetor com as notas de Estatística (85), Matemática (90) e Computação (95). Aceda e imprima o nome, a idade e a nota de Computação.
2. Na lista `dados_estudante`, acrescente um elemento `status` = “Aprovado”. Substitua a nota de Estatística por 88. Imprima a lista completa.
3. Crie duas listas, `estudante1` e `estudante2`, com a mesma estrutura:
 - `estudante1`: “Maria”, 22, notas 78, 85, 90, status “Aprovado”.
 - `estudante2`: “Carlos”, 23, notas 70, 75, 80, status “Aprovado”.

Combine-as numa lista `turma` e imprima-a.

4. Usando a lista `turma`:

- (a) Extraia e imprima o nome do segundo estudante.
- (b) Calcule a média das notas do primeiro estudante.
- (c) Mude o status do segundo estudante para “Reprovado” e imprima a lista.

5. Crie uma lista `estatistica_aplicada` com duas listas internas, `turma1` e `turma2`, cada uma com dois estudantes (mesma estrutura de antes). `turma1` deve conter `estudante1` e `estudante2`; `turma2`, dois novos estudantes. Aceda e imprima o nome do primeiro estudante da `turma2`, a média das notas do segundo estudante da `turma1` e a lista completa.

6. Uma fábrica produz 4 produtos e regista, para cada um, o nome, a quantidade produzida mensalmente, o custo de produção unitário (€) e o preço de venda unitário (€). Crie a lista `produtos_fabrica` e:

- (a) Calcule e acrescente à lista a margem de lucro mensal de cada produto (preço menos custo, vezes a quantidade). Imprima-as.
- (b) Identifique o produto com maior margem de lucro mensal.
- (c) Aumente em 5% o preço de venda do segundo produto e imprima a sua nova margem.
- (d) Crie uma lista com os produtos cuja margem mensal supera 10% do custo de produção mensal.

7. Investigadores registaram, para três estudantes, o número de mensagens trocadas com o ChatGPT e o tempo total de uso (minutos) numa semana:

```
chatgpt_logs <- list(
  estudante1 = list(nome = "Ana", mensagens = c(12, 15, 9), tempo_total = 35),
  estudante2 = list(nome = "Bruno", mensagens = c(7, 6, 5, 12), tempo_total = 42),
  estudante3 = list(nome = "Carla", mensagens = c(10, 20), tempo_total = 28)
)
```

- (a) Quantas mensagens enviou o Bruno na terceira sessão?
- (b) Calcule o total de mensagens de cada estudante.
- (c) Acrescente a cada estudante um elemento `media_tempo_sessao` com o tempo médio por sessão.
- (d) Calcule a média de mensagens por sessão de cada um. Quem usou o ChatGPT de forma mais intensiva?

8. Três perfis de Instagram sobre estatística publicaram vários conteúdos numa semana:

```
instagram <- list(
  biostats_girl = list(temas = c("café", "dados", "meme"),
    visualizacoes = c(1200, 1800, 2400)),
  r_nerd = list(temas = c("dica_R", "ggplot2", "shiny"),
    visualizacoes = c(800, 1900, 2100)),
  estatistica_hoje = list(temas = c("história", "carreira", "código"),
    visualizacoes = c(1500, 1600, 2000))
)
```

- (a) Qual foi o tema mais visto do perfil `r_nerd`?
- (b) Calcule o total de visualizações de cada perfil e guarde-o num novo elemento `total_views`.
- (c) Ordene os perfis do mais para o menos visto.
- (d) Calcule a média de visualizações por publicação de cada perfil. Qual teve melhor desempenho médio?

9. Os resultados de três turmas na disciplina de Probabilidades incluem, por turma, as notas dos alunos em dois testes (T1 e T2) e se entregaram o projeto:

```
avaliacoes <- list(  
  turmaA = list(notas_t1 = c(14, 12, 16), notas_t2 = c(15, 13, 14),  
                projeto = c(TRUE, TRUE, FALSE)),  
  turmaB = list(notas_t1 = c(10, 11, 9), notas_t2 = c(13, 12, 10),  
                projeto = c(TRUE, FALSE, FALSE)),  
  turmaC = list(notas_t1 = c(17, 18, 19), notas_t2 = c(18, 17, 20),  
                projeto = c(TRUE, TRUE, TRUE))  
)
```

- (a) Calcule a média final de cada aluno em cada turma.
- (b) Quantos alunos, por turma, entregaram o projeto e tiveram média final ≥ 13 ?
- (c) Para cada turma, calcule a média geral das notas finais e a percentagem de aprovados (quem entregou o projeto e teve média ≥ 10).
- (d) Qual foi a turma com melhor desempenho global?

Capítulo 5

Estruturas de decisão

Até agora, os nossos programas executavam sempre a mesma sequência de comandos, do princípio ao fim, sem alternativas. Mas um programa útil precisa, muitas vezes, de *decidir*: se a nota é suficiente, aprova; caso contrário, reprova. Se o número é par, faz uma coisa; se é ímpar, faz outra. É esta capacidade de escolher caminhos diferentes consoante as circunstâncias que separa uma simples lista de contas de um verdadeiro programa.

As **estruturas de decisão** (ou de seleção) controlam o fluxo de execução com base em condições. E, felizmente, já temos as peças de que precisamos: as condições são exatamente as expressões lógicas que aprendemos no Capítulo 3 — aquelas que devolvem `TRUE` ou `FALSE`. Vamos agora usá-las para dirigir o programa. As estruturas mais comuns são o `if`, o `else` e o `else if`.

5.1 A condicional `if`

O `if` é a estrutura mais elementar: executa um bloco de código **apenas se** uma condição for verdadeira. Se a condição for falsa, o bloco é simplesmente ignorado e o programa continua.

```
if (condição) {  
  # código executado apenas se a condição for TRUE  
}
```

Um exemplo concreto: verificar se alguém é maior de idade.

```
idade <- 18  
  
if (idade >= 18) {  
  print("Maior de idade")  
}  
## [1] "Maior de idade"
```

Como `idade` vale 18, a condição `idade >= 18` é verdadeira, e o R executa o código dentro das chavetas.

Repare na anatomia da instrução: a condição vai entre **parênteses** () e o bloco de código a executar vai entre **chavetas** { }. Habitue-se a esta pontuação desde já — esquecer uma chaveta é um dos erros mais frequentes.

5.2 A estrutura if...else

O `if` sozinho responde a “faz isto *se*...”. Mas muitas vezes queremos também dizer “...*caso contrário*, faz aquilo”. Para isso serve o `else`, que acrescenta um caminho alternativo, executado quando a condição é falsa.

```
if (condição) {  
  # código executado se a condição for TRUE  
} else {  
  # código executado se a condição for FALSE  
}
```

Reformulando o exemplo anterior para cobrir os dois casos:

```
idade <- 16  
  
if (idade >= 18) {  
  print("Maior de idade")  
} else {  
  print("Menor de idade")  
}  
## [1] "Menor de idade"
```

Agora, como `idade` vale 16, a condição é falsa e o R executa o bloco do `else`. Note que exatamente um dos dois blocos é executado — nunca ambos, nunca nenhum.

5.3 A condicional else if

E quando há mais de dois caminhos? Uma nota pode ser “Excelente”, “Bom” ou “Insuficiente” — três hipóteses, não duas. O `else if` permite encadear várias condições. O R avalia-as por ordem e, assim que uma é verdadeira, executa o respetivo bloco e ignora todos os restantes.

```

if (condição1) {
  # se condição1 for TRUE
} else if (condição2) {
  # se condição2 for TRUE (e a primeira for FALSE)
} else {
  # se nenhuma das anteriores for TRUE
}

```

Classifiquemos uma nota:

```

nota <- 85

if (nota >= 90) {
  print("Excelente")
} else if (nota >= 70) {
  print("Bom")
} else {
  print("Insuficiente")
}

## [1] "Bom"

```

A **ordem** das condições importa. O R testa-as de cima para baixo e pára na primeira verdadeira. Por isso, escrevemos primeiro `nota >= 90` e só depois `nota >= 70`: uma nota de 95 satisfaz ambas, mas queremos que seja “Excelente”. Se invertêssemos a ordem, uma nota de 95 satisfaria logo `nota >= 70` e seria erradamente rotulada de “Bom”.

5.4 A função `ifelse()`

Todas as estruturas anteriores decidem sobre **um único valor** de cada vez. Mas lembremos de que, em R, trabalhamos constantemente com vetores. E se quiséssemos classificar *cada elemento* de um vetor? Escrever um `if` para cada um seria absurdo.

É aqui que entra a função `ifelse()`, a versão **vetorizada** da decisão. Aplica uma condição a cada elemento de um vetor de uma só vez, devolvendo um valor consoante a condição seja verdadeira ou falsa:

```

resultado <- ifelse(condição, valor_se_true, valor_se_false)

```

Vejamos:

```
valores <- c(4, 6, 9, 3)
ifelse(valores > 5, "maior que 5", "não é maior que 5")
## [1] "não é maior que 5" "maior que 5"      "maior que 5"
## [4] "não é maior que 5"
```

A função avaliou `valores > 5` para cada elemento e devolveu, elemento a elemento, a etiqueta correspondente. É uma ferramenta indispensável para criar novas variáveis a partir de condições — algo que faremos muitas vezes ao trabalhar com dados.

Não confunda `if` com `ifelse()`. O `if` decide sobre **uma** condição e executa um **bloco** de código; espera uma condição de comprimento 1. O `ifelse()` é uma **função** que percorre um vetor e devolve um vetor. Regra prática: se está a decidir o rumo do programa, use `if`; se está a transformar um vetor elemento a elemento, use `ifelse()`.

5.5 Erros comuns e exemplos de análise

Ler e depurar código de decisão é tão importante como escrevê-lo. Os exemplos seguintes treinam esse olhar crítico — começando por identificar erros.

Exemplo 1. Que erro há no código abaixo?

```
if (x %% 2 = 0) {
  print("Par")
} else {
  print("Ímpar")
}
```

O erro está no `=`: dentro da condição queremos **testar** a igualdade, não atribuir. A forma correta usa `==`:

```
if (x %% 2 == 0) {
  print("Par")
} else {
  print("Ímpar")
}
```

Exemplo 2. Que erros há neste código?

```
if (a > 0) {
  print("Positivo")
}
```

```

if (a %% 5 = 0)
  print("Divisível por 5")
} else if (a == 0)
  print("Zero")
else if {
  print("Negativo")
}

```

Há vários problemas: o `=` em vez de `==`, chavetas em falta e um `else if` sem condição (que devia ser apenas `else`). A versão corrigida:

```

if (a > 0) {
  print("Positivo")
  if (a %% 5 == 0) {
    print("Divisível por 5")
  }
} else if (a == 0) {
  print("Zero")
} else {
  print("Negativo")
}

```

Use sempre chavetas `{ }`, mesmo quando o bloco tem uma só linha. É verdade que o R as dispensa nesse caso, mas incluí-las torna o código mais claro, evita surpresas quando mais tarde acrescentamos linhas ao bloco, e previne muitos dos erros que vimos acima.

Exemplo 3. Quais são os valores de `x` e `y` no fim da execução?

```

x <- 1
y <- 0
if (x == 1) {
  y <- y - 1
}
if (y == 1) {
  x <- x + 1
}

```

Vale a pena seguir o raciocínio passo a passo, como faria o R. A primeira condição (`x == 1`) é verdadeira, logo `y` passa a `-1`. A segunda (`y == 1`) é falsa (porque `y` vale agora `-1`), logo `x` mantém-se em 1. No fim: `x = 1` e `y = -1`.

Exemplo 4. Considere o programa seguinte.

```
if (x == 1) {  
  x <- x + 1  
  if (x == 1) {  
    x <- x + 1  
  } else {  
    x <- x - 1  
  }  
} else {  
  x <- x - 1  
}
```

- Se $x = 1$, qual será o valor final de x ?
- Que valor teria de ter x para que, no fim, valesse -1 ?
- Há uma parte do programa que **nunca** é executada. Qual, e porquê?

Sugestão para o terceiro ponto: dentro do primeiro `if`, começamos por fazer `x <- x + 1`. Que valor tem x imediatamente a seguir? Pode a condição `if (x == 1)` interior alguma vez ser verdadeira nesse ponto? Percorrer o código “à mão”, anotando o valor das variáveis a cada passo, é a técnica mais fiável para responder a estas perguntas — e um hábito que vale a pena cultivar.

5.6 Exercícios

1. Escreva um programa que leia um número e exiba “O número é positivo” se for maior que zero.
2. Escreva um programa que leia um inteiro e imprima “Par” se for par e “Ímpar” caso contrário.
3. Escreva um programa que leia um número não negativo e indique se está no intervalo $[0, 10]$, $[11, 20]$, ou se é maior que 20 (intervalos inclusivos). Por exemplo, para 15, deve exibir “O número está no intervalo $[11, 20]$ ”.
4. Escreva um programa que leia uma idade e a classifique como “Criança” (0–12), “Adolescente” (13–17), “Adulto” (18–64) ou “Idoso” (≥ 65). Se a idade for negativa, exiba “Idade inválida”.
5. Escreva um programa que leia o preço de um produto e aplique um desconto: 5% se o preço for inferior a 100 €; 10% se estiver entre 100 € e 500 € (inclusive); 15% se for superior a 500 €. Se o preço for negativo, exiba “Preço inválido”. Por exemplo, para 350, deve exibir “O preço com desconto é 315 €”.
6. Escreva um programa que leia as coordenadas (x, y) de um ponto e determine o quadrante em que se encontra. Se estiver sobre um eixo, indique “Eixo X” ou “Eixo Y”. Por exemplo, para $(-3, 2)$, deve exibir “Segundo quadrante”.

7. Escreva um programa que leia um ano e determine se é bissexto (divisível por 4, mas não por 100, exceto se for divisível por 400). Se o ano for negativo, exiba “Ano inválido”. Por exemplo, para 2000, deve exibir “Ano bissexto”.
8. Escreva um programa que leia um inteiro entre 0 e 999 e o descreva em centenas, dezenas e unidades. Por exemplo, para 304, deve exibir “3 centenas 0 dezenas e 4 unidades”. Fora de $[0, 999]$, exiba “Número fora do intervalo”.
9. Escreva um programa que leia um inteiro positivo (≤ 5) e escreva a sua representação em numeração romana. Fora de $[1, 5]$, exiba “Número fora do intervalo”. Por exemplo, para 3, deve exibir “III”.
10. Escreva um programa que leia quatro inteiros (um de cada vez) e exiba, após cada leitura, o menor número lido até ao momento.
11. Crie um vetor com os números de 1 a 10 e use `ifelse()` para rotular cada número como “par” ou “ímpar”.
12. Crie o vetor de notas `c(50, 85, 70, 40, 60, 90)`. Use `ifelse()` para marcar como “Aprovado” quem tem nota ≥ 50 e “Reprovado” os restantes. Depois, modifique o programa para que as notas entre 50 e 59 sejam “Aprovado com Restrição”.
13. Dado um vetor de idades, use `ifelse()` (encadeado) para as categorizar em “Criança” (≤ 12), “Adolescente” (13-18) e “Adulto” (> 18).

Capítulo 6

Funções

Ao longo dos capítulos anteriores usámos dezenas de funções — `mean()`, `sum()`, `c()`, `factor()`, `subset()`. Chegou o momento de virar a mesa e aprender a escrever as *nossas próprias* funções. Esta é, talvez, a competência mais transformadora de toda a programação: a partir do momento em que sabemos criar funções, deixamos de ser apenas utilizadores da linguagem e passamos a estendê-la.

A motivação é simples e profunda. Imagine que precisa de calcular o índice de massa corporal de dez pessoas. Poderia repetir a fórmula dez vezes — mas, se se enganasse na fórmula, teria de corrigir dez lugares, e se quisesse mudar algo, outros dez. A alternativa é escrever a fórmula **uma vez**, dentro de uma função, e chamá-la dez vezes. A este princípio dá-se por vezes o nome de “não te repitas”: tudo o que se repete deve ser encapsulado num só lugar.

Uma função é, então, um bloco de código com nome, que realiza uma tarefa específica e só corre quando é chamado. As suas vantagens resumem-se a três palavras:

- **Reutilização** — uma vez definida, chama-se as vezes que forem precisas, em qualquer parte do programa.
- **Modularização** — divide um problema grande em peças pequenas, cada uma com uma responsabilidade clara, tornando o código mais organizado e legível.
- **Abstração** — depois de escrita e testada, podemos usar a função sem voltar a pensar em como funciona por dentro. `mean()` calcula uma média sem nos obrigar a saber como.

6.1 Anatomia de uma função

A estrutura de uma função em R é a seguinte:

```
nome_da_funcao <- function(argumentos) {  
  # corpo: o código que realiza a tarefa  
  resultado <- ...  
  return(resultado)  # o valor devolvido  
}
```

Quatro elementos a identificar:

- **Nome** — o identificador pelo qual chamaremos a função (segue as regras dos nomes de variáveis do Capítulo 3).
- **Argumentos** — os valores de entrada que a função recebe. Podem ser obrigatórios ou opcionais.
- **Corpo** — o bloco de código, entre chavetas, que faz o trabalho.
- **Valor devolvido** — o resultado que a função entrega, indicado por `return()`.

Vejamos um primeiro exemplo completo: uma função que calcula a área de um retângulo.

```
calcula_area <- function(largura, altura) {  
  area <- largura * altura  
  return(area)  
}  
  
# Chamar a função  
calcula_area(4, 3)  
## [1] 12
```

Definimos a função uma vez e, a partir daí, calcular a área de qualquer retângulo é uma questão de a chamar com novos valores. É este o ganho.

6.1.1 Variáveis locais e globais

Há aqui uma subtileza importante, que evita muita confusão mais tarde. Dentro da função `calcula_area`, as variáveis `largura`, `altura` e `area` são **locais**: existem apenas enquanto a função corre e desaparecem quando ela termina. Não são visíveis fora da função, nem interferem com variáveis de igual nome que existam no resto do programa.

```
x <- 10  
  
minha_funcao <- function() {  
  x <- 5          # este 'x' é local: só existe dentro da função
```

```
    return(x)
  }

minha_funcao()
## [1] 5
x           # o 'x' global não foi afetado
## [1] 10
```

Este isolamento é uma vantagem, não um obstáculo. Significa que podemos escrever uma função sem receio de que os seus nomes internos colidam com o resto do programa — cada função tem o seu “mundo” próprio. É uma das razões pelas quais as funções tornam o código mais seguro e mais fácil de manter.

6.2 Argumentos: mais flexibilidade

As funções tornam-se muito mais úteis quando aprendemos a controlar como os seus argumentos são passados.

6.2.1 Valores por omissão

Podemos dar a um argumento um **valor por omissão**, usado sempre que quem chama a função não o especifica. Isto torna certos argumentos opcionais:

```
calcula_area <- function(largura, altura = 2) {
  return(largura * altura)
}

calcula_area(4)      # 'altura' não foi dado, usa-se o valor por omissão (2)
## [1] 8
```

6.2.2 Argumentos por nome

Ao chamar uma função, podemos passar os argumentos **pelo nome**, e não apenas pela posição. Isto liberta-nos da ordem e torna as chamadas mais legíveis:

```
calcula_area <- function(largura, altura = 2) {
  return(largura * altura)
}
```

```
calcula_area(altura = 4, largura = 3)  # a ordem já não importa
## [1] 12
```

Passar argumentos pelo nome é uma excelente prática quando uma função tem muitos argumentos, ou quando queremos especificar apenas alguns dos opcionais. Compare `rnorm(100, 0, 1)` com `rnorm(n = 100, mean = 0, sd = 1)`: a segunda forma dispensa quem lê o código de decorar a ordem dos argumentos.

6.3 Validar entradas com `stop()`

Uma boa função protege-se contra usos indevidos. A função `stop()` interrompe a execução e emite uma mensagem de erro personalizada, útil quando uma condição necessária não é satisfeita:

```
f <- function(x) {
  if (x < 0) {
    stop("Erro: x não pode ser negativo")
  }
  return(sqrt(x))
}

f(-2)
## Error in f(-2): Erro: x não pode ser negativo
```

Ao verificar `x < 0` logo no início, a função recusa liminarmente uma entrada inválida (a raiz quadrada de um negativo), em vez de devolver um `NaN` silencioso que poderia contaminar cálculos posteriores. Antecipar e sinalizar erros é a marca de uma função bem escrita.

6.4 Exercícios de análise

Antes dos exercícios de construção, alguns exercícios de leitura — para consolidar a compreensão do que as funções fazem.

Exemplo 1. Qual é o resultado da chamada?

```
dados_estudante <- function(nome, altura = 167) {
  print(paste("O(A) estudante", nome, "tem", altura, "centímetros de altura."))
}

dados_estudante("Joana", 160)
```

Exemplo 2. Qual é a sintaxe correta para definir uma função que soma dois números?

- (a) `soma <- function(x, y) { return(x + y) }`
- (b) `function soma(x, y) { return(x + y) }`
- (c) `def soma(x, y) { return(x + y) }`
- (d) `soma(x, y) = function { return(x + y) }`

Exemplo 3. Considerando a função `dados_estudante` do Exemplo 1, quais das chamadas seguintes estão corretas?

```
dados_estudante("Joana", 160)
dados_estudante(altura = 160, nome = "Joana")
dados_estudante(nome = "Joana", 160)
dados_estudante(altura = 160, "Joana")
dados_estudante(160)
```

A última chamada, `dados_estudante(160)`, é sintaticamente válida mas logicamente absurda: 160 será interpretado como o `nome` (por ser o primeiro argumento) e a `altura` usará o valor por omissão, 167 — resultando em “O(A) estudante 160 tem 167 centímetros de altura.” Um bom lembrete de que o R faz o que lhe pedimos, não o que queremos dizer.

Exemplo 4. Qual é o valor de `resultado`?

```
adi <- function(a, b) {
  return(c(a + 5, b + 5))
}

resultado <- adi(3, 2)
```

6.5 Exercícios

1. Crie uma função `quadrado()` que receba um número `x` e devolva o seu quadrado. Teste-a com 3, 5 e 7.
2. Crie uma função `hipotenusa(a, b)` que calcule a hipotenusa de um triângulo retângulo a partir dos dois catetos, usando o Teorema de Pitágoras.
3. Crie uma função `divisao_segura(numerador, denominador)` que devolva a divisão, ou uma mensagem de erro apropriada se o denominador for zero.
4. Crie uma função que receba dois inteiros e devolva o maior. Teste-a lendo dois números do utilizador. Depois, crie uma segunda função que devolva o menor, aproveitando a primeira.

5. Crie uma função que devolva o maior de três inteiros, reutilizando a função `maior()` do exercício anterior.
6. Crie uma função `analise_vetor()` que receba um vetor numérico e devolva um vetor com a soma, a média, o desvio padrão e o máximo. Aplique-a a `c(4, 8, 15, 16, 23, 42)`. *Funções úteis:* `sum()`, `mean()`, `sd()`, `max()`.
7. Crie uma função `classificar_numero()` que classifique um inteiro como “positivo”, “negativo” ou “zero”.
8. Crie uma função que calcule o Índice de Massa Corporal a partir do peso (kg) e da altura (m) e o classifique segundo a tabela:

IMC (kg/m ²)	Classificação
< 18,5	Baixo peso
18,5 – 24,9	Peso normal
25 – 29,9	Excesso de peso
30 – 34,9	Obesidade grau I
35 – 39,9	Obesidade grau II
≥ 40	Obesidade grau III

Recorde que $IMC = \frac{\text{Peso}}{\text{Altura}^2}$.

9. Crie funções para converter euros noutras moedas, com base em taxas de câmbio. Escreva depois um programa que peça um valor em euros e imprima a conversão para dólar, libra, iene e real.
10. Crie uma função que calcule o intervalo de confiança a $(1 - \alpha) \times 100\%$ para o valor médio (para amostras grandes, de dimensão $n \geq 30$), dado por

$$\left(\bar{x} - z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \bar{x} + z_{1-\frac{\alpha}{2}} \frac{s}{\sqrt{n}} \right),$$

onde s é o desvio padrão amostral e $z_{1-\frac{\alpha}{2}}$ é o quantil de probabilidade $1 - \alpha/2$ (use `qnorm()`).

```
IC <- function(amostra, alpha) {
  # o seu código aqui
}
```

11. Generalize a função anterior para calcular o intervalo de confiança para vários valores de α de uma só vez (por exemplo, 0,001, 0,01, 0,05 e 0,10).

```
ICs <- function(amostra, alphas) {  
  # o seu código aqui  
}
```

12. Crie uma função `calcula_moda()` que receba um vetor (numérico ou de caracteres) e devolva a moda — o valor mais frequente. Se houver mais do que uma moda, deve devolvê-las todas. *Sugestão:* Use a função `table()` para contar ocorrências.

Teste com `c(1, 2, 2, 3, 3, 4, 4)` e com `c("A", "B", "A", "C", "C", "C", "B")`.

13. A distância euclidiana entre dois pontos $A(x_1, y_1)$ e $B(x_2, y_2)$ é

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Crie uma função `distancia_euclidiana()` que a calcule, e teste-a com $A(2, 3)$ e $B(6, 9)$. Generalize-a depois para pontos em qualquer dimensão e teste-a com $A(1, 2, 3)$ e $B(4, 5, 6)$.

14. Crie uma função `cria_matriz_aleatoria(linhas, colunas)` que devolva uma matriz com números aleatórios uniformes entre 0 e 1. Crie depois uma função `analisa_matriz()` que devolva a soma, a média e o traço (se a matriz for quadrada) de uma matriz dada. Teste-as com uma matriz 5×5 . Por fim, generalize `analisa_matriz()` para devolver também o determinante, apenas quando a matriz for quadrada; caso contrário, deve informar que não é possível calculá-lo.

Com as funções, fechamos a caixa de ferramentas fundamental da programação em R. Temos agora tudo o que é preciso — valores, estruturas de dados, decisões e funções — para, nos próximos capítulos, automatizar tarefas com ciclos, ler dados reais e, finalmente, mergulhar na probabilidade e na simulação.

Capítulo 7

Scripts em R

Nos capítulos anteriores escrevemos código diretamente na consola e definimos as nossas próprias funções (Capítulo 6). Enquanto experimentamos, a consola é cómoda; mas o trabalho que fica apenas na consola perde-se quando fechamos a sessão e é difícil de repetir com exatidão. Chegou o momento de guardar o código num lugar permanente: um *script*.

Um **script** é simplesmente um ficheiro de texto, com a extensão `.R`, que reúne uma sequência de instruções — variáveis, funções, blocos de código — pensada para ser guardada, lida e executada de novo sempre que for preciso. Os scripts são a base de duas ideias centrais em ciência de dados: a **automatização** de tarefas repetitivas e a **reprodutibilidade** da análise, isto é, a garantia de que qualquer pessoa — incluindo o nosso próprio “eu” de daqui a seis meses — consegue obter exatamente os mesmos resultados a partir dos mesmos dados.

Neste capítulo aprendemos a guardar código num ficheiro `.R`, a executá-lo com `source()`, a compreender o papel do diretório de trabalho e a adotar hábitos que tornam a nossa análise reprodutível.

7.1 Criar um script

No RStudio, cria-se um novo script pelo menu *File > New File > R Script* (ou com o atalho `Ctrl+Shift+N` / `Cmd+Shift+N`). Escreve-se o código no editor e guarda-se o ficheiro com a extensão `.R`.

Guarde, por exemplo, o código seguinte num ficheiro chamado `meu_script.R`:

```
produto <- function(x, y) {  
  return(x * y)  
}
```


Repare que guardar o script **não executa** o código — apenas o preserva. Definir a função e poder usá-la são dois passos distintos.

7.2 Executar um script

Para correr um script guardado, usamos a função `source()`, na consola ou dentro de outro script. O R lê o ficheiro e executa, uma a uma, todas as suas linhas.

Se o ficheiro `meu_script.R` estiver no **diretório de trabalho atual**, basta:

```
source("meu_script.R")
```

Se estiver noutra pasta, indicamos o caminho completo:

```
source("/caminho/para/o/seu/script/meu_script.R")
```

7.2.1 O que acontece quando executamos um script?

Ao chamar `source()`, todas as funções, variáveis e resultados definidos no script passam a estar disponíveis no ambiente de trabalho. Depois de executar o `meu_script.R` acima, a função `produto()` fica pronta a usar:

```
produto(2, 3)
## [1] 6
```

É esta a lógica que torna os scripts tão úteis: escrevemos e testamos um conjunto de funções uma vez, guardamo-las num ficheiro e, em qualquer análise futura, basta uma linha — `source("as_minhas_funcoes.R")` — para as ter todas à disposição.

No RStudio, o botão **Source** (canto superior direito do editor) executa o script inteiro, e **Ctrl+Enter** / **Cmd+Enter** corre apenas a linha ou o bloco selecionado. Correr linha a linha é a melhor forma de perceber e depurar um script enquanto o escrevemos.

7.3 O diretório de trabalho

O **diretório de trabalho** (*working directory*) é a pasta que o R toma como ponto de referência: é aí que procura os ficheiros a ler e onde guarda os resultados, a menos que indiquemos um caminho completo. Duas funções bastam para o gerir:

- `getwd()` — devolve o caminho do diretório de trabalho atual.

- `setwd("caminho/da/pasta")` — muda o diretório de trabalho para o caminho indicado.

```
getwd()
## [1] "C:/Users/Utilizador/Documents"
```

Com o diretório de trabalho bem definido, referimo-nos aos ficheiros pelo nome, sem escrever o caminho todo. Se `dados.csv` estiver no diretório de trabalho:

```
dados <- read.csv("dados.csv")
```

Evite espalhar chamadas a `setwd("C:/Users/...")` pelos seus scripts. Um caminho absoluto só existe no seu computador: quando o script chega às mãos de outra pessoa (ou muda de máquina), deixa de funcionar — e a reprodutibilidade, que era o objetivo, perde-se. A boa prática é trabalhar dentro de um **Projeto do RStudio** (*File > New Project*), que fixa automaticamente o diretório de trabalho na pasta do projeto e permite usar apenas caminhos **relativos**, como `"dados/dados.csv"`.

7.4 Exercícios

1. Crie os seguintes scripts passo a passo:

- (a) Crie um script `quadrado.R` com funções para calcular o perímetro e a área de um quadrado, dado o comprimento do lado. Noutro script, use `quadrado.R` para calcular o perímetro e a área de um quadrado de lado 5.
- (b) Crie um script `estatistica.R` com funções para ordenar uma amostra e calcular a média, a variância e o desvio padrão de um vetor numérico.
- (c) Crie um programa que use o script `estatistica.R` para calcular a mediana, a variância e o desvio padrão da amostra `amostra <- c(1, 2, 3, 4, 5, 6, 7)`.

2. Crie um script `circulo.R` com duas funções:

- `area_circulo(r)` — calcula a área de um círculo de raio r ;
- `circunferencia(r)` — calcula o perímetro (circunferência) de um círculo de raio r .

Depois, noutro script, use as funções de `circulo.R` para calcular a área e o perímetro de um círculo de raio 10.

3. Crie um script `analise.R` com funções para calcular:

- o valor mínimo (`minimo`) e máximo (`maximo`) de um vetor numérico;
- a amplitude (`amplitude`), definida como a diferença entre o máximo e o mínimo.

Depois, num script separado, use `analise.R` para calcular o mínimo, o máximo e a amplitude do vetor `dados <- c(8, 15, 22, 3, 18, 7)`.

4. Crie um script `categorias.R` com uma função `contar_categorias()` que receba um vetor categórico e devolva uma tabela de frequências de cada categoria. Use-o depois para analisar o vetor `idades <- c("Lisboa", "Porto", "Lisboa", "Coimbra", "Porto", "Porto", "Lisboa")` e apresente a frequência de cada cidade.

5. Crie um script `escore_z.R` com uma função `calcular_escore_z(valor, media, desvio_padrao)` que calcule o valor padronizado (*escore z*) de um valor, dado por $z = \frac{x - \mu}{\sigma}$. Num script separado, use-o para calcular o *escore z* dos valores `valores <- c(5, 6, 7, 8, 9)`, assumindo média 7 e desvio padrão 1,5.

6. Crie um script `binomial.R` com uma função `probabilidade_binomial(n, p, x)` que calcule a probabilidade de ocorrerem exatamente x sucessos em n provas independentes de um ensaio de Bernoulli:

$$P(X = x) = \binom{n}{x} p^x (1 - p)^{n-x},$$

onde $\binom{n}{x}$ é o coeficiente binomial (em R, `choose(n, x)`) e p é a probabilidade de sucesso em cada prova. Num script separado, use-o para calcular a probabilidade de obter exatamente 3 sucessos em 5 provas, com $p = 0,6$.

Com os scripts guardámos o nosso código de forma permanente e reutilizável. No próximo capítulo damos o passo seguinte e igualmente decisivo: trazer para dentro do R **dados reais**, guardados em ficheiros de texto, CSV ou Excel.

Capítulo 8

Leitura de dados

Até aqui os dados com que trabalhámos foram sempre escritos à mão, dentro do próprio código. Mas a estatística vive de dados **reais** — recolhidos em inquéritos, sensores, registos administrativos ou plataformas online — e esses dados chegam-nos guardados em ficheiros. Saber importá-los para o R é, por isso, a ponte entre a linguagem que aprendemos e a análise que queremos fazer. É neste capítulo que o livro se torna verdadeiramente aplicado.

Seja qual for o formato, o processo de leitura segue quase sempre os mesmos três passos:

1. **Indicar onde está o ficheiro** — o seu nome ou caminho.
2. **Descrever as suas características** — separador de campos, separador decimal, presença de cabeçalho, codificação, valores em falta.
3. **Ler e guardar os dados num objeto** — quase sempre uma *data frame* (Capítulo 4), a estrutura ideal para manipular e analisar dados tabulares.

Neste capítulo aprendemos a ler dados a partir do teclado e de ficheiros de texto, CSV e Excel, com atenção às particularidades dos ficheiros portugueses (vírgula decimal, acentos), e a guardar resultados de volta em disco.

8.1 Ler dados do teclado

A função `scan()` lê valores da entrada padrão (o teclado) ou de um ficheiro e devolve-os num vetor. É útil para introduzir manualmente pequenas quantidades de dados:

```
# Pede ao utilizador uma série de números
numeros <- scan()
```

Depois de escrever `scan()` e premir *Enter*, introduzem-se os valores separados por espaços; uma linha vazia (novo *Enter*) termina a entrada.

8.2 O diretório de trabalho

Como vimos no Capítulo 7, o **diretório de trabalho** é a pasta que o R usa como referência para ler e guardar ficheiros. Vale a pena recordar as duas funções que o gerem:

- `getwd()` — devolve o caminho do diretório de trabalho atual;
- `setwd("caminho/da/pasta")` — define um novo diretório de trabalho.

```
getwd()
## [1] "C:/Users/Utilizador/Documents"
```

Com o diretório bem definido, se o ficheiro `dados.csv` estiver nessa pasta, lê-se simplesmente pelo nome, sem o caminho completo:

```
dados <- read.csv("dados.csv")
head(dados)      # mostra as primeiras linhas
```

No RStudio não é sequer necessário escrever `setwd()`: o menu *Session > Set Working Directory > To Source File Location* fixa o diretório na pasta do script. Ainda melhor, trabalhar dentro de um **Projeto do RStudio** dispensa por completo caminhos absolutos.

8.3 A função `read.table()`

A `read.table()` é a função mais versátil para ler ficheiros de texto em formato tabular. Importa os dados e permite configurá-los ao pormenor. (Pode descarregar o ficheiro `dados.txt` a partir do [site da disciplina](#) para praticar.)

```
dados <- read.table(file = "dados.txt", header = TRUE, sep = "")
print(dados)
```

Os seus argumentos mais usados são:

- `file` — nome ou caminho do ficheiro a ler.
- `header` — indica se a primeira linha contém os nomes das colunas (`header = TRUE`). **Por omissão é FALSE.**
- `sep` — o símbolo que separa os campos. **Por omissão é o espaço em branco** (`sep = ""`, que trata qualquer sequência de espaços ou tabulações como separador); pode ser `","`, `" "`, `"\t"` (tabulação), etc.
- `dec` — o símbolo que separa a parte inteira da decimal (`"."` ou `","`).

- `nrows` — número máximo de linhas a ler.
- `na.strings` — que valores no ficheiro devem ser interpretados como NA (por exemplo, "NA", "N/A").
- `skip` — número de linhas a ignorar no início do ficheiro.
- `comment.char` — símbolo que marca comentários; linhas iniciadas por ele são ignoradas.

É frequente confundir os separadores por omissão. Ao contrário do que se poderia pensar, o separador por omissão de `read.table()` **não é a vírgula, mas sim o espaço em branco**. A vírgula é o separador por omissão da `read.csv()`, que veremos já a seguir. Confundir os dois é uma das causas mais comuns de leituras que “correm bem” mas produzem uma única coluna com tudo lá dentro.

Para consultar todos os argumentos e os seus valores por omissão:

```
args(read.table)
## function (file, header = FALSE, sep = "", quote = "\"'", dec = ".",
##     numerals = c("allow.loss", "warn.loss", "no.loss"), row.names,
##     col.names, as.is = !stringsAsFactors, na.strings = "NA", ...)
## NULL
```

Se o ficheiro for um CSV, basta indicar o separador apropriado:

```
dados_csv <- read.table("dados.csv", header = TRUE, sep = ",")
```

8.4 A função `read.csv()`

A `read.csv()` está otimizada para ficheiros CSV (*Comma-Separated Values*), ficheiros de texto em que cada linha é um registo e os valores são separados por vírgulas. Na prática, é a `read.table()` com outros valores por omissão: separador ",", e `header = TRUE`.

```
dados_csv <- read.csv("dados.csv", header = TRUE)
```

8.5 A função `read.csv2()`

A `read.csv2()` destina-se a ficheiros CSV “à europeia”: separador de campos **ponto e vírgula** (;) e separador decimal **vírgula** (,). É o formato que a maioria dos programas exporta em Portugal, precisamente porque cá a vírgula é o separador decimal.

```
dados_csv2 <- read.csv2("dados.csv", header = TRUE)
```

Esta distinção é essencial em Portugal. Um valor como 1.234,56 num ficheiro português significa “mil duzentos e trinta e quatro vírgula cinquenta e seis”. Se o lermos com `read.csv()` (que espera a vírgula como separador de campos e o ponto como decimal), o R interpreta tudo mal. A regra prática: ficheiros gerados por sistemas portugueses leem-se, quase sempre, com `read.csv2()`.

8.6 A função `read_excel()`

Para ler ficheiros Excel (`.xls` e `.xlsx`) usamos a função `read_excel()` do pacote `readxl`, que precisa de ser instalado e carregado antes da primeira utilização:

```
install.packages("readxl")  # apenas uma vez
library(readxl)

# Sintaxe básica
read_excel(path, sheet = NULL, range = NULL, col_names = TRUE,
           col_types = NULL, na = "", trim_ws = TRUE, skip = 0, n_max = Inf)
```

Os seus parâmetros principais são:

- `path` — caminho do ficheiro Excel (obrigatório); pode ser local ou um URL.
- `sheet` — nome ou número da folha a ler. Por omissão, lê a primeira.
- `range` — intervalo específico a ler, como "A1:D10". Por omissão (`NULL`), lê a folha inteira.
- `col_names` — se a primeira linha deve ser usada como nomes das colunas (`TRUE`, por omissão).
- `col_types` — tipos das colunas ("numeric", "date", "text", "guess", ...). Por omissão (`NULL`), a função adivinha.
- `na` — cadeias a interpretar como valores em falta (`NA`).
- `trim_ws` — se remove espaços no início e no fim dos valores de texto (`TRUE`, por omissão).
- `skip`, `n_max` — linhas a ignorar no início e número máximo de linhas a ler.

Suponha um ficheiro `instagram.xlsx`. Para ler a folha `Sheet1` e guardá-la numa *data frame* chamada `dados_instagram`:

```
dados_instagram <- read_excel(path = "instagram.xlsx",
                              sheet = "Sheet1",
                              col_names = TRUE)
```

No RStudio, o botão **Import Dataset** (separador *Environment*) permite importar ficheiros de texto e Excel através de uma janela, mostrando uma pré-visualização e gerando automaticamente o código de leitura. É uma excelente forma de acertar os argumentos (separador, decimal, codificação) sem tentativa e erro.

8.7 Ler dados a partir de um URL

O R também lê dados diretamente da internet. Basta passar o endereço do ficheiro à função de leitura apropriada ao formato:

```
url <- "https://exemplo.com/dados.csv"
dados_online <- read.table(url, header = TRUE, sep = ",")
```

8.8 Guardar resultados em disco

Ler dados é só metade da história: depois de os analisar, queremos muitas vezes **guardar** o resultado. As funções `write.table()` e `write.csv()` fazem o percurso inverso das que vimos, escrevendo uma *data frame* num ficheiro de texto:

```
write.csv(dados, file = "resultado.csv", row.names = FALSE)
```

O argumento `row.names = FALSE` evita que o R acrescente uma coluna extra com os números das linhas — um pormenor pequeno, mas que poupa confusão a quem voltar a ler o ficheiro.

Os acentos do português dependem da **codificação** do ficheiro. Se, ao ler, os caracteres acentuados aparecerem trocados (por exemplo, ã§ em vez de ç), especifique a codificação com o argumento `fileEncoding`, tipicamente `fileEncoding = "UTF-8"` ou `fileEncoding = "latin1"`, conforme a origem do ficheiro.

8.9 Exercícios

1. Um instituto de climatologia registou as temperaturas mínima e máxima durante 3 dias. Os dados estão no ficheiro `temperaturas.txt`, com separador de tabulação e cabeçalho.

- (a) Leia `temperaturas.txt` com `read.table()`.
- (b) Veja as 6 primeiras linhas com `head()`.
- (c) Qual foi a temperatura máxima no segundo dia?

- (d) Crie uma coluna `diferenca_temp` com a diferença entre a temperatura máxima e a mínima e guarde o resultado num ficheiro `temperaturas_nova.txt`. *Sugestão:* `write.table(x = , file = , row.names = )`.
2. Um ficheiro `.csv` exportado de um sistema português contém despesas e receitas mensais: [financas.csv](#). Os valores usam vírgula como separador decimal e ponto e vírgula como separador de campos.
- (a) Importe os dados com `read.csv2()`.
- (b) Calcule a soma total das receitas.
- (c) Crie uma coluna `saldo = receitas - despesas`.
- (d) Guarde o resultado num ficheiro `financas_nova.csv`. *Sugestão:* `write.csv2(x = , file = , row.names = )`.
3. Um laboratório gerou um ficheiro sem cabeçalho com 4 colunas: `experimento`, `tempo`, `concentração` e `erro`, separadas por espaços: [resultados_lab.txt](#).
- (a) Leia os dados com `read.table(..., header = FALSE)`.
- (b) Atribua nomes às colunas: `"experimento"`, `"tempo"`, `"concentracao"`, `"erro"`.
- (c) Qual a concentração do 5.º experimento?
4. Um ficheiro `.csv` contém uma entrada incorreta na coluna `salario` (“quatro mil”), o que impede que a coluna seja lida como numérica: [dados_com_erros.csv](#).
- (a) Importe com `read.csv()` e verifique a classe da coluna `salario`.
- (b) Corrija os dados e converta `salario` em `numeric`.
5. Leia o ficheiro `instagram.xlsx` ([aqui](#); lembre-se de instalar o pacote `readxl`) e responda:
- (a) Mostre as 6 primeiras linhas do conjunto de dados.
- (b) **Tabela de frequências.** Construa uma tabela de frequências para cada variável. Para as variáveis numéricas, agrupe em classes. O código seguinte fá-lo para o `indice_influencia`, usando a Regra de Sturges para o número de classes:

```

# -----
# Índice de influência
# -----

# Número de classes pela Regra de Sturges
n <- length(dados_instagram$indice_influencia) # número de observações
k <- ceiling(1 + 3.322 * log10(n))             # número de classes (arredondado)

# Limites das classes
min_val <- min(dados_instagram$indice_influencia)
max_val <- max(dados_instagram$indice_influencia)
intervalos <- seq(min_val, max_val, length.out = k + 1)

# Tabela de frequências absolutas
tabela_if <- cut(dados_instagram$indice_influencia,
                 breaks = intervalos, include.lowest = TRUE)
tabela_if <- table(tabela_if)

# Converter em data frame para uma leitura mais organizada
tabela_if_df <- as.data.frame(tabela_if)
names(tabela_if_df) <- c("Intervalo", "Frequencia")

# Frequência relativa
tabela_if_df$Freq_Relativa <- round(tabela_if_df$Frequencia / n, digits = 3)

# Frequências acumuladas (absoluta e relativa)
tabela_if_df$Freq_Abs_Acum <- cumsum(tabela_if_df$Frequencia)
tabela_if_df$Freq_Rela_Acum <- cumsum(tabela_if_df$Freq_Relativa)

tabela_if_df

```

- (c) **Representação gráfica.** Crie o gráfico adequado a cada variável: histogramas (`hist()`) para variáveis contínuas, gráficos de barras (`barplot()`) para variáveis categóricas, e caixas de bigodes (`boxplot()`) para observar a dispersão.
- (d) **Medidas descritivas.** Determine, para cada variável, a média, a mediana, a variância, o desvio padrão, o coeficiente de variação, o primeiro e o terceiro quartis (Q_1 e Q_3), a amplitude e a amplitude interquartil. Indique quando alguma destas medidas não se aplica.
- (e) Calcule a média e a mediana do índice de influência **por gênero**. Comente as diferenças

e discuta se sugerem tendências distintas de influência.

- (f) Calcule a média e a mediana da média de gostos por género. Avalie se há diferenças relevantes e discuta possíveis implicações.
- (g) **Comparação de dispersão.** Compare a dispersão do índice de influência e do número de publicações usando a variância e o desvio padrão. Justifique qual apresenta maior dispersão e interprete o resultado.
- (h) **Percentis.** Identifique os percentis 10%, 25%, 75% e 90% do número de seguidores. Comente como podem servir para categorizar influenciadores por nível de popularidade.
- (i) **Outliers.** Investigue a presença de valores atípicos nas variáveis número de publicações, número de seguidores e média de gostos.
- (j) **Correlação.** Calcule a correlação entre o índice de influência e a média de gostos. Comente se um índice de influência elevado tende a associar-se a mais gostos. *Sugestão:* Use a função `cor()`.

6. Importe o ficheiro `dataset.csv` ([aqui](#)) e guarde-o numa *data frame*. Em seguida:

- (a) Mostre as 6 primeiras linhas.
- (b) Escolha uma variável categórica, construa a sua tabela de frequências e represente a distribuição num gráfico de barras.
- (c) Para cada variável numérica, crie um histograma (distribuição) e uma caixa de bigodes (dispersão e outliers).
- (d) Calcule as principais medidas descritivas de uma variável numérica à sua escolha: média, mediana, variância, desvio padrão, coeficiente de variação, quartis (Q_1 e Q_3), amplitude e amplitude interquartil.
- (e) Escolha uma variável numérica e uma categórica e compare a numérica entre os grupos da categórica:
 - calcule a média e a mediana da variável numérica em cada grupo;
 - use caixas de bigodes para comparar visualmente as distribuições.

Com os dados dentro do R, o passo seguinte é transformá-los e resumi-los. No próximo capítulo conhecemos o operador *pipe*, que torna essas sequências de operações muito mais legíveis.

Capítulo 9

O operador *pipe*

Depois de trazer os dados para o R (Capítulo 8), raramente lhes aplicamos uma única operação. O mais comum é encadear várias — filtrar, transformar, resumir — umas a seguir às outras. Escrever esse encadeamento de forma legível é o problema que o operador *pipe* resolve com elegância.

Considere uma tarefa simples: calcular a raiz quadrada da soma de um vetor. Sem *pipe*, escrevemos as funções aninhadas umas dentro das outras:

```
x <- c(1, 2, 3, 4)
sqrt(sum(x))
## [1] 3.16
```

O problema é que lemos este código **de dentro para fora**: primeiro `sum`, depois `sqrt`, embora a escrita seja a ordem inversa. Com duas funções ainda se gere; com cinco, torna-se um quebra-cabeças. O *pipe* permite escrever e ler as operações pela ordem em que acontecem, **da esquerda para a direita**.

9.1 O *pipe* do `magrittr` (`%>%`)

O *pipe* foi popularizado pelo pacote `magrittr`, de Stefan Milton Bache (2014). O nome é uma homenagem ao pintor surrealista René Magritte e à sua obra *Ceci n'est pas une pipe* (“Isto não é um cachimbo”) — um trocadilho com a ideia de fazer os dados “fluírem” por uma sequência de operações.

```
install.packages("magrittr")
library(magrittr)
```

O operador %>% pega no **valor à sua esquerda** e passa-o como **primeiro argumento** da função à sua direita. O exemplo anterior reescreve-se assim:

```
library(magrittr)
x %>% sum() %>% sqrt()
## [1] 3.16
```

Lê-se de forma quase literal: “pega em **x**, soma, e depois tira a raiz quadrada”. A vantagem cresce com o número de passos. Para calcular a raiz quadrada da soma dos logaritmos dos valores absolutos de um vetor:

```
# Sem pipe: lê-se de dentro para fora
resultado <- sqrt(sum(log(abs(x))))

# Com pipe: lê-se na ordem das operações
resultado <- x %>%
  abs() %>%
  log() %>%
  sum() %>%
  sqrt()
```

9.1.1 O ponto (.) como marcador de posição

Por vezes queremos passar o valor da esquerda para um argumento que **não é o primeiro**. Nesses casos, usamos um ponto (.) para marcar a posição exata onde o valor deve entrar.

Por exemplo, na função `lm()`, que ajusta um modelo linear, os dados vão no argumento `data =` (o segundo). Usamos o `.` para lá os colocar:

```
airquality %>%
  na.omit() %>% # remove valores em falta
  lm(Ozone ~ Wind + Temp + Solar.R, data = .) %>% # ajusta o modelo linear
  summary() # resume o modelo
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -64.3421    23.0547  -2.79   0.0062 **
## Wind         -3.3336     0.6544  -5.09  1.5e-06 ***
## Temp          1.6521     0.2535   6.52  2.4e-09 ***
## Solar.R       0.0598     0.0232   2.58  0.0112 *
## ---
```

```
## Residual standard error: 21.2 on 107 degrees of freedom
## Multiple R-squared:  0.606, Adjusted R-squared:  0.595
```

O conjunto `airquality` é limpo de valores em falta com `na.omit()`, o resultado entra como `data` em `lm()`, e o modelo é finalmente resumido — toda a cadeia se lê de uma só vez, de cima para baixo.

9.2 O *pipe* nativo do R (`|>`)

Desde a versão 4.1 (2021), o próprio R passou a incluir um *pipe* nativo, escrito `|>`, que **não exige nenhum pacote**. O seu comportamento essencial é o mesmo: passa o valor da esquerda como primeiro argumento da função à direita.

```
x |> sum() |> sqrt()
## [1] 3.16
```

Como faz parte da base do R e é hoje a forma recomendada em código novo, vale a pena conhecer as duas diferenças principais em relação ao `%>%`:

- O `|>` **exige sempre os parênteses** da função: escreve-se `x |> sqrt()`, e não `x |> sqrt`.
- Para colocar o valor num argumento que não o primeiro, o `|>` usa o marcador `_` (sublinhado), em vez do `.`, e esse argumento tem de ser **nomeado** (a partir da versão 4.2). O exemplo do modelo linear fica:

```
airquality |>
  na.omit() |>
  lm(Ozone ~ Wind + Temp + Solar.R, data = _) |>
  summary()
```

Qual usar? Para código novo e simples, o `|>` é a escolha natural: é nativo, não depende de pacotes e será compreendido por qualquer instalação recente do R. O `%>%` do `magrittr` continua indispensável quando se usam os verbos do `dplyr` (como veremos) ou os seus atalhos (`add()`, `extract()`, `divide_by()`, ...), e oferece mais flexibilidade com o marcador `..`. Neste capítulo, os exercícios que recorrem a essas funções auxiliares usam, por isso, o `%>%`.

No RStudio, o atalho `Ctrl+Shift+M` / `Cmd+Shift+M` insere o operador *pipe*. Em *Tools > Global Options > Code* pode escolher se esse atalho insere `%>%` ou `|>`.

9.3 Exercícios

1. Reescreva a expressão seguinte usando o %>%.

```
round(mean(sum(1:10) / 3), digits = 1)
```

Sugestão: use a função `magrittr::divide_by()`. Consulte a ajuda da função.

2. Reescreva o código seguinte usando o %>%.

```
x <- rnorm(100)
x.pos <- x[x > 0]
media <- mean(x.pos)
saida <- round(media, 2)
```

Sugestão: use a função `magrittr::extract()`. Consulte a ajuda da função.

3. Sem executar, indique qual é o resultado do código seguinte. Consulte a ajuda das funções, se precisar.

```
2 %>%
  add(2) %>%
  c(6, NA) %>%
  mean(na.rm = TRUE) %>%
  equals(5)
```

4. No conjunto `mtcars`, selecione as colunas `mpg`, `hp` e `wt`, filtre os carros com mais de 100 cavalos (`hp > 100`) e ordene-os por `wt` (peso) de forma crescente. Reescreva o código seguinte usando o *pipe*:

```
install.packages("dplyr")
library(dplyr)
result <- arrange(filter(select(mtcars, mpg, hp, wt), hp > 100), wt)
print(result)
```

5. No conjunto `iris`, filtre as linhas com `Sepal.Length > 5`, selecione as colunas `Sepal.Length` e `Species` e calcule a média de `Sepal.Length` por espécie. Reescreva o código seguinte usando o *pipe*:

```
result <- aggregate(Sepal.Length ~ Species,
                    data = select(filter(iris, Sepal.Length > 5),
```

```
          Sepal.Length, Species),  
        mean)  
print(result)
```

6. No conjunto `airquality`, remova as linhas com valores em falta, selecione as colunas `Ozone` e `Wind` e calcule a média de `Ozone` nas observações em que `Wind > 10`. Reescreva o código seguinte usando o *pipe*. *Sugestão*: Use a função `summarise()` para calcular a média.

```
cleaned_data <- na.omit(airquality)  
filtered_data <- filter(cleaned_data, Wind > 10)  
result <- mean(filtered_data$Ozone)  
print(result)
```

7. No conjunto `mtcars`, crie uma coluna `hp_per_wt` igual à razão entre `hp` e `wt`, filtre os carros com essa razão maior que 30 e selecione as colunas `mpg`, `hp_per_wt` e `gear`. Reescreva o código seguinte usando o *pipe*. *Sugestão*: Use a função `mutate()` para criar a nova coluna.

```
mtcars$hp_per_wt <- mtcars$hp / mtcars$wt  
filtered_data <- filter(mtcars, hp_per_wt > 30)  
result <- select(filtered_data, mpg, hp_per_wt, gear)  
print(result)
```

O *pipe* encadeia operações que corremos **uma vez**. Mas muitas tarefas exigem **repetir** o mesmo cálculo várias vezes — e é aí que entram os ciclos, tema dos dois capítulos seguintes.

Capítulo 10

O ciclo while

Até aqui, cada instrução do nosso código corria uma só vez. Mas há tarefas que exigem **repetição**: somar os números até um certo limite, pedir dados ao utilizador até ele decidir parar, refinar um cálculo até atingir a precisão desejada. Para estas situações, o R oferece os **ciclos** (ou *loops*), e o primeiro que estudamos é o **while**.

O **while** repete um bloco de código **enquanto** uma condição lógica se mantiver verdadeira. É a estrutura ideal quando não sabemos de antemão quantas repetições serão precisas — sabemos apenas a *condição* que faz o ciclo parar.

```
while (condição) {  
  # bloco de código a repetir  
}
```

- **condição** — uma expressão lógica, avaliada antes de cada repetição. Enquanto for **TRUE**, o bloco corre; assim que passar a **FALSE**, o ciclo termina.
- **bloco de código** — as instruções que se repetem.

O funcionamento é um ciclo de três passos: avalia a condição; se for verdadeira, executa o bloco; volta a avaliar a condição. Repete-se assim até a condição se tornar falsa, momento em que o programa segue para a instrução seguinte ao **while**.

Neste capítulo aprendemos a repetir código com **while**, a garantir que o ciclo termina, a interrompê-lo com **break** e a reconhecer e evitar os ciclos infinitos.

10.1 Exemplos

Somar números até um limite. O ciclo seguinte soma todos os inteiros de 1 a 10:

```

limite <- 10
soma <- 0
contador <- 1

while (contador <= limite) {
  soma <- soma + contador
  contador <- contador + 1
}

print(paste("A soma dos números de 1 a", limite, "é:", soma))
## [1] "A soma dos números de 1 a 10 é: 55"

```

Em cada repetição, o valor de `contador` é somado a `soma` e depois incrementado. Quando `contador` ultrapassa `limite`, a condição torna-se falsa e o ciclo termina. A linha `contador <- contador + 1` é o que garante o progresso rumo à paragem — e é precisamente aí que reside o erro mais comum, como o próximo exemplo mostra.

Escrever a tabuada de um número. O programa seguinte deveria imprimir a tabuada de um número dado pelo utilizador — mas **nunca termina**. Consegue ver porquê?

```

n <- as.numeric(readline("Introduza um número inteiro: "))

print(paste("Tabuada do", n, ":"))
i <- 1
while (i <= 10) {
  print(paste(n, "x", i, "=", n * i))
  i + 1          # <- o erro está aqui
}

```

A expressão `i + 1` **calcula** o novo valor, mas não o **guarda** em `i`: o resultado é simplesmente descartado. Como `i` nunca muda, a condição `i <= 10` mantém-se eternamente verdadeira e o ciclo corre para sempre. A correção é atribuir o resultado de volta à variável:

```

i <- i + 1

```

Sempre que um `while` parece não terminar, o primeiro suspeito é uma variável de controlo que não está a ser atualizada.

Interromper com `break`. A instrução `break` sai imediatamente do ciclo, mesmo que a condição ainda seja verdadeira:

```
limite <- 10
soma <- 0
contador <- 1

while (contador <= limite) {
  soma <- soma + contador
  print(contador)
  if (contador == 3) {
    break          # sai do ciclo quando contador chega a 3
  }
  contador <- contador + 1
}
## [1] 1
## [1] 2
## [1] 3
```

Há duas instruções irmãs do **break**. A **next** não termina o ciclo — apenas **salta** para a repetição seguinte, ignorando o resto do bloco na iteração atual. E, além do **while**, o R tem o **repeat**, que repete indefinidamente e só para quando encontra um **break**; é útil quando queremos executar o bloco **pelo menos uma vez** antes de testar a condição de paragem.

10.2 Considerações importantes

- **Condição de paragem.** Garanta que a condição acaba por se tornar falsa. Um **while** cuja condição nunca falha é um **ciclo infinito**, que bloqueia o programa (no RStudio, interrompe-se com *Esc* ou o botão de *stop*).
- **Atualizar a variável de controlo.** A variável que a condição testa tem de ser atualizada dentro do ciclo, no sentido correto — foi o que faltou no exemplo da tabuada.
- **Desempenho.** Em R, os ciclos são muitas vezes mais lentos do que as operações **vetorizadas**. Para grandes volumes de dados, prefira funções como **sum()**, **mean()** ou a família **apply** a percorrer os elementos um a um.

10.3 Exercícios

Nos exercícios seguintes, use um ciclo **while**.

1. Peça ao utilizador um inteiro n e imprima todos os inteiros de 0 até n , um por linha.
2. Peça um inteiro, calcule e mostre o seu quadrado; depois pergunte ao utilizador se quer continuar ou terminar. O ciclo deve prosseguir até o utilizador escolher terminar.

3. Peça um inteiro n e calcule a soma dos primeiros n inteiros (de 1 a n).
4. Peça um inteiro n e conte quantos números pares existem no intervalo de 0 a n .
5. Some inteiros positivos introduzidos pelo utilizador até que ele insira um número negativo, momento em que o programa para e apresenta a soma dos positivos.
6. Peça um inteiro positivo e faça uma contagem decrescente até zero, imprimindo cada número.
7. Peça um inteiro negativo e conte de forma crescente até zero, imprimindo cada número.
8. Peça um número ao utilizador: se for positivo, conte de forma decrescente até zero; se for negativo, conte de forma crescente até zero. Imprima cada número.
9. Peça um inteiro não negativo e calcule o seu fatorial. Recorde que $0! = 1$ e que, por exemplo, $4! = 4 \times 3 \times 2 \times 1 = 24$.
10. Leia um inteiro positivo e calcule a soma dos seus algarismos. Por exemplo, para 23, o resultado é $2 + 3 = 5$. *Sugestão:* Os operadores `/%` (divisão inteira) e `%%` (resto) permitem extrair os algarismos um a um.
11. Gere e imprima os termos da sucessão de Fibonacci até atingir um valor máximo indicado pelo utilizador. A sucessão começa em 1,1 e cada termo é a soma dos dois anteriores: 1, 1, 2, 3, 5, 8, 13, ...

O `while` repete enquanto uma condição se verifica. Quando, pelo contrário, sabemos **exatamente** sobre que valores queremos iterar — os elementos de um vetor, as linhas de uma matriz —, há uma estrutura mais natural: o ciclo `for`, tema do próximo capítulo.

Capítulo 11

O ciclo for

O ciclo `while` do capítulo anterior repete-se enquanto uma condição se mantém — ideal quando não sabemos quantas repetições serão precisas. Muitas vezes, porém, sabemos exatamente sobre que valores queremos trabalhar: os elementos de um vetor, as linhas de uma matriz, as colunas de uma *data frame*. Para esses casos existe o ciclo `for`, que percorre, um a um, os elementos de uma sequência.

```
for (variável in sequência) {  
  # bloco de código a executar em cada iteração  
}
```

- **variável** — a variável de controlo, que em cada iteração assume o valor do elemento seguinte da sequência;
- **sequência** — a estrutura a percorrer (um vetor, uma lista, um intervalo como `1:10`, ...);
- **bloco de código** — as instruções executadas em cada passagem.

O funcionamento é direto: a variável de controlo toma sucessivamente cada valor da sequência, o bloco corre uma vez para cada um, e o ciclo termina quando todos os elementos tiverem sido processados. Ao contrário do `while`, o número de iterações fica determinado à partida.

Neste capítulo aprendemos a percorrer sequências com `for`, a iterar por índices, a trabalhar sobre matrizes e *data frames*, e a fazê-lo de forma correta e eficiente (percorrendo com `seq_along()` e pré-allocando os resultados).

11.1 Exemplos

Imprimir uma sequência. O caso mais simples percorre um intervalo:

```
for (i in 0:10) {
  print(i)
}
## [1] 0
## [1] 1
## [1] 2
## ... (até 10)
```

Somar os elementos de um vetor. A variável de controlo assume cada valor do vetor:

```
numeros <- c(1, 2, 3, 4, 5)
soma <- 0

for (num in numeros) {
  soma <- soma + num
}

print(paste("A soma dos números é:", soma))
## [1] "A soma dos números é: 15"
```

Iterar por índices. Quando precisamos de **modificar** os elementos nas suas posições, iteramos sobre os índices em vez dos valores. Aqui, `seq_along(numeros)` gera as posições 1, 2, ..., `length(numeros)`:

```
numeros <- c(10, 20, 30, 40, 50)

for (i in seq_along(numeros)) {
  numeros[i] <- numeros[i] * 2
}

print(numeros)
## [1] 20 40 60 80 100
```

É tentador escrever `for (i in 1:length(numeros))`, e é assim que aparece em muito código por aí — mas esconde uma armadilha. Se o vetor estiver **vazio**, `length(numeros)` é 0 e `1:0` não produz uma sequência vazia, como seria de esperar, mas sim o vetor `c(1, 0)` — pelo que o ciclo corre duas vezes, com índices inválidos, gerando erros difíceis de diagnosticar. A função `seq_along(numeros)` (ou `seq_len(n)`) evita o problema, devolvendo corretamente uma sequência vazia quando não há elementos. Prefira sempre `seq_along()` a `1:length()`.

Iterar sobre uma matriz. O `for` percorre matrizes por linha ou por coluna. Aqui calculamos a soma de cada linha, guardando o resultado num vetor **pré-alocado**:

```
matriz <- matrix(1:9, nrow = 3, ncol = 3)
soma_linhas <- numeric(nrow(matriz))      # vetor de zeros, do tamanho certo

for (i in 1:nrow(matriz)) {
  soma_linhas[i] <- sum(matriz[i, ])
}

print(soma_linhas)
## [1] 12 15 18
```

Médias das colunas de uma *data frame*.

```
dados <- data.frame(
  A = c(1, 2, 3),
  B = c(4, 5, 6),
  C = c(7, 8, 9)
)

medias <- numeric(ncol(dados))

for (col in 1:ncol(dados)) {
  medias[col] <- mean(dados[, col])
}

print(medias)
## [1] 2 5 8
```

Repare que, nos dois últimos exemplos, criámos o vetor de resultados **antes** do ciclo, já com o tamanho final (`numeric(nrow(matriz))`). Este hábito — a **pré-alocação** — é importante: fazer o vetor “crescer” dentro do ciclo (por exemplo, com `c()` a cada iteração) obriga o R a copiá-lo repetidamente e torna o código muito mais lento em problemas grandes.

11.2 Exercícios

Nos exercícios seguintes, use um ciclo `for`.

1. Imprima todos os números de 1 a 10.
2. Imprima o quadrado de cada número de 1 a 5.
3. Some todos os elementos do vetor `numeros <- c(2, 4, 6, 8, 10)`.

4. Calcule a média das notas do vetor `notas <- c(85, 90, 78, 92, 88)`.
5. Imprima todos os números ímpares do vetor `valores <- c(1, 3, 4, 6, 9, 11, 14)`.
6. Leia um inteiro e imprima a sua tabuada de 1 a 10.
7. Leia um inteiro n e construa o vetor `c(1, 2, ..., n-1, n)`.
8. Leia um inteiro e determine se é um **número perfeito** — aquele cuja soma dos divisores próprios é igual ao próprio número (por exemplo, 6 tem divisores próprios 1, 2 e 3, e $1+2+3 = 6$).
9. Encontre e imprima todos os números perfeitos entre 1 e 1000.
10. Leia dois inteiros n e m e imprima os números entre 0 e n (inclusive), com um intervalo de m entre eles.
11. Calcule a média de um conjunto de notas dadas pelo utilizador. O número de notas é indicado antes da sua introdução. Eis um exemplo da interação:

```
Introduza o número de notas: 4
Introduza a nota: 12
Introduza a nota: 13
Introduza a nota: 15.5
Introduza a nota: 16
A média das notas é 14.125
```

12. Crie uma matriz 4×4 com os números de 1 a 16. Usando o ciclo `for`, calcule:

- (a) a média da terceira coluna;
- (b) a média da terceira linha;
- (c) a média de cada coluna;
- (d) a média de cada linha;
- (e) a média dos números pares de cada coluna;
- (f) a média dos números ímpares de cada linha.

Sugestão: Use estruturas de decisão (`if...else...`, Capítulo 5) sempre que precise de condicionar a lógica dentro do ciclo.

Os ciclos `for` e `while` dão-nos controlo total sobre a repetição. No entanto, em R, muitas dessas repetições podem ser escritas de forma mais curta e rápida com a família de funções `apply` — o tema que exploramos a seguir.

Capítulo 12

A família apply

Os ciclos `for` e `while` (Capítulos 10 e 11) dão-nos controlo total sobre a repetição, mas em R há muitas vezes um caminho mais curto — e mais rápido. A família de funções `apply` foi desenhada precisamente para **aplicar uma função a cada elemento de um objeto** sem escrever um ciclo explícito. O resultado é código mais conciso, mais legível e, tipicamente, mais eficiente.

A ideia é sempre a mesma: em vez de descrevermos *como* percorrer os elementos (inicializar um contador, incrementá-lo, indexar), dizemos apenas *o que* queremos fazer a cada um. Esta forma de pensar — separar a operação da mecânica da iteração — chama-se **programação funcional**, e é uma das marcas do estilo idiomático em R.

As quatro funções mais usadas resumem-se assim:

Função	Sintaxe	O que faz	Entrada	Saída
<code>apply</code>	<code>apply(X, MARGIN, FUN)</code>	Aplica FUN às linhas ou colunas	Matriz ou <i>data frame</i>	vetor, lista ou <i>array</i>
<code>lapply</code>	<code>lapply(X, FUN)</code>	Aplica FUN a cada elemento	Lista, vetor ou <i>data frame</i>	lista
<code>sapply</code>	<code>sapply(X, FUN)</code>	Como <code>lapply</code> , mas simplifica	Lista, vetor ou <i>data frame</i>	vetor ou matriz
<code>tapply</code>	<code>tapply(X, INDEX, FUN)</code>	Aplica FUN a cada grupo	Vetor	<i>array</i>

Neste capítulo aprendemos a substituir ciclos por `apply` (linhas/colunas de uma matriz), `lapply` e `sapply` (elementos de uma lista ou vetor) e `tapply` (grupos definidos por um fator), escrevendo código mais curto e mais expressivo.

12.1 A função `apply()`

A `apply()` aplica uma função às **margens** de uma matriz ou *data frame*: às linhas, às colunas, ou a ambas.

```
apply(X, MARGIN, FUN)
```

- `X` — a matriz ou *data frame*;
- `MARGIN` — 1 para as linhas, 2 para as colunas, `c(1, 2)` para cada célula;
- `FUN` — a função a aplicar.

Para calcular a soma, a média e a raiz quadrada de cada coluna de uma matriz:

```
matriz <- matrix(1:9, nrow = 3)
```

```
apply(matriz, 2, sum)
```

```
## [1]  6 15 24
```

```
apply(matriz, 2, mean)
```

```
## [1]  2  5  8
```

```
apply(matriz, 2, sqrt)
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 1.00 2.00 2.65
```

```
## [2,] 1.41 2.24 2.83
```

```
## [3,] 1.73 2.45 3.00
```

O segundo argumento é a chave: `apply(matriz, 1, sum)` somaria cada **linha**; `apply(matriz, 2, sum)`, cada **coluna**.

12.2 A função `lapply()`

A `lapply()` aplica uma função a cada elemento de uma lista ou vetor e devolve **sempre uma lista** — daí o `l` do nome.

```
lapply(X, FUN, ...)
```

Por exemplo, para pôr uma série de nomes em minúsculas:

```
nomes <- c("ANA", "JOAO", "PAULO", "FILIPA")
nomes_minusc <- lapply(nomes, tolower)
str(nomes_minusc)
## List of 4
## $ : chr "ana"
## $ : chr "joao"
## $ : chr "paulo"
## $ : chr "filipa"
```

A função aplica-se também a cada elemento de uma lista — aqui, a raiz quadrada de dois vetores:

```
dados <- list(a = 1:4, b = 5:8)
lapply(dados, sqrt)
## $a
## [1] 1.00 1.41 1.73 2.00
##
## $b
## [1] 2.24 2.45 2.65 2.83
```

12.3 A função `sapply()`

A `sapply()` faz o mesmo que a `lapply()`, mas **simplifica** o resultado sempre que possível — para um vetor ou uma matriz — em vez de devolver uma lista. O **s** vem de *simplify*.

```
dados <- 1:5
f <- function(x) x^2

lapply(dados, f)      # devolve uma lista com cinco elementos
## [[1]]
## [1] 1
## ... (até [[5]] com 25)

sapply(dados, f)      # devolve um vetor
## [1] 1 4 9 16 25
```

A simplificação da `sapply()` é cômoda, mas tem um senão: quando os resultados não têm todos o mesmo comprimento, a função devolve silenciosamente uma lista, e o nosso código a jusante pode partir-se sem aviso claro. Em programas onde a robustez importa, prefira a `vapply()`,

que obriga a declarar de antemão o tipo e o tamanho do resultado de cada elemento e falha imediatamente se algo não corresponder — tornando o erro visível e fácil de localizar.

12.4 A função `tapply()`

A `tapply()` aplica uma função a **subconjuntos** de um vetor, definidos por um fator. É a ferramenta natural para calcular estatísticas por grupo.

```
tapply(X, INDEX, FUN, ...)
```

- `X` — o vetor de dados;
- `INDEX` — o fator (ou lista de fatores) que define os grupos;
- `FUN` — a função a aplicar a cada grupo.

Usemos o conjunto `iris`, um clássico introduzido por Ronald A. Fisher em 1936. Contém medições morfológicas (comprimento e largura de sépalas e pétalas) de três espécies de íris (*setosa*, *versicolor* e *virginica*). Vejamos as primeiras linhas:

```
head(iris)
```

	<i>Sepal.Length</i>	<i>Sepal.Width</i>	<i>Petal.Length</i>	<i>Petal.Width</i>	<i>Species</i>
## 1	5.1	3.5	1.4	0.2	<i>setosa</i>
## 2	4.9	3.0	1.4	0.2	<i>setosa</i>
## 3	4.7	3.2	1.3	0.2	<i>setosa</i>
## 4	4.6	3.1	1.5	0.2	<i>setosa</i>
## 5	5.0	3.6	1.4	0.2	<i>setosa</i>
## 6	5.4	3.9	1.7	0.4	<i>setosa</i>

Para obter o comprimento médio da pétala **em cada espécie**, uma única linha basta:

```
tapply(iris$Petal.Length, iris$Species, mean)
```

	<i>setosa</i>	<i>versicolor</i>	<i>virginica</i>
##	1.46	4.26	5.55

O resultado revela de imediato o que distingue as espécies: as pétalas da *virginica* são, em média, quase quatro vezes mais compridas do que as da *setosa*.

A família tem outros membros úteis. A `vapply()` é uma `sapply()` mais segura (com tipo de saída declarado); a `mapply()` aplica uma função a **vários** vetores em paralelo, elemento a elemento; e a `Map()` faz o mesmo devolvendo uma lista. Dominadas as quatro funções deste capítulo, estas variantes seguem-se com naturalidade.

12.5 Exercícios

1. Crie um vetor `nomes` com “Alice”, “Bruno”, “Carla” e “Daniel”. Use `lapply()` para os converter em maiúsculas e verifique o resultado.

```
nomes <- c("Alice", "Bruno", "Carla", "Daniel")  
# A sua solução aqui
```

2. Crie um vetor `palavras` com “cão”, “gato”, “elefante” e “leão”. Use `sapply()` para calcular o comprimento (número de caracteres) de cada palavra. *Sugestão:* veja `?nchar`.

```
palavras <- c("cão", "gato", "elefante", "leão")  
# A sua solução aqui
```

3. Crie uma matriz 3×3 chamada `matriz_exemplo` com os números de 1 a 9. Use `apply()` para calcular a soma de cada linha e, depois, a média de cada coluna.

```
matriz_exemplo <- matrix(1:9, nrow = 3)  
# A sua solução aqui
```

4. No conjunto `mtcars`, use `tapply()` para calcular o consumo médio (`mpg`) para cada número de cilindros (`cyl`).

```
data(mtcars)  
# A sua solução aqui
```

5. Crie uma lista `minha_lista` com três elementos: um vetor numérico de 1 a 5, um vetor de caracteres `c("A", "B", "C")` e uma matriz 2×2 com os valores de 1 a 4. Use `lapply()` para calcular o comprimento de cada elemento e, depois, `sapply()` para simplificar o resultado num vetor.

```
minha_lista <- list(1:5, c("A", "B", "C"), matrix(1:4, nrow = 2))  
# A sua solução aqui
```

6. Crie um vetor `numeros` de 1 a 10. Use `sapply()` com uma função personalizada que devolva “par” se o número for par e “ímpar” caso contrário.

```
numeros <- 1:10  
# A sua solução aqui
```

7. No conjunto `iris`, use `tapply()` para calcular a média do comprimento das pétalas (`Petal.Length`) por espécie e a variância da largura das sépalas (`Sepal.Width`) por espécie.

```
data(iris)
# A sua solução aqui
```

8. No conjunto `mtcars`, use `apply()` para normalizar (escala de 0 a 1) todas as colunas numéricas e converta o resultado numa *data frame*. Depois, use `lapply()` para aplicar `summary()` a cada coluna normalizada. O valor normalizado é $\frac{x - \min(x)}{\max(x) - \min(x)}$.

```
data(mtcars)
# A sua solução aqui
```

9. Um professor quer analisar o desempenho de 30 alunos em quatro exames.

- Crie uma *data frame* `notas_alunos` com 30 linhas (alunos) e 4 colunas (exames), preenchida com valores aleatórios entre 0 e 100 (use `runif()`).
- Use `apply()` para calcular a média de cada aluno nos quatro exames.
- Use `sapply()` para calcular a nota mínima e máxima de cada exame.

10. Um gestor de *marketing* analisa o número de clientes de quatro lojas ao longo de um mês.

- Crie uma matriz `frequencia_lojas` com 4 linhas (lojas) e 30 colunas (dias), preenchida com valores aleatórios entre 0 e 200 (use `sample()`).
- Use `apply()` para calcular a média diária de clientes em cada loja.
- Use `apply()` para calcular o total de clientes de cada loja no mês.
- Converta a matriz numa *data frame* e use `sapply()` para calcular o desvio padrão do número de clientes em cada dia (cada coluna).

Com a família `apply` fechamos as ferramentas essenciais de programação. O próximo capítulo é uma pausa para consolidar tudo o que vimos até aqui, através de exercícios que combinam estruturas de dados, decisões, ciclos e funções.

Capítulo 13

Exercícios de revisão

Chegámos ao fim da primeira parte do livro. Até aqui construímos, peça a peça, as ferramentas fundamentais da programação em R: os tipos e as estruturas de dados (Capítulo 4), as estruturas de decisão (Capítulo 5), as funções (Capítulo 6), os ciclos (Capítulos 10 e 11) e a família `apply` (Capítulo 12). Isoladamente, cada uma destas ferramentas é simples; a sua força vem de as combinarmos para resolver problemas reais.

Este capítulo é uma pausa para consolidar. Os problemas que se seguem são deliberadamente integradores — cada um exige juntar várias ferramentas — e inspiram-se em contextos concretos: satisfação de clientes, produtividade de equipas, simulação de amostras, temperaturas urbanas e até o efeito da radiação na memória de um computador. Resolvê-los sem consultar as soluções é a melhor forma de medir o que ficou realmente aprendido.

Consolidar, através de problemas aplicados, a combinação de estruturas de dados, funções, estruturas de decisão, ciclos e a família `apply`.

1. Um investigador analisa uma sondagem que avalia a satisfação de clientes com três serviços (A, B e C). Cada linha representa um cliente e cada coluna a pontuação (de 1 a 10) atribuída a cada serviço:

```
satisfacao <- data.frame(  
  servico_A = c(8, 5, 9, 6, 7, 4, 10, 6, 3, 7),  
  servico_B = c(7, 6, 8, 5, 8, 5, 9, 7, 4, 6),  
  servico_C = c(9, 4, 10, 6, 6, 3, 10, 5, 5, 8)  
)
```

- (a) Escreva uma função `calcula_media()` que receba a *data frame* `satisfacao` e devolva um vetor com a média de satisfação de cada cliente (média das 3 colunas).
- (b) Acrescente à *data frame* uma coluna `media` com os valores calculados.

- (c) Com base na coluna `media`, use um ciclo `for` com `if...else` para classificar cada cliente como "Satisfeito" (média ≥ 7) ou "Insatisfeito" (caso contrário). Acrescente essa informação numa nova coluna `classificacao`.

2. Uma empresa registou o número de tarefas concluídas por 15 colaboradores ao longo de 5 dias úteis. A meta é realizar pelo menos 20 tarefas por dia; pretende-se saber quem atingiu a meta em pelo menos 3 dos 5 dias. Cada linha é um colaborador, cada coluna um dia:

```
tarefa_funcionarios <- data.frame(
  dia1 = c(22, 18, 19, 25, 30, 15, 21, 24, 20, 19, 28, 26, 12, 20, 18),
  dia2 = c(18, 21, 23, 20, 27, 18, 20, 25, 16, 21, 22, 29, 17, 19, 20),
  dia3 = c(25, 17, 15, 18, 26, 16, 19, 30, 21, 18, 24, 20, 14, 23, 22),
  dia4 = c(20, 19, 22, 26, 28, 20, 23, 22, 17, 25, 19, 18, 19, 21, 24),
  dia5 = c(19, 22, 18, 27, 29, 14, 24, 28, 20, 23, 25, 27, 16, 17, 18)
)
```

- (a) Escreva uma função `verifica_meta()` que receba a *data frame* e devolva um vetor lógico indicando, para cada colaborador, se atingiu a meta em pelo menos 3 dias.
- (b) Com base nesse vetor lógico, use um `for` com `if...else` para criar um vetor `classificacao` com os valores "Atingiu Meta" (3 ou mais dias) ou "Não Atingiu Meta" (caso contrário).
- (c) Acrescente o vetor `classificacao` como nova coluna da *data frame*.

3. Um estatístico estuda o comportamento da média amostral. Simulou 6 amostras de tamanho 15 (valores entre 0 e 100) e quer calcular a média de cada uma, interrompendo o processo assim que encontrar uma média superior a 75. Os dados estão na matriz:

```
amostras <- matrix(
  c(
    52, 68, 75, 48, 60, 55, 72, 81, 64, 70, 45, 69, 74, 60, 71,
    49, 50, 63, 41, 59, 55, 67, 62, 58, 66, 61, 54, 65, 60, 57,
    80, 82, 85, 79, 90, 88, 92, 85, 87, 84, 86, 81, 89, 83, 91,
    30, 42, 51, 39, 35, 48, 46, 52, 41, 37, 45, 43, 38, 44, 36,
    69, 74, 70, 66, 68, 72, 71, 73, 65, 67, 75, 64, 63, 62, 76,
    55, 58, 61, 56, 60, 63, 57, 62, 64, 59, 65, 66, 68, 67, 69
  ),
  nrow = 6,
  byrow = TRUE
)
```


- (a) Crie uma função `calcula_media_amostra()` que receba a matriz `amostras` e um índice, e devolva a média dessa linha.
- (b) Escreva um ciclo `while` que percorra as linhas, calculando a média de cada amostra. Se alguma média exceder 75, o programa imprime "Média Excedeu Limite" e para; caso contrário, continua até processar todas as amostras.

4. Crie uma lista `listas_transformadas` de comprimento 7, em que o elemento de índice i contém a sequência

$$x_j = j^2 + i, \quad j = 1, 2, \dots, i.$$

Por exemplo, para $i = 3$ o vetor é $[1^2 + 3, 2^2 + 3, 3^2 + 3] = [4, 7, 12]$.

- (a) Construa a lista `listas_transformadas` com um ciclo `for`.
 - (b) Verifique o comprimento do último vetor da lista.
 - (c) Escreva uma função `soma_elementos()` que receba uma lista e devolva um vetor com a soma de cada vetor nela contido.
 - (d) Use `soma_elementos()` para somar os elementos de cada vetor de `listas_transformadas`.
5. Crie dois vetores x e y , ambos de comprimento 60, com inteiros aleatórios escolhidos com reposição entre 0 e 999.

- (a) Gere x e y com `sample()` e `replace = TRUE`.
- (b) Selecione os valores de y menores que 500 cujos correspondentes em x (mesma posição) sejam múltiplos de 3.
- (c) Crie o vetor `dif_absoluta` com o valor absoluto da diferença $|x_i - y_i|$ em cada posição.
- (d) Calcule a média e o desvio padrão de `dif_absoluta`.

6. Seja x um vetor de 80 observações de uma distribuição normal de média 5 e desvio padrão 2.

- (a) Gere x com `rnorm()`.
- (b) Calcule a média amostral $\bar{x}$.
- (c) Crie o vetor `resultado` cujo i -ésimo elemento é $z_i = \sqrt{|x_i^3 - \bar{x}^2|}$.
- (d) Calcule o máximo, o mínimo e a média de `resultado`.

7. Um climatologista estuda as temperaturas médias diárias em 12 cidades ao longo de uma semana. Quer a temperatura média semanal de cada cidade e verificar se está acima ou abaixo de 25 °C. Cada linha é uma cidade, cada coluna um dia:

```
temperaturas_cidades <- data.frame(
  segunda = c(28, 22, 25, 19, 30, 24, 21, 26, 27, 23, 31, 20),
  terca   = c(29, 24, 23, 20, 32, 25, 22, 27, 26, 24, 33, 21),
  quarta  = c(30, 25, 22, 18, 33, 26, 23, 28, 25, 22, 34, 22),
  quinta  = c(31, 23, 24, 17, 31, 27, 20, 29, 28, 21, 32, 23),
  sexta   = c(27, 21, 26, 16, 29, 28, 19, 30, 29, 25, 30, 24),
  sabado  = c(26, 22, 27, 15, 28, 26, 18, 28, 27, 26, 29, 25),
  domingo = c(25, 23, 28, 14, 27, 25, 17, 27, 26, 27, 28, 26)
)
```

- Escreva uma função `calcula_media_temperaturas()` que receba a *data frame* e devolva um vetor `media_semanal` com a temperatura média semanal de cada cidade.
- Use um ciclo `while` para percorrer `media_semanal` e criar um vetor `classificacao` que classifique cada cidade como "Acima de 25°C" (média > 25) ou "Abaixo de 25°C" (caso contrário).
- Acrescente `media_semanal` e `classificacao` como novas colunas da *data frame*.

8. Os computadores podem sofrer alterações aleatórias em *bits* da memória (*bit flips*) devido à ação de partículas de alta energia, como os raios gama — um fenómeno relevante em contextos críticos como a aviação, a medicina ou a criptografia. Um engenheiro simula o impacto da radiação em blocos de memória RAM, procurando os blocos mais afetados:

```
set.seed(2025)

# Estado original de 1000 bits (0 ou 1)
original <- sample(c(0, 1), size = 1000, replace = TRUE)

# Estado final após exposição à radiação:
# cada bit tem 3% de probabilidade de ser alterado (invertido)
flipado <- ifelse(runif(1000) < 0.03, 1 - original, original)

# Divisão dos bits em blocos de memória (100 bits por bloco)
blocos <- rep(1:10, each = 100)
```

- Crie um vetor `erros` com o número de *bits* alterados, comparando `original` com `flipado`.

- (b) Crie um fator `bloco_memoria` com os valores de 1 a 10, indicando o bloco de cada *bit* (como em `blocos`).
- (c) Calcule o número de *bit flips* em cada bloco de memória. *Sugestão:* Use a função `tapply()`.

Consolidadas as bases, a segunda parte do livro vira-se para a **visualização**: no próximo capítulo aprendemos a representar dados graficamente com o R base.

Capítulo 14

Gráficos com o R base

Começa aqui a segunda parte do livro, dedicada à **visualização**. Um gráfico bem feito faz num instante o que uma tabela de números raramente consegue: revela um padrão, uma tendência, uma assimetria, um valor atípico. A visualização não é um enfeite que se acrescenta no fim de uma análise — é uma ferramenta de raciocínio, e muitas vezes o primeiro passo para compreender os dados.

O R inclui, de origem, um sistema de gráficos poderoso e flexível, a que se costuma chamar *R base*. Neste capítulo percorremos os tipos de gráfico mais comuns — barras, circular, histograma, caixa de bigodes, dispersão e linhas — associando cada um ao tipo de variável e à pergunta que responde. Mais adiante (Capítulo 16) conheceremos o `ggplot2`, uma alternativa moderna; mas dominar o R base é indispensável, pela sua ubiquidade e pela rapidez com que produz um gráfico exploratório.

Neste capítulo aprendemos a construir e personalizar os gráficos essenciais do R base e, sobretudo, a escolher o gráfico certo para cada tipo de dados: barras e gráficos circulares para variáveis categóricas; histogramas e caixas de bigodes para variáveis quantitativas; dispersão e linhas para relações entre variáveis.

14.1 Gráfico de barras

Um **gráfico de barras** representa uma variável categórica: cada barra corresponde a uma categoria e a sua altura indica a frequência (absoluta ou relativa) dessa categoria. A largura das barras é constante e não tem significado estatístico.

```
barplot(altura,  
        names.arg = NULL,    # rótulos do eixo x  
        main = NULL,         # título  
        xlab = NULL, ylab = NULL,
```

```
col = NULL,          # cor(es) das barras
beside = FALSE)      # barras lado a lado (TRUE) ou empilhadas (FALSE)
```

O argumento `altura` pode ser um vetor de frequências (gráfico simples) ou uma matriz (barras agrupadas ou empilhadas). O caminho habitual é obter as frequências com `table()` e passá-las a `barplot()`:

```
# Cores preferidas de nove pessoas
cores <- c("Azul", "Vermelho", "Verde", "Azul", "Verde",
           "Vermelho", "Azul", "Verde", "Azul")

frequencia_absoluta <- table(cores)

barplot(frequencia_absoluta,
        main = "Cores preferidas",
        xlab = "Cor",
        ylab = "Frequência absoluta",
        col = c("steelblue", "seagreen", "firebrick"),
        border = "black")
```

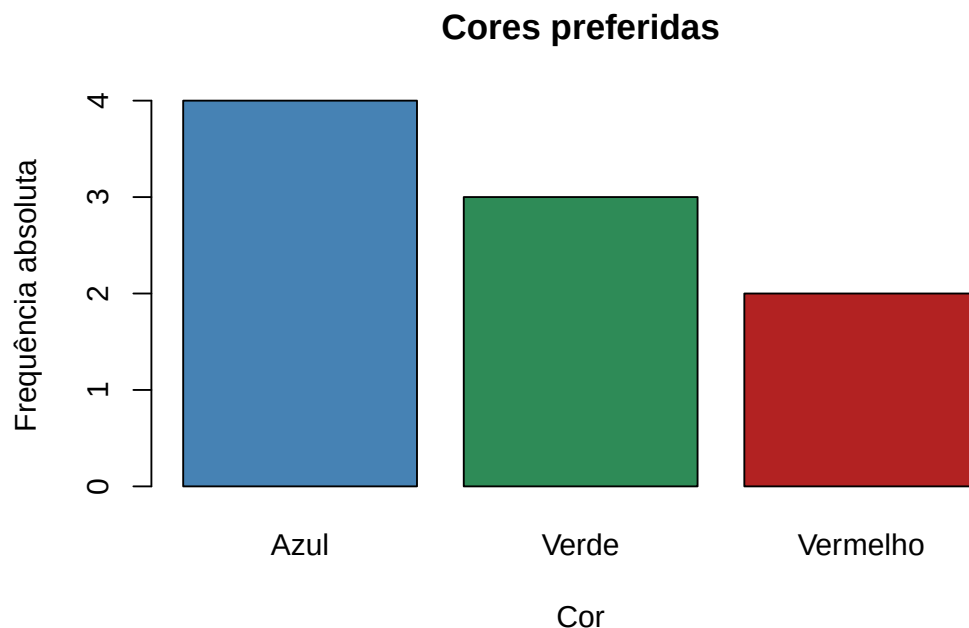


Figura 14.1: Gráfico de barras com frequências absolutas.

Para representar as **frequências relativas**, basta dividir pela dimensão da amostra:

```
frequencia_relativa <- frequencia_absoluta / length(cores)

barplot(frequencia_relativa,
        main = "Cores preferidas",
        xlab = "Cor",
        ylab = "Frequência relativa",
        col = c("steelblue", "seagreen", "firebrick"),
        border = "black")
```

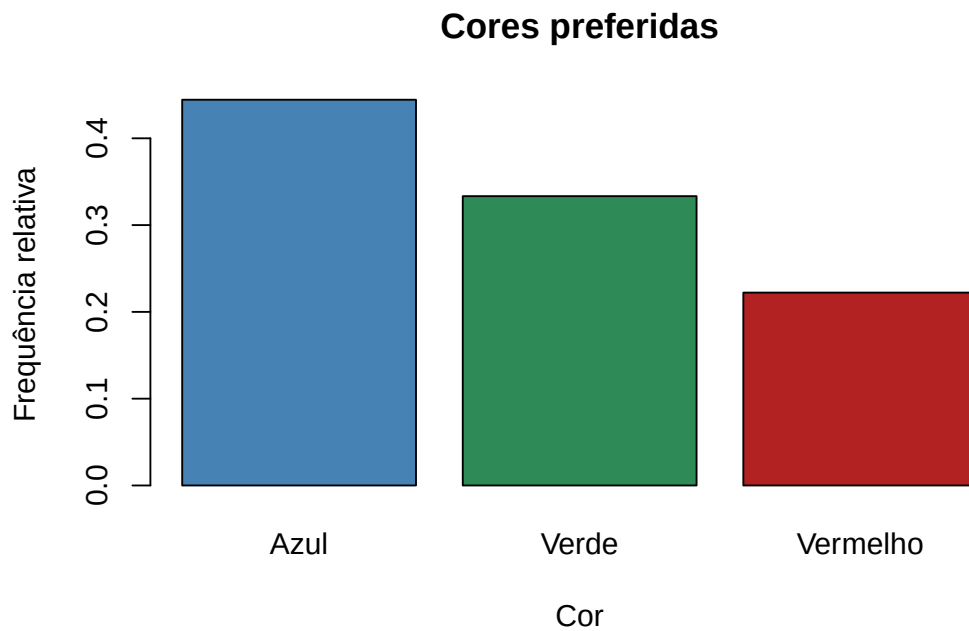


Figura 14.2: O mesmo gráfico com frequências relativas.

Repare que os dois gráficos têm exatamente a mesma forma — muda apenas a escala do eixo vertical. As frequências relativas são úteis quando se querem comparar amostras de dimensões diferentes, para as quais as frequências absolutas seriam enganadoras.

O argumento `horiz = TRUE` desenha as barras na horizontal, o que ajuda quando os rótulos das categorias são longos:

```
barplot(frequencia_absoluta,
        main = "Cores preferidas",
        xlab = "Frequência absoluta",
        col = c("steelblue", "seagreen", "firebrick"),
        horiz = TRUE)
```

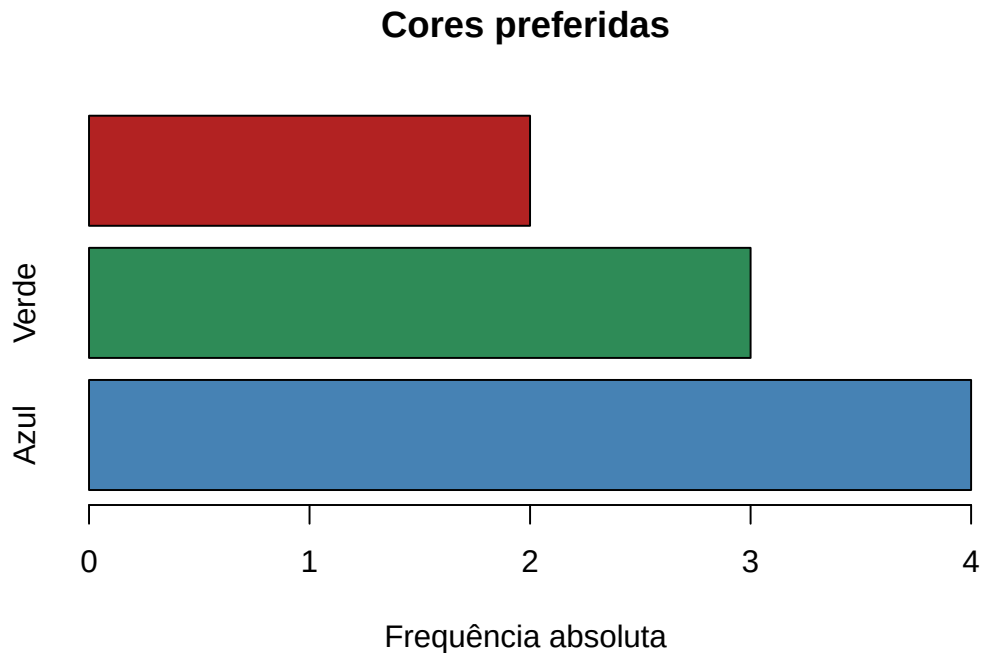


Figura 14.3: Gráfico de barras horizontal.

14.1.1 Barras agrupadas e empilhadas

Quando queremos comparar **duas variáveis** ao mesmo tempo, organizamos os dados numa matriz. Comparemos, por exemplo, a quantidade de amostras de dois tipos de rocha em três regiões:

```
regioes <- c("Região 1", "Região 2", "Região 3")
granito <- c(15, 10, 25)
basalto <- c(20, 15, 10)

dados_rochas <- rbind(granito, basalto)

barplot(dados_rochas,
        beside = TRUE,
        names.arg = regioes,
        col = c("lightblue", "lightcoral"),
        legend.text = c("Granito", "Basalto"),
        main = "Amostras de rocha por região",
        xlab = "Região",
        ylab = "Número de amostras")
```

Usámos `rbind()` para juntar os dois vetores numa matriz e `beside = TRUE` para pôr as barras

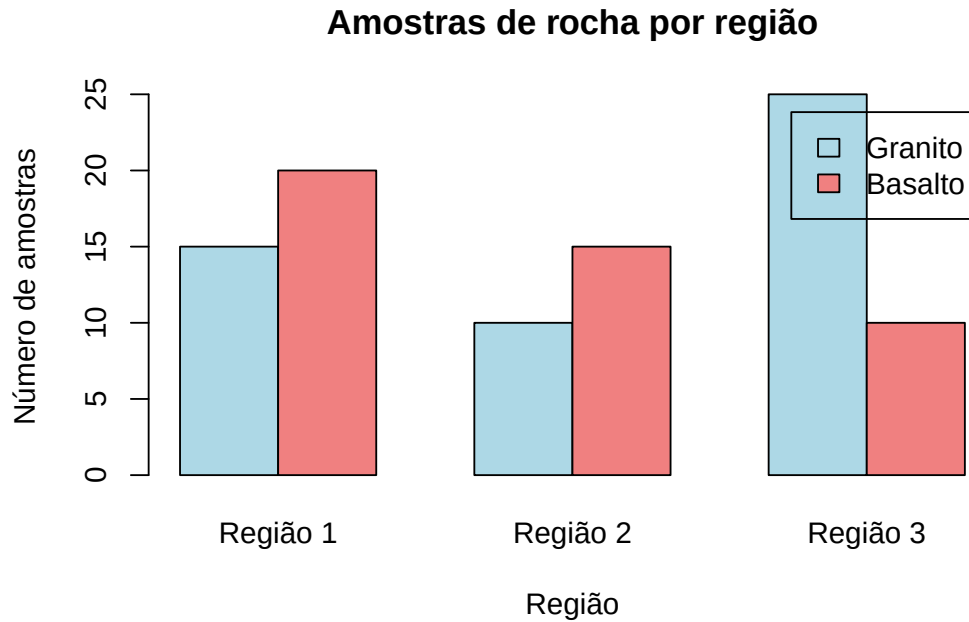


Figura 14.4: Barras agrupadas: cada região mostra os dois tipos de rocha lado a lado.

lado a lado; `legend.text` gera a legenda. Removendo `beside = TRUE`, as barras passam a estar **empilhadas** — cada barra mostra o total da região, repartido pelos dois tipos de rocha:

```
barplot(dados_rochas,
        names.arg = regioes,
        col = c("lightblue", "lightcoral"),
        legend.text = c("Granito", "Basalto"),
        main = "Amostras de rocha por região",
        xlab = "Região",
        ylab = "Número de amostras")
```

14.2 Gráfico circular

O **gráfico circular** (ou de setores) mostra a proporção de cada categoria face ao total: cada fatia é proporcional à percentagem que representa.

```
pie(x, labels = NULL, main = NULL, col = NULL, radius = 1)
```

```
pie(frequencia_relativa,
    main = "Cores preferidas",
    col = c("steelblue", "seagreen", "firebrick"),
```

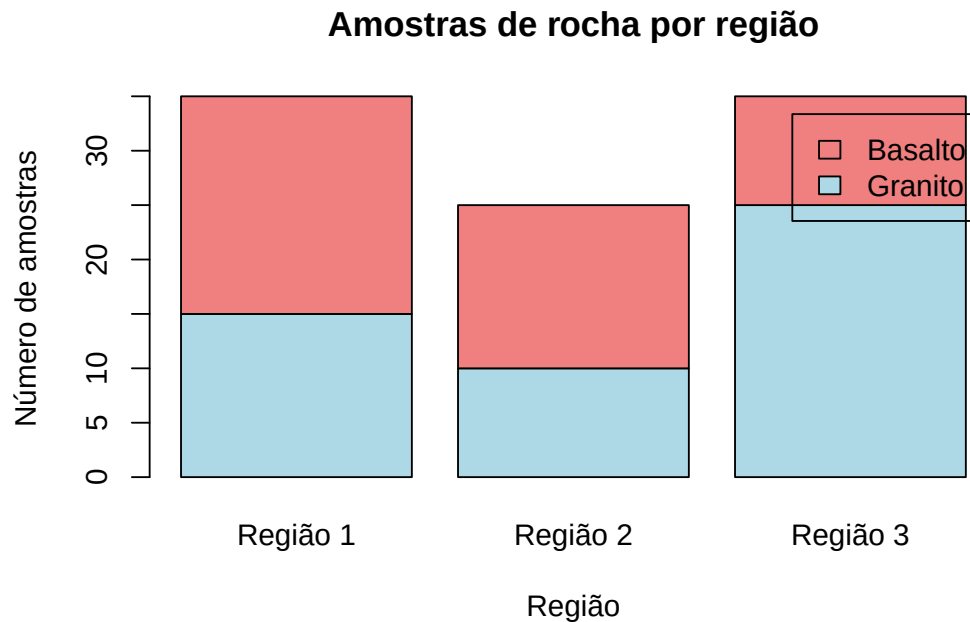



Figura 14.5: As mesmas barras, agora empilhadas.

```
labels = paste(names(frequencia_relativa),
               round(frequencia_relativa * 100, 1), "%"))
```

O gráfico circular é popular, mas deve usar-se com moderação. O olho humano compara comprimentos (barras) com muito mais precisão do que ângulos ou áreas (fatias), pelo que diferenças pequenas entre categorias são difíceis de ler num gráfico circular, e tornam-se ilegíveis quando há muitas fatias. Como regra prática: para comparar frequências, um gráfico de barras é quase sempre a escolha mais clara.

14.3 Histograma

O **histograma** é o gráfico por excelência para uma variável quantitativa contínua. Divide-se o eixo horizontal em classes (intervalos) e desenha-se, sobre cada classe, um retângulo cuja altura traduz a frequência das observações nela contidas. É a forma mais direta de ver a **distribuição** dos dados: onde se concentram, se são simétricos, se têm valores atípicos.

```
hist(x, breaks = "Sturges",
     main = NULL, xlab = NULL, ylab = NULL,
     col = NULL, border = NULL)
```

O argumento `breaks` controla as classes: pode ser um número, ou um método automático ("Sturges", "Scott", "FD"). Por omissão, o R usa a regra de Sturges e classes do tipo $(a, b]$

Cores preferidas

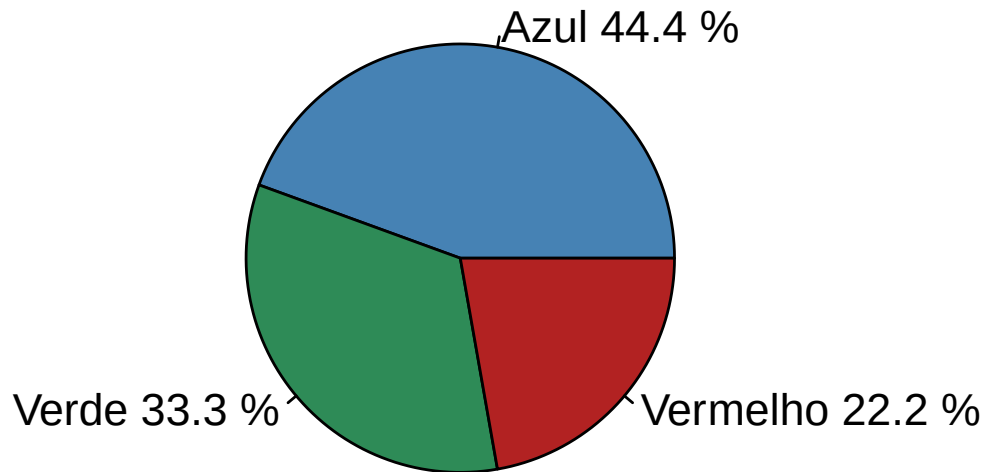


Figura 14.6: Gráfico circular das cores preferidas, com percentagens.

(abertas à esquerda, fechadas à direita).

```
# Massa (kg) de 40 bicicletas
bicicletas <- c(4.3, 6.8, 9.2, 7.2, 8.7, 8.6, 6.6, 5.2, 8.1, 10.9,
               7.4, 4.5, 3.8, 7.6, 6.8, 7.8, 8.4, 7.5, 10.5, 6.0,
               7.7, 8.1, 7.0, 8.2, 8.4, 8.8, 6.7, 8.2, 9.4, 7.7,
               6.3, 7.7, 9.1, 7.9, 7.9, 9.4, 8.2, 6.7, 8.2, 6.5)

h <- hist(bicicletas,
          breaks = "Sturges",
          main = "Massa das bicicletas",
          xlab = "Massa (kg)",
          ylab = "Frequência absoluta",
          col = "lightblue",
          border = "black",
          ylim = c(0, 12),
          labels = TRUE)
```

Guardar o resultado de `hist()` num objeto (aqui, `h`) dá acesso aos valores usados na sua construção — os limites das classes e as contagens:

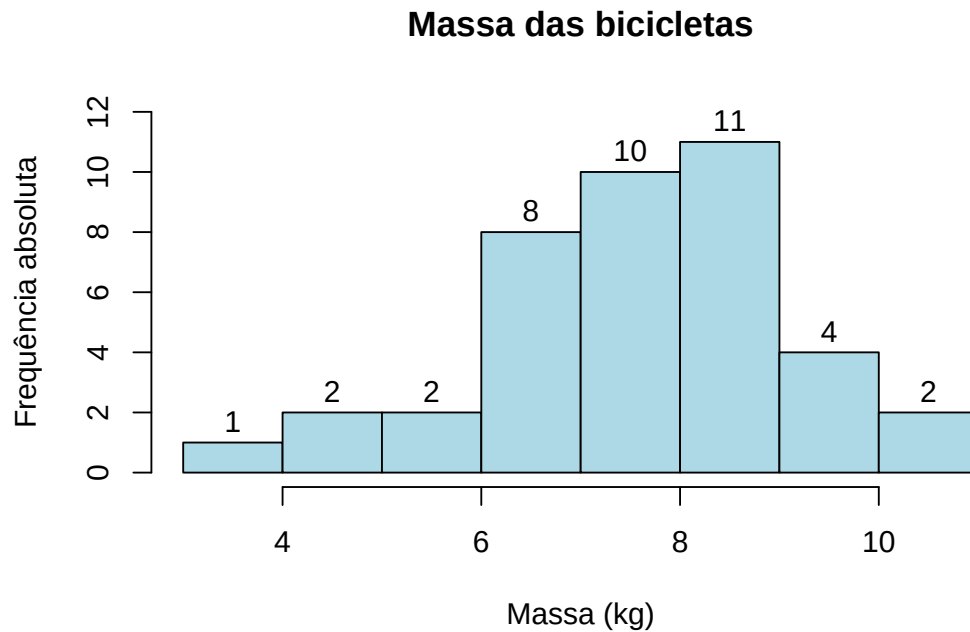


Figura 14.7: Histograma da massa de 40 bicicletas (frequências absolutas).

```
h$breaks      # limites das classes
## [1]  3  4  5  6  7  8  9 10 11

h$counts      # frequência absoluta em cada classe
## [1]  1  2  2  8 10 11  4  2
```

14.3.1 Densidade e curvas suaves

Com o argumento `freq = FALSE`, o histograma passa a representar a **densidade** em vez das contagens:

```
hist(bicicletas,
     main = "Massa das bicicletas",
     xlab = "Massa (kg)",
     ylab = "Densidade",
     freq = FALSE,
     col = "lightblue",
     border = "black")

# Sobrepor uma estimativa suave da densidade
lines(density(bicicletas), col = "red", lwd = 2)
```

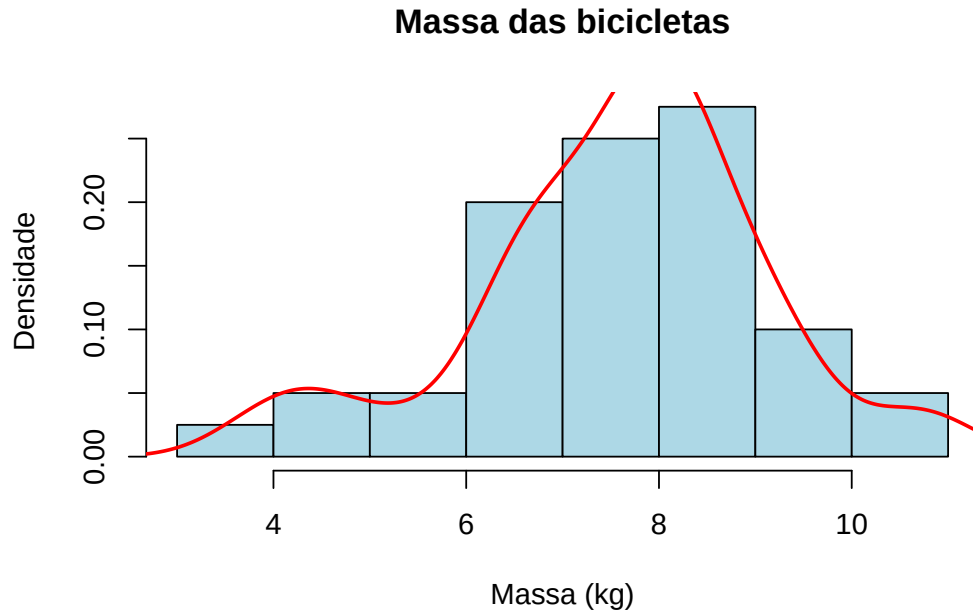


Figura 14.8: Histograma em escala de densidade, com a curva de densidade estimada.

É comum ler-se que `freq = FALSE` produz “frequências relativas”, mas isso não é rigorosamente exato. O que a opção desenha é a **densidade**: as alturas são escaladas de modo a que a *área total* do histograma seja igual a 1. Só quando todas as classes têm a mesma amplitude é que a altura fica proporcional à frequência relativa. A distinção importa porque é a escala de densidade — e não a de frequências — que permite sobrepôr ao histograma uma curva de densidade, como fizemos com `density()`, ou a curva de um modelo teórico.

14.4 Caixa de bigodes (*boxplot*)

A **caixa de bigodes** resume a distribuição de uma variável a partir dos seus quartis. A caixa vai do primeiro quartil (Q_1) ao terceiro (Q_3), com um traço na mediana (Q_2); os “bigodes” estendem-se a partir da caixa e os pontos para lá deles assinalam possíveis valores atípicos.

```
boxplot(x,
  main = NULL, xlab = NULL, ylab = NULL,
  col = NULL, border = NULL,
  horizontal = FALSE)
```

Ao contrário do que por vezes se diz, os bigodes **não** marcam necessariamente o mínimo e o máximo. Por convenção, cada bigode estende-se até à observação mais extrema que ainda esteja dentro de $1.5 \times \text{AIQ}$ da caixa, onde $\text{AIQ} = Q_3 - Q_1$ é a amplitude interquartil. As observações que caem para além dos bigodes são desenhadas individualmente e interpretadas como candidatas a valores atípicos. É esta regra que torna a caixa de bigodes tão útil para detetar *outliers*.

```
boxplot(bicicletas,
        main = "Massa das bicicletas",
        ylab = "Massa (kg)",
        col = "lightblue",
        border = "darkblue")
```

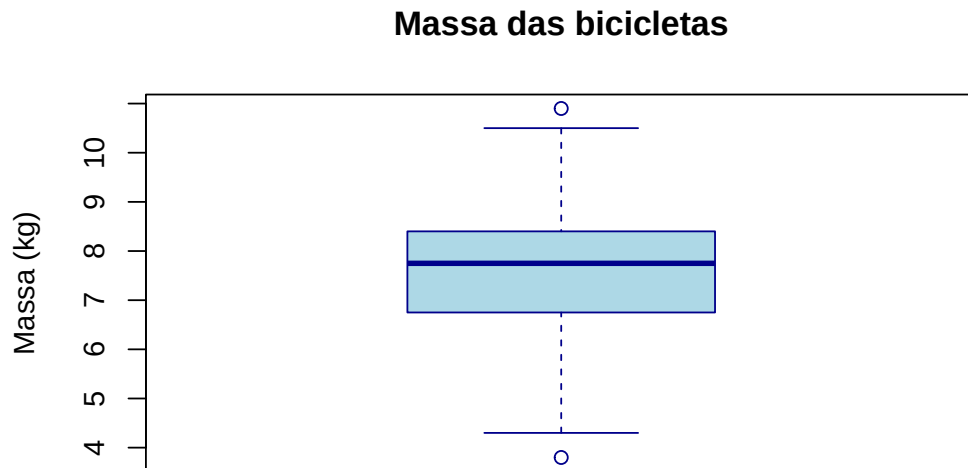


Figura 14.9: Caixa de bigodes da massa das bicicletas.

O argumento `horizontal = TRUE` roda o gráfico, e a função `par(mfrow = ...)` permite dispor vários gráficos no mesmo painel:

```
par(mfrow = c(1, 2)) # painel de 1 linha, 2 colunas

boxplot(bicicletas, main = "Vertical", col = "lightblue")
boxplot(bicicletas, main = "Horizontal", col = "lightgreen", horizontal = TRUE)
```

```
par(mfrow = c(1, 1)) # repor o painel original
```

O verdadeiro valor da caixa de bigodes revela-se ao **comparar grupos**. Usando a notação de fórmula `variável ~ grupo`, comparamos o comprimento da sépala entre as três espécies de íris:

```
boxplot(Sepal.Length ~ Species,
        data = iris,
        main = "Comprimento da sépala por espécie",
        xlab = "Espécie",
        ylab = "Comprimento da sépala (cm)",
        col = c("lightblue", "lightgreen", "lightpink"))
```

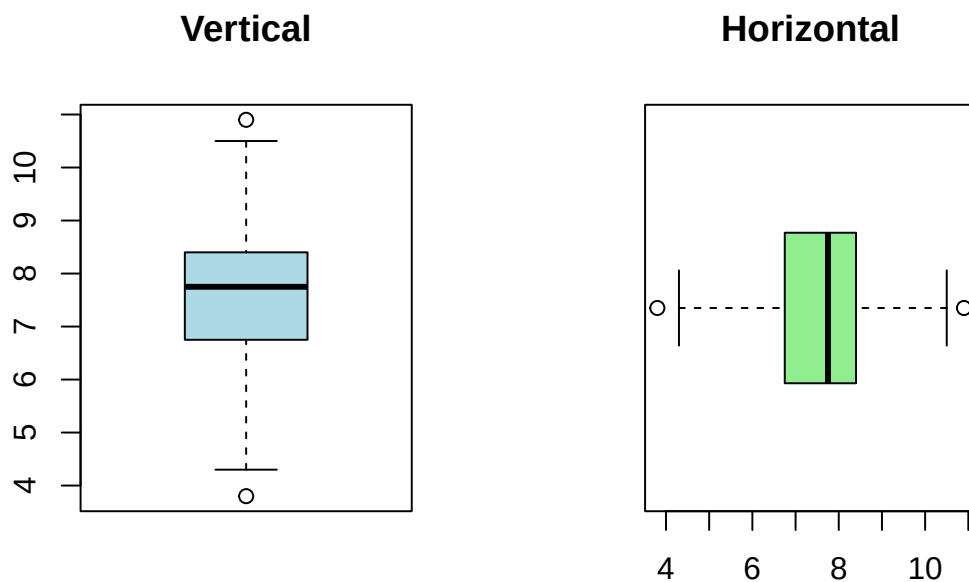


Figura 14.10: Dois gráficos no mesmo painel, com `par(mfrow)`.

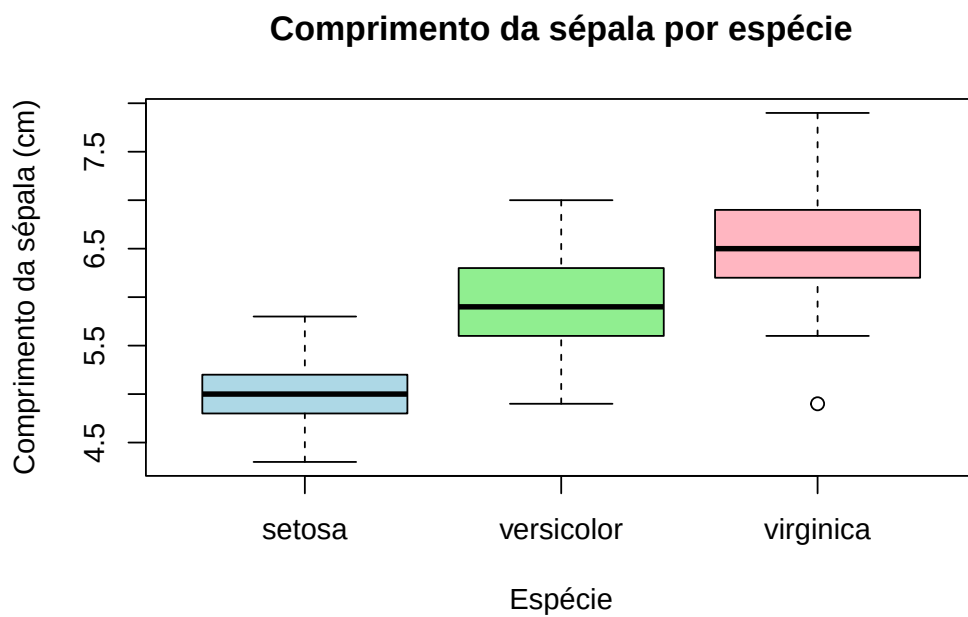


Figura 14.11: Comprimento da sépala por espécie no conjunto iris.

A leitura é imediata: a *virginica* tem sépalas tipicamente mais compridas, a *setosa* as mais curtas, e a dispersão dentro de cada espécie é relativamente semelhante.

14.5 Gráfico de dispersão

O **gráfico de dispersão** representa duas variáveis quantitativas, uma em cada eixo, e é a ferramenta natural para investigar a **relação** entre elas: se crescem juntas, se uma decresce quando a outra cresce, se há agrupamentos ou pontos afastados.

```
plot(x, y,
     main = NULL, xlab = NULL, ylab = NULL,
     col = NULL,    # cor dos pontos
     pch = NULL,    # símbolo (ex.: 19 = ponto cheio)
     cex = 1)      # tamanho dos pontos
```

```
set.seed(1)
x <- rnorm(100)
y <- x + rnorm(100, sd = 0.5)

plot(x, y,
     main = "Dispersão de X e Y",
     xlab = "Variável X",
     ylab = "Variável Y",
     col = "steelblue",
     pch = 19)
```

Podemos acrescentar a **reta de regressão** dos mínimos quadrados com `abline()` aplicada a um modelo linear `lm()`, o que ajuda a ver a tendência global:

```
plot(x, y,
     main = "Dispersão com reta de regressão",
     xlab = "Variável X",
     ylab = "Variável Y",
     col = "steelblue",
     pch = 19)

abline(lm(y ~ x), col = "seagreen", lwd = 2)
```

Quando há **grupos** nos dados, podemos distingui-los por cor, acrescentando pontos com `points()` e identificando-os com uma legenda:

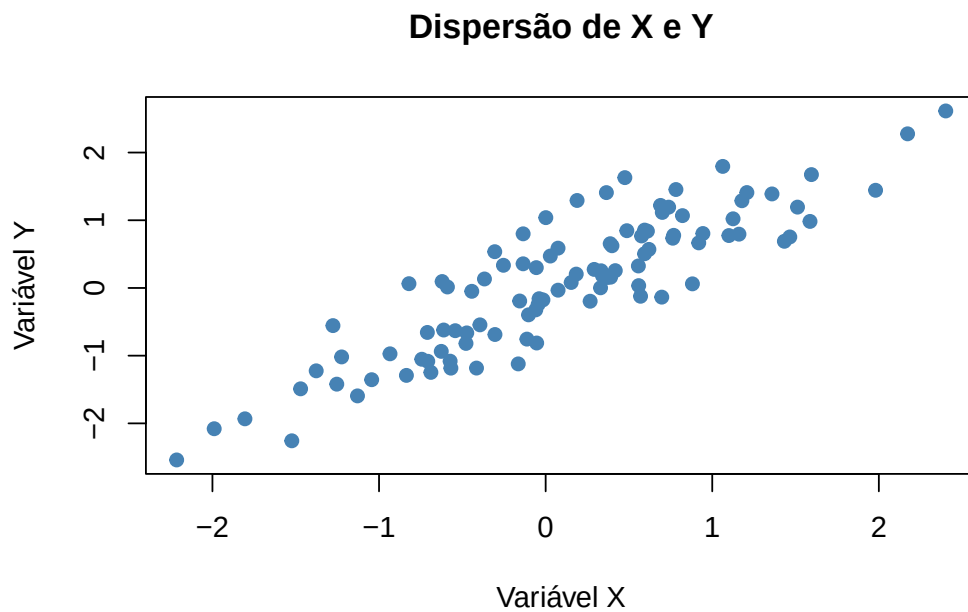


Figura 14.12: Gráfico de dispersão de duas variáveis relacionadas.

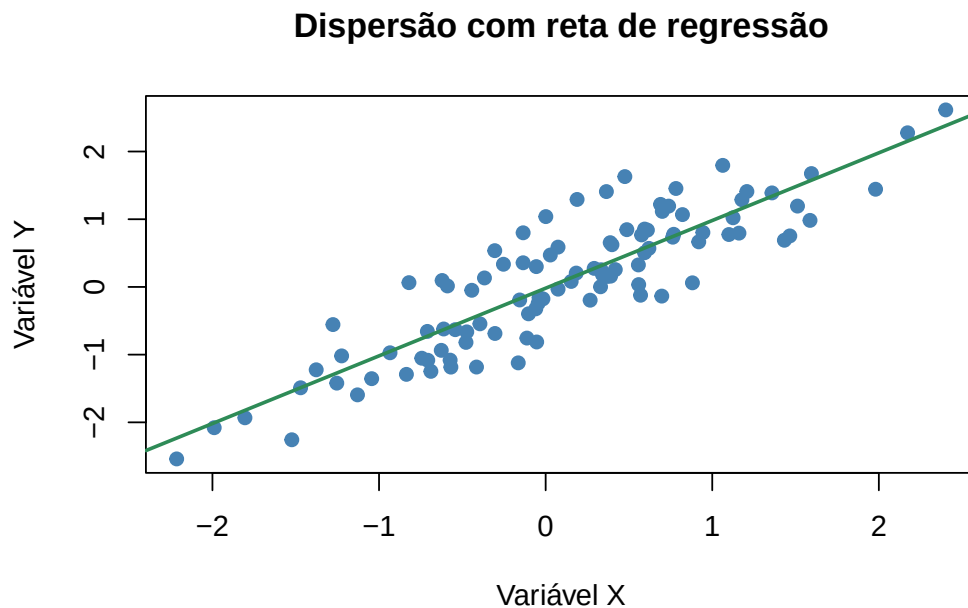


Figura 14.13: O mesmo gráfico com a reta de regressão dos mínimos quadrados.


```

set.seed(123)
x1 <- rnorm(50, mean = 5); y1 <- 3 * x1 + rnorm(50)
x2 <- rnorm(50, mean = 7); y2 <- 3 * x2 + rnorm(50)

plot(x1, y1,
     main = "Dispersão com dois grupos",
     xlab = "Eixo X", ylab = "Eixo Y",
     col = "firebrick", pch = 16,
     xlim = range(c(x1, x2)), ylim = range(c(y1, y2)))
points(x2, y2, col = "steelblue", pch = 16)
legend("topleft",
     legend = c("Grupo 1", "Grupo 2"),
     col = c("firebrick", "steelblue"), pch = 16)

```

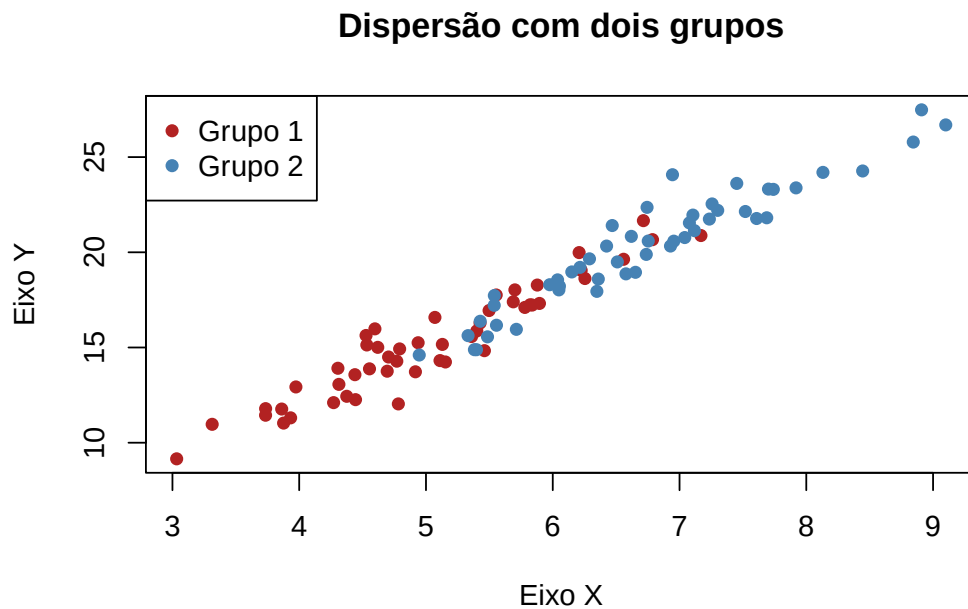


Figura 14.14: Dois grupos distinguidos por cor.

Ao sobrepor um segundo conjunto de pontos com `points()`, os limites dos eixos ficam fixados pelo **primeiro** `plot()`. Se os pontos do segundo grupo caírem fora desses limites, ficam invisíveis. Por isso definimos `xlim` e `ylim` à partida, com `range()`, de modo a abranger todos os dados.

14.6 Gráfico de linhas

O **gráfico de linhas** liga os pontos por segmentos e é indicado para mostrar a evolução de uma variável ao longo de uma sequência ordenada — em especial o tempo (séries temporais).

Obtém-se com `plot()` e o argumento `type`:

```
plot(x, y,  
     type = "l",    # "l" = linha, "p" = pontos, "b"/"o" = ambos  
     lty = 1,       # tipo de traço (1 = contínuo, 2 = tracejado)  
     lwd = 1)       # espessura da linha
```

```
tempo <- 1:10  
valores <- c(3, 5, 7, 9, 11, 10, 8, 6, 4, 2)
```

```
plot(tempo, valores,  
     type = "o",  
     main = "Evolução ao longo do tempo",  
     xlab = "Tempo",  
     ylab = "Valor",  
     col = "steelblue",  
     lwd = 2,  
     pch = 19)
```

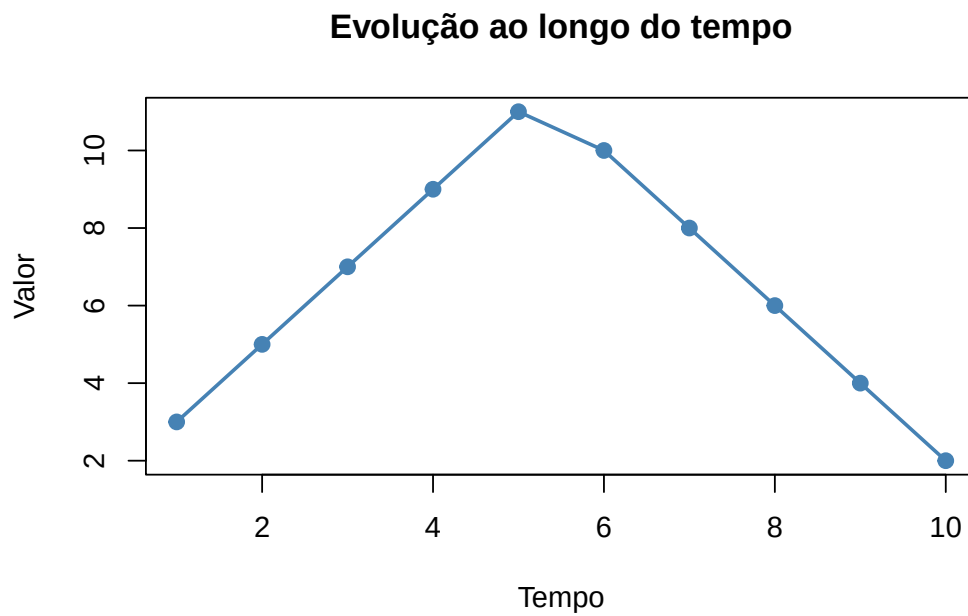


Figura 14.15: Gráfico de linhas com pontos e linha.

14.7 Exercícios

1. No conjunto `iris`, crie um gráfico de barras com a contagem de cada espécie (título “Contagem de espécies”, eixo x “Espécie”, eixo y “Frequência absoluta”, cores azul, verde e amarelo).

Depois, refaça-o com frequências relativas.

2. No conjunto `airquality`, crie um gráfico de barras com a temperatura média (`Temp`) por mês. *Sugestão:* Use a função `tapply()` para obter a média mensal. (Título “Média de temperatura por mês”, eixo x “Mês”, eixo y “Temperatura média (°F)”, cor azul.)
3. Crie um gráfico circular das espécies de flores do conjunto `iris`, identificando cada fatia com o nome da espécie e a respectiva percentagem.
4. No conjunto `Titanic`, crie um gráfico circular com a proporção de sobreviventes e não sobreviventes. *Sugestão:* transforme `Titanic` numa *data frame* com `as.data.frame()`.
5. Crie um histograma da variável `Sepal.Length` (conjunto `iris`). Personalize-o com cor, ajuste o número de classes (`breaks`) e acrescente título e rótulos aos eixos.
6. Um investigador estuda a distribuição de alturas de plantas de duas espécies (A e B), com 100 amostras de cada. Gere os dados com `rnorm()` (médias e desvios padrão diferentes), desenhe um histograma **sobreposto** das duas distribuições (com `add = TRUE`) e acrescente uma legenda. Use cores translúcidas para distinguir as espécies:

```
altura_A <- rnorm(100, mean = 150, sd = 10)
altura_B <- rnorm(100, mean = 160, sd = 15)

# Cores translúcidas: rgb(vermelho, verde, azul, transparência)
# col = rgb(1, 0, 0, 0.5) e col = rgb(0, 0, 1, 0.5)
legend("topright",
      legend = c("Espécie A", "Espécie B"),
      fill = c(rgb(1, 0, 0, 0.5), rgb(0, 0, 1, 0.5)))
```

7. Um professor aplicou um exame a 200 alunos (notas de 0 a 20) e quer analisar a distribuição das notas. Gere as notas com `rnorm(200, mean = 14, sd = 4)` (use `set.seed(1234)`), desenhe o histograma, calcule a média, a mediana e o desvio padrão, e marque a média e a mediana no gráfico com `abline()`. Junte uma legenda:

```
legend("topright",
      legend = c(paste("Média:", round(media_nota, 2)),
                paste("Mediana:", round(mediana_nota, 2)),
                paste("Desvio padrão:", round(desvio_nota, 2))),
      bty = "n")
```

8. No conjunto `airquality`, crie histogramas da variável `Ozono` (`Ozone`) separadamente para cada mês (de maio a setembro). Use `par(mfrow = c(3, 2))` para os dispor no mesmo painel e compare a distribuição dos níveis de ozono ao longo dos meses.

9. No conjunto `iris`, crie uma caixa de bigodes que compare o comprimento da pétala (`Petal.Length`) entre as três espécies. Personalize com cores diferentes por grupo e acrescente título e rótulos.

10. No conjunto `airquality`, crie uma caixa de bigodes dos níveis de ozono (`Ozone`) por mês (`Month`) e acrescente uma linha horizontal na mediana global com `abline()`. Comente como o ozono varia ao longo do tempo.

11. Estude a relação entre a altura e o peso de um grupo de pessoas.

- Crie um vetor `altura` com 30 valores aleatórios entre 150 e 200 cm (use `runif()`).
- Crie um vetor `peso` aproximadamente proporcional à altura: `peso <- altura * 0.4 + runif(30, -5, 5)`.
- Desenhe um gráfico de dispersão (altura no eixo x, peso no eixo y), com título e rótulos.
- Que tendência observa? Comente a correlação entre as duas variáveis.

12. Analise o crescimento de uma população ao longo de 20 anos.

- Crie um vetor `anos` de 2000 a 2019.
- Crie um vetor `populacao` a partir de 100 000 pessoas em 2000, com crescimento de 2% ao ano e uma pequena flutuação aleatória: `populacao <- 100000 * cumprod(1.02 + runif(20, -0.01, 0.01))`.
- Desenhe um gráfico de linhas com a evolução da população ao longo dos anos.

Os gráficos do R base são rápidos e diretos, ideais para a exploração. No próximo capítulo aprofundamos a **manipulação de dados** — o passo que quase sempre precede um bom gráfico — e, mais adiante, conheceremos o `ggplot2`, que leva a visualização a outro nível de expressividade.

Capítulo 15

Manipulação de dados

Ler dados para dentro do R (Capítulo 8) é apenas o começo. Antes de os analisar ou representar graficamente, quase sempre é preciso *prepará-los*: escolher colunas, filtrar linhas, criar variáveis novas, resumir por grupos, juntar tabelas de origens diferentes. A este trabalho — que costuma ocupar a maior parte do tempo de uma análise — chama-se **manipulação de dados**.

O R base já permite fazer tudo isto (vimo-lo no Capítulo 4), mas de forma por vezes verbosa. Neste capítulo conhecemos o **dplyr**, um pacote do **tidyverse** — a coleção de pacotes, criada em torno de Hadley Wickham, que uniformizou a ciência de dados em R. O **dplyr** oferece um pequeno conjunto de “verbos”, cada um com um nome que descreve o que faz, que se combinam com o operador *pipe* (Capítulo 9) para exprimir manipulações complexas de forma legível.

Este capítulo usa vários pacotes do **tidyverse**. Instale-os uma vez com `install.packages("tidyverse")` (ou, individualmente, `install.packages(c("tibble", "dplyr", "stringr"))`) e carregue-os no início da sessão.

15.1 Tibbles

O **tidyverse** trabalha com uma versão modernizada do *data frame* chamada **tibble**. Um *tibble* guarda os mesmos dados de um *data frame* — é, na verdade, um *data frame* — mas com alguns comportamentos mais previsíveis e cómodos. As diferenças principais são:

Característica	<i>Data frame</i>	<i>Tibble</i>
Impressão na consola	Mostra todas as linhas	Mostra um resumo (10 linhas) e o tipo de cada coluna
Texto → fatores	Mantém texto como texto (desde o R 4.0)	Mantém sempre texto como texto

Característica	<i>Data frame</i>	<i>Tibble</i>
Nomes das colunas	Torna-os únicos e sem espaços	Aceita nomes não convencionais (com crases)
Indexação com [	Pode devolver um vetor ou um <i>data frame</i>	Devolve sempre um <i>tibble</i>
Integração	Base do R	Otimizado para o <i>tidyverse</i>

Lê-se com frequência que “os *data frames* convertem texto em fatores por omissão”. Isto foi verdade durante muitos anos, mas **mudou no R 4.0** (2020): desde então, `data.frame()` e `read.table()` usam `stringsAsFactors = FALSE`, ou seja, mantêm o texto como texto, tal como os *tibbles*. É um bom exemplo de como convém confirmar afirmações sobre a linguagem na documentação atual, e não confiar em material antigo.

Criar um *tibble* é como criar um *data frame*, mas a função permite referir colunas acabadas de definir:

```
library(tibble)

tb <- tibble(
  x = 1:5,
  y = c("A", "B", "C", "D", "E"),
  z = x * 2                                # pode usar-se a coluna 'x' recém-criada
)

tb
## # A tibble: 5 × 3
##       x y       z
##   <int> <chr> <dbl>
## 1     1 A         2
## 2     2 B         4
## 3     3 C         6
## 4     4 D         8
## 5     5 E        10
```

Para converter um *data frame* já existente, usa-se `as_tibble()`.

15.2 Os verbos do dplyr

O *dplyr* organiza a manipulação em torno de um punhado de verbos, cada um com uma função clara:

- `select()` — escolhe colunas;
- `arrange()` — ordena linhas;
- `filter()` — filtra linhas por condições;
- `mutate()` — cria ou modifica colunas;
- `group_by()` — agrupa por categorias;
- `summarize()` — resume os dados (em geral, após `group_by()`).

Neste capítulo aprendemos a selecionar, ordenar, filtrar, transformar, agrupar e resumir dados com o `dplyr`, e a combinar tabelas com as funções de junção — sempre com o operador *pipe* a tornar as sequências de operações legíveis.

Vamos explorá-los com o conjunto `starwars`, que acompanha o `dplyr` e descreve as personagens da saga. Guardamo-lo em `sw`:

```
library(dplyr)

sw <- starwars
glimpse(sw)  # visão geral das colunas e dos seus tipos
## Rows: 87
## Columns: 14
## $ name      <chr> "Luke Skywalker", "C-3PO", "R2-D2", "Darth Vader", "Leia Organa", "Owen
## $ height    <int> 172, 167, 96, 202, 150, 178, 165, 97, 183, 182, 188, 180, 228, 180, 173
## $ mass      <dbl> 77.0, 75.0, 32.0, 136.0, 49.0, 120.0, 75.0, 32.0, 84.0, 77.0, 84.0, NA,
## $ hair_color <chr> "blond", NA, NA, "none", "brown", "brown, grey", "brown", NA, "black",
## $ skin_color <chr> "fair", "gold", "white, blue", "white", "light", "light", "light", "whi
## $ eye_color  <chr> "blue", "yellow", "red", "yellow", "brown", "blue", "blue", "red", "bro
## $ birth_year <dbl> 19.0, 112.0, 33.0, 41.9, 19.0, 52.0, 47.0, NA, 24.0, 57.0, 41.9, 64.0,
## $ sex        <chr> "male", "none", "none", "male", "female", "male", "female", "none", "ma
## $ gender     <chr> "masculine", "masculine", "masculine", "masculine", "feminine", "mascul
## $ homeworld  <chr> "Tatooine", "Tatooine", "Naboo", "Tatooine", "Alderaan", "Tatooine", "T
## $ species    <chr> "Human", "Droid", "Droid", "Human", "Human", "Human", "Human", "Droid",
## $ films      <list> <"A New Hope", "The Empire Strikes Back", "Return of the Jedi", "Reven
## $ vehicles   <list> <"Snowspeeder", "Imperial Speeder Bike">, <>, <>, <>, "Imperial Speede
## $ starships  <list> <"X-wing", "Imperial shuttle">, <>, <>, "TIE Advanced x1", <>, <>, <>,
```

15.2.1 Selecionar colunas com `select()`

A `select()` escolhe colunas pelo nome, sem aspas:

```

select(sw, name)                                # uma coluna
## # A tibble: 87 x 1
##   name
##   <chr>
## 1 Luke Skywalker
## 2 C-3PO
## 3 R2-D2
## 4 Darth Vader
## 5 Leia Organa
## 6 Owen Lars
## 7 Beru Whitesun Lars
## 8 R5-D4
## 9 Biggs Darklighter
## 10 Obi-Wan Kenobi
## # i 77 more rows
select(sw, name, mass, hair_color)              # várias colunas
## # A tibble: 87 x 3
##   name                mass hair_color
##   <chr>              <dbl> <chr>
## 1 Luke Skywalker      77 blond
## 2 C-3PO               75 <NA>
## 3 R2-D2              32 <NA>
## 4 Darth Vader       136 none
## 5 Leia Organa        49 brown
## 6 Owen Lars         120 brown, grey
## 7 Beru Whitesun Lars  75 brown
## 8 R5-D4              32 <NA>
## 9 Biggs Darklighter  84 black
## 10 Obi-Wan Kenobi    77 auburn, white
## # i 77 more rows
select(sw, name:hair_color)                    # um intervalo de colunas
## # A tibble: 87 x 4
##   name                height mass hair_color
##   <chr>              <int> <dbl> <chr>
## 1 Luke Skywalker     172    77 blond
## 2 C-3PO              167    75 <NA>
## 3 R2-D2              96    32 <NA>
## 4 Darth Vader       202   136 none
## 5 Leia Organa       150    49 brown
## 6 Owen Lars         178   120 brown, grey

```



```
## 7 Beru Whitesun Lars      165      75 brown
## 8 R5-D4                   97       32 <NA>
## 9 Biggs Darklighter      183      84 black
## 10 Obi-Wan Kenobi         182      77 auburn, white
## # i 77 more rows
```

Há funções auxiliares muito úteis: `starts_with()`, `ends_with()` e `contains()` seleccionam colunas pelo nome. Um sinal `-` remove colunas:

```
select(sw, ends_with("color"))      # colunas terminadas em "color"
## # A tibble: 87 x 3
##   hair_color    skin_color eye_color
##   <chr>         <chr>      <chr>
## 1 blond        fair        blue
## 2 <NA>         gold        yellow
## 3 <NA>         white, blue red
## 4 none        white        yellow
## 5 brown        light        brown
## 6 brown, grey  light        blue
## 7 brown        light        blue
## 8 <NA>         white, red  red
## 9 black        light        brown
## 10 auburn, white fair        blue-gray
## # i 77 more rows
select(sw, -ends_with("color"))     # todas menos essas
## # A tibble: 87 x 11
##   name          height  mass birth_year sex    gender homeworld species films vehi
##   <chr>          <int> <dbl>    <dbl> <chr> <chr>   <chr>   <chr> <lis> <lis>
## 1 Luke Skywalker    172    77      19   male  masculi~ Tatooine Human  <chr> <chr>
## 2 C-3PO             167    75     112  none  masculi~ Tatooine Droid  <chr> <chr>
## 3 R2-D2              96    32      33  none  masculi~ Naboo   Droid  <chr> <chr>
## 4 Darth Vader       202   136     41.9 male  masculi~ Tatooine Human  <chr> <chr>
## 5 Leia Organa       150    49      19  female feminine Alderaan Human  <chr> <chr>
## 6 Owen Lars         178   120      52   male  masculi~ Tatooine Human  <chr> <chr>
## 7 Beru Whitesun Lars 165    75      47  female feminine Tatooine Human  <chr> <chr>
## 8 R5-D4              97    32      NA   none  masculi~ Tatooine Droid  <chr> <chr>
## 9 Biggs Darklighter 183    84      24   male  masculi~ Tatooine Human  <chr> <chr>
## 10 Obi-Wan Kenobi    182    77      57   male  masculi~ Stewjon Human  <chr> <chr>
## # i 77 more rows
```

15.2.1.1 Exercícios

Use a base `sw` nos exercícios seguintes.

1. Aplique `glimpse()` à base `sw`. O que faz esta função?
2. Crie uma tabela `sw_simples` apenas com as colunas `name`, `gender` e `films`.
3. Selecione as colunas `hair_color`, `skin_color` e `eye_color` usando a função auxiliar `contains()`.
4. Escreva todas as formas diferentes que conseguir de devolver a base `sw` sem as colunas `hair_color`, `skin_color` e `eye_color`.

15.2.2 Ordenar com `arrange()`

A `arrange()` ordena as linhas por uma ou mais colunas; `desc()` inverte a ordem:

```
arrange(sw, mass)                                # massa crescente
## # A tibble: 87 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
## 1 Ratts T~      79    15 none      grey, blue unknown      NA male  mascu~ Aleen Mi~
## 2 Yoda          66    17 white      green      brown      896 male  mascu~ <NA>
## 3 Wicket ~      88    20 brown      brown      brown      8 male  mascu~ Endor
## 4 R2-D2          96    32 <NA>      white, bl~ red      33 none  mascu~ Naboo
## 5 R5-D4          97    32 <NA>      white, red red      NA none  mascu~ Tatooine
## 6 Sebulba      112    40 none      grey, red  orange      NA male  mascu~ Malastare
## 7 Padmé A~     185    45 brown      light      brown      46 fema~ femin~ Naboo
## 8 Dud Bolt      94    45 none      blue, grey yellow      NA male  mascu~ Vulpter
## 9 Wat Tam~     193    48 none      green, gr~ unknown      NA male  mascu~ Skako
## 10 Sly Moo~     178    48 none      pale      white      NA <NA>  <NA>  Umbara
## # i 77 more rows
## # i 2 more variables: vehicles <list>, starships <list>
arrange(sw, desc(mass))                          # massa decrescente
## # A tibble: 87 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
## 1 Jabba D~     175  1358 <NA>      green-tan~ orange      600 herm~ mascu~ Nal Hutta
## 2 Grievous     216   159 none      brown, wh~ green, y~      NA male  mascu~ Kalee
## 3 IG-88        200   140 none      metal      red      15 none  mascu~ <NA>
## 4 Darth V~     202   136 none      white      yellow     41.9 male  mascu~ Tatooine
## 5 Tarfful      234   136 brown      brown      blue      NA male  mascu~ Kashyyyk
```

```
## 6 Owen La~      178    120 brown, gr~ light      blue      52    male  mascu~ Tatooine
## 7 Bossk         190    113 none      green      red        53    male  mascu~ Trandosha
## 8 Chewbac~     228    112 brown      unknown    blue      200    male  mascu~ Kashyyyk
## 9 Jek Ton~     180    110 brown      fair       blue      NA     <NA> <NA>   Bestine
## 10 Dexter ~    198    102 none      brown      yellow     NA     male  mascu~ Ojom
## # i 77 more rows
## # i 2 more variables: vehicles <list>, starships <list>
arrange(sw, desc(height), desc(mass))      # por altura e, em empate, por massa
## # A tibble: 87 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
## 1 Yarael ~    264    NA none      white      yellow     NA     male  mascu~ Quermia
## 2 Tarfful    234    136 brown      brown      blue      NA     male  mascu~ Kashyyyk
## 3 Lama Su    229    88 none      grey       black      NA     male  mascu~ Kamino
## 4 Chewbac~   228    112 brown      unknown    blue      200    male  mascu~ Kashyyyk
## 5 Roos Ta~   224    82 none      grey       orange     NA     male  mascu~ Naboo
## 6 Grievous   216    159 none      brown, wh~ green, y~  NA     male  mascu~ Kalee
## 7 Taun We    213    NA none      grey       black      NA     fema~ femin~ Kamino
## 8 Tion Me~   206    80 none      grey       black      NA     male  mascu~ Utapau
## 9 Rugor N~   206    NA none      green      orange     NA     male  mascu~ Naboo
## 10 Darth V~  202    136 none      white      yellow     41.9   male  mascu~ Tatooine
## # i 77 more rows
## # i 2 more variables: vehicles <list>, starships <list>
```

15.2.2.1 Exercícios

1. Ordene `sw` por `mass` (crescente) e, em empate, por `birth_year` (decrecente). Guarde em `sw_ordenados`.
2. Selecione as colunas `name` e `birth_year` e ordene por `birth_year` decrescente. Faça-o de três formas: com funções aninhadas, com um objeto intermédio e com o *pipe*.

15.2.3 Filtrar linhas com `filter()`

A `filter()` seleciona linhas que satisfazem uma ou mais condições lógicas:

```
sw %>% filter(height > 170)                # uma condição
## # A tibble: 55 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
```

```
## 1 Luke Sk~ 172 77 blond fair blue 19 male mascu~ Tatooine
## 2 Darth V~ 202 136 none white yellow 41.9 male mascu~ Tatooine
## 3 Owen La~ 178 120 brown, gr~ light blue 52 male mascu~ Tatooine
## 4 Biggs D~ 183 84 black light brown 24 male mascu~ Tatooine
## 5 Obi-Wan~ 182 77 auburn, w~ fair blue-gray 57 male mascu~ Stewjon
## 6 Anakin ~ 188 84 blond fair blue 41.9 male mascu~ Tatooine
## 7 Wilhuff~ 180 NA auburn, g~ fair blue 64 male mascu~ Eriadu
## 8 Chewbac~ 228 112 brown unknown blue 200 male mascu~ Kashyyyk
## 9 Han Solo 180 80 brown fair brown 29 male mascu~ Coreellia
## 10 Greedo 173 74 <NA> green black 44 male mascu~ Rodia
## # i 45 more rows
## # i 2 more variables: vehicles <list>, starships <list>
sw %>% filter(height > 170) %>% select(name, height)
## # A tibble: 55 x 2
##   name          height
##   <chr>         <int>
## 1 Luke Skywalker 172
## 2 Darth Vader    202
## 3 Owen Lars      178
## 4 Biggs Darklighter 183
## 5 Obi-Wan Kenobi 182
## 6 Anakin Skywalker 188
## 7 Wilhuff Tarkin 180
## 8 Chewbacca      228
## 9 Han Solo       180
## 10 Greedo        173
## # i 45 more rows
sw %>% filter(height > 170, mass > 80) # duas condições (E lógico)
## # A tibble: 21 x 14
##   name      height mass hair_color skin_color eye_color birth_year sex gender homeworld
##   <chr>    <int> <dbl> <chr>    <chr>    <chr>    <dbl> <chr> <chr> <chr>
## 1 Darth V~ 202 136 none white yellow 41.9 male mascu~ Tatooine
## 2 Owen La~ 178 120 brown, gr~ light blue 52 male mascu~ Tatooine
## 3 Biggs D~ 183 84 black light brown 24 male mascu~ Tatooine
## 4 Anakin ~ 188 84 blond fair blue 41.9 male mascu~ Tatooine
## 5 Chewbac~ 228 112 brown unknown blue 200 male mascu~ Kashyyyk
## 6 Jabba D~ 175 1358 <NA> green-tan~ orange 600 herm~ mascu~ Nal Hutta
## 7 Jek Ton~ 180 110 brown fair blue NA <NA> <NA> Bestine ~
## 8 IG-88    200 140 none metal red 15 none mascu~ <NA>
## 9 Bossk    190 113 none green red 53 male mascu~ Trandosha
```

```
## 10 Ackbar      180      83 none      brown mot~ orange      41   male  mascu~ Mon Cala
## # i 11 more rows
## # i 2 more variables: vehicles <list>, starships <list>
```

Para variáveis categóricas, comparamos com `==` ou testamos a pertença a um conjunto com `%in%`:

```
sw %>% filter(hair_color == "black" | hair_color == "brown")
## # A tibble: 31 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr> <chr>
## 1 Leia Or~    150   49  brown      light      brown        19  fema~ femin~ Alderaan
## 2 Beru Wh~    165   75  brown      light      blue         47  fema~ femin~ Tatooine
## 3 Biggs D~    183   84  black      light      brown        24  male  mascu~ Tatooine
## 4 Chewbac~    228  112  brown      unknown    blue        200  male  mascu~ Kashyyyk
## 5 Han Solo    180   80  brown      fair       brown        29  male  mascu~ Corellia
## 6 Wedge A~    170   77  brown      fair       hazel        21  male  mascu~ Corellia
## 7 Jek Ton~    180  110  brown      fair       blue         NA   <NA> <NA> Bestine ~
## 8 Boba Fe~    183  78.2 black      fair       brown       31.5 male  mascu~ Kamino
## 9 Lando C~    177   79  black      dark       brown        31  male  mascu~ Socorro
## 10 Arvel C~    NA    NA  brown      fair       brown        NA   male  mascu~ <NA>
## # i 21 more rows
## # i 2 more variables: vehicles <list>, starships <list>
sw %>% filter(hair_color %in% c("black", "brown")) # forma equivalente, mais curta
## # A tibble: 31 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr> <chr>
## 1 Leia Or~    150   49  brown      light      brown        19  fema~ femin~ Alderaan
## 2 Beru Wh~    165   75  brown      light      blue         47  fema~ femin~ Tatooine
## 3 Biggs D~    183   84  black      light      brown        24  male  mascu~ Tatooine
## 4 Chewbac~    228  112  brown      unknown    blue        200  male  mascu~ Kashyyyk
## 5 Han Solo    180   80  brown      fair       brown        29  male  mascu~ Corellia
## 6 Wedge A~    170   77  brown      fair       hazel        21  male  mascu~ Corellia
## 7 Jek Ton~    180  110  brown      fair       blue         NA   <NA> <NA> Bestine ~
## 8 Boba Fe~    183  78.2 black      fair       brown       31.5 male  mascu~ Kamino
## 9 Lando C~    177   79  black      dark       brown        31  male  mascu~ Socorro
## 10 Arvel C~    NA    NA  brown      fair       brown        NA   male  mascu~ <NA>
## # i 21 more rows
## # i 2 more variables: vehicles <list>, starships <list>
```

Quando queremos procurar um padrão *dentro* do texto — e não uma correspondência exata —

recorremos à `str_detect()`, do pacote `stringr`, que indica, para cada elemento, se contém o padrão:

```
library(stringr)

sw %>% filter(str_detect(hair_color, "grey")) # cabelo que contenha "grey"
## # A tibble: 3 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
## 1 Owen Lars    178   120 brown, gr~ light      blue        52 male  mascu~ Tatooine
## 2 Wilhuff ~    180    NA auburn, g~ fair      blue        64 male  mascu~ Eriadu
## 3 Palpatine    170    75 grey      pale      yellow      82 male  mascu~ Naboo
## # i 2 more variables: vehicles <list>, starships <list>
```

15.2.3.1 Exercícios

Use a base `sw`.

1. Crie um objeto `humanos` apenas com personagens humanas.
2. Crie um objeto `altos_fortes` com personagens de mais de 200 cm e mais de 100 kg.
3. Devolva tabelas (*tibbles*) apenas com:
 - (a) personagens humanas nascidas mais de 100 anos antes da batalha de Yavin (`birth_year > 100`);
 - (b) personagens cuja cor (de pele ou olhos) contenha "light" ou "red";
 - (c) personagens com mais de 100 kg, ordenadas por altura decrescente, mostrando só `name`, `mass` e `height`;
 - (d) personagens da espécie "Human" ou "Droid" com mais de 170 cm;
 - (e) personagens sem informação de altura e de massa (NA em ambas).

15.2.4 Criar e modificar colunas com `mutate()`

A `mutate()` cria colunas novas ou altera colunas existentes. Se o nome já existir, a coluna é substituída; se for novo, a coluna é acrescentada no fim. Por exemplo, para converter a altura de centímetros para metros:

```
sw %>% mutate(height = height / 100) # substitui a coluna existente
## # A tibble: 87 x 14
##   name      height  mass hair_color skin_color eye_color birth_year sex  gender homeworld
##   <chr>      <dbl> <dbl> <chr>      <chr>      <chr>      <dbl> <chr> <chr>  <chr>
```

```
## 1 Luke Sk~ 1.72 77 blond fair blue 19 male mascu~ Tatooine
## 2 C-3PO 1.67 75 <NA> gold yellow 112 none mascu~ Tatooine
## 3 R2-D2 0.96 32 <NA> white, bl~ red 33 none mascu~ Naboo
## 4 Darth V~ 2.02 136 none white yellow 41.9 male mascu~ Tatooine
## 5 Leia Or~ 1.5 49 brown light brown 19 fema~ femin~ Alderaan
## 6 Owen La~ 1.78 120 brown, gr~ light blue 52 male mascu~ Tatooine
## 7 Beru Wh~ 1.65 75 brown light blue 47 fema~ femin~ Tatooine
## 8 R5-D4 0.97 32 <NA> white, red red NA none mascu~ Tatooine
## 9 Biggs D~ 1.83 84 black light brown 24 male mascu~ Tatooine
## 10 Obi-Wan~ 1.82 77 auburn, w~ fair blue-gray 57 male mascu~ Stewjon
## # i 77 more rows
## # i 2 more variables: vehicles <list>, starships <list>
sw %>% mutate(height_meters = height / 100) # cria uma nova coluna
## # A tibble: 87 x 15
## name height mass hair_color skin_color eye_color birth_year sex gender homeworld
## <chr> <int> <dbl> <chr> <chr> <chr> <dbl> <chr> <chr> <chr>
## 1 Luke Sk~ 172 77 blond fair blue 19 male mascu~ Tatooine
## 2 C-3PO 167 75 <NA> gold yellow 112 none mascu~ Tatooine
## 3 R2-D2 96 32 <NA> white, bl~ red 33 none mascu~ Naboo
## 4 Darth V~ 202 136 none white yellow 41.9 male mascu~ Tatooine
## 5 Leia Or~ 150 49 brown light brown 19 fema~ femin~ Alderaan
## 6 Owen La~ 178 120 brown, gr~ light blue 52 male mascu~ Tatooine
## 7 Beru Wh~ 165 75 brown light blue 47 fema~ femin~ Tatooine
## 8 R5-D4 97 32 <NA> white, red red NA none mascu~ Tatooine
## 9 Biggs D~ 183 84 black light brown 24 male mascu~ Tatooine
## 10 Obi-Wan~ 182 77 auburn, w~ fair blue-gray 57 male mascu~ Stewjon
## # i 77 more rows
## # i 3 more variables: vehicles <list>, starships <list>, height_meters <dbl>
```

Podemos combinar várias colunas numa expressão. Calculemos o índice de massa corporal (IMC) de cada personagem, $IMC = massa/altura^2$ (com a altura em metros), devolvendo NA sempre que falte a altura ou a massa:

```
sw %>%
  mutate(IMC = mass / (height / 100)^2) %>%
  select(name, height, mass, IMC)
## # A tibble: 87 x 4
## name height mass IMC
## <chr> <int> <dbl> <dbl>
## 1 Luke Skywalker 172 77 26.0
```



```
## 2 C-3PO          167    75  26.9
## 3 R2-D2           96    32  34.7
## 4 Darth Vader    202   136  33.3
## 5 Leia Organa    150    49  21.8
## 6 Owen Lars      178   120  37.9
## 7 Beru Whitesun Lars 165    75  27.5
## 8 R5-D4           97    32  34.0
## 9 Biggs Darklighter 183    84  25.1
## 10 Obi-Wan Kenobi  182    77  23.2
## # i 77 more rows
```

Não foi preciso tratar os NA explicitamente: qualquer operação aritmética que envolva um NA devolve NA, pelo que as personagens sem altura ou sem massa ficam automaticamente com IMC igual a NA. Esta propagação silenciosa dos valores em falta é uma característica do R que convém ter sempre presente.

15.2.4.1 Exercícios

1. Crie a coluna `dif_altura_peso` (diferença entre altura e massa) e guarde a tabela em `starwars_dif`. Depois, filtre as personagens cuja altura excede a massa e ordene por `dif_altura_peso` crescente.
2. Crie as colunas: (a) `indice_massa_altura = mass / height`; (b) `indice_massa_medio = mean(mass, na.rm = TRUE)`; (c) `indice_relativo = (indice_massa_altura - indice_massa_medio) / indice_massa_medio`; (d) `acima_media = ifelse(indice_massa_altura > indice_massa_medio, "sim", "não")`.
3. Crie uma coluna que classifique cada personagem como "recente" (nascida menos de 100 anos antes da batalha de Yavin) ou "antiga" (100 anos ou mais).

15.2.5 Resumir com `summarize()` e `group_by()`

A `summarize()` reduz uma ou mais colunas a estatísticas — uma média, uma mediana, uma variância. Combinada com `group_by()`, calcula essas estatísticas **para cada grupo**, o que é a essência da análise por categorias:

```
# Uma estatística sobre toda a tabela
sw %>% summarize(media_massa = mean(mass, na.rm = TRUE))
## # A tibble: 1 x 1
##   media_massa
##   <dbl>
```



```
## 1          97.3

# A mesma estatística, agora por espécie
sw %>%
  group_by(species) %>%
  summarize(media_massa = mean(mass, na.rm = TRUE))
## # A tibble: 38 x 2
##   species    media_massa
##   <chr>         <dbl>
## 1 Aleena         15
## 2 Besalisk      102
## 3 Cerean         82
## 4 Chagrian      NaN
## 5 Clawdite       55
## 6 Droid         69.8
## 7 Dug           40
## 8 Ewok           20
## 9 Geonosian      80
## 10 Gungan        74
## # i 28 more rows
```

Podemos calcular várias estatísticas ao mesmo tempo:

```
sw %>% summarize(
  media_massa      = mean(mass, na.rm = TRUE),
  mediana_massa    = median(mass, na.rm = TRUE),
  variancia_massa  = var(mass, na.rm = TRUE)
)
## # A tibble: 1 x 3
##   media_massa mediana_massa variancia_massa
##   <dbl>         <dbl>         <dbl>
## 1    97.3           79           28716.
```

E resumir uma variável dentro de cada categoria de outra — aqui, a altura média por cor de cabelo:

```
sw %>%
  filter(!is.na(hair_color), !is.na(height)) %>%
  group_by(hair_color) %>%
  summarize(media_altura = mean(height))
```

```
## # A tibble: 11 x 2
##   hair_color      media_altura
##   <chr>          <dbl>
## 1 auburn          150
## 2 auburn, grey    180
## 3 auburn, white   182
## 4 black           174.
## 5 blond           177.
## 6 blonde          168
## 7 brown           177.
## 8 brown, grey     178
## 9 grey            170
## 10 none           181.
## 11 white          156
```

Repare no papel de `na.rm = TRUE`. Funções como `mean()` ou `var()` devolvem NA se houver um único valor em falta nos dados; `na.rm = TRUE` manda ignorá-los no cálculo. Ignorar valores em falta nem sempre é inócuo — pode enviesar resultados —, mas é frequentemente o que se pretende ao resumir dados reais, que quase nunca estão completos.

15.2.5.1 Exercícios

Use a base `sw`.

1. Calcule a altura média e mediana das personagens.
2. Calcule a massa média das personagens com mais de 175 cm.
3. Na mesma tabela, apresente a massa média das personagens com menos de 175 cm e a das personagens com 175 cm ou mais.
4. Devolva tabelas (*tibbles*) com: (a) a altura média por espécie; (b) a massa média e mediana por espécie; (c) apenas o nome das personagens que participaram em mais de 2 filmes.

15.3 Juntar tabelas

Em ciência de dados, a informação está muitas vezes repartida por várias tabelas que partilham uma **coluna-chave** — um identificador comum. As funções de junção do `dplyr` combinam-nas, e distinguem-se pela forma como tratam as linhas sem correspondência:

- `left_join()` — mantém todas as linhas da tabela da esquerda;
- `right_join()` — mantém todas as linhas da tabela da direita;

- `full_join()` — mantém todas as linhas de ambas.

Onde não há correspondência, as colunas em falta são preenchidas com `NA`.

Para ilustrar, criamos dois subconjuntos de `sw` — as personagens altas e as personagens humanas — ligados pela chave `name`:

```
personagens_altos <- sw %>%
  filter(height > 180) %>%
  select(name, height)

personagens_humanos <- sw %>%
  filter(species == "Human") %>%
  select(name, species)
```

A `left_join()` mantém **todas** as personagens altas e acrescenta a espécie apenas às que também são humanas (as restantes ficam com `NA` em `species`):

```
left_join(personagens_altos, personagens_humanos, by = "name")
## # A tibble: 39 x 3
##   name                height species
##   <chr>              <int> <chr>
## 1 Darth Vader        202 Human
## 2 Biggs Darklighter  183 Human
## 3 Obi-Wan Kenobi     182 Human
## 4 Anakin Skywalker   188 Human
## 5 Chewbacca          228 <NA>
## 6 Boba Fett          183 Human
## 7 IG-88              200 <NA>
## 8 Bossk              190 <NA>
## 9 Qui-Gon Jinn       193 Human
## 10 Nute Gunray       191 <NA>
## # i 29 more rows
```

A `right_join()` faz o inverso: mantém todas as personagens humanas e acrescenta a altura às que também são altas:

```
right_join(personagens_altos, personagens_humanos, by = "name")
## # A tibble: 35 x 3
##   name                height species
##   <chr>              <int> <chr>
```

```
## 1 Darth Vader      202 Human
## 2 Biggs Darklighter 183 Human
## 3 Obi-Wan Kenobi    182 Human
## 4 Anakin Skywalker  188 Human
## 5 Boba Fett         183 Human
## 6 Qui-Gon Jinn      193 Human
## 7 Padmé Amidala     185 Human
## 8 Ric Olié          183 Human
## 9 Quarsh Panaka     183 Human
## 10 Mace Windu        188 Human
## # i 25 more rows
```

A `full_join()` mantém todas as personagens de ambas as tabelas:

```
full_join(personagens_altos, personagens_humanos, by = "name")
## # A tibble: 59 x 3
##   name          height species
##   <chr>          <int> <chr>
## 1 Darth Vader      202 Human
## 2 Biggs Darklighter 183 Human
## 3 Obi-Wan Kenobi    182 Human
## 4 Anakin Skywalker  188 Human
## 5 Chewbacca        228 <NA>
## 6 Boba Fett         183 Human
## 7 IG-88            200 <NA>
## 8 Bossk            190 <NA>
## 9 Qui-Gon Jinn      193 Human
## 10 Nute Gunray      191 <NA>
## # i 49 more rows
```

As três junções são a versão em `dplyr` das combinações de tabelas que já tínhamos visto com a função `merge()` do R base (Capítulo 4). A vantagem aqui é a clareza: o nome de cada função diz exatamente que linhas serão preservadas.

15.3.0.1 Exercícios

1. Crie `personagens_altos` (altura > 180 cm) e `personagens_leves` (massa < 75 kg) e combine-as com `left_join()` pela coluna `name`. Observe que as personagens altas sem correspondência ficam com NA na massa.
2. Crie `personagens_humanos` (espécie "Human") e `cor_cabelo` (personagens com `hair_color` não NA) e combine-as com `right_join()` por `name`.

3. Crie `especies_personagens` (espécie não NA) e `cor_olhos` (`eye_color` não NA) e combine-as com `full_join()` por `name`.

Com os dados lidos, limpos e organizados, estamos prontos para o passo mais recompensador: representá-los graficamente com toda a expressividade. É o que faremos no próximo capítulo, com o `ggplot2`.

Capítulo 16

O pacote ggplot2

No Capítulo 14 aprendemos a desenhar gráficos com o R base, acrescentando elementos ao gráfico um comando de cada vez. Neste capítulo conhecemos uma abordagem diferente e muito influente: a do `ggplot2`, que não nos pede que desenhemos o gráfico, mas que o **descrevamos**.

16.1 Um pouco de história

A ideia que está por trás do `ggplot2` nasceu num livro: *The Grammar of Graphics*, do estatístico e informático Leland Wilkinson, publicado em 1999 (com segunda edição em 2005). Wilkinson propôs que qualquer gráfico estatístico, por mais complexo, pode ser descrito como a combinação de um pequeno número de componentes independentes — os dados, os mapeamentos para propriedades visuais, as formas geométricas, as escalas, o sistema de coordenadas. Tal como a gramática de uma língua permite formar infinitas frases a partir de poucas regras, esta “gramática dos gráficos” permite construir uma enorme variedade de visualizações a partir de poucos elementos.

Em meados da década de 2000, Hadley Wickham implementou e adaptou estas ideias num pacote de R, a que chamou `ggplot2` — o “gg” vem precisamente de *grammar of graphics*. A sua adaptação, publicada em 2010 no artigo *A Layered Grammar of Graphics*, organiza a construção de um gráfico em **camadas** que se somam umas às outras. O `ggplot2` tornou-se, desde então, um dos pacotes mais usados do R e uma das pedras angulares do **tidyverse**.

16.2 Porque é importante

O que distingue o `ggplot2` da abordagem do R base é a mudança de perspetiva. Com o R base, pensamos em *instruções*: “desenha estes pontos, agora acrescenta uma legenda, agora uma linha”. Com o `ggplot2`, pensamos em *relações*: “a variável `wt` mapeia-se no eixo horizontal, a

`mpg` no eixo vertical, e cada observação é um ponto”. Descrevemos a correspondência entre os dados e as propriedades visuais, e o pacote encarrega-se do resto — escalas, eixos, legendas.

Desta filosofia decorrem as suas grandes vantagens: a **consistência** (a mesma gramática serve para todos os tipos de gráfico), a facilidade de construir gráficos sofisticados por acumulação de camadas, resultados com **qualidade de publicação** logo por omissão, e um vasto ecossistema de extensões. É, por tudo isto, a ferramenta de eleição para a visualização de dados em R.

Este capítulo usa o `ggplot2`, incluído no `tidyverse`. Instale-o com `install.packages("ggplot2")` (ou `install.packages("tidyverse")`) e carregue-o com `library(ggplot2)`.

16.3 A gramática, camada a camada

Um gráfico em `ggplot2` compõe-se dos seguintes elementos:

1. **Dados** (*data*) — o conjunto de dados a visualizar (um *data frame* ou *tibble*).
2. **Estética** (*aes*) — os mapeamentos das variáveis para propriedades visuais: a posição (`x`, `y`), a cor (`colour`), o tamanho (`size`), a forma (`shape`), o preenchimento (`fill`).
3. **Geometrias** (*geoms*) — o tipo de objeto que representa os dados: pontos (`geom_point()`), barras (`geom_bar()`), linhas (`geom_line()`), caixas (`geom_boxplot()`), etc.
4. **Escalas** (*scales*) — como os valores dos dados se traduzem em propriedades visuais (que cores, que intervalos nos eixos).
5. **Facetas** (*facets*) — a divisão do gráfico em subgráficos, um por grupo.
6. **Temas** (*themes*) — a aparência geral (tipos de letra, fundos, grelhas).

As camadas somam-se literalmente com o operador `+`. A estrutura mínima é sempre a mesma: `ggplot()` recebe os dados e a estética, e uma geometria acrescenta os objetos visuais.

```
library(ggplot2)

ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```

Aqui, `mtcars` são os dados, `aes(x = wt, y = mpg)` diz que o peso vai no eixo horizontal e o consumo no vertical, e `geom_point()` representa cada carro por um ponto.

16.4 Exemplos

Dispersão com uma terceira variável. A força da estética revela-se quando mapeamos *mais* variáveis. Ao colorir os pontos pelo número de cilindros, acrescentamos uma terceira dimensão de

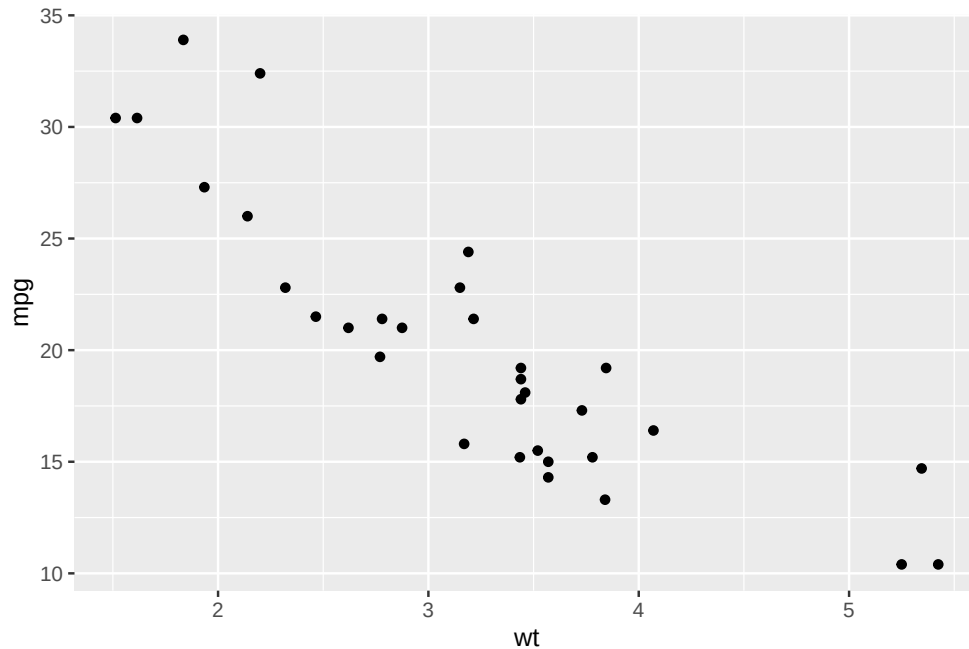


Figura 16.1: A estrutura mínima: dados, estética e uma geometria.

informação sem mudar de gráfico. Como `cyl` é, na verdade, uma variável categórica, convertemo-la num fator:

```
ggplot(mtcars, aes(x = wt, y = mpg, colour = factor(cyl))) +
  geom_point(size = 2) +
  labs(title = "Consumo em função do peso",
       x = "Peso (1000 lb)", y = "Consumo (milhas/galão)",
       colour = "Cilindros")
```

Gráfico de barras. Com uma variável categórica, `geom_bar()` conta automaticamente as observações de cada categoria:

```
ggplot(mpg, aes(x = class)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Modelos por classe", x = "Classe", y = "Contagem")
```

Barras proporcionais. Mapeando `fill` para uma segunda variável categórica e usando `position = "fill"`, cada barra passa a mostrar **proporções** — ideal para comparar a composição entre grupos. Usamos o conjunto `diamonds`, que acompanha o `ggplot2`:

```
ggplot(diamonds, aes(x = cut, fill = clarity)) +
  geom_bar(position = "fill") +
```

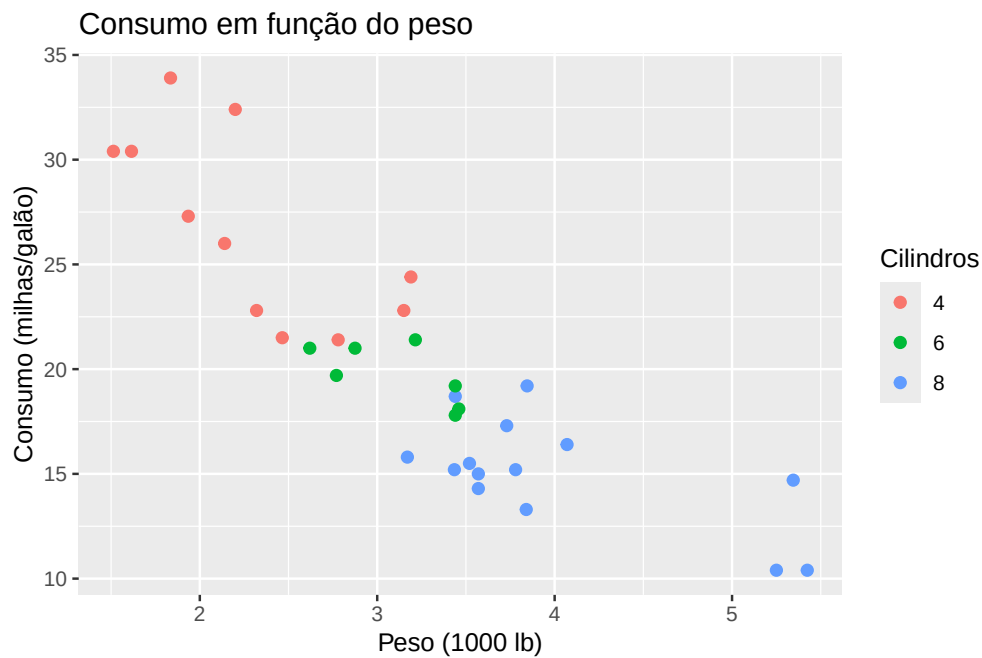



Figura 16.2: Consumo vs peso, com a cor a codificar o número de cilindros.

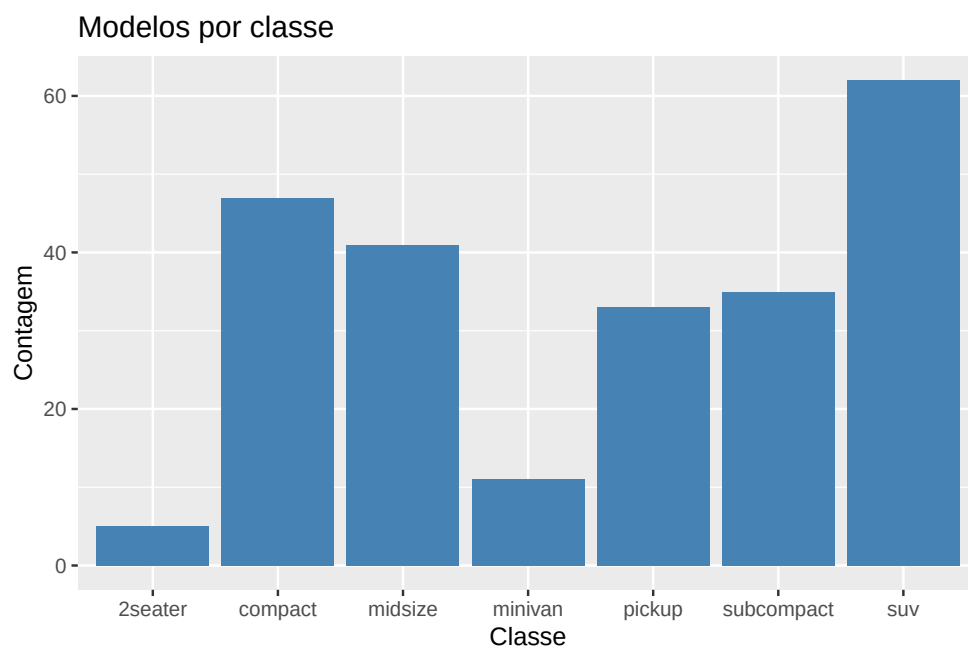


Figura 16.3: Número de modelos por classe de automóvel (conjunto mpg).

```
labs(title = "Pureza por qualidade de corte",
     x = "Corte", y = "Proporção", fill = "Pureza")
```

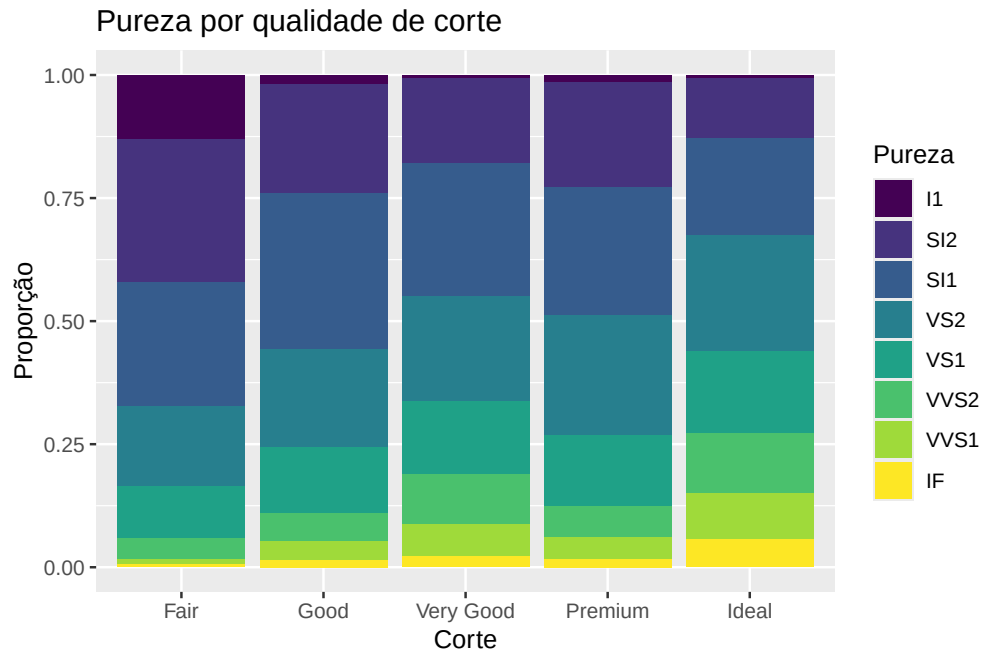


Figura 16.4: Composição da pureza (clarity) em cada qualidade de corte (cut).

Caixas de bigodes por grupo. A `geom_boxplot()` compara a distribuição de uma variável quantitativa entre categorias:

```
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = "Consumo em autoestrada por classe",
       x = "Classe", y = "Milhas por galão (autoestrada)")
```

Tendência com regressão. Somando uma camada `geom_smooth()`, o `ggplot2` ajusta e desenha uma tendência — com `method = "lm"`, a reta de regressão dos mínimos quadrados, rodeada da sua banda de confiança:

```
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(title = "Consumo vs peso, com reta de regressão",
       x = "Peso (1000 lb)", y = "Consumo (milhas/galão)")
```

Facetas. As facetas dividem o gráfico numa grelha de subgráficos, um por grupo, partilhando

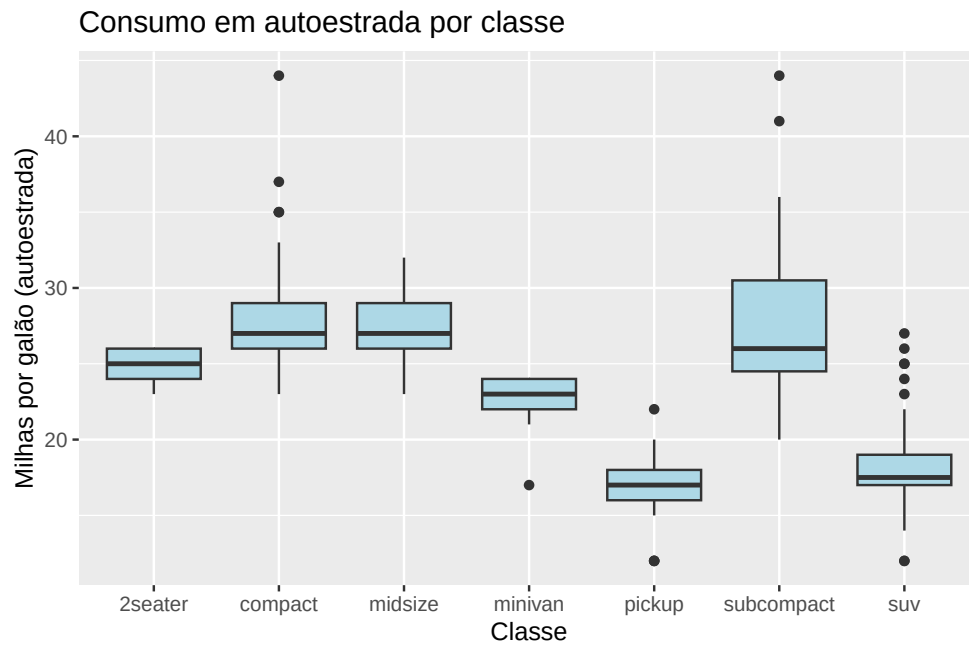


Figura 16.5: Consumo em autoestrada por classe de automóvel.

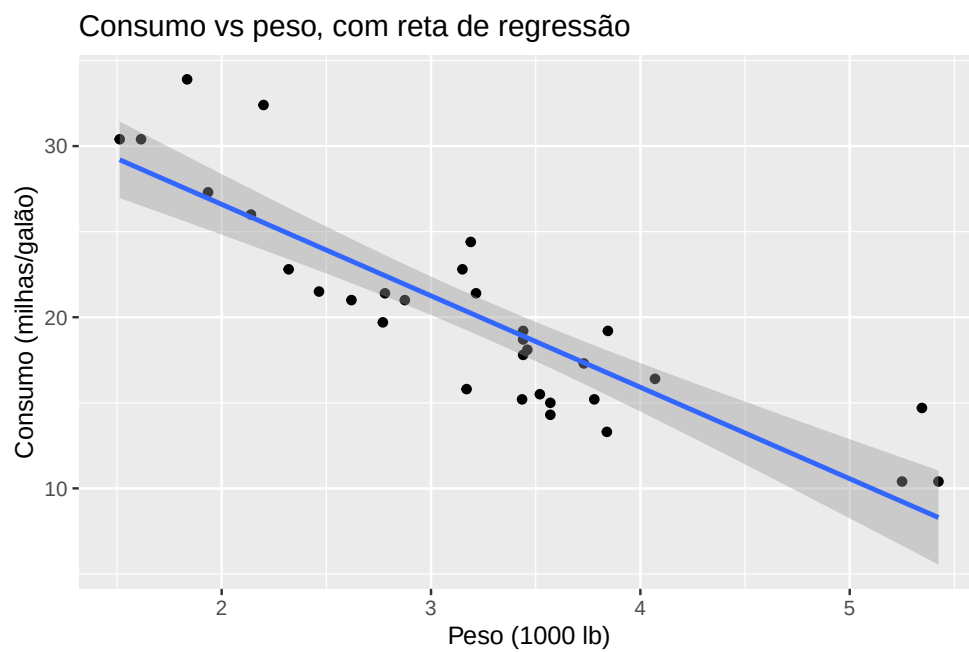


Figura 16.6: Dispersão com reta de regressão e banda de confiança.

os eixos — o que torna as comparações imediatas. Aqui, a relação entre cilindrada e consumo, separada por classe:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  facet_wrap(~ class) +
  labs(x = "Cilindrada (l)", y = "Consumo (autoestrada)")
```

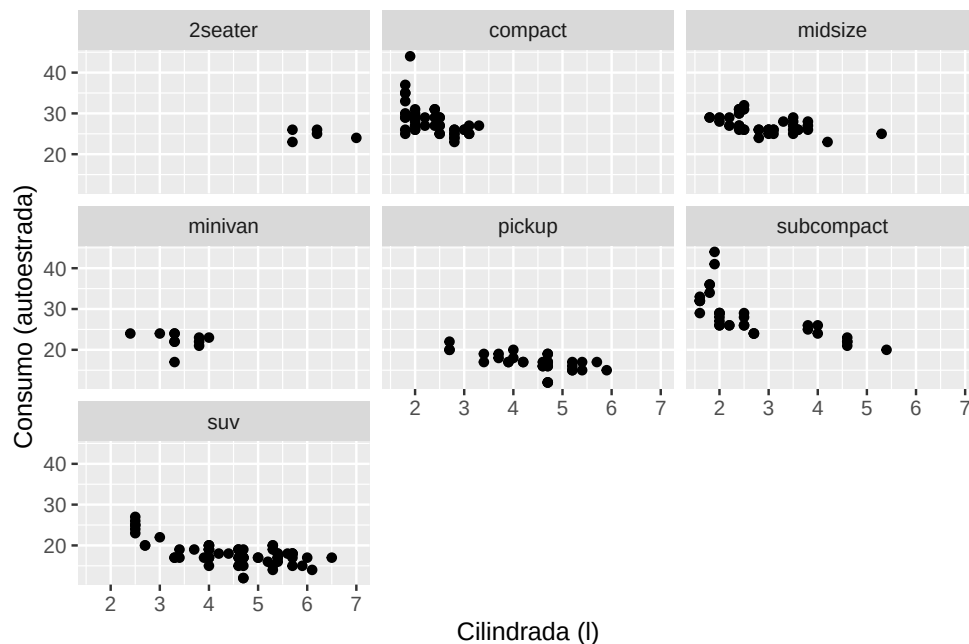


Figura 16.7: Subgráfico por classe, usando a função `facet_wrap()`.

Temas. Um tema controla toda a aparência do gráfico. Trocar `theme_minimal()` por outro tema muda o aspeto sem tocar nos dados nem na estética:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(colour = "steelblue") +
  theme_minimal() +
  labs(x = "Cilindrada (l)", y = "Consumo (autoestrada)")
```

Alguns hábitos tornam qualquer gráfico melhor: dar títulos e rótulos claros com `labs(title = ..., x = ..., y = ...)`; evitar sobrecarregar o gráfico com informação a mais; escolher cores com bom contraste (e legíveis por quem tem daltonismo, por exemplo com `scale_colour_viridis_d()`); e usar facetas em vez de amontoar tudo num só painel quando há muitos grupos.

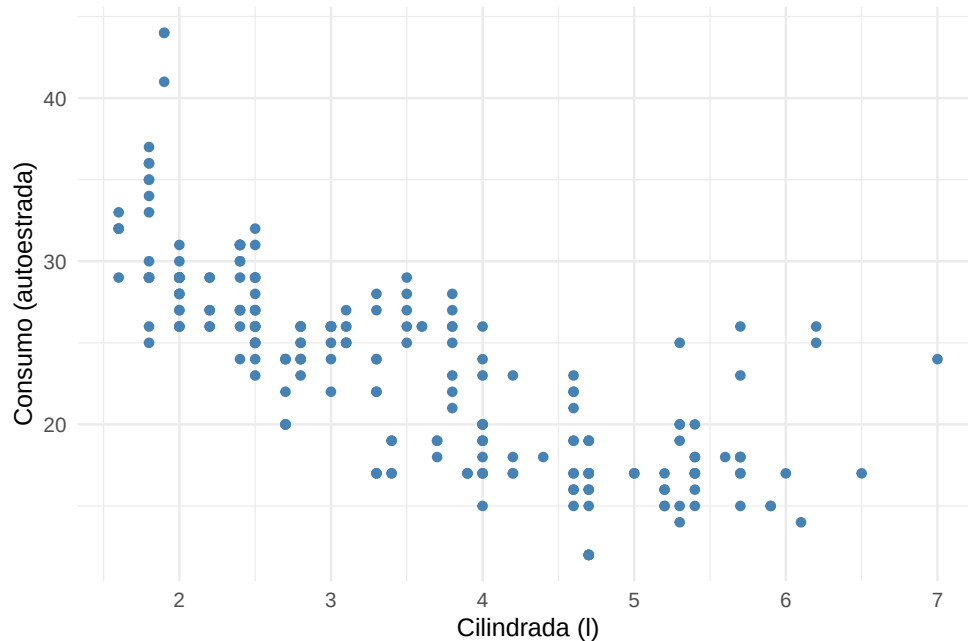


Figura 16.8: O mesmo gráfico com o tema `theme_minimal()`.

16.5 Exercícios

Nos exercícios seguintes, use o `ggplot2` e os conjuntos de dados indicados.

1. Com o conjunto `mtcars`, faça um gráfico de dispersão entre `wt` e `mpg`, colorindo os pontos pela variável `cyl`.
2. Com o conjunto `mtcars`, faça um gráfico de dispersão entre `mpg` e `hp`, colorindo os pontos por `cyl` e variando a forma (`shape`) dos pontos por `am`.
3. Com o conjunto `mpg`, faça um gráfico de barras da variável `drv` e altere a cor das barras com `fill`.
4. Com o conjunto `diamonds`, faça um gráfico de barras da variável `cut`, preenchendo (`fill`) pelas categorias de `clarity`. Use `position = "fill"` para comparar as proporções relativas.
5. Com o conjunto `mpg`, desenhe um histograma da variável `hwy` e sobreponha-lhe uma curva de densidade com `geom_density()`. Explore o argumento `alpha` para controlar a transparência.
6. Faça uma caixa de bigodes de `hwy` por `manufacturer` (conjunto `mpg`), ordenando os fabricantes pela média de `hwy`. *Sugestão:* `forcats::fct_reorder()`.
7. Com o conjunto `iris`, faça caixas de bigodes de `Sepal.Length` por espécie e sobreponha um ponto com a média de cada grupo, usando `stat_summary()`.
8. Com o conjunto `mpg`, ajuste uma curva LOESS a `displ` vs `hwy` com `geom_smooth()`, separando por `drv` com `facet_grid()`.

9. Exporte um dos seus gráficos para um ficheiro `.png` com `ggsave("grafico.png")`.
10. Escolha um conjunto de dados à sua escolha (ou um *tibble* pequeno) e crie uma visualização que combine pelo menos **dois** tipos de geometria (`geom`) diferentes — por exemplo, pontos e uma linha de tendência.

Dominadas a leitura, a manipulação e a visualização de dados, encerra-se a parte aplicada de tratamento de dados. A partir daqui, o livro entra no seu tema central: a **aleatoriedade** e a **simulação**. Começamos, no próximo capítulo, pela ferramenta mais elementar para gerar acaso no R — a função `sample()`.

Capítulo 17

A função `sample()`

Começa aqui a parte central do livro: a **aleatoriedade** e a **simulação**. Simular é imitar, no computador, um fenómeno sujeito ao acaso — lançar um dado, extrair uma amostra, esperar numa fila — e observar o que acontece quando o repetimos muitas vezes. É uma forma poderosa de estudar a probabilidade: em vez de resolver um problema apenas com fórmulas, podemos *fazê-lo acontecer* milhares de vezes e medir o resultado.

A ferramenta mais elementar para gerar acaso no R é a função `sample()`, que extrai elementos ao acaso de um conjunto. Com ela lançam-se dados e moedas, baralham-se cartas, sorteiam-se amostras — os alicerces de quase todas as simulações que se seguem.

```
sample(x, size, replace = FALSE, prob = NULL)
```

- `x` — o vetor de onde se extraem os elementos;
- `size` — a dimensão da amostra a extrair;
- `replace` — se a extração é **com** reposição (`TRUE`) ou **sem** reposição (`FALSE`, por omissão);
- `prob` — um vetor de probabilidades associadas a cada elemento de `x`; se omitido, todos são igualmente prováveis.

Neste capítulo aprendemos a extrair amostras aleatórias com `sample()` — com e sem reposição, com probabilidades iguais ou diferentes — e a tornar as simulações reproduzíveis com `set.seed()`.

17.1 Extrair amostras

Sem reposição. Cada elemento sai no máximo uma vez, como ao retirar bolas de um saco sem as repor. Extraíamos 5 números de uma população de 1 a 10:

```
pop <- 1:10
sample(pop, size = 5, replace = FALSE)
## [1] 5 2 4 10 8      # um resultado possível
```

Com reposição. Cada elemento pode sair mais do que uma vez, como ao lançar repetidamente o mesmo dado. É `replace = TRUE` que permite repetições:

```
sample(pop, size = 5, replace = TRUE)
## [1] 6 8 9 9 6      # o 6 e o 9 repetem-se
```

Sem reposição, a dimensão da amostra não pode exceder o número de elementos disponíveis: `sample(1:10, size = 15)` gera um erro, porque não há 15 valores distintos para extrair. Com `replace = TRUE`, pelo contrário, podemos extrair amostras tão grandes quanto quisermos.

Com probabilidades diferentes. O argumento `prob` permite que uns elementos sejam mais prováveis do que outros — indispensável para simular dados ou moedas enviesados. Aqui, os dois últimos valores têm metade da probabilidade dos restantes:

```
prob <- c(0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.05, 0.05)
sample(pop, size = 5, prob = prob)
## [1] 2 4 6 5 3      # um resultado possível
```

As probabilidades em `prob` não têm de somar 1: o R normaliza-as automaticamente, dividindo cada uma pela soma total. Assim, `prob = c(2, 1, 1)` é equivalente a `prob = c(0.5, 0.25, 0.25)` — o que conta são as proporções relativas entre os elementos.

17.2 Reprodutibilidade: `set.seed()`

Há uma subtilidade fundamental em tudo o que envolve o acaso no computador. Os números que o R diz serem “aleatórios” são, na verdade, **pseudoaleatórios**: resultam de um algoritmo determinístico que, partindo de um valor inicial (a *semente*), produz uma sequência que *parece* aleatória. Isto tem uma consequência muito útil: se fixarmos a semente, obtemos sempre a mesma sequência.

A função `set.seed()` fixa essa semente. Executar `set.seed()` com o mesmo número antes de uma simulação garante que ela produz exatamente o mesmo resultado de cada vez — em qualquer computador:

```
set.seed(42)
sample(1:10, size = 5)
```



```
## [1] 1 5 10 8 2

set.seed(42)      # a mesma semente...
sample(1:10, size = 5)
## [1] 1 5 10 8 2  # ...produz exatamente o mesmo resultado
```

Adquira o hábito de começar qualquer simulação com `set.seed()`. Não torna os resultados menos aleatórios — a sequência continua a comportar-se como se fosse acaso —, mas torna o seu trabalho **reprodutível**: quem executar o seu código obterá os mesmos números, poderá confirmar as suas conclusões e depurar erros. A reprodutibilidade é uma exigência central da ciência, e no computador consegue-se com uma linha.

Cuidado com uma armadilha de `sample()`: quando o primeiro argumento é um **único número** positivo `n`, o R interpreta-o como `1:n`. Assim, `sample(10)` devolve uma permutação de 1 a 10, e não o número 10. Isto raramente é o que se quer quando `x` é calculado dentro de um programa e pode, por acaso, ter comprimento 1. Para amostrar com segurança dos *elementos* de um vetor `x`, use `x[sample(length(x), size)]`, que indexa sempre pelas posições.

17.3 Exercícios

1. Crie um vetor com os números de 1 a 20 e extraia uma amostra aleatória de 5 elementos, sem reposição.
2. Considere uma população representada pelos números de 1 a 10. Extraia uma amostra de 10 elementos com reposição.
3. Crie um vetor com as letras A, B, C, D, E e extraia uma amostra de 3 letras, em que a probabilidade de cada uma é dada por `c(0.1, 0.2, 0.3, 0.25, 0.15)`.
4. Crie um vetor com os números de 1 a 10 e use `sample()` para o reordenar aleatoriamente (uma permutação).
5. Crie um vetor com cinco frutas (“Maçã”, “Banana”, “Laranja”, “Uva”, “Pera”). Selecione aleatoriamente uma fruta e, depois, uma amostra de 3 frutas.
6. É responsável pelo controlo de qualidade numa fábrica com 1000 produtos, numerados de 1 a 1000. Extraia uma amostra aleatória de 50 produtos para inspeção, sem reposição.
7. Simule o lançamento de dois dados equilibrados 10 000 vezes e registe a soma das faces. Use `sample()` e, no fim, faça um histograma das somas obtidas.
8. Tem 200 estudantes distribuídos por três turmas — A, B e C — com 50, 100 e 50 alunos, respetivamente. Usando `sample()`, extraia uma amostra de 20 alunos que mantenha a proporção das turmas.

9. Um cartão de Bingo contém 24 números aleatórios entre 1 e 75 (excluindo a casa central “livre”). Crie 5 cartões de Bingo distintos com `sample()`.

10. Num ensaio clínico, 30 pacientes têm de ser aleatorizados em dois grupos: tratamento (20 pacientes) e controlo (10). Faça a aleatorização com `sample()`. *Sugestão:* selecione os 20 do tratamento e obtenha os restantes com `setdiff()`.

Com a `sample()` sabemos gerar acaso. Falta-nos a linguagem para o descrever rigorosamente: as **variáveis aleatórias** e a **probabilidade**, tema do próximo capítulo.

Capítulo 18

Probabilidade e variáveis aleatórias

No capítulo anterior aprendemos a *gerar* acaso. Agora precisamos da linguagem matemática para o *descrever* com rigor — a linguagem das **variáveis aleatórias**, que está no coração de toda a estatística e de toda a simulação.

Uma **variável aleatória** é uma função que atribui um valor numérico a cada resultado possível de uma experiência aleatória. Formalmente, é uma função que faz corresponder a cada elemento do espaço de resultados Ω um número real. Ao lançar um dado, por exemplo, podemos definir a variável aleatória X como o número da face voltada para cima, que assume valores de 1 a 6.

As variáveis aleatórias são fundamentais porque permitem quantificar e analisar matematicamente fenômenos incertos. Classificam-se em dois grandes tipos:

- **discretas** — assumem valores num conjunto finito ou numerável, como o número de caras em três lançamentos de uma moeda;
- **contínuas** — assumem qualquer valor num intervalo, como a altura de uma pessoa.

Neste capítulo revemos as ferramentas que descrevem uma variável aleatória — função de probabilidade, função densidade, função de distribuição, valor esperado e variância — e aprendemos a calculá-las no R, com particular destaque para a integração numérica com `integrate()`.

18.1 Como se descreve uma variável aleatória

A distribuição de uma variável aleatória descreve-se de formas diferentes consoante ela seja discreta ou contínua.

Uma variável **discreta** descreve-se pela sua **função massa de probabilidade** (f.m.p.), $f_X(x) = P(X = x)$, que dá a probabilidade de cada valor. Para ser uma f.m.p. válida, tem de satisfazer duas condições:

$$f_X(x) \geq 0 \quad \forall x, \quad \sum_x f_X(x) = 1.$$

Uma variável **contínua** descreve-se pela sua **função densidade de probabilidade** (f.d.p.), $f_X(x)$. Aqui, a probabilidade não está em cada ponto (que é nula), mas na **área sob a curva**: $P(a < X < b) = \int_a^b f_X(x) dx$. As condições análogas são:

$$f_X(x) \geq 0 \quad \forall x, \quad \int_{-\infty}^{+\infty} f_X(x) dx = 1.$$

Em qualquer dos casos, a **função de distribuição** (acumulada), $F_X(x) = P(X \leq x)$, acumula a probabilidade até x — somando, no caso discreto, ou integrando, no contínuo:

$$F_X(x) = \sum_{t \leq x} f_X(t) \quad \text{ou} \quad F_X(x) = \int_{-\infty}^x f_X(t) dt.$$

Finalmente, dois números resumem a distribuição: o **valor esperado** $E(X)$ (o centro, a “média teórica”) e a **variância** $V(X)$ (a dispersão em torno desse centro). As suas definições espelham a natureza discreta ou contínua da variável:

$$E(X) = \sum_x x f_X(x) \quad \text{ou} \quad E(X) = \int_{-\infty}^{+\infty} x f_X(x) dx,$$

$$V(X) = E[(X - E(X))^2] = \sum_x (x - E(X))^2 f_X(x) \quad \text{ou} \quad \int_{-\infty}^{+\infty} (x - E(X))^2 f_X(x) dx.$$

Repare no paralelo perfeito entre os dois mundos: onde a variável discreta **soma** ($\sum$), a contínua **integra** ($\int$). É a mesma ideia — acumular probabilidade — expressa nas duas linguagens. Guardar este paralelo em mente torna toda a matéria mais simples.

18.2 Exemplo: uma variável discreta

Numa empresa, o número de reclamações recebidas por dia é uma variável aleatória discreta X com a seguinte f.m.p.:

$$f_X(x) = P(X = x) = \begin{cases} 0,10, & x = 0 \\ 0,25, & x = 1 \\ 0,30, & x = 2 \\ 0,20, & x = 3 \\ 0,15, & x = 4 \end{cases}$$

Queremos (a) confirmar que f_X é uma f.m.p., (b) calcular o número esperado e a variância das reclamações, e (c) a probabilidade de haver mais de 2 reclamações num dia.

Para (a), verificamos que as probabilidades são não negativas e somam 1:

```
X_valores <- c(0, 1, 2, 3, 4)
P_X <- c(0.10, 0.25, 0.30, 0.20, 0.15)

sum(P_X)          # deve dar 1
## [1] 1
```

Para (b), aplicamos diretamente as definições de $E(X)$ e $V(X)$:

```
E_X <- sum(X_valores * P_X)          # valor esperado
Var_X <- sum((X_valores - E_X)^2 * P_X) # variância
E_X
## [1] 2.05
Var_X
## [1] 1.448
```

E para (c), somamos as probabilidades dos valores superiores a 2:

```
P_X_maior_2 <- sum(P_X[X_valores > 2])
P_X_maior_2
## [1] 0.35
```

Em média, a empresa recebe cerca de 2 reclamações por dia, e a probabilidade de receber mais de 2 é de 0,35.

18.3 Exemplo: uma variável contínua

Seja X uma variável contínua com f.d.p.

$$f(x) = \begin{cases} 2e^{-2x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

(trata-se de uma distribuição exponencial). Queremos (a) confirmar que é uma f.d.p., (b) calcular $P(X > 2)$ e (c) $P(0,3 < X < 0,7)$.

Começamos por definir a função em R:

```
f1 <- function(x) {
  ifelse(x < 0, 0, 2 * exp(-2 * x))
}
```

O seu gráfico mostra a forma típica de decaimento da exponencial:

```
plot(f1, from = 0, to = 5,
     main = "Densidade exponencial",
     xlab = "x", ylab = "f(x)", col = "steelblue", lwd = 2)
```

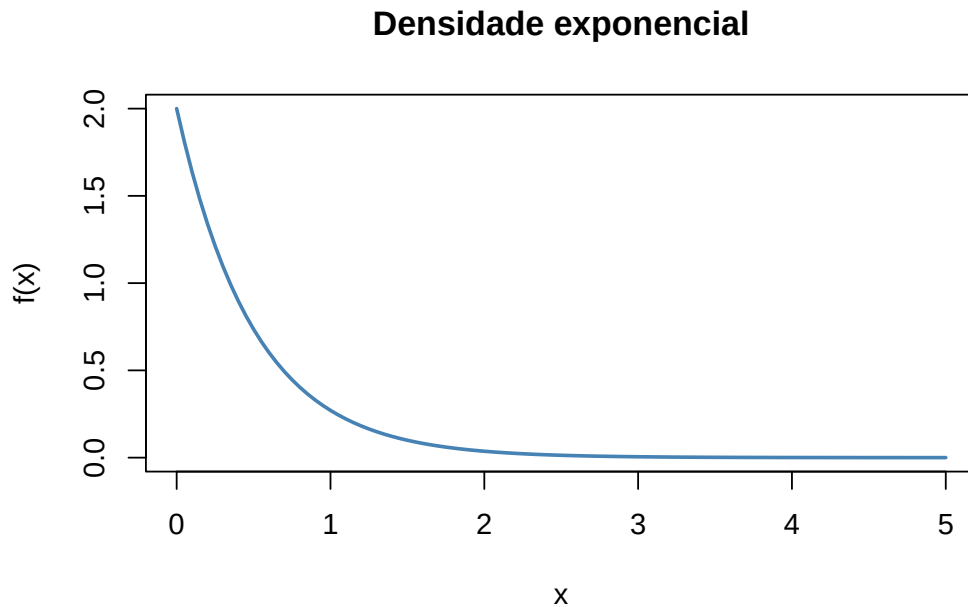


Figura 18.1: Função densidade da distribuição exponencial de parâmetro 2.

Para (a), a condição $\int_{-\infty}^{+\infty} f(x) dx = 1$ verifica-se por integração numérica com `integrate()`:

```
integrate(f1, lower = 0, upper = Inf)
## 1 with absolute error < 5e-07
```

As alíneas (b) e (c) correspondem a áreas sob a curva, ou seja, a integrais nos intervalos pedidos:

$$P(X > 2) = \int_2^{+\infty} 2e^{-2x} dx, \quad P(0,3 < X < 0,7) = \int_{0,3}^{0,7} 2e^{-2x} dx.$$

```
integrate(f1, lower = 2, upper = Inf)      # P(X > 2)
## 0.01832 with absolute error < 2.8e-06
integrate(f1, lower = 0.3, upper = 0.7)    # P(0.3 < X < 0.7)
## 0.3022 with absolute error < 3.4e-15
```

A `integrate()` devolve um objeto com vários elementos; o valor do integral está em `$value`. Assim, para usar o resultado num cálculo posterior, escrevemos, por exemplo, `integrate(f1, 0, Inf)$value`. Guardar este pormenor evita muita frustração quando se encadeiam contas.

Quando a forma da distribuição é conhecida, a sua função de distribuição pode ser escrita de forma fechada, sem integrar. Para esta exponencial, $F_X(x) = 1 - e^{-2x}$ para $x \geq 0$, e $P(X > 2) = 1 - F_X(2) = e^{-4} \approx 0,018$ — exatamente o valor que a integração numérica devolveu. Comparar o cálculo analítico com o numérico é uma excelente forma de confirmar que ambos estão certos.

18.4 Exemplo: uma densidade definida por troços

A procura diária de arroz num supermercado, em centenas de quilogramas, é uma variável aleatória X com f.d.p.

$$f(x) = \begin{cases} \frac{2}{3}x, & 0 \leq x < 1 \\ -\frac{x}{3} + 1, & 1 \leq x < 3 \\ 0, & x < 0 \text{ ou } x \geq 3 \end{cases}$$

Queremos (a) a probabilidade de se venderem mais de 150 kg num dia e (b) a venda esperada em 30 dias. Definimos a função por troços com `ifelse()` encadeados:

```
f2 <- function(x) {
  ifelse(x >= 0 & x < 1, (2/3) * x,
        ifelse(x >= 1 & x < 3, -x/3 + 1, 0))
}
```

Confirmamos que é uma f.d.p. (o integral vale 1) e vemos o seu gráfico — uma densidade triangular:

```
integrate(f2, 0, 3)
## 1 with absolute error < 1.1e-15

plot(f2, from = -1, to = 4,
     main = "Densidade da procura de arroz",
     xlab = "x (centenas de kg)", ylab = "f(x)", col = "steelblue", lwd = 2)
```

Para (a), 150 kg correspondem a $x = 1,5$ centenas de quilogramas, pelo que queremos $P(X > 1,5) = \int_{1,5}^3 f(x) dx$. Podemos assinalar essa área no gráfico:

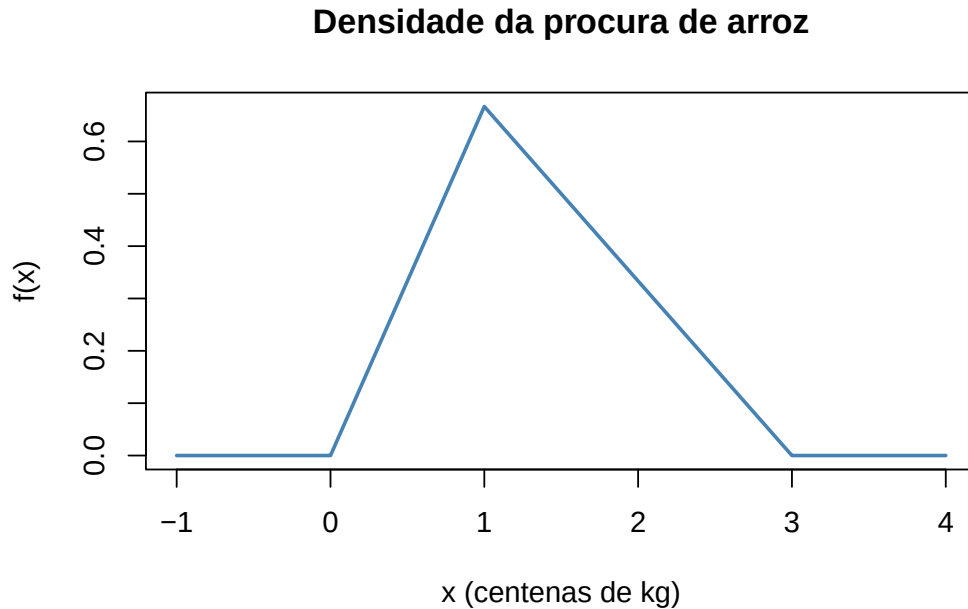


Figura 18.2: Densidade da procura de arroz, definida por dois troços.

```
plot(f2, from = -1, to = 4,
     main = "P(X > 1,5)", xlab = "x", ylab = "f(x)", col = "steelblue", lwd = 2)
polygon(x = c(1.5, 1.5, 3), y = c(0, f2(1.5), 0), density = 15, col = "firebrick")
```

```
integrate(f2, 1.5, 3)      # P(X > 1,5)
## 0.375 with absolute error < 4.2e-15
```

Para (b), a venda esperada num dia é $E(X) = \int_{-\infty}^{+\infty} x f(x) dx$; definimos a função integranda $x f(x)$ e integramos. A venda esperada em 30 dias é 30 vezes esse valor:

```
ef2 <- function(x) x * f2(x)

E_X <- integrate(ef2, 0, 3)$value
E_X      # venda esperada num dia (centenas de kg)
## [1] 1.333
30 * E_X      # venda esperada em 30 dias
## [1] 40
```

Note-se que os exemplos deste capítulo são simples e poderiam resolver-se analiticamente, sem métodos numéricos — foram escolhidos apenas para ilustrar as funções do R. O verdadeiro valor da integração numérica revela-se nos problemas complexos, em que a solução analítica é difícil ou impossível de obter. É aí que ferramentas como a `integrate()` se tornam indispensáveis.

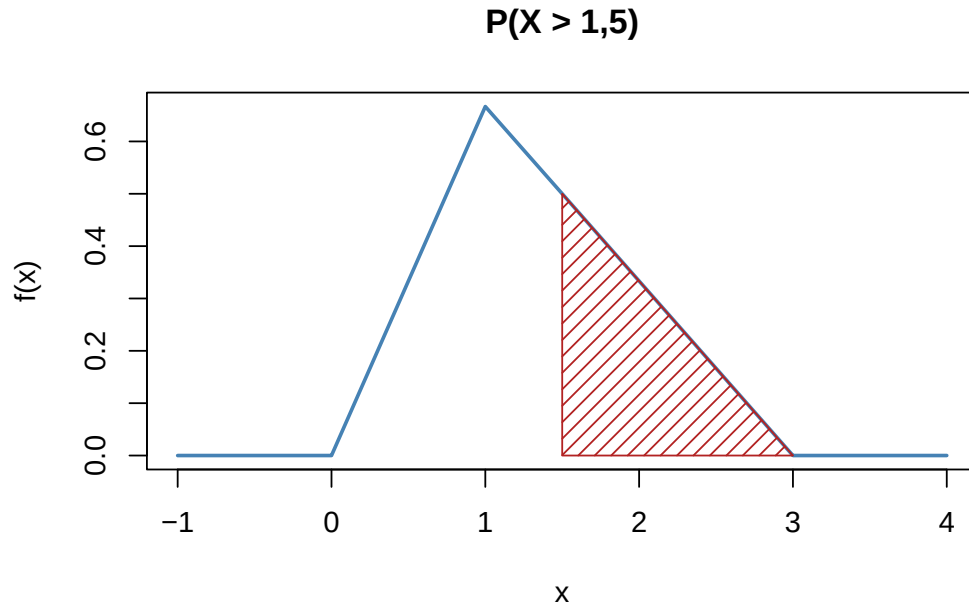


Figura 18.3: A área sombreada representa $P(X > 1,5)$.

18.5 Exercícios

1. O número de produtos defeituosos numa linha de produção, ao fim de cada hora, é uma variável aleatória discreta X com f.m.p.:

$$P(X = x) = \begin{cases} 1/10, & x = 0 \\ 2/10, & x = 1 \\ 3/10, & x = 2 \\ 2/10, & x = 3 \\ 2/10, & x = 4 \end{cases}$$

- (a) Verifique se é uma f.m.p.
- (b) Calcule o valor esperado e a variância do número de produtos defeituosos por hora.
- (c) Calcule a probabilidade de haver, no máximo, 2 produtos defeituosos numa hora.

2. Um inquérito numa universidade registou o número de cadeiras em que cada aluno está inscrito. A variável aleatória Z tem f.m.p.:

$$P(Z = z) = \begin{cases} 0,05, & z = 1 \\ 0,15, & z = 2 \\ 0,35, & z = 3 \\ 0,25, & z = 4 \\ 0,20, & z = 5 \end{cases}$$

- (a) Verifique se é uma f.m.p.
- (b) Calcule o valor esperado e a variância do número de cadeiras.
- (c) Calcule a probabilidade de um aluno estar inscrito em mais de 3 cadeiras.

3. Numa dada localidade, a distribuição do rendimento, em u.m. (unidades monetárias), é uma variável aleatória X com **função densidade de probabilidade**:

$$f_X(x) = \begin{cases} \frac{1}{10}x + \frac{1}{10}, & 0 \leq x \leq 2 \\ -\frac{3}{40}x + \frac{9}{20}, & 2 < x \leq 6 \\ 0, & x < 0 \text{ ou } x > 6 \end{cases}$$

- (a) Mostre que f_X é uma f.d.p.
- (b) Qual o rendimento médio nesta localidade?
- (c) Calcule a probabilidade de encontrar uma pessoa com rendimento acima de 4,5 u.m. e assinale o resultado no gráfico da densidade.

4. Uma variável aleatória contínua X tem distribuição uniforme no intervalo $[10, 20]$.

- (a) Apresente o gráfico da f.d.p.
- (b) Calcule $P(X < 15)$.
- (c) Calcule $P(12 \leq X \leq 18)$.
- (d) Calcule $E(X)$ e $V(X)$.

5. O tempo de espera (em minutos) num balcão de atendimento é uma variável aleatória contínua Y com f.d.p.

$$f_Y(y) = \begin{cases} 0,125y, & 0 \leq y \leq 4 \\ 0, & \text{caso contrário} \end{cases}$$

- (a) Crie uma função para a função de distribuição acumulada de Y .
- (b) Calcule $P(Y > 3)$.

- (c) Calcule $P(1 < Y < 3)$.
- (d) Calcule $P(Y < 2)$.
- (e) Calcule $P(Y < 3 \mid Y > 1)$ (probabilidade condicional).

6. A duração, em anos, de uma lâmpada especial é uma variável aleatória contínua X com densidade

$$f_X(x) = \begin{cases} 3e^{-3x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- (a) Crie uma função para a função de distribuição acumulada de X .
- (b) Calcule $P(X > 2)$.
- (c) Calcule $P(0,5 < X < 1,2)$.
- (d) Calcule $P(X > 3)$.
- (e) Calcule $P(X < 3 \mid X > 1)$.

Descrita a probabilidade de uma variável aleatória, o passo natural é conhecer as **distribuições** mais importantes — a binomial, a normal, a exponencial — e as funções que o R lhes dedica. É esse o tema do próximo capítulo.

Capítulo 19

Simulação

Chegámos ao coração do livro. Simular é usar o computador para *imitar* um fenómeno aleatório — repeti-lo milhares ou milhões de vezes — e, a partir dos resultados observados, responder a perguntas que seriam difíceis, ou mesmo impossíveis, de resolver só com papel e caneta.

O fundamento teórico da simulação é a **Lei dos Grandes Números**: se observarmos uma grande amostra de variáveis aleatórias independentes e identicamente distribuídas (i.i.d.) com média finita, a média dessas observações converge para a média verdadeira da distribuição à medida que a dimensão da amostra aumenta. Por outras palavras, em vez de calcular analiticamente uma média (ou uma probabilidade, que é a média de uma variável indicadora), podemos gerar uma amostra grande e simplesmente calcular a média observada — que será uma boa estimativa do valor verdadeiro.

A eficácia deste método depende de três fatores:

1. **Identificar as variáveis aleatórias certas** — que distribuições descrevem corretamente o fenómeno que estamos a modelar.
2. **Gerá-las com fidelidade** — os algoritmos de geração de números pseudoaleatórios têm de produzir amostras que representem bem as distribuições pretendidas.
3. **Escolher uma dimensão de amostra adequada** — grande o suficiente para que a estimativa seja fiável, sabendo que amostras maiores reduzem a variabilidade.

Neste capítulo compreendemos o princípio da simulação através da Lei dos Grandes Números, vemos como o R gera números pseudoaleatórios (e o que isso significa) e resolvemos, por simulação, um problema cuja solução analítica seria muito trabalhosa.

19.1 A simulação reproduz a teoria

Exemplo (estimar a média de uma uniforme). A média da distribuição uniforme em $[0, 1]$ é, sabidamente, $\frac{1}{2}$. Pela Lei dos Grandes Números, a média de uma amostra $X_1, \dots, X_n$ dessa

Tabela 19.1: Cinco estimativas independentes de uma probabilidade, para diferentes valores de n . À medida que n aumenta, as estimativas tendem a concentrar-se em torno do valor teórico.

n	Rép. 1	Rép. 2	Rép. 3	Rép. 4	Rép. 5
100	0.485	0.481	0.484	0.569	0.441
1 000	0.497	0.506	0.480	0.498	0.499
10 000	0.502	0.501	0.499	0.498	0.498
100 000	0.502	0.499	0.500	0.498	0.499

distribuição, $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$, deve aproximar-se de 0,5 à medida que n cresce.

A Tabela 19.1 mostra cinco repetições independentes da mesma experiência de simulação, para diferentes valores de n . Note-se como as médias estão sempre próximas de 0.5, mas com uma variação que **diminui** claramente à medida que n aumenta:

Para gerar uma amostra uniforme em $[0, 1]$ usa-se a função `runif()`:

```
runif(100, min = 0, max = 1)
```

Aqui não havia necessidade real de simular, pois conhecíamos a média analiticamente — o exemplo serve apenas para mostrar que a simulação reproduz a teoria. A lição a reter é esta: por maior que seja a amostra, a média amostral **não** será exatamente igual à média verdadeira; a simulação introduz uma variabilidade que temos de considerar, e que só se reduz aumentando n .

19.2 Quando a simulação vale mesmo a pena

Vejamos agora um problema fácil de descrever, mas cuja solução analítica seria penosa.

O problema. Dois empregados de balcão, A e B, começam a atender clientes ao mesmo tempo e combinam fazer uma pausa assim que ambos tiverem atendido 10 clientes. Provavelmente um terminará antes do outro e terá de esperar. Quanto tempo, em média, terá um deles de esperar pelo outro?

Modelamos os tempos de atendimento de cada cliente como variáveis i.i.d. com distribuição exponencial de taxa $\lambda = 0,3$ clientes por minuto. Então, o tempo total que cada empregado leva a atender 10 clientes é a soma de 10 exponenciais i.i.d., que segue uma distribuição **gama** de forma $k = 10$ e taxa $\lambda = 0,3$. Sejam X e Y esses tempos para A e B; queremos $E(|X - Y|)$.

A solução analítica exigiria um integral duplo sobre uma região não retangular. A simulação resolve o problema em poucas linhas: geramos muitos pares (X, Y) , calculamos $Z = |X - Y|$ em cada um, e estimamos $E(|X - Y|)$ pela média dos valores de Z .

```
set.seed(123)

# Tempos para atender 10 clientes: soma de 10 exponenciais = Gama(forma=10, taxa=0.3)
x <- rgamma(10000, shape = 10, rate = 0.3)
y <- rgamma(10000, shape = 10, rate = 0.3)

z <- abs(x - y)      # tempo de espera em cada réplica
mean(z)              # estimativa de  $E(|X - Y|)$ 
## [1] 11.76
```

A simulação estima um tempo médio de espera de cerca de 11,7 minutos. O histograma mostra a distribuição completa desses tempos:

```
hist(z,
      main = "Tempo de espera",
      xlab = "Tempo de espera (minutos)",
      ylab = "Frequência absoluta",
      col = "lightblue", border = "black")
```

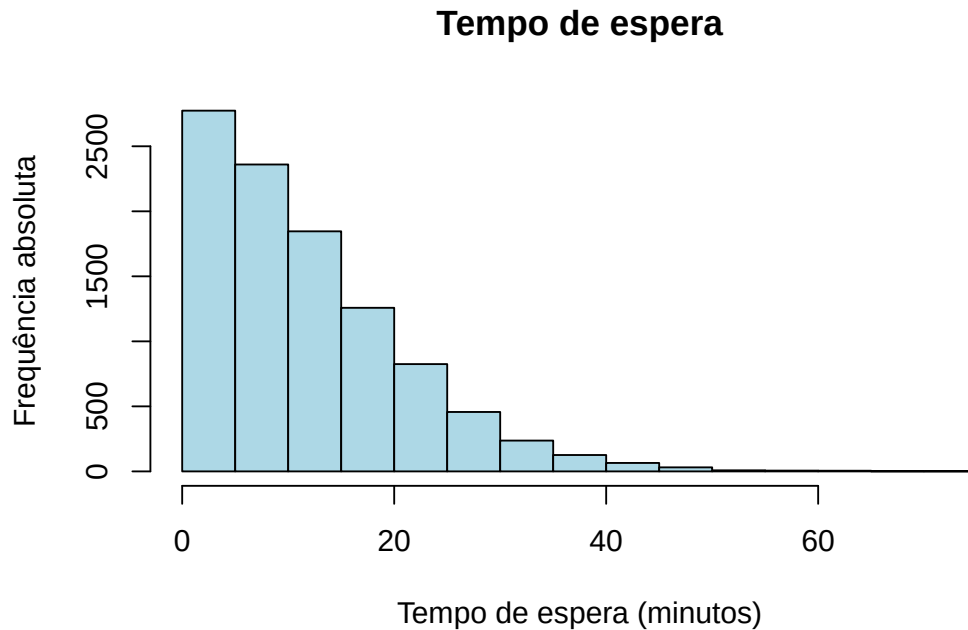


Figura 19.1: Distribuição do tempo de espera de um empregado pelo outro.

19.3 Números pseudoaleatórios

Vale a pena perceber o que o computador faz quando lhe pedimos “acaso”.

Números **verdadeiramente aleatórios** derivam de processos físicos imprevisíveis — o ruído térmico de um circuito, o decaimento radioativo, a radiação cósmica. Obtê-los é difícil e, para a maioria das aplicações, desnecessário.

Números **pseudoaleatórios**, pelo contrário, são produzidos por um algoritmo determinístico que, a partir de um valor inicial — a **semente** (*seed*) —, gera uma sequência que *parece* aleatória. Como o algoritmo é determinístico, a mesma semente produz sempre a mesma sequência. Não sendo verdadeiramente aleatórios, são rápidos de gerar, suficientemente imprevisíveis para quase tudo e, sobretudo, **reprodutíveis** — uma vantagem decisiva para a investigação e para a depuração de código.

19.3.1 O método congruencial multiplicativo

Um dos geradores clássicos é o **método congruencial multiplicativo** (ou de Lehmer). A partir de uma semente x_0 , os valores seguintes obtêm-se pela recorrência

$$x_n = (a \cdot x_{n-1}) \bmod m,$$

onde a e m são inteiros positivos dados e x_n é o resto da divisão inteira de $a \cdot x_{n-1}$ por m . Cada quociente x_n/m é um número pseudoaleatório, aproximando uma variável uniforme em $(0, 1)$.

Para servir numa simulação, as constantes a e m devem garantir que a sequência *pareça* aleatória, tenha um **período longo** antes de se repetir e seja eficiente de calcular. Com $m = 17$, $a = 5$ e $x_0 = 7$:

$$x_1 = (5 \times 7) \bmod 17 = 1, \quad x_2 = (5 \times 1) \bmod 17 = 5, \quad x_3 = (5 \times 5) \bmod 17 = 8, \dots$$

Continuando, obtém-se a sequência

$$7, 1, 5, 8, 6, 13, 14, 2, 10, 16, 12, 9, 11, 4, 3, 15, \underbrace{7}_{\text{repete}}, 1, 5, 8, \dots$$

com período 16.

Não é por acaso que o período é exatamente 16. Como $m = 17$ é primo e $a = 5$ é uma *raiz primitiva* módulo 17, a sequência percorre **todos** os valores de 1 a $m - 1$ antes de repetir — o chamado período completo, $m - 1$. Note-se ainda que, sendo o gerador multiplicativo, o valor 0 nunca aparece: os x_n estão sempre entre 1 e $m - 1$, e os x_n/m em $(0, 1)$. Escolher bem a e m é, por isto, uma questão delicada — os geradores usados na prática têm períodos astronomicamente longos.

19.4 A função `set.seed()`

No R, a semente do gerador fixa-se com `set.seed()`. Executar `set.seed()` com o mesmo número antes de uma operação aleatória garante que ela produz sempre os mesmos resultados:

```
set.seed(123)
sample(1:100, 5)
## [1] 31 79 51 14 67
```

Reexecutando o mesmo código com a mesma semente, obtêm-se exatamente os mesmos cinco números. O gerador que o R usa por omissão é o *Mersenne Twister*, com um período verdadeiramente enorme; quando não fixamos a semente, o R constrói uma a partir da hora e da sessão, pelo que cada execução parte de um ponto diferente.

Fixar a semente é um hábito, não uma limitação: os resultados continuam a comportar-se como acaso, mas tornam-se reproduzíveis. Todas as simulações deste livro que devam dar sempre o mesmo resultado começam por `set.seed()`.

19.5 Exercícios

1. Fixe uma semente à sua escolha com `set.seed()`.

- Gere duas amostras de dimensão 10 dos números de 1 a 50, sem reposição.
- Mude a semente e gere novamente as amostras.
- Compare os resultados antes e depois de mudar a semente.

2. Fixe a semente com `set.seed(123)`.

- Simule 100 lançamentos de um dado (1 a 6) e conte a frequência de cada face.
- Repita o processo sem redefinir a semente.
- Volte a fixar `set.seed(123)` e repita. Compare com a primeira simulação.

3. Use `set.seed()` para gerar 50 valores de uma normal $N(\mu = 10, \sigma = 2)$ com `rnorm(50, mean = 10, sd = 2)`.

- Com a mesma semente, gere 50 valores de uma uniforme $U(0, 1)$ com `runif(50)`.
- Mude a semente e repita. Compare os conjuntos de dados gerados.

4. Fixe a semente com `set.seed(42)`.

- Divida os números de 1 a 100 em dois grupos aleatórios de 50 (use `sample()` e `setdiff()`).
 - Repita a divisão sem fixar a semente.
 - Volte a fixar `set.seed(42)` e repita. Verifique se os grupos são os mesmos.
5. Use `set.seed()` para simular as respostas de 50 participantes a uma pergunta de escolha múltipla com 4 opções (A, B, C, D).
- Mude a semente e repita a simulação.
 - Verifique se a distribuição das respostas muda ao alterar, ou ao manter, a semente.

Sabemos agora simular e reproduzir o acaso. No próximo capítulo passamos em revista as **distribuições de probabilidade** mais importantes e as funções que o R lhes dedica — o vocabulário indispensável de qualquer simulação.

Capítulo 20

Distribuições de probabilidade

No capítulo anterior vimos que simular consiste, no essencial, em gerar variáveis aleatórias com a distribuição certa. Chega, por isso, o momento de conhecer as **distribuições de probabilidade** mais importantes e as funções que o R lhes dedica. Cada uma modela um tipo de fenómeno — contagens, tempos de espera, medições — e para cada uma o R oferece, de forma sistemática, quatro funções.

O R inclui as distribuições univariadas mais comuns. Para cada distribuição existem quatro funções, distinguidas por uma letra inicial:

O *nome* é a abreviatura da distribuição: `binom`, `geom`, `pois`, `unif`, `exp`, `norm`, e assim por diante.

Vale a pena fixar esta lógica de quatro letras, porque se aplica a *todas* as distribuições do R. Se sabe usar `dnorm`, `pnorm`, `qnorm` e `rnorm`, sabe usar as funções de qualquer outra distribuição — muda apenas a raiz do nome e, eventualmente, os parâmetros. É uma das grandes comodidades da linguagem.

Neste capítulo percorremos as principais distribuições — uniforme, Bernoulli, binomial, geométrica, Poisson (discretas), uniforme, exponencial e normal (contínuas) —, vendo para cada uma a definição, o valor esperado e a variância, o cálculo de probabilidades com `d/p/q` e a geração de amostras com `r`, comparando sempre a teoria com a simulação.

Tabela 20.1: Convenção usada em R para as funções associadas a distribuições de probabilidade.

Prefixo	O que devolve
<code>dnome(...)</code>	A função de probabilidade, no caso discreto, ou a função densidade, no caso contínuo.
<code>pnome(...)</code>	A função de distribuição acumulada, isto é, $F(x) = P(X \leq x)$.
<code>qnome(...)</code>	O quantil correspondente a uma dada probabilidade; em termos práticos, a inversa da função de <code>pnome</code> .
<code>rnome(...)</code>	Uma amostra aleatória gerada a partir da distribuição.

20.1 Primeiros exemplos

Exemplo 1 (lançar três moedas). Simulemos o lançamento de três moedas equilibradas e a contagem do número de caras, X . Queremos estimar $P(X = 1)$ e $E(X)$; e depois repetir para uma moeda enviesada com $P(\text{cara}) = 3/4$.

```
set.seed(123)
n <- 10000
sim1 <- numeric(n)  # indicador de "saiu exatamente 1 cara"
sim2 <- numeric(n)  # número de caras
for (i in 1:n) {
  moedas <- sample(0:1, 3, replace = TRUE)
  sim1[i] <- if (sum(moedas) == 1) 1 else 0
  sim2[i] <- sum(moedas)
}
mean(sim1)  # estimativa de  $P(X = 1)$ 
## [1] 0.3821
mean(sim2)  # estimativa de  $E(X)$ 
## [1] 1.4928
```

Para a moeda enviesada, basta indicar as probabilidades em `sample()`:

```
set.seed(123)
n <- 10000
sim1 <- numeric(n)
sim2 <- numeric(n)
for (i in 1:n) {
  moedas <- sample(c(0, 1), 3, prob = c(1/4, 3/4), replace = TRUE)
  sim1[i] <- if (sum(moedas) == 1) 1 else 0
  sim2[i] <- sum(moedas)
}
mean(sim1)
## [1] 0.1384
mean(sim2)
## [1] 2.2503
```

Mas o número de caras em três lançamentos é, por definição, uma Binomial($n = 3, p$). Podemos, por isso, gerar as observações diretamente com `rbinom()`, sem o ciclo:

```
set.seed(123)
valores <- rbinom(10000, size = 3, prob = 0.5)
sum(valores == 1) / length(valores)  #  $P(X = 1)$ 
## [1] 0.383
mean(valores)                        #  $E(X)$ 
## [1] 1.4897
```

E, para a moeda enviesada, com $X \sim \text{Binomial}(3, 3/4)$:

```
set.seed(123)
valores <- rbinom(10000, size = 3, prob = 3/4)
sum(valores == 1) / length(valores)
## [1] 0.1365
mean(valores)
## [1] 2.2558
```

Exemplo 2 (tempo até ao autocarro). O tempo até à chegada de um autocarro segue uma distribuição exponencial de média 30 minutos. Estimemos por simulação a probabilidade de o autocarro chegar nos primeiros 20 minutos e comparemos com o valor exato dado por `pexp()`:

```
set.seed(123)
valores <- rexp(10000, rate = 1/30)
sum(valores < 20) / length(valores)  # estimativa de  $P(X < 20)$ 
## [1] 0.4832
pexp(20, rate = 1/30)                # valor exato
## [1] 0.4865829
```

Exemplo 3 (cartas até ao primeiro ás). Retiram-se cartas de um baralho, com reposição, até sair um ás. Simulemos a média e a variância do número de cartas necessárias:

```
set.seed(123)
n <- 10000
simlist <- numeric(n)
for (i in 1:n) {
  ct <- 0
  as <- 0
  while (as == 0) {
    carta <- sample(1:52, 1, replace = TRUE)  # ases: 1, 2, 3, 4
    ct <- ct + 1
    if (carta <= 4) as <- 1
  }
  simlist[i] <- ct
}
```

```

}
  simlist[i] <- ct
}
mean(simlist)
## [1] 12.8081
var(simlist)
## [1] 147.5318

```

O número de provas de Bernoulli até ao primeiro sucesso (aparecer um ás, com $p = 4/52$) segue uma distribuição **geométrica**. Como veremos, o R conta os *insucessos* até ao primeiro sucesso, pelo que somamos 1 para obter o número de cartas:

```

set.seed(123)
valores <- rgeom(10000, prob = 4/52) + 1
mean(valores)
## [1] 13.0108
var(valores)
## [1] 152.0335

```

20.2 Função de distribuição empírica

A **função de distribuição empírica** é a função de distribuição construída a partir dos *dados observados*: para cada valor x , dá a proporção de observações da amostra que são menores ou iguais a x . É uma ferramenta natural para visualizar uma distribuição amostral e compará-la com um modelo teórico.

Formalmente, para uma amostra de dimensão n ,

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(X_i \leq x),$$

onde $\mathbb{I}(X_i \leq x)$ é a função indicadora, que vale 1 se $X_i \leq x$ e 0 caso contrário. Ordenando os dados por ordem crescente, $x_{(1)} \leq \dots \leq x_{(n)}$, a função é uma escada que vale 0 antes de $x_{(1)}$, sobe $\frac{1}{n}$ em cada observação (ou $\frac{k}{n}$ se um valor se repetir k vezes) e chega a 1 em $x_{(n)}$.

20.2.1 A função `ecdf()`

No R, a função de distribuição empírica calcula-se com `ecdf()`:

```
dados <- c(3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)

Fn <- ecdf(dados) # cria a função de distribuição empírica
plot(Fn, main = "Função de distribuição empírica",
     xlab = "x", ylab = "Fn(x)", col = "blue", lwd = 2)
```

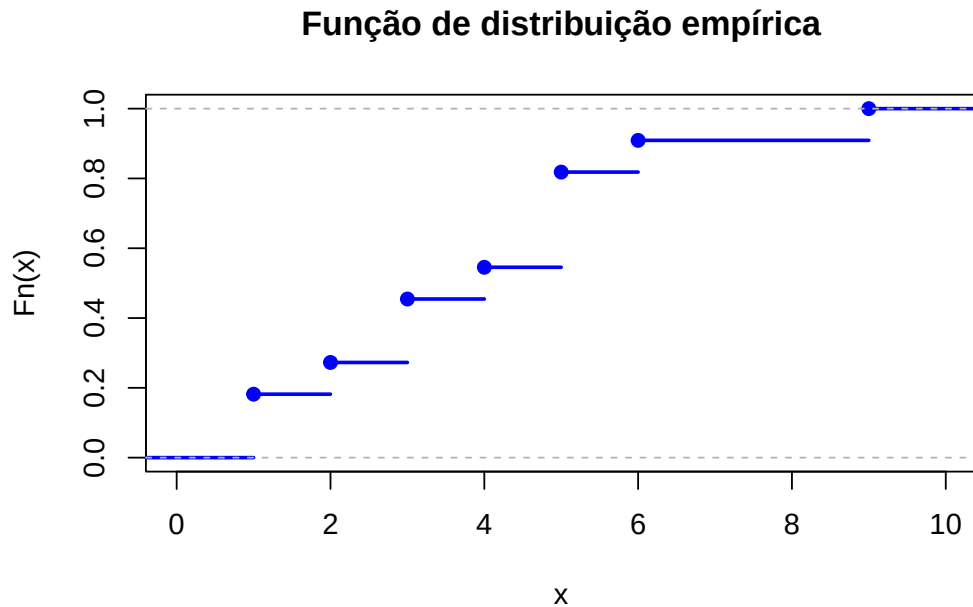


Figura 20.1: Função de distribuição empírica de um pequeno conjunto de dados.

O objeto devolvido por `ecdf()` é, ele próprio, uma função: `Fn(20)` devolve a proporção de observações ≤ 20 . Retomando o Exemplo 2, podemos estimar $P(X \leq 20)$ e compará-la com o valor exato:

```
set.seed(123)
valores <- rexp(10000, rate = 1/30)
Fn <- ecdf(valores)
Fn(20) # estimativa de P(X <= 20)
## [1] 0.4832
pexp(20, rate = 1/30) # valor exato
## [1] 0.4865829
```

20.3 Modelos discretos

Um modelo probabilístico discreto descreve fenômenos em que a variável aleatória assume apenas valores isolados — num conjunto finito ou numerável — e especifica a probabilidade de cada um. Vejamos os principais.

20.4 Distribuição uniforme discreta

Definição. A variável aleatória X tem **distribuição uniforme discreta** no conjunto $\{x_1, x_2, \dots, x_n\}$ se todos os valores forem igualmente prováveis:

$$P(X = x) = \begin{cases} \frac{1}{n}, & x \in \{x_1, \dots, x_n\} \\ 0, & \text{caso contrário} \end{cases}$$

Notação e momentos. $X \sim \text{Uniforme Discreta}(\{x_1, \dots, x_n\})$, com

$$E(X) = \frac{1}{n} \sum_{i=1}^n x_i, \quad V(X) = \frac{1}{n} \sum_{i=1}^n x_i^2 - \left(\frac{1}{n} \sum_{i=1}^n x_i \right)^2.$$

No caso particular, muito comum, em que os valores são os inteiros consecutivos $\{1, 2, \dots, n\}$, estas fórmulas simplificam-se para

$$E(X) = \frac{n+1}{2}, \quad V(X) = \frac{n^2-1}{12}.$$

Atenção, porém: estas versões simplificadas só valem para $\{1, \dots, n\}$. Para um conjunto qualquer de valores, há que usar as fórmulas gerais acima.

O R não tem uma função dedicada a esta distribuição — dada a sua simplicidade —, mas a `sample()` gera-a diretamente. Por exemplo, 15 valores de uma uniforme discreta em $\{1, \dots, 10\}$:

```
sample(1:10, size = 15, replace = TRUE)
## [1] 2 3 4 4 4 5 7 9 4 2 6 7 1 6 9
```

20.4.1 Exercícios

1. Crie uma variável aleatória uniforme discreta X com valores de 1 a 10.

- Simule 1000 realizações de X .
- Verifique se a frequência relativa de cada valor se aproxima da probabilidade teórica $P(X = x) = \frac{1}{10}$.
- Represente os resultados num gráfico de barras.

2. Defina X uniforme discreta com valores de 5 a 15.

- Calcule $E(X)$ e $V(X)$.
- Simule 10 000 realizações e compare a média e a variância empíricas com as teóricas.

3. Considere duas variáveis uniformes discretas independentes X e Y , com valores de 1 a 6 (um par de dados).

- Gere 5000 pares (X, Y) .
- Calcule a média e a variância da soma $Z = X + Y$.
- Estime $P(Z > 8)$.

4. Defina X uniforme discreta com valores de -3 a 3 .

- Crie a variável $Y = X^2 + 2X$.
- Simule 5000 valores de X e calcule $E(Y)$ e $V(Y)$.

5. Defina X uniforme discreta com valores de -5 a 5 .

- Crie $Y = 3X^3 - 2X^2 + X$.
- Simule 5000 valores de X e calcule a média empírica de Y e a proporção de valores de Y positivos.

20.5 Distribuição de Bernoulli

Prova de Bernoulli. Uma experiência diz-se uma **prova de Bernoulli** se tiver apenas dois resultados: um sucesso, com probabilidade p ($0 \leq p \leq 1$), e um insucesso, com probabilidade $1 - p$. Lançar uma moeda, verificar se uma peça é defeituosa ou se uma perfuração encontra petróleo são exemplos típicos.

Definição. A variável X que conta o número de sucessos numa prova de Bernoulli tem **distribuição de Bernoulli** de parâmetro p , com f.m.p.

$$P(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0, 1\}.$$

Notação e momentos. $X \sim \text{Bernoulli}(p)$, com $E(X) = p$ e $V(X) = p(1 - p)$.

20.5.1 Cálculo de probabilidades

A Bernoulli é o caso $n = 1$ da binomial, pelo que se usam as funções `*binom` com `size = 1`. Seja $X \sim \text{Bernoulli}(p = 0,5)$:

- $P(X = 0) \rightarrow \text{dbinom}(0, \text{size} = 1, \text{prob} = 0.5) = 0,5$
- $P(X = 1) \rightarrow \text{dbinom}(1, \text{size} = 1, \text{prob} = 0.5) = 0,5$
- $P(X \leq 1) \rightarrow \text{pbinom}(1, \text{size} = 1, \text{prob} = 0.5) = 1$
- $P(X > 0) \rightarrow \text{pbinom}(0, \text{size} = 1, \text{prob} = 0.5, \text{lower.tail} = \text{FALSE}) = 0,5$
- amostra de dimensão 5 $\rightarrow \text{rbinom}(5, \text{size} = 1, \text{prob} = 0.5)$

20.5.2 Exercícios

1. Considere o lançamento de uma moeda equilibrada e a variável $X =$ “número de caras em 1 lançamento”.

- (a) Simule a experiência, determinando a percentagem de caras para $n_1 = 5$, $n_2 = 10$, $n_3 = 100$ e $n_4 = 1000$ lançamentos.
- (b) Para cada amostra, calcule a média e a variância e compare com $E(X)$ e $V(X)$.

2. Seja $X \sim \text{Bernoulli}$ com $P(X = 1) = 0,7$.

- Simule 1000 valores de X .
- Calcule a frequência relativa de $X = 1$ e $X = 0$ e compare com as probabilidades teóricas.

3. Defina $X \sim \text{Bernoulli}$ com $P(X = 1) = 0,4$.

- Simule 10 000 valores.
- Compare a média empírica com $E(X) = p$ e a variância empírica com $V(X) = p(1 - p)$.

4. Considere 5 variáveis $X_1, \dots, X_5$, cada uma Bernoulli com $p = 0,5$.

- Simule 5000 realizações de cada.
- Calcule a soma $S = X_1 + \dots + X_5$.
- Compare a frequência relativa de cada valor de S (de 0 a 5) com a binomial teórica, `dbinom(0:5, size = 5, prob = 0.5)`.

20.6 Distribuição binomial

Definição. A variável X que conta o número de sucessos em n provas de Bernoulli independentes, todas com probabilidade de sucesso p , tem **distribuição binomial** de parâmetros (n, p) , com f.m.p.

$$P(X = x) = \binom{n}{x} p^x (1 - p)^{n-x}, \quad x = 0, 1, \dots, n,$$

onde $\binom{n}{x} = \frac{n!}{x!(n-x)!}$ é o número de combinações (em R, `choose(n, x)`).

Notação e momentos. $X \sim \text{Binomial}(n, p)$, com $E(X) = np$ e $V(X) = np(1 - p)$.

20.6.1 Cálculo de probabilidades

Seja $X \sim \text{Binomial}(n = 20, p = 0,1)$:

- $P(X = 4) \rightarrow \text{dbinom}(4, \text{size} = 20, \text{prob} = 0.1) = 0,0898$
- $P(X \leq 4) \rightarrow \text{pbinom}(4, \text{size} = 20, \text{prob} = 0.1) = 0,9568$
- $P(X > 4) \rightarrow \text{pbinom}(4, \text{size} = 20, \text{prob} = 0.1, \text{lower.tail} = \text{FALSE}) = 0,0432$
- amostra de dimensão 5 $\rightarrow \text{rbinom}(5, \text{size} = 20, \text{prob} = 0.1)$

20.6.2 Função massa de probabilidade (teórica)

```
n <- 20; p <- 0.1
x <- 0:20
teorico <- data.frame(x = x, y = dbinom(x, size = n, prob = p))

plot(teorico$x, teorico$y,
     main = "Binomial(n = 20, p = 0.1)",
     xlab = "Número de sucessos", ylab = "Probabilidade",
     pch = 19, col = "blue")
grid(nx = 21, ny = NULL)
```

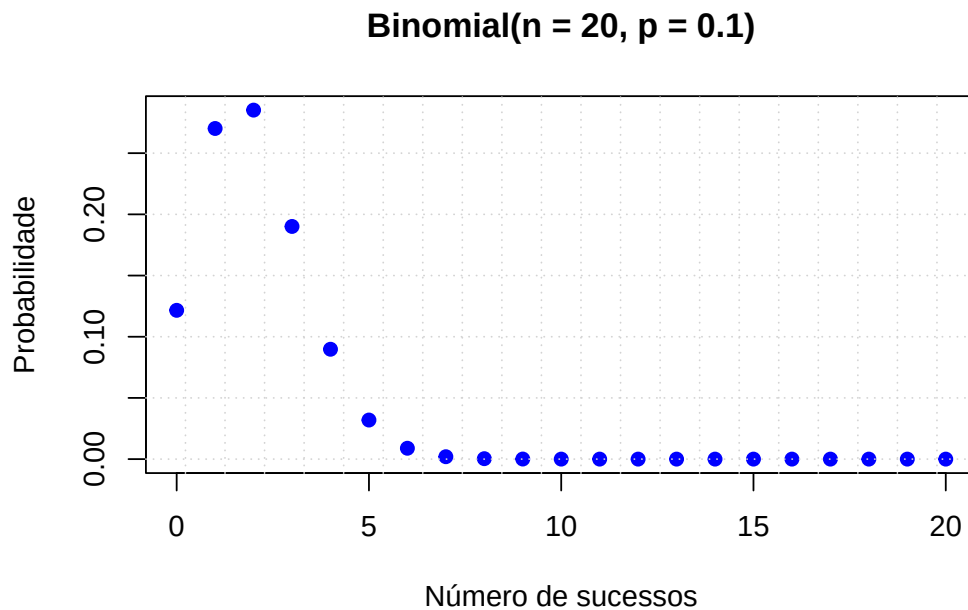


Figura 20.2: Função massa de probabilidade da Binomial(20, 0.1).

20.6.3 Função massa de probabilidade (simulação)

```
set.seed(1234)
n <- 20; p <- 0.1; k <- 1000          # k = número de simulações

dados <- rbinom(k, size = n, prob = p)
frequencia_relativa <- table(dados) / length(dados)

barplot(frequencia_relativa,
        main = "Amostra da Binomial(20, 0.1)",
        col = "lightblue",
        xlab = "Número de sucessos", ylab = "Frequência relativa",
        ylim = c(0, 0.3))
grid()
```

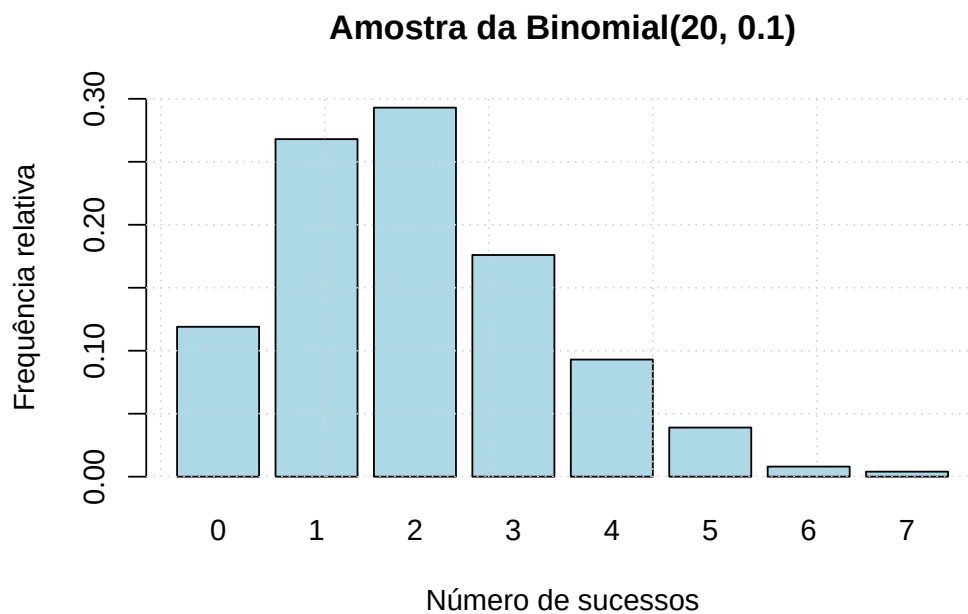


Figura 20.3: Frequências relativas de uma amostra de 1000 valores da Binomial(20, 0.1).

20.6.4 Comparação entre teoria e simulação

```
set.seed(1234)
n <- 20; p <- 0.1; k <- 1000

dados <- rbinom(k, size = n, prob = p)
```

```
frequencia_relativa <- table(factor(dados, levels = 0:n)) / length(dados)
teorico <- data.frame(x = 0:n, y = dbinom(0:n, size = n, prob = p))

posicoes_barra <- barplot(frequencia_relativa,
  main = "Amostra da Binomial(20, 0.1)",
  col = "lightblue",
  xlab = "Número de sucessos", ylab = "Frequência relativa",
  ylim = c(-0.01, 0.3))
points(posicoes_barra, teorico$y, col = "magenta", pch = 19)
grid()
```

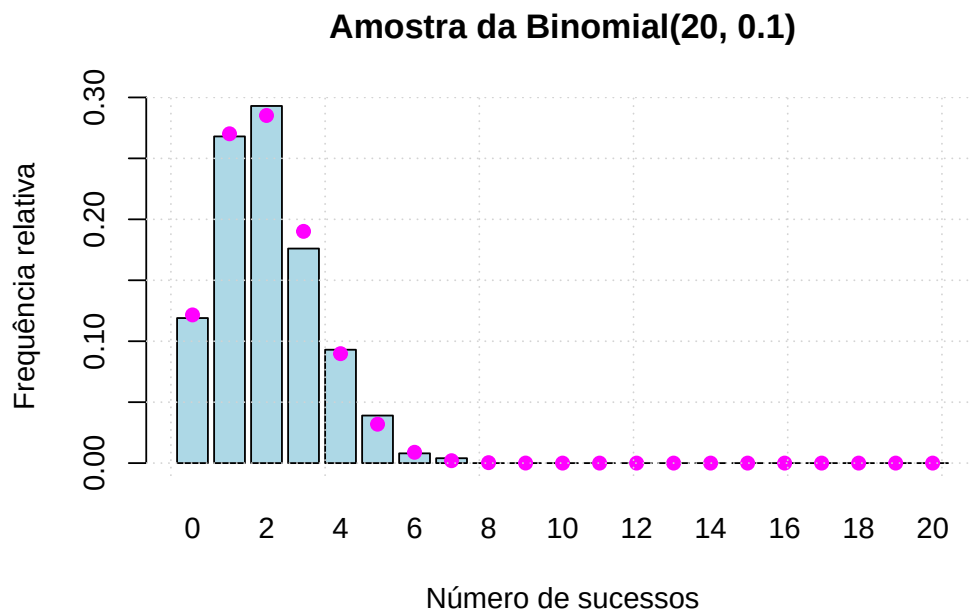


Figura 20.4: Frequências relativas simuladas (barras) e probabilidades teóricas (pontos).

20.6.5 Função de distribuição

```
n <- 20; p <- 0.1
x <- 0:n
cdf_values <- pbinom(x, size = n, prob = p)

plot(x, cdf_values, type = "s", lwd = 2, col = "blue",
  xlab = "Número de sucessos", ylab = "F(x)",
  main = "Função de distribuição da Binomial(20, 0.1)")
```

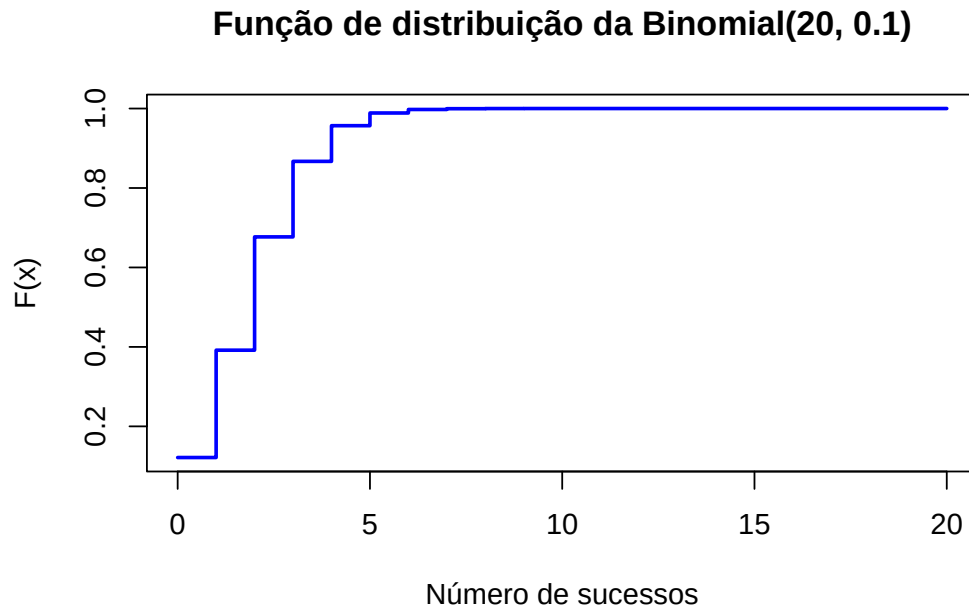


Figura 20.5: Função de distribuição da Binomial(20, 0.1).

20.6.6 Função de distribuição empírica

```
n <- 20; p <- 0.1
set.seed(1234)
amostra <- rbinom(1000, size = n, prob = p)

Fn <- ecdf(amostra)
plot(Fn, main = "Função de distribuição empírica",
     xlab = "x", ylab = "Fn(x)", col = "blue")
```

Assim, para $X \sim \text{Binomial}(20, 0.1)$, o valor teórico $P(X \leq 4) = \text{pbinom}(4, 20, 0.1) = 0.9568$ é bem aproximado pela estimativa empírica $\text{Fn}(4) \approx 0.956$.

20.6.7 Exercícios

1. Considere 10 lançamentos de uma moeda equilibrada ($p = 0.5$).
 - (a) Calcule a probabilidade de obter exatamente 6 caras.
 - (b) Calcule a probabilidade de obter no máximo 4 caras.
 - (c) Calcule a probabilidade de obter mais de 7 caras.
2. Um dado equilibrado é lançado 12 vezes; o sucesso é “sair 6” ($p = 1/6$).

Função de distribuição empírica

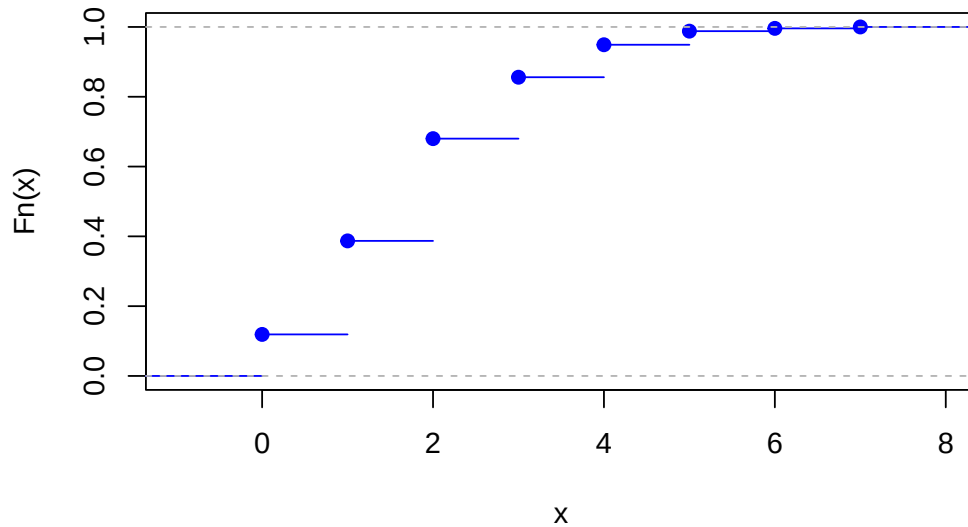


Figura 20.6: Função de distribuição empírica de uma amostra da Binomial(20, 0.1).

- Calcule a probabilidade de, no máximo, cinco 6.
 - Gere uma amostra de 1000 experiências e registre o número de sucessos em cada uma.
 - Faça um gráfico de barras das frequências relativas e sobreponha a distribuição teórica de X .
 - Use a função de distribuição empírica para estimar a probabilidade da alínea (a) e compare com o valor teórico.
 - Calcule a média e a variância da amostra.
 - Compare com os valores teóricos $E(X) = np$ e $V(X) = np(1 - p)$.
- 3.** Considere $X =$ “número de caras em 7 lançamentos” de uma moeda equilibrada.
- Calcule a probabilidade de sair 2 vezes cara.
 - Simule a experiência para $n_1 = 5$, $n_2 = 10$, $n_3 = 100$ e $n_4 = 1000$ repetições; para cada caso, determine a percentagem de vezes em que saíram 3 caras.
 - Para cada amostra, calcule a média e a variância e compare com $E(X)$ e $V(X)$.
- 4.** Um teste de escolha múltipla tem 10 questões, cada uma com 4 alternativas e apenas uma correta ($p = 0,25$).
- Calcule a probabilidade de acertar exatamente k questões, para $k = 0, \dots, 10$.
 - Represente graficamente a distribuição de probabilidades.
 - Identifique o valor de k mais provável.

5. Uma equipa de basquetebol acerta cada lance livre com probabilidade 0,6 e tenta 15 lances num jogo.

- (a) Calcule a probabilidade de acertar exatamente 9.
- (b) Calcule a probabilidade de acertar entre 8 e 12 (inclusive).
- (c) Gere 500 jogos e estime a proporção em que a equipa acerta entre 8 e 12.

6. Num processo de fabrico, X é o número de peças defeituosas num lote de 40, com $p = 0,05$ de uma peça ser defeituosa.

- (a) Com `set.seed(123)`, gere 10 000 observações de X .
- (b) Conte os lotes com exatamente 2 peças defeituosas.
- (c) Compare a proporção correspondente com $P(X = 2)$ para $X \sim \text{Binomial}(40, 0,05)$.

7. Numa loja, cada cliente compra com probabilidade $p = 0,3$; entram 25 clientes. Seja X o número de compradores.

- (a) Com `set.seed(456)`, gere 5000 observações de X .
- (b) Conte os períodos com pelo menos 10 compradores.
- (c) Compare a proporção com $P(X \geq 10)$ para $X \sim \text{Binomial}(25, 0,3)$.
- (d) Use a função de distribuição empírica para estimar essa probabilidade e compare.
- (e) Calcule o número médio de compradores na amostra.
- (f) Compare a média amostral com $E(X)$.

8. O número de acertos num alvo em 30 tentativas, com probabilidade de acerto 0,4, é $X \sim \text{Binomial}(30, 0,4)$. Com `set.seed(123)`, gere uma amostra de dimensão 700.

- (a) Faça um gráfico de barras das frequências relativas e sobreponha a distribuição teórica de X .
- (b) Com a função de distribuição empírica, estime $P(X > 15)$ e calcule também o valor teórico.

20.7 Distribuição geométrica

Definição. A variável $X =$ “número de provas de Bernoulli independentes (com probabilidade de sucesso p) realizadas até ao primeiro sucesso” tem **distribuição geométrica** de parâmetro p , com f.m.p.

$$P(X = x) = p(1 - p)^{x-1}, \quad x = 1, 2, 3, \dots$$

Notação e momentos. $X \sim \text{Geométrica}(p)$, com $E(X) = \frac{1}{p}$ e $V(X) = \frac{1-p}{p^2}$.

Há uma segunda forma, equivalente, de definir a distribuição — e é a que o R adota.

Definição (convenção do R). A variável $Y = X - 1$ “número de *insucessos* antes do primeiro sucesso” tem f.m.p.

$$P(Y = y) = p(1 - p)^y, \quad y = 0, 1, 2, \dots,$$

$$\text{com } E(Y) = \frac{1-p}{p} \text{ e } V(Y) = \frac{1-p}{p^2}.$$

Esta é uma fonte frequente de erros. As funções `dgeom()`, `pgeom()` e `rgeom()` do R usam a convenção de Y — o **número de insucessos** antes do primeiro sucesso, com suporte $0, 1, 2, \dots$ — e não o número de provas X , com suporte $1, 2, \dots$. A relação é simples, $X = Y + 1$: para obter o número de provas, some 1 ao resultado de `rgeom()`; e para calcular $P(X = k)$, use `dgeom(k - 1, p)`.

20.7.1 Cálculo de probabilidades

Nas funções do R (convenção Y , número de insucessos), seja $Y \sim \text{Geométrica}(p = 0,5)$:

- $P(Y = 0) \rightarrow \text{dgeom}(0, \text{prob} = 0.5) = 0,5$
- $P(Y = 1) \rightarrow \text{dgeom}(1, \text{prob} = 0.5) = 0,25$
- $P(Y \leq 1) \rightarrow \text{pgeom}(1, \text{prob} = 0.5) = 0,75$
- $P(Y > 1) \rightarrow \text{pgeom}(1, \text{prob} = 0.5, \text{lower.tail} = \text{FALSE}) = 0,25$

Exemplo. Seja X o número de lançamentos de um dado equilibrado até sair a primeira face 2. Qual a probabilidade de a face 2 sair no terceiro lançamento? Como no R a geométrica conta insucessos, $P(X = 3) = P(Y = 2)$:

```
dgeom(2, prob = 1/6)      # P(X = 3) = P(Y = 2)
## [1] 0.09645062
```

20.7.2 Exercícios

1. Um dado equilibrado é lançado até sair um 6 ($p = 1/6$).

- Simule 1000 experiências e registre o número de tentativas em cada uma.
- Compare as frequências relativas com as probabilidades teóricas (`dgeom()`, atendendo à convenção do R).

2. Com `set.seed(123)`, simule 5000 valores de uma geométrica com $p = 0,2$.

- Estime $P(\text{número de tentativas} \leq 5)$.
 - Compare com o valor teórico (`pgeom()`).
3. Gere a distribuição geométrica para $p = 0,1$, $p = 0,5$ e $p = 0,9$.
- Faça gráficos de barras para comparar.
 - Descreva como p afeta a forma e o decaimento da distribuição.
4. Defina uma geométrica com $p = 0,3$.
- Com `set.seed(123)`, simule 10 000 valores e calcule a média e a variância empíricas.
 - Compare com $E(X) = \frac{1-p}{p}$ e $V(X) = \frac{1-p}{p^2}$ (convenção do R).
5. Numa sondagem, 80% das pessoas concordam com uma afirmação ($p = 0,8$).
- Simule o número de entrevistas até encontrar uma pessoa que discorde ($1 - p = 0,2$).
 - Em 5000 simulações, calcule a média e a variância do número de entrevistas.
 - Visualize a distribuição empírica.
6. Numa central de apoio técnico, cada tentativa resolve o problema com $p = 0,3$.
- (a) Qual a probabilidade de o resolver em no máximo 5 tentativas?
 - (b) Qual a probabilidade de precisar de mais de 8 tentativas?
 - (c) Simule 500 casos com `rgeom()` e estime a frequência relativa de casos com ≤ 5 tentativas; compare com (a).
 - (d) Calcule a média e o desvio padrão da amostra e compare com $\mu = \frac{1-p}{p}$ e $\sigma = \sqrt{\frac{1-p}{p^2}}$.
 - (e) Compare graficamente a distribuição teórica com a frequência relativa observada.
7. Num sistema de autenticação biométrica, cada tentativa tem $p = 0,2$ de sucesso. O tempo de processamento associado a X tentativas é $T = 2X^2 + 3X + 5$ (ms).
- (a) Simule 1000 valores de $X \sim \text{Geom}(0,2)$.
 - (b) Calcule o vetor de tempos T .
 - (c) Faça um histograma dos tempos.
 - (d) Calcule a média e o desvio padrão de T .
 - (e) Estime a probabilidade de um acesso demorar mais de 150 ms.
8. Um medicamento elimina uma célula-alvo com $p = 0,1$ por tentativa. Seja X o número de falhas antes do sucesso; o custo de um ensaio é $C = 0,5X^2 + 20$ (euros).

- (a) Simule 5000 ensaios, com $X \sim \text{Geom}(0,1)$.
- (b) Calcule o vetor de custos C .
- (c) Faça uma caixa de bigodes dos custos.
- (d) Calcule a média e o desvio padrão dos custos, e a percentagem de ensaios com custo superior a 200 € e inferior a 50 €.

9. Num sistema de alta disponibilidade, cada servidor está operacional com probabilidade 0,1; procura-se sequencialmente até encontrar um funcional. O tempo de resposta é $T = \sqrt{X^2 + 3X + 1} + \log(X + 1)$ (ms).

- (a) Simule 3000 valores de $X \sim \text{Geom}(0,1)$.
- (b) Calcule T para cada valor.
- (c) Faça um gráfico de dispersão entre X e T .
- (d) Calcule a média, a mediana e o desvio padrão de T .
- (e) Estime a probabilidade de $T > 10$ ms.
- (f) Calcule os quantis 25%, 50% e 75% de T .

20.8 Distribuição de Poisson

Considere-se a contagem do número de ocorrências de um acontecimento num intervalo (de tempo, comprimento, área, volume) que verifica três propriedades: as ocorrências em intervalos disjuntos são independentes (ausência de memória); a probabilidade de ocorrência é a mesma para intervalos de igual amplitude; e a probabilidade de mais de uma ocorrência num intervalo muito pequeno é desprezável. Uma tal experiência é um **processo de Poisson**.

Definição. A variável $X =$ “número de ocorrências por unidade de tempo ou espaço” tem **distribuição de Poisson** de parâmetro $\lambda > 0$, com f.m.p.

$$P(X = x) = \frac{e^{-\lambda} \lambda^x}{x!}, \quad x = 0, 1, 2, \dots$$

Notação e momentos. $X \sim \text{Poisson}(\lambda)$, com $E(X) = V(X) = \lambda$ (o parâmetro é o número médio de ocorrências por unidade).

Aditividade. Se $X_1, \dots, X_n$ são independentes, com $X_i \sim \text{Poisson}(\lambda_i)$, então $\sum_i X_i \sim \text{Poisson}(\sum_i \lambda_i)$. Esta propriedade é o que permite, por exemplo, mudar a unidade de tempo: se ocorrem em média 3 acontecimentos por mês, ocorrem 3/2 por quinzena — a mesma família, com o parâmetro reescalado.

20.8.1 Cálculo de probabilidades

Seja $X \sim \text{Poisson}(\lambda = 5)$:

- $P(X = 4) \rightarrow \text{dpois}(4, 5) = 0,1755$
- $P(X \leq 4) \rightarrow \text{ppois}(4, 5) = 0,4405$
- $P(X > 4) \rightarrow \text{ppois}(4, 5, \text{lower.tail} = \text{FALSE}) = 0,5595$

20.8.2 Função massa de probabilidade (teórica)

O painel seguinte mostra como a forma da Poisson muda com λ — assimétrica para valores pequenos, cada vez mais simétrica à medida que λ cresce. (Usa os pacotes `ggplot2`, `latex2exp` e `gridExtra`.)

```
library(ggplot2); library(latex2exp); library(gridExtra)

lambdas <- c(0.1, 1, 2.5, 5, 15, 30)
x <- 0:50
plots <- list()
for (i in seq_along(lambdas)) {
  teorico <- data.frame(x = x, y = dpois(x, lambda = lambdas[i]))
  plots[[i]] <- ggplot(teorico) +
    geom_point(aes(x = x, y = y), color = "blue") +
    scale_x_continuous(breaks = seq(0, 50, by = 10)) +
    labs(title = TeX(paste0("$Poisson(\\lambda=", lambdas[i], ")$")),
         x = "x", y = "Probabilidade") +
    theme_light()
}
grid.arrange(grobs = plots, nrow = 2, ncol = 3)
```

20.8.3 Função massa de probabilidade (simulação)

```
set.seed(1234)
lambdas <- c(0.1, 1, 2.5, 5, 15, 30)
n <- 1000
plots <- list()
for (i in seq_along(lambdas)) {
  dados <- data.frame(X = rpois(n, lambda = lambdas[i]))
  plots[[i]] <- ggplot(dados) +
    geom_bar(aes(x = X, y = after_stat(prop)), fill = "lightblue") +
    labs(title = TeX(paste0("$Poisson(\\lambda=", lambdas[i], ")$")),
         x = "x", y = "Frequência relativa") +
    theme_light()
}
```

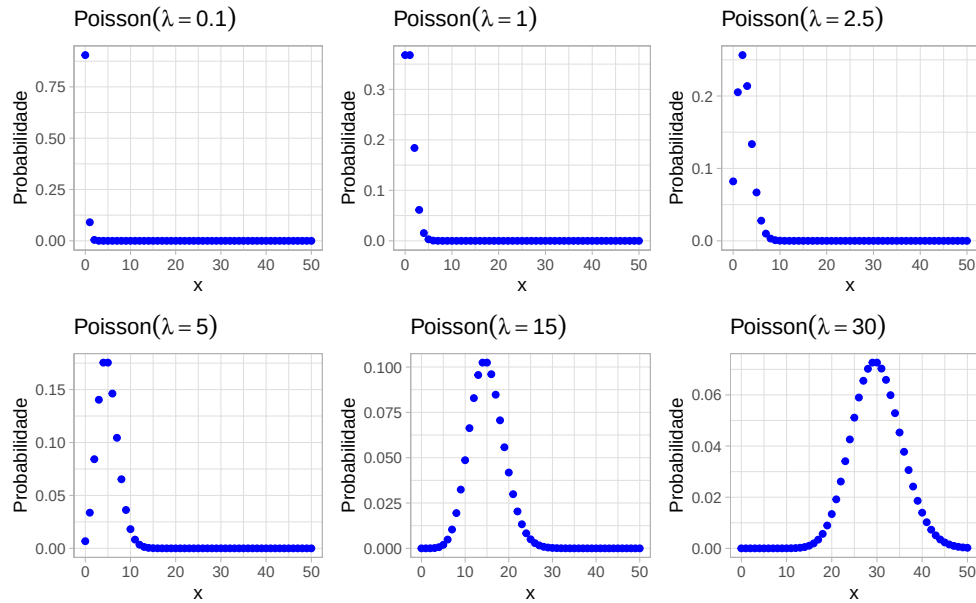


Figura 20.7: Função massa de probabilidade da Poisson para vários valores de lambda.

```
}
grid.arrange(grobs = plots, nrow = 2, ncol = 3)
```

20.8.4 Função de distribuição

```
lambda <- 5
x <- 0:15
y <- ppois(x, lambda = lambda)

plot(x, y, type = "s", lwd = 2, col = "blue",
     main = "Função de distribuição da Poisson(5)",
     xlab = "x", ylab = "F(x)")
```

Exemplo. Numa região ocorrem, em média, 3 sismos por mês; o número mensal de sismos segue uma Poisson. Sendo $X \sim \text{Poisson}(\lambda = 3)$:

```
dpois(2, lambda = 3)           #  $P(X = 2)$ 
## [1] 0.2240418
ppois(4, lambda = 3, lower.tail = FALSE) #  $P(X > 4)$ 
## [1] 0.1847368
```

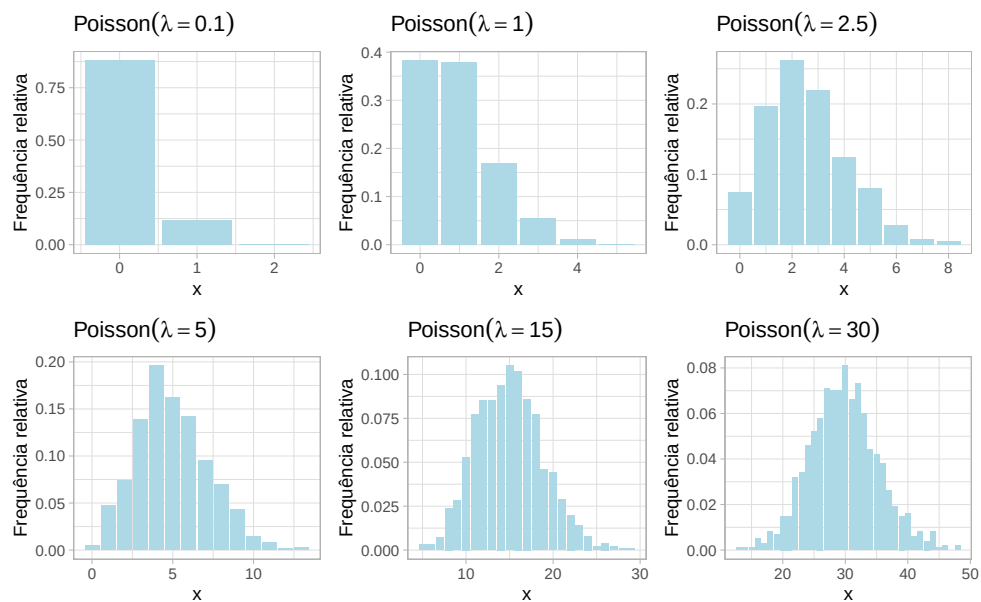


Figura 20.8: Amostras simuladas da Poisson para vários lambda.

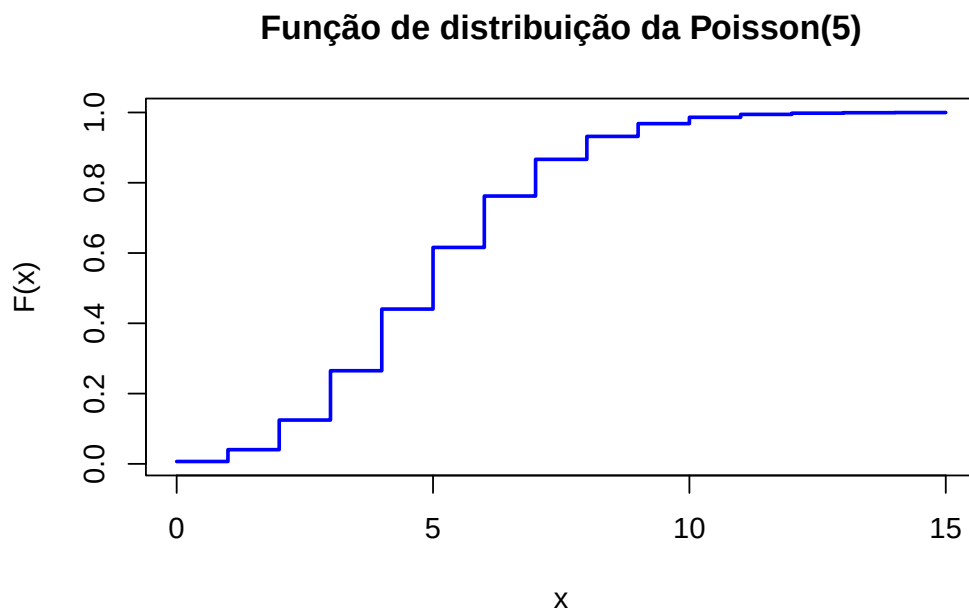


Figura 20.9: Função de distribuição da Poisson(5).

Para a probabilidade de pelo menos 1 sismo em **2 semanas**, usamos a aditividade: em 2 semanas (meio mês) o parâmetro é $\lambda = 1,5$, e $P(\tilde{X} \geq 1) = 1 - P(\tilde{X} = 0)$:

```
1 - dpois(0, lambda = 1.5)           # P(X >= 1) em 2 semanas
## [1] 0.7768698
```

20.8.5 Exercícios

- Uma fábrica produz, em média, 4 defeitos por dia (Poisson).
 - Qual a probabilidade de exatamente 5 defeitos num dia?
 - Qual a probabilidade de 3 ou menos defeitos num dia?
- Simule o número de defeitos em 30 dias com `rpois()` e $\lambda = 4$.
 - Gere uma amostra de dimensão 30.
 - Calcule a média e o desvio padrão da amostra.
 - Compare com os valores teóricos.
- Uma linha de apoio recebe, em média, 20 pedidos por cada 10 minutos (Poisson).
 - Calcule a probabilidade de, em 10 minutos, chegarem exatamente 20 pedidos.
 - Simule para $n_1 = 5$, $n_2 = 10$, $n_3 = 100$ e $n_4 = 1000$ repetições; para cada caso, determine a percentagem de vezes em que chegam exatamente 20 pedidos.
 - Para cada amostra, calcule a média e a variância e compare com $E(X)$ e $V(X)$.
- Uma loja recebe, em média, 12 clientes por hora (Poisson).
 - Qual a probabilidade de receber no máximo 10 clientes numa hora?
 - Qual a probabilidade de receber mais de 15?
 - Simule 500 horas: (i) gere a amostra; (ii) calcule a frequência relativa de horas com ≤ 10 clientes e compare com (a).
 - Calcule a média e o desvio padrão da amostra e compare com os teóricos ($\lambda = 12$).
 - Compare graficamente a distribuição teórica com a frequência relativa observada.
- Numa área de conservação, avistam-se em média 8 aves por hora ($\lambda = 8$).
 - Simule o número de aves em 1000 horas.
 - Construa a função de distribuição empírica (`ecdf()`).
 - Compare-a com a função de distribuição teórica (`ppois()`).

6. Com `set.seed(543)`, gere 2400 observações de $Y \sim \text{Poisson}(\lambda = 6)$.
- Faça um histograma de frequências relativas e sobreponha a distribuição de Y .
 - Com a função de distribuição empírica, estime $P(Y > 5)$ e compare com o teórico.
7. Para $\lambda = 5$, represente $P(X = x)$ para x de 0 a 15.
- Use `dpois()` para as probabilidades.
 - Faça um gráfico de barras.
8. Para $\lambda = 50$, use a aproximação normal:
- Calcule $P(X \geq 55)$ com a Poisson.
 - Calcule a mesma probabilidade com a aproximação $N(\mu = 50, \sigma^2 = 50)$.
 - Compare os resultados.
9. O número de acidentes por dia numa autoestrada segue uma Poisson com $\lambda = 2$.
- Simule 1000 amostras de dimensão 30.
 - Calcule a média de cada amostra.
 - Faça o histograma das médias amostrais e sobreponha a densidade de uma normal de média $\mu = 2$ e desvio padrão $\sigma = \sqrt{\lambda/n}$.
10. Num hospital, atendem-se em média 5 pacientes por hora ($\lambda = 5$). Com `set.seed(456)`:
- Simule 1000 amostras de dimensão 50, 100 e 1000.
 - Para cada dimensão, calcule a média de cada amostra.
 - Faça o histograma das médias e sobreponha a densidade normal de média λ e desvio padrão $\sqrt{\lambda/n}$.
 - Comente a aproximação à normal à medida que n cresce, relacionando com o Teorema do Limite Central.

20.9 Distribuição uniforme contínua

Definição. A variável contínua X tem **distribuição uniforme** no intervalo (a, b) , com $a < b$, se a sua densidade for constante nesse intervalo:

$$f_X(x) = \begin{cases} \frac{1}{b-a}, & a \leq x \leq b \\ 0, & \text{caso contrário} \end{cases}, \quad F_X(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x < b \\ 1, & x \geq b \end{cases}$$

Notação e momentos. $X \sim \text{Uniforme}(a, b)$, com $E(X) = \frac{a+b}{2}$ e $V(X) = \frac{(b-a)^2}{12}$.

20.9.1 Cálculo de probabilidades

Seja $X \sim \text{Uniforme}(0, 1)$:

- $P(X \leq 0,5) \rightarrow \text{punif}(0.5, \text{min} = 0, \text{max} = 1) = 0,5$
- $P(X > 0,5) \rightarrow \text{punif}(0.5, \text{min} = 0, \text{max} = 1, \text{lower.tail} = \text{FALSE}) = 0,5$

20.9.2 Densidade, simulação e comparação

```
x_vals <- seq(0, 1, length.out = 100)
plot(x_vals, dunif(x_vals, 0, 1), type = "l", col = "red", lwd = 2,
     main = "Densidade da Uniforme(0, 1)", xlab = "x", ylab = "Densidade")
```

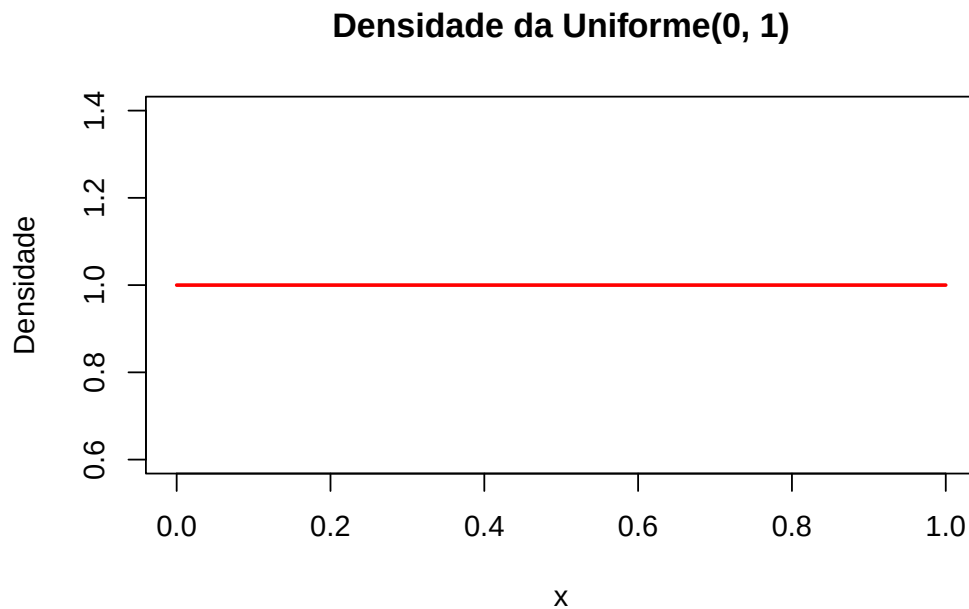


Figura 20.10: Densidade teórica da Uniforme(0, 1).

```
set.seed(123)
dados <- runif(10000, min = 0, max = 1)

hist(dados, probability = TRUE,
     main = "Amostra da Uniforme(0, 1)",
     xlab = "x", ylab = "Densidade",
     col = "lightblue", border = "darkblue")
curve(dunif(x, 0, 1), add = TRUE, col = "red", lwd = 2)
```

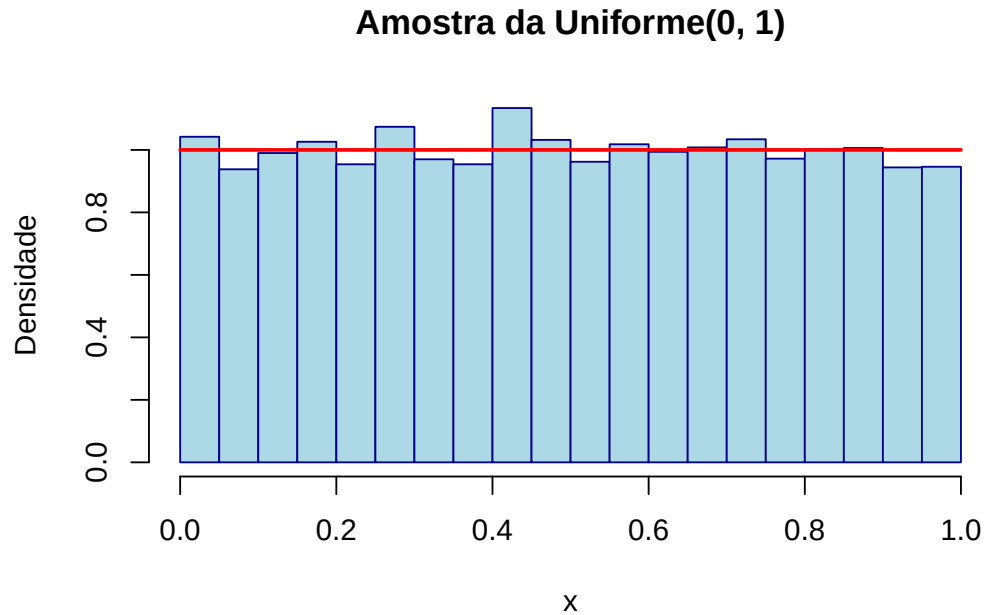



Figura 20.11: Histograma de uma amostra da Uniforme(0, 1) com a densidade teórica sobreposta.

```
x_vals <- seq(0, 1, length.out = 100)
plot(x_vals, punif(x_vals, 0, 1), type = "l", col = "blue", lwd = 2,
     main = "Função de distribuição da Uniforme(0, 1)", xlab = "x", ylab = "F(x)")
```

20.9.3 Exercícios

1. Simule 1000 valores de $X \sim U(0, 1)$.
 - (a) Calcule a média e a variância dos valores simulados.
 - (b) Compare com os valores teóricos ($E(X) = 0,5$, $V(X) = 1/12$).
2. Simule 500 valores de $X \sim U(-3, 7)$.
 - (a) Faça o histograma dos valores.
 - (b) Sobreponha a densidade teórica.
3. O peso de uma barra de chocolate (que deveria pesar 100 g) tem distribuição uniforme em $[85, 105]$ gramas.
 - (a) Qual a probabilidade de uma barra pesar menos de 100 g?

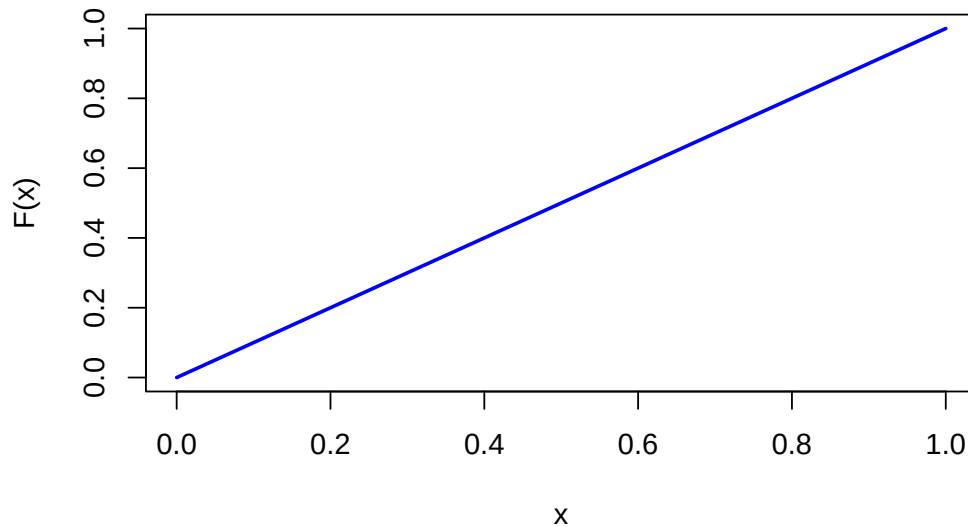
Função de distribuição da Uniforme(0, 1)

Figura 20.12: Função de distribuição da Uniforme(0, 1).

- (b) Simule para $n_1 = 5$, $n_2 = 10$, $n_3 = 100$ e $n_4 = 1000$; para cada caso, determine a percentagem de barras com peso inferior a 100 g.
4. Simule 10 000 valores de $X \sim U(2, 8)$.
- (a) Estime $P(X > 5)$.
- (b) Compare com o valor teórico (`punif()`).
5. Seja $Y \sim U(10, 20)$. Simule 1000 valores.
- (a) Calcule a proporção empírica de valores em $[12, 15]$.
- (b) Compare com o valor teórico (`punif()`).
6. Simule 1000 amostras de dimensão 30 de $X \sim U(-5, 5)$.
- (a) Calcule a média de cada amostra.
- (b) Faça o histograma das médias e sobreponha a densidade normal de média 0 e variância $\frac{(5-(-5))^2}{12 \cdot n}$.
7. Seja $Z = 3X + 2$, com $X \sim U(0, 1)$. Simule 1000 valores de X e transforme-os em Z .
- (a) Calcule a média e a variância de Z .
- (b) Compare com $E(Z) = 3E(X) + 2$ e $V(Z) = 9V(X)$.

8. Uma fábrica produz itens com peso uniforme em $[100, 120]$ g. Simule 2000 itens.

- (a) Calcule a proporção com peso inferior a 105 g.
- (b) Compare a densidade empírica com a teórica.

9. Com `set.seed(123)`, gere amostras de dimensão $n = 100, 1000, 10\,000, 100\,000$ de $W \sim U(0, 1)$. Para cada uma, calcule a média amostral e compare com o valor esperado teórico. Comente a convergência (Lei dos Grandes Números).

10. O tempo de entrega de um *drone* (minutos) segue $X \sim U(10, 30)$. Com `set.seed(1430)`, gere 8000 amostras de dimensão $n = 100$.

- (a) Calcule a soma de cada amostra, $S_n = \sum_{i=1}^n X_i$.
- (b) Faça o histograma das somas e sobreponha uma normal de média $nE(X)$ e desvio padrão $\sqrt{nV(X)}$.
- (c) Calcule a média de cada amostra, $\bar{X}_n$.
- (d) Faça o histograma das médias e sobreponha uma normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$.

11. Num *data center*, o número de servidores que falham por hora segue uma $\text{Poisson}(\lambda = 4)$; o tempo de reparação de cada um segue $U(2, 6)$ horas, e a energia gasta é $E_i = T_i^2$. Simule 1000 horas e, em cada uma, calcule o consumo total $E_{\text{total}} = \sum_{i=1}^N T_i^2$, onde N é o número de servidores falhados.

20.10 Distribuição exponencial

A distribuição exponencial modela **tempos** — a duração de um equipamento, o intervalo entre ocorrências consecutivas de um mesmo tipo de acontecimento (chegadas de clientes, avarias, colisões).

Definição. A variável contínua X tem **distribuição exponencial** de parâmetro $\lambda > 0$ se

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x}, & x > 0 \\ 0, & x \leq 0 \end{cases}, \quad F_X(x) = \begin{cases} 1 - e^{-\lambda x}, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

Notação e momentos. $X \sim \text{Exponencial}(\lambda)$, com $E(X) = \frac{1}{\lambda}$ e $V(X) = \frac{1}{\lambda^2}$.

A exponencial é a única distribuição contínua **sem memória**: $P(X > s+t \mid X > s) = P(X > t)$. Por palavras: se um componente já durou s horas, a probabilidade de durar mais t é a mesma que teria em novo. É esta propriedade que a liga naturalmente ao processo de Poisson — os intervalos entre ocorrências de uma $\text{Poisson}(\lambda)$ são exponenciais(λ).

20.10.1 Cálculo de probabilidades

Seja $X \sim \text{Exponencial}(\lambda = 1)$:

- $P(X \leq 0,5) \rightarrow \text{pexp}(0.5, \text{rate} = 1) = 0,3935$
- $P(X > 0,5) \rightarrow \text{pexp}(0.5, \text{rate} = 1, \text{lower.tail} = \text{FALSE}) = 0,6065$

20.10.2 Densidade, simulação e comparação

```
x_vals <- seq(0, 10, length.out = 100)
plot(x_vals, dexp(x_vals, rate = 1), type = "l", col = "red", lwd = 2,
     main = "Densidade da Exponencial(1)", xlab = "x", ylab = "Densidade")
```

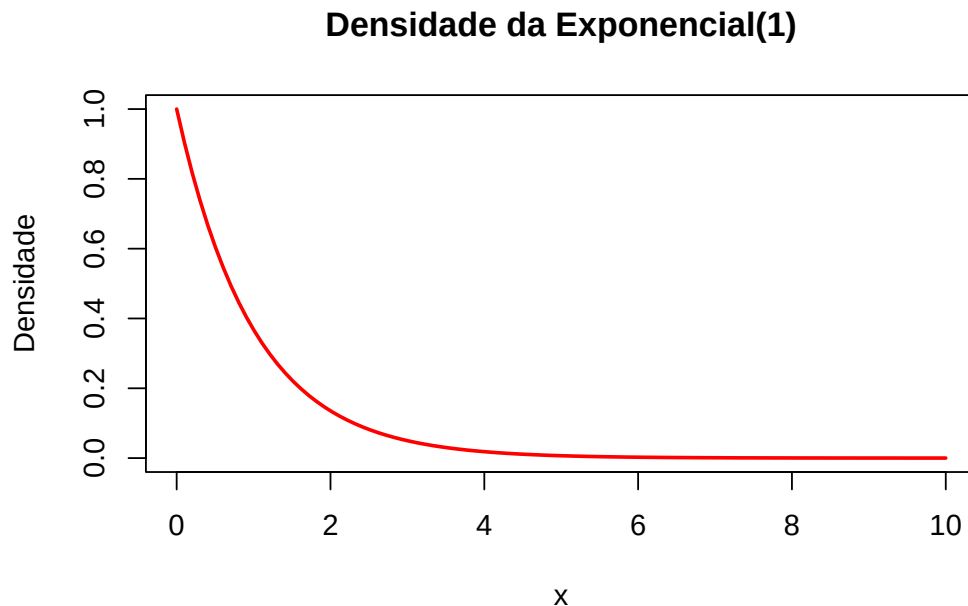


Figura 20.13: Densidade teórica da Exponencial(1).

```
set.seed(123)
dados <- rexp(10000, rate = 1)

hist(dados, probability = TRUE,
     main = "Amostra da Exponencial(1)",
     xlab = "x", ylab = "Densidade",
     col = "lightblue", border = "darkblue")
curve(dexp(x, rate = 1), add = TRUE, col = "red", lwd = 2)
```

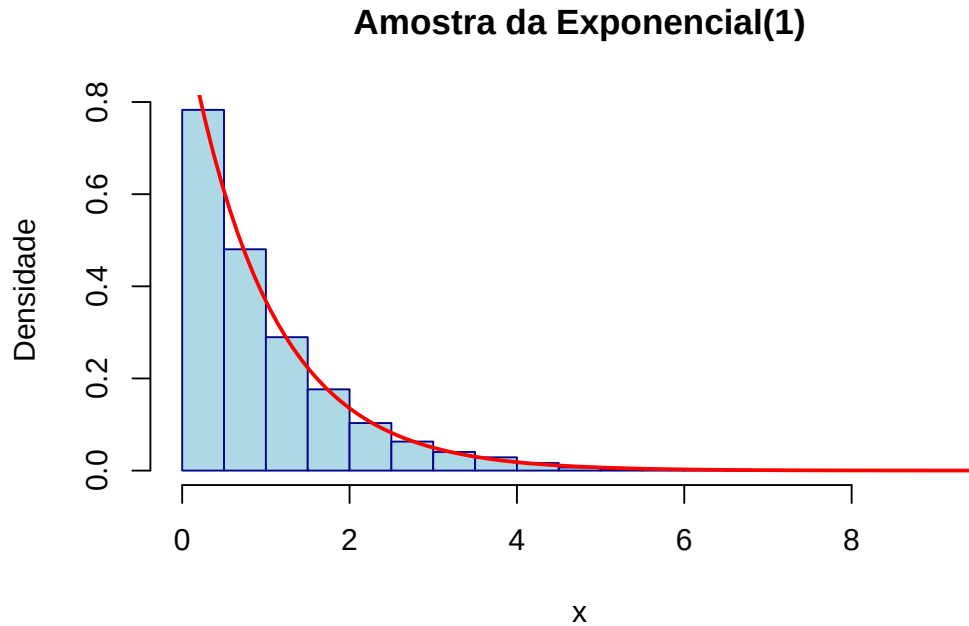


Figura 20.14: Histograma de uma amostra da Exponencial(1) com a densidade teórica sobreposta.

```
x_vals <- seq(0, 10, length.out = 100)
plot(x_vals, pexp(x_vals, rate = 1), type = "l", col = "red", lwd = 2,
     main = "Função de distribuição da Exponencial(1)", xlab = "x", ylab = "F(x)")
```

20.10.3 Exercícios

1. Os intervalos entre chamadas numa central seguem uma exponencial com $\lambda = 2$ (chamadas por minuto).

- (a) Simule 1000 intervalos.
- (b) Calcule o tempo médio e compare com $1/\lambda$.
- (c) Faça o histograma e sobreponha a densidade teórica.

2. Numa fila, o tempo entre chegadas segue uma exponencial com $\lambda = 0,5$.

- (a) Calcule $P(\text{tempo} > 3)$ teoricamente.
- (b) Simule 5000 intervalos e estime essa probabilidade.
- (c) Compare.

3. O tempo de atendimento segue uma exponencial com $\lambda = 0,25$ por minuto.

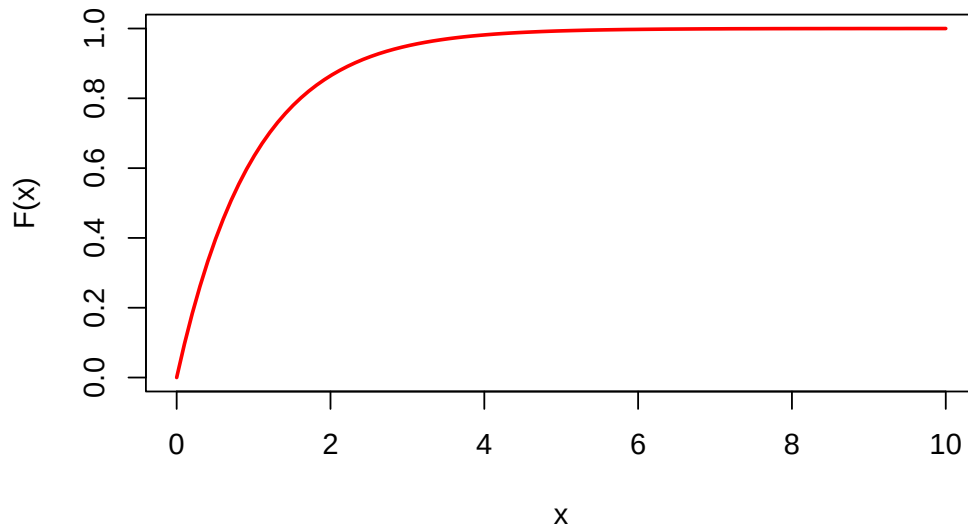
Função de distribuição da Exponencial(1)

Figura 20.15: Função de distribuição da Exponencial(1).

- (a) Simule os tempos para $n = 50$ clientes.
 - (b) Calcule o tempo total.
 - (c) Após 1000 simulações, faça o histograma do tempo total e sobreponha uma normal de média n/λ e desvio padrão $\sqrt{n/\lambda^2}$.
- 4.** Dois processos independentes têm tempos exponenciais com $\lambda_1 = 3$ e $\lambda_2 = 5$.
- (a) Simule 1000 tempos de cada.
 - (b) Some os tempos.
 - (c) Faça o histograma da soma e discuta se ainda é exponencial.
- 5.** Simule 5000 amostras de dimensão 30 de $X \sim \text{Exponencial}(\lambda = 1,5)$.
- (a) Calcule a média de cada amostra.
 - (b) Faça o histograma das médias e sobreponha uma normal de média $1/\lambda$ e desvio padrão $1/(\lambda\sqrt{n})$.
 - (c) Explique a ligação ao Teorema do Limite Central.
- 6.** O tempo até à primeira avaria de uma máquina segue uma exponencial com $\lambda = 0,1$.
- (a) Simule o tempo de avaria de 10 000 máquinas.
 - (b) Estime a probabilidade de avariar em menos de 15 horas.
 - (c) Compare com o valor teórico (função de distribuição).

7. Numa estrada, os intervalos entre acidentes seguem uma exponencial com $\lambda = 0,8$ (acidentes por hora).

- (a) Simule os intervalos para 1000 horas.
- (b) Estime a probabilidade de um intervalo ser inferior a 2 horas.
- (c) Faça o histograma e compare com a densidade teórica.

8. Considere $X_1 \sim \text{Exponencial}(0,5)$ e $X_2 = 2X_1 + 1$.

- (a) Simule 5000 pares (X_1, X_2) .
- (b) Calcule a média e a variância de X_2 .
- (c) Faça o gráfico de dispersão de X_1 e X_2 e comente a relação.

9. Os intervalos entre chamadas seguem uma exponencial com $\lambda = 2$.

- (a) Simule 5000 amostras de dimensão $n = 5$.
- (b) Para cada amostra, calcule a soma $S = \sum_{i=1}^n X_i$.
- (c) Compare o histograma de S com a densidade de uma Gama($k = 5, \theta = 1/\lambda$).
- (d) Compare a média e a variância empíricas de S com $E(S) = k\theta$ e $V(S) = k\theta^2$.

10. O tempo até à avaria de uma máquina segue uma exponencial com $\lambda = 0,5$; observa-se o tempo até 10 avarias consecutivas.

- (a) Simule 10 000 observações de $S = \sum_{i=1}^{10} X_i$.
- (b) Compare o histograma com a densidade de uma Gama($k = 10, \theta = 1/\lambda$).
- (c) Calcule $P(S > 25)$ teoricamente (densidade gama) e compare com a estimativa empírica.

11. O tempo de uma tarefa é a soma de $n = 7$ tempos exponenciais com $\lambda = 3$.

- (a) Simule 5000 valores de $S = \sum_{i=1}^n X_i$.
- (b) Ajuste-os a uma Gama($k = 7, \theta = 1/\lambda$).
- (c) Faça o histograma e compare com a densidade teórica.
- (d) Estime o percentil 90 de S e compare com o teórico (`qgamma()` e `quantile()`).

20.11 Distribuição normal

A distribuição normal é a mais importante da estatística. Vejamos alguns exemplos, a começar pela normal padrão $N(\mu = 0, \sigma = 1)$, que é o que as funções assumem por omissão:

```
dnorm(-1)      # densidade em x = -1
## [1] 0.2419707
pnorm(-1)      # P(X <= -1)
## [1] 0.1586553
qnorm(0.975)   # x tal que P(X <= x) = 0.975
## [1] 1.959964
rnorm(10)      # amostra de 10 valores (varia a cada execução)
```

O valor de `dnorm(-1)` é a densidade da normal padrão no ponto $x = -1$. A densidade da normal $N(\mu, \sigma)$ é

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2},$$

pelo que o mesmo valor se obtém substituindo diretamente na fórmula:

```
mu <- 0; sigma <- 1; x <- -1
(1 / (sigma * sqrt(2 * pi))) * exp(-1/2 * ((x - mu) / sigma)^2)
## [1] 0.242
```

As funções aceitam os argumentos `mean` e `sd` para outras médias e desvios padrão, que podem ser passados de várias formas (por nome, abreviados ou por posição):

```
qnorm(0.975, mean = 100, sd = 8)   # todas estas três chamadas...
qnorm(0.975, m = 100, s = 8)       # ...são equivalentes
qnorm(0.975, 100, 8)
## [1] 115.6797
```

No R, o segundo parâmetro da normal é o **desvio padrão** (σ), e não a variância (σ^2). Assim, para $X \sim N(\mu = 100, \sigma^2 = 64)$ escreve-se `sd = 8`, e não `sd = 64`. Confundir os dois é um erro comum e silencioso — o R não avisa, apenas devolve resultados errados.

Os cálculos de probabilidades que outrora exigiam tabelas fazem-se agora diretamente. Seja $X \sim N(\mu = 100, \sigma = 10)$:

```
pnorm(95, 100, 10)                  # P(X < 95)
## [1] 0.3085375
pnorm(110, 100, 10) - pnorm(90, 100, 10) # P(90 < X < 110)
## [1] 0.6826895
pnorm(95, 100, 10, lower.tail = FALSE) # P(X > 95)
## [1] 0.6914625
```


20.11.1 Densidade e função de distribuição

```
par(mfrow = c(1, 2))
plot(dnorm, from = -3, to = 3, xlab = "x", ylab = "Densidade")
title("Densidade\N(0, 1)")
plot(pnorm, from = -3, to = 3, xlab = "x", ylab = "F(x)")
title("Função de distribuição\N(0, 1)")
```

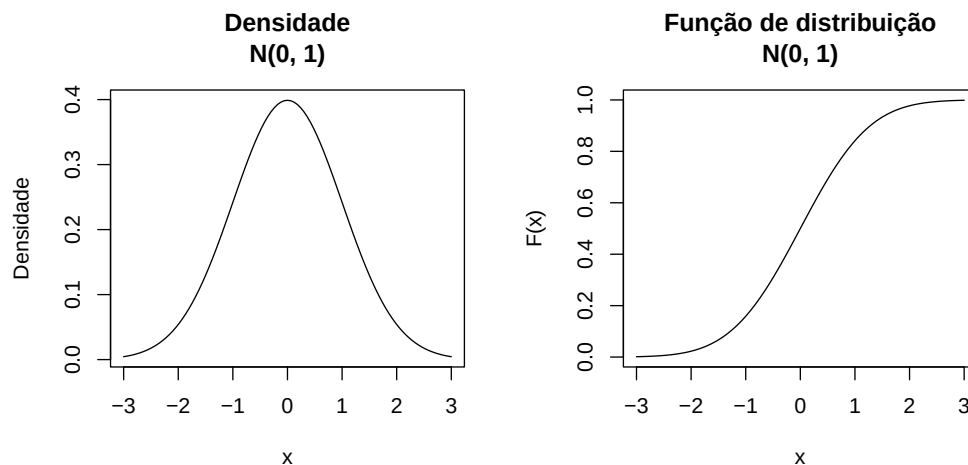


Figura 20.16: Densidade e função de distribuição da normal padrão.

```
par(mfrow = c(1, 1))
```

O efeito dos parâmetros vê-se bem comparando várias normais: a média desloca a curva; o desvio padrão controla a sua largura:

```
plot(function(x) dnorm(x, 100, 8), 60, 140, ylab = "f(x)", xlab = "x")
plot(function(x) dnorm(x, 90, 8), 60, 140, add = TRUE, col = 2)
plot(function(x) dnorm(x, 100, 15), 60, 140, add = TRUE, col = 3)
legend("topright", fill = 1:3,
      legend = c("N(100, 64)", "N(90, 64)", "N(100, 225)"))
```

20.11.2 Exercícios

1. O peso de um mineral (g) segue uma normal de média $\mu = 50$ e desvio padrão $\sigma = 5$.

(a) Simule 1000 pesos.

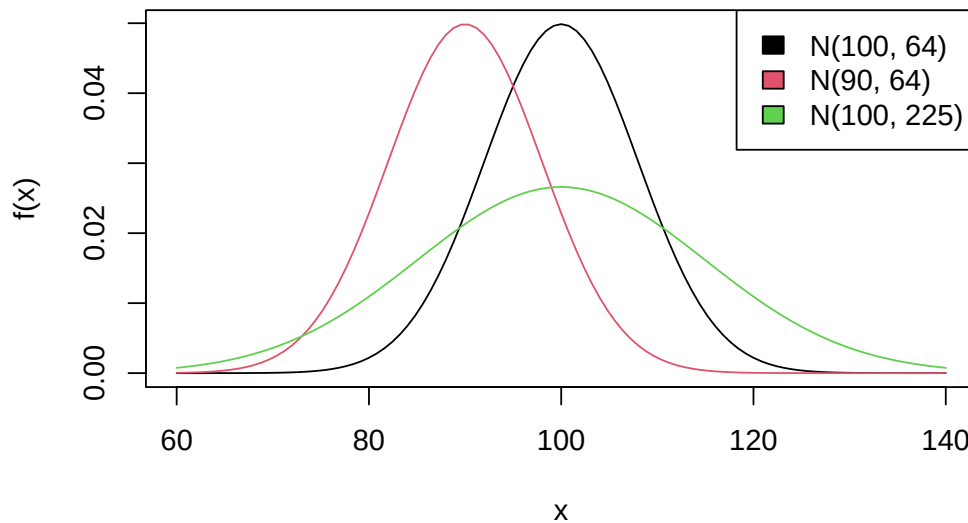


Figura 20.17: Três normais com médias e desvios padrão diferentes.

- (b) Compare a média e o desvio padrão simulados com os teóricos.
 - (c) Faça o histograma e sobreponha a densidade teórica.
- 2.** Sejam $X_1 \sim N(5, 2)$ e $X_2 \sim N(10, 3)$ independentes.
- (a) Simule 1000 pares.
 - (b) Calcule $S = X_1 + X_2$.
 - (c) Faça o histograma de S e sobreponha uma normal de média $\mu_1 + \mu_2$ e variância $\sigma_1^2 + \sigma_2^2$.
- 3.** Sejam $X_1, X_2 \sim N(0, 1)$ independentes.
- (a) Simule 5000 pares.
 - (b) Calcule $Q = X_1^2 + X_2^2$.
 - (c) Compare o histograma de Q com a densidade de uma qui-quadrado com 2 graus de liberdade (`dchisq()`).
- 4.** Estude a média de amostras de uma normal padrão $N(0, 1)$.
- (a) Extraia 1000 amostras de cada dimensão $n = 5, 10, 30, 100$.
 - (b) Para cada amostra, calcule a média.
 - (c) Faça o histograma das médias e sobreponha a densidade de uma t -Student com $n - 1$ graus de liberdade e a de uma normal de média 0 e variância $1/n$.
 - (d) Comente como a t -Student se aproxima da normal quando n aumenta, e o efeito de n na variabilidade das médias.

5. Simule 1000 amostras de dimensão 20 de $X \sim N(0, 1)$.

- (a) Calcule a soma dos quadrados de cada amostra.
- (b) Compare o histograma com a densidade de uma qui-quadrado com 20 graus de liberdade.
- (c) Interprete os resultados.

6. Simule 5000 amostras de dimensão 30 de $X \sim N(\mu = 10, \sigma = 4)$.

- (a) Calcule a média de cada amostra.
- (b) Faça o histograma das médias e sobreponha uma normal de média 10 e desvio padrão $4/\sqrt{30}$.
- (c) Explique a ligação ao Teorema do Limite Central.

7. Sejam $X_1 \sim N(0, 1)$ e $X_2 = 0,5X_1 + Z$, com $Z \sim N(0, 1)$ independente.

- (a) Simule 5000 pares.
- (b) Calcule $S = X_1 + X_2$.
- (c) Faça o histograma de S .

8. Simule 1000 amostras de dimensão 10 de $X \sim N(0, 1)$. Para cada uma, calcule a média $\bar{X}$ e a variância S^2 , e construa $T = \frac{\bar{X}}{S/\sqrt{n}}$. Compare o histograma de T com a densidade de uma t -Student com $n - 1$ graus de liberdade.

9. Seja $X \sim N(0, 1)$. Simule 1000 amostras de dimensão $n = 10$. Para cada uma:

- (a) Calcule a variância amostral S^2 .
- (b) Construa a estatística $Q = \frac{(n-1)S^2}{\sigma^2}$, com $\sigma^2 = 1$.
- (c) Compare o histograma de Q com a densidade de uma $\chi^2(n-1)$.
- (d) Compare a média e a variância empíricas de Q com $E(Q) = n-1$ e $V(Q) = 2(n-1)$.

Conhecidas as distribuições e as funções `rnome` que geram amostras diretamente, uma pergunta natural impõe-se: e quando a distribuição de que precisamos não tem uma função `r` pronta no R? É esse o problema que os próximos capítulos resolvem, a começar pelo método da transformada inversa.

Capítulo 21

Exercícios extra

Antes de avançarmos para os métodos de geração de variáveis aleatórias, este capítulo propõe uma bateria de exercícios que consolidam tudo o que vimos sobre distribuições e simulação. Os primeiros são de aquecimento — gerar amostras, calcular médias e variâncias, comparar frequências empíricas com probabilidades teóricas. Os últimos antecipam um dos resultados mais belos da estatística, o **Teorema do Limite Central**: ao somar ou promediar muitas variáveis independentes, a distribuição do resultado aproxima-se de uma normal, seja qual for a distribuição de partida.

Consolidar, através de exercícios aplicados, a geração de amostras com as funções `runif`, o cálculo de probabilidades teóricas e empíricas, e a observação da Lei dos Grandes Números e do Teorema do Limite Central por simulação.

1. Com `set.seed(123)`, simule 1000 lançamentos de uma moeda com probabilidade 0,5 de sair cara. Conte o número de caras e faça um gráfico de barras dos resultados.

```
set.seed(123)
n <- 1000
p <- 0.5

amostra <- rbinom(n, 1, p)      # 1000 provas de Bernoulli
sum(amostra)                   # número de caras

freq_relativa <- table(amostra) / length(amostra)
barplot(freq_relativa, col = "lightblue")
```

2. Com `set.seed(123)`, gere 5000 observações de uma binomial com $n = 10$ e $p = 0,3$. Calcule a média e a variância.

```
set.seed(123)
amostra <- rbinom(5000, 10, 0.3)
mean(amostra)
var(amostra)
```

3. Com `set.seed(123)`, gere 2300 observações de uma Poisson com $\lambda = 4$. Calcule a média e o desvio padrão.

```
set.seed(123)
amostra <- rpois(2300, 4)
mean(amostra)
sd(amostra)
```

4. Num processo de qualidade, X é o número de produtos defeituosos num lote de 50, com probabilidade 0,1 de um produto ser defeituoso. Com `set.seed(123)`, gere 10 000 observações de X , conte os lotes com exatamente 5 defeituosos e compare a proporção com $P(X = 5)$ para $X \sim \text{Binomial}(50, 0,1)$.

```
set.seed(123)
amostra <- rbinom(10000, 50, 0.1)

freq_rela <- mean(amostra == 5)           # proporção empírica
prob_teorica <- dbinom(5, 50, 0.1)       # probabilidade teórica

data.frame(empirica = freq_rela, teorica = prob_teorica)
```

5. Com `set.seed(123)`, gere 5000 observações de $X \sim \text{Binomial}(20, 0,7)$.

- (a) Faça um histograma de frequências relativas e sobreponha a distribuição de X .
- (b) Use a função de distribuição empírica para estimar $P(X \leq 10)$ e compare com o valor teórico.

```
set.seed(123)
amostra <- rbinom(5000, 20, 0.7)

hist(amostra, freq = FALSE, col = "lightblue")
lines(0:20, dbinom(0:20, 20, 0.7), col = "magenta", type = "p")

Fn <- ecdf(amostra)
data.frame(empirica = Fn(10), teorica = pbinom(10, 20, 0.7))
```

6. Com `set.seed(543)`, gere 2400 observações de $Y \sim \text{Poisson}(\lambda = 6)$.

- (a) Faça um histograma de frequências relativas e sobreponha a distribuição de Y .
- (b) Use a função de distribuição empírica para estimar $P(Y > 5)$ e compare com o valor teórico.

```
set.seed(543)
amostra <- rpois(2400, 6)

hist(amostra, freq = FALSE, col = "lightblue")
lines(0:15, dpois(0:15, 6), col = "magenta", type = "p")

Fn <- ecdf(amostra)
data.frame(empirica = 1 - Fn(5), teorica = ppois(5, 6, lower.tail = FALSE))
```

7. Com `set.seed(345)`, gere 3450 observações de $Z \sim U(0, 1)$. Estime $P(Z \leq 0,5)$ pela função de distribuição empírica e compare com o valor teórico.

```
set.seed(345)
amostra <- runif(3450, 0, 1)

hist(amostra, freq = FALSE, col = "lightblue")
curve(dunif(x, 0, 1), col = "red", add = TRUE)

Fn <- ecdf(amostra)
data.frame(empirica = Fn(0.5), teorica = punif(0.5, 0, 1))
```

8. Com `set.seed(123)`, gere 3467 observações de $W \sim N(0, 1)$.

- (a) Faça um histograma de frequências relativas e sobreponha a densidade de W .
- (b) Use a função de distribuição empírica para estimar $P(W > 1)$ e compare com o valor teórico.

9. Com `set.seed(123)`, gere 1234 observações de $V \sim \text{Exponencial}(\lambda = 0,5)$.

- (a) Faça um histograma de frequências relativas e sobreponha a densidade de V .
- (b) Use a função de distribuição empírica para estimar $P(V > 2)$ e compare com o valor teórico.

10. O número de acertos num alvo em 30 tentativas, com probabilidade de acerto 0,4, é $X \sim \text{Binomial}(30, 0,4)$. Com `set.seed(123)`, gere uma amostra de dimensão 700.

- (a) Faça um histograma de frequências relativas e sobreponha a distribuição de X .
- (b) Com a função de distribuição empírica, estime $P(X > 15)$ e calcule também o valor teórico.

11. Com `set.seed(123)`, gere amostras de dimensão crescente $n = 100, 1000, 10\,000, 100\,000$ de $X \sim \text{Poisson}(\lambda = 3)$. Para cada dimensão, calcule a média amostral e compare-a com o valor esperado teórico. Comente a convergência (Lei dos Grandes Números).

```
set.seed(123)
tamanhos <- c(100, 1000, 10000, 100000)
lambda <- 3

media_amostra <- numeric(length(tamanhos))
for (i in seq_along(tamanhos)) {
  media_amostra[i] <- mean(rpois(tamanhos[i], lambda = lambda))
}

data.frame(Tamanho = tamanhos, Media_amostra = media_amostra, Media_teorica = lambda)
```

12. Com `set.seed(123)`, gere amostras de dimensão crescente $n = 100, 1000, 10\,000, 100\,000$ de $W \sim U(0, 1)$. Para cada dimensão, calcule a média amostral e compare-a com o valor esperado teórico. Comente a convergência.

13. Um grupo de estudantes conduz um estudo sobre a satisfação com um novo curso. Cada aluno indica se está satisfeito, com probabilidade 0,75. Com `set.seed(42)`, simule amostras de dimensão crescente $n = 100, 500, 1000, 5000, 10\,000$ de $X \sim \text{Binomial}$, onde X é o número de satisfeitos. Para cada dimensão, calcule a proporção de satisfeitos e compare-a com a probabilidade teórica (0,75).

14. Com `set.seed(1058)`, gere 9060 amostras de dimensão 9 de $X \sim \text{Binomial}(41, 0,81)$. Calcule a média de cada amostra, obtendo uma amostra de médias; calcule ainda $E(X)$ e compare-o com a média das médias.

```
set.seed(1058)
m <- 9060      # número de amostras
n <- 9         # dimensão de cada amostra

amostra <- matrix(rbinom(m * n, 41, 0.81), nrow = m, ncol = n)
media_amostra <- rowMeans(amostra)

data.frame(empirica = mean(media_amostra), teorica = 41 * 0.81)
```

15. Num hospital, o tempo de atendimento segue uma exponencial de média 30 minutos. Com `set.seed(456)`, simule 1000 amostras de dimensão 50, 100 e 1000. Para cada dimensão, calcule

a média de cada amostra, faça o histograma das médias e compare com a densidade normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$. Comente a aplicação do Teorema do Limite Central.

16. O tempo de espera (minutos) no balcão de informações de um banco segue $X \sim U(5, 20)$. Com `set.seed(1430)`, gere 8000 amostras de dimensão $n = 100$.

- Calcule a soma de cada amostra, $S_n = \sum_{i=1}^n X_i$.
- Faça o histograma das somas e sobreponha uma normal de média $nE(X)$ e desvio padrão $\sqrt{nV(X)}$.
- Calcule a média de cada amostra, $\bar{X}_n$.
- Faça o histograma das médias e sobreponha uma normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$.

```
set.seed(1430)
m <- 8000; n <- 100
a <- 5; b <- 20

media <- (a + b) / 2
variancia <- (b - a)^2 / 12

amostras <- matrix(runif(m * n, a, b), nrow = m, ncol = n)

# Soma de cada amostra + curva normal
soma_amostras <- rowSums(amostras)
hist(soma_amostras, freq = FALSE, col = "lightblue")
curve(dnorm(x, n * media, sqrt(variancia * n)), col = "red", add = TRUE, lwd = 2)

# Média de cada amostra + curva normal
media_amostras <- rowMeans(amostras)
hist(media_amostras, freq = FALSE, col = "lightblue")
curve(dnorm(x, media, sqrt(variancia / n)), col = "red", add = TRUE, lwd = 2)
```

17. O tempo de atendimento de doentes graves segue $X \sim \text{Exponencial}(\lambda = 0,21)$. Com `set.seed(1580)`, gere 1234 amostras de dimensão $n = 50$.

- Calcule a soma de cada amostra, $S_n = \sum_{i=1}^n X_i$.
- Faça o histograma das somas e sobreponha uma normal de média $nE(X)$ e desvio padrão $\sqrt{nV(X)}$.
- Calcule a soma padronizada $\frac{S_n - E(S_n)}{\sqrt{V(S_n)}}$ e faça o histograma, sobrepondo uma normal padrão.

- (d) Calcule a média de cada amostra, $\bar{X}_n$.
- (e) Faça o histograma das médias e sobreponha uma normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$.

18. A altura (cm) dos alunos de uma escola segue $X \sim N(\mu = 170, \sigma = 10)$. Com `set.seed(678)`, gere 9876 amostras de dimensão $n = 80$.

- (a) Calcule a soma de cada amostra, S_n .
- (b) Faça o histograma das somas e sobreponha uma normal de média $nE(X)$ e desvio padrão $\sqrt{nV(X)}$.
- (c) Calcule a soma padronizada e faça o histograma, sobrepondo uma normal padrão.
- (d) Calcule a média de cada amostra, $\bar{X}_n$.
- (e) Faça o histograma das médias e sobreponha uma normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$.
- (f) Faça o histograma da média padronizada $\frac{\bar{X}_n - E(\bar{X}_n)}{\sqrt{V(\bar{X}_n)}}$ e sobreponha uma normal padrão.

19. A chegada de clientes a uma loja numa hora, com uma taxa média de 20 clientes por hora, segue $X \sim \text{Poisson}(\lambda = 20)$. Com `set.seed(1222)`, gere 8050 amostras de dimensão 30.

- (a) Calcule a soma de cada amostra, S_n .
- (b) Faça o histograma das somas e sobreponha uma normal de média $nE(X)$ e desvio padrão $\sqrt{nV(X)}$.
- (c) Calcule a soma padronizada e faça o histograma, sobrepondo uma normal padrão.
- (d) Calcule a média de cada amostra, $\bar{X}_n$.
- (e) Faça o histograma das médias e sobreponha uma normal de média $E(X)$ e desvio padrão $\sqrt{V(X)/n}$.
- (f) Faça o histograma da média padronizada e sobreponha uma normal padrão.

Os últimos exercícios mostram, por simulação, um dos resultados centrais da estatística. No próximo capítulo mudamos de perspectiva: em vez de usar as funções `rnome` já feitas, aprendemos a **construí-las** — a gerar nós próprios variáveis aleatórias de qualquer distribuição, a começar pelo método da transformada inversa.

Capítulo 22

Método da transformada inversa

Nos capítulos anteriores gerámos variáveis aleatórias com as funções `rnome` do R — `rbinom`, `rpois`, `rnorm`, e as demais. Mas de onde vêm esses números? E o que fazer quando precisamos de uma distribuição para a qual o R **não** oferece uma função pronta? A resposta a ambas as perguntas começa com uma observação notável: se soubermos gerar números uniformes em $(0, 1)$ — a matéria-prima de todo o acaso no computador —, podemos transformá-los em amostras de *qualquer* distribuição. O **método da transformada inversa** é a primeira e mais fundamental dessas transformações.

Vale a pena dominá-lo por três razões: permite gerar amostras de **distribuições não implementadas** no R; dá **flexibilidade** para simulações personalizadas; e, sobretudo, esclarece os **fundamentos** da geração de variáveis aleatórias — implementá-lo à mão é compreender o que as funções `rnome` fazem por dentro.

Neste capítulo aprendemos o resultado que está na base da geração de variáveis aleatórias — a transformada integral de probabilidade — e aplicamo-lo para gerar, a partir de números uniformes, variáveis discretas (arbitrárias, Bernoulli, binomial, geométrica, Poisson) e contínuas (exponencial, uniforme, Weibull, Cauchy, e outras).

22.1 O resultado fundamental

Tudo assenta no seguinte teorema, conhecido como **transformada integral de probabilidade**.

Teorema. Seja F uma função de distribuição qualquer e defina-se a sua *inversa generalizada* por

$$F^{-1}(u) = \inf\{x : F(x) \geq u\}, \quad 0 < u < 1.$$

Se $U \sim U(0, 1)$, então a variável aleatória $X = F^{-1}(U)$ tem função de distribuição F . Reciprocamente, se X é contínua com função de distribuição F , então $F(X) \sim U(0, 1)$.

A demonstração é curta no caso em que F é contínua e estritamente crescente (e, por isso, invertível no sentido usual). A chave é a equivalência $F^{-1}(u) \leq x \iff u \leq F(x)$. Então

$$P(X \leq x) = P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x),$$

onde a última igualdade decorre de $U \sim U(0, 1)$ e $0 \leq F(x) \leq 1$ (para uma uniforme, $P(U \leq a) = a$). Logo $X = F^{-1}(U)$ tem mesmo a distribuição F .

A intuição é geométrica: a função de distribuição F transforma o eixo dos x na escala de probabilidades $[0, 1]$; inverter esse mapa é sortear uma “altura” U uniforme em $[0, 1]$ e ler qual o x que lhe corresponde. Regiões onde F sobe depressa (isto é, onde a densidade é alta) recebem, naturalmente, mais valores.

Deste teorema resulta uma receita geral:

Receita da transformada inversa.

1. Determine a função de distribuição $F(x) = P(X \leq x)$.
2. Resolva a equação $u = F(x)$ em ordem a x , obtendo a inversa $x = F^{-1}(u)$.
3. Gere $U \sim U(0, 1)$ e devolva $X = F^{-1}(U)$.

Vejamos como isto se concretiza, primeiro para variáveis discretas e depois para contínuas.

22.2 Variáveis aleatórias discretas

O método da transformada inversa também se aplica a variáveis aleatórias discretas. A diferença principal é que, neste caso, a função de distribuição não é contínua: ela é uma função em escada, com saltos nos pontos que pertencem ao suporte da variável aleatória.

Suponhamos que X toma valores

$$x_1 < x_2 < x_3 < \dots$$

com probabilidades

$$P(X = x_i) = p_i, \quad i = 1, 2, \dots,$$

onde $p_i \geq 0$ e $\sum_i p_i = 1$. A função de distribuição acumulada é dada por

$$F(x_i) = P(X \leq x_i) = p_1 + \dots + p_i.$$

Neste contexto, a inversa usual de F não existe no sentido clássico, pois F não é uma função contínua nem estritamente crescente. Por isso, usamos a **inversa generalizada**, definida por

$$F^{-1}(u) = \inf\{x \in \mathbb{R} : F(x) \geq u\}, \quad 0 < u < 1.$$

Assim, se $U \sim U(0, 1)$, definimos

$$X = F^{-1}(U).$$

Em termos práticos, isto significa que escolhemos o menor valor x_i para o qual a função de distribuição acumulada já atingiu ou ultrapassou o valor observado de U . Equivalentemente,

$$X = x_i \quad \text{se} \quad F(x_{i-1}) < U \leq F(x_i),$$

com a convenção $F(x_0) = 0$.

Esta formulação é particularmente natural no caso discreto: o intervalo

$$(F(x_{i-1}), F(x_i)]$$

tem comprimento

$$F(x_i) - F(x_{i-1}) = p_i.$$

Como U é uniforme em $(0, 1)$, a probabilidade de U cair nesse intervalo é precisamente p_i . Portanto,

$$P(X = x_i) = P(F(x_{i-1}) < U \leq F(x_i)) = F(x_i) - F(x_{i-1}) = p_i.$$

Logo, a variável $X = F^{-1}(U)$ tem exactamente a distribuição pretendida.

Como U é uma variável contínua, a probabilidade de U tomar exactamente um dos valores $F(x_i)$ é zero. Por isso, pequenas diferenças na escolha dos extremos dos intervalos não alteram a distribuição gerada. No entanto, a convenção

$$F^{-1}(u) = \inf\{x : F(x) \geq u\}$$

é a mais rigorosa e a mais usada em teoria da probabilidade.

```
# Esquema do algoritmo
# 1. Gerar  $u \sim U(0, 1)$ 
# 2. Percorrer os valores, acumulando probabilidades, até  $F(x_i) \geq u$ 
# 3. Devolver esse  $x_i$ 
```

Exemplo 1 (distribuição arbitrária). Seja X discreta com valores possíveis 1, 2, 3, 4 e probabilidades $p_1 = 0,20$, $p_2 = 0,15$, $p_3 = 0,25$, $p_4 = 0,40$. A sua função de distribuição é

$$F(x) = \begin{cases} 0, & x < 1 \\ 0,20, & 1 \leq x < 2 \\ 0,35, & 2 \leq x < 3 \\ 0,60, & 3 \leq x < 4 \\ 1, & x \geq 4. \end{cases}$$

Gerando U , devolvemos 1 se $0 < U \leq 0,20$; 2 se $0,20 < U \leq 0,35$; 3 se $0,35 < U \leq 0,60$; e 4 se $0,60 < U \leq 1$. Na prática computacional, devido à continuidade de U , escrever as desigualdades com $<$ ou $\leq$ nos pontos de corte não altera a distribuição gerada.

```
gerar_inversa <- function() {
  u <- runif(1)
  if (u <= 0.20)      return(1)
  else if (u <= 0.35) return(2)
  else if (u <= 0.60) return(3)
  else               return(4)
}

set.seed(123)
valores <- replicate(10000, gerar_inversa())
table(valores) / 10000      # frequências relativas ~ (0.20, 0.15, 0.25, 0.40)
## valores
##      1      2      3      4
## 0.1998 0.1499 0.2550 0.3953
```

Exemplo 2 (uniforme discreta, algoritmo geral). Para X uniforme em $\{1, 2, \dots, 10\}$ (todas as probabilidades iguais a $1/10$), o algoritmo geral percorre os valores acumulando probabilidade até ultrapassar U :

```
gerar_va_inversa <- function() {
  u <- runif(1)
  p <- 1/10      # P(X = 1)
  F <- p         # função de distribuição acumulada
  X <- 1
  while (u > F) { # avança enquanto U ainda não foi ultrapassado
    X <- X + 1
    F <- F + p
  }
```

```

}
  return(X)
}

set.seed(123)
valores <- replicate(5000, gerar_va_inversa())
barplot(table(valores) / 5000, col = "lightblue")

```

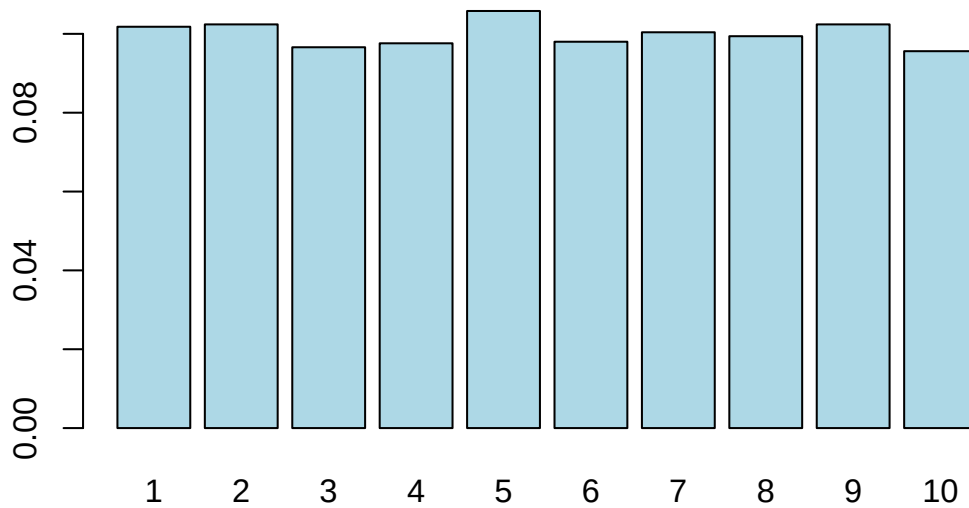


Figura 22.1: Amostra de uma uniforme discreta gerada por transformada inversa.

Exemplo 3 (Bernoulli). Para $X \sim \text{Bernoulli}(p)$, a receita reduz-se a: gerar U e devolver 1 se $U \leq p$, e 0 caso contrário.

```

gerar_bernoulli_inversa <- function(p) {
  U <- runif(1)
  if (U <= p) 1 else 0
}

set.seed(123)
valores <- replicate(100, gerar_bernoulli_inversa(0.8))
mean(valores)      # ~ 0.8
## [1] 0.82

```

Exemplo 4 (binomial). Uma $\text{Binomial}(n, p)$ é a soma de n Bernoullis i.i.d., pelo que basta somar n chamadas ao gerador anterior:

```

gerar_binomial_inversa <- function(n, p) {
  sum(replicate(n, gerar_bernoulli_inversa(p)))
}

set.seed(123)
valores <- replicate(10000, gerar_binomial_inversa(10, 0.5))
hist(valores, freq = FALSE, breaks = 11, col = "lightblue",
     main = "Binomial(10, 0.5) por transformada inversa", xlab = "x")
points(0:10, dbinom(0:10, 10, 0.5), col = "magenta", pch = 19)

```

Binomial(10, 0.5) por transformada inversa

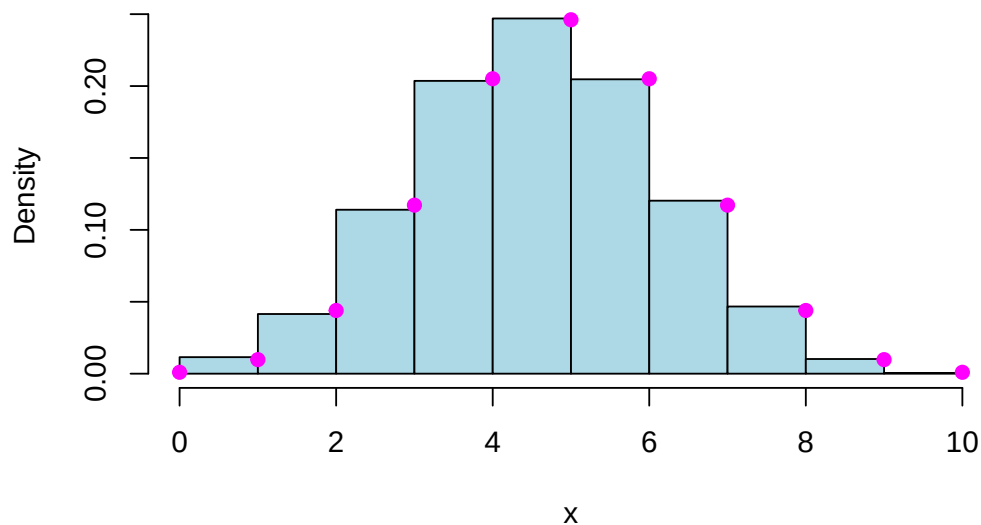


Figura 22.2: Amostra de uma Binomial(10, 0.5) obtida somando Bernoullis.

Exemplo 5 (geométrica). Recorde que a geométrica do R conta o número de *insucessos* Y antes do primeiro sucesso, com $P(Y = y) = p(1-p)^y$ e $F(y) = 1 - (1-p)^{y+1}$, $y \geq 0$. Invertendo $u = F(y)$ obtém-se um algoritmo direto, sem ciclos: resolvendo $1 - (1-p)^{y+1} = u$ chega-se a

$$Y = \left\lceil \frac{\ln(1-U)}{\ln(1-p)} \right\rceil - 1.$$

```

gerar_geometrica_inversa <- function(p) {
  U <- runif(1)
  ceiling(log(1 - U) / log(1 - p)) - 1
}

set.seed(123)

```

```
valores <- replicate(10000, gerar_geometrica_inversa(0.5))
hist(valores, freq = FALSE, col = "lightblue",
     main = "Geométrica(0.5) por transformada inversa", xlab = "x")
points(0:10, dgeom(0:10, 0.5), col = "magenta", pch = 19)
```

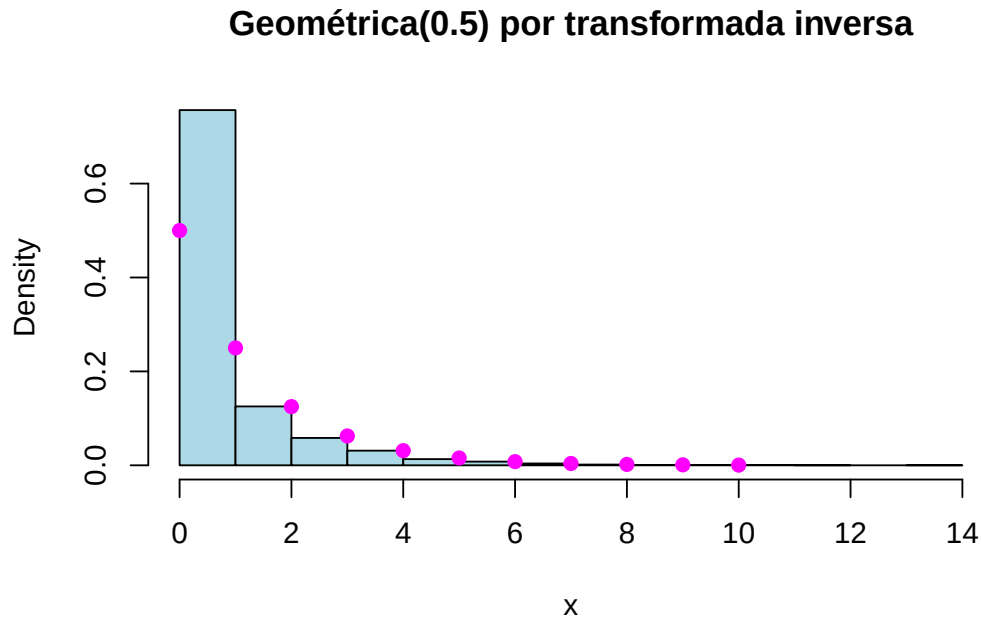


Figura 22.3: Amostra de uma geométrica (número de insucessos) por transformada inversa.

A distribuição geométrica aparece frequentemente com duas convenções diferentes. Em R, a função `rgeom()` usa a convenção

$$Y = 0, 1, 2, \dots,$$

em que Y representa o **número de insucessos antes do primeiro sucesso**. Neste caso,

$$P(Y = y) = p(1 - p)^y, \quad y = 0, 1, 2, \dots$$

e a função de distribuição é

$$F_Y(y) = P(Y \leq y) = 1 - (1 - p)^{y+1}.$$

Noutra convenção, também muito usada em Probabilidade, a variável aleatória conta o **número de provas até ao primeiro sucesso**. Nesse caso,

$$X = 1, 2, 3, \dots$$

e

$$P(X = k) = (1 - p)^{k-1}p, \quad k = 1, 2, 3, \dots$$

As duas versões estão relacionadas por

$$X = Y + 1.$$

Assim, em R:

```
# Número de insucessos antes do primeiro sucesso
rgeom(n, prob = p)

# Número de provas até ao primeiro sucesso
rgeom(n, prob = p) + 1
```

Por isso, quando um exercício mencionar uma variável geométrica, é essencial verificar qual das duas convenções está a ser usada.

Exemplo 6 (Poisson). Para a Poisson, o algoritmo acumula probabilidades como no Exemplo 2, mas usa a recursão que liga probabilidades consecutivas,

$$p_{i+1} = \frac{\lambda}{i+1} p_i, \quad p_0 = e^{-\lambda},$$

o que evita recalcular fatoriais e exponenciais a cada passo:

```
gerar_poisson_inversa <- function(lambda) {
  U <- runif(1)
  p <- exp(-lambda)    # P(X = 0)
  F <- p               # função de distribuição acumulada
  X <- 0
  while (U > F) {
    X <- X + 1
    p <- p * lambda / X  # recursão: p_i -> p_{i+1}
    F <- F + p
  }
  return(X)
}

set.seed(123)
valores <- replicate(10000, gerar_poisson_inversa(3))
```

```
hist(valores, freq = FALSE, breaks = 12, col = "lightblue",  
     main = "Poisson(3) por transformada inversa", xlab = "x")  
points(0:10, dpois(0:10, 3), col = "magenta", pch = 19)
```

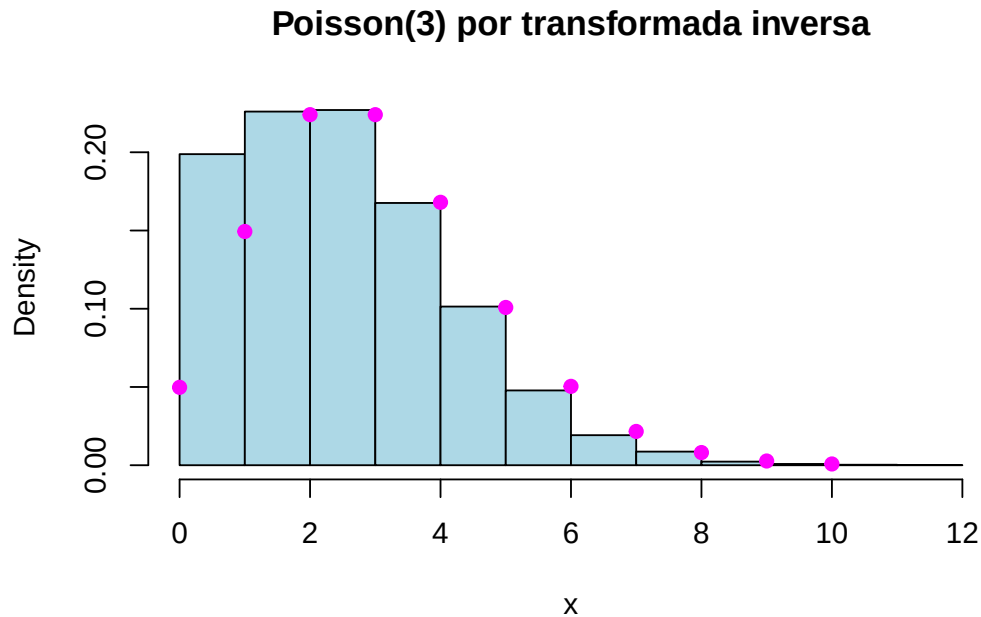


Figura 22.4: Amostra de uma Poisson(3) por transformada inversa.

22.2.1 Exercícios

1. Seja X com $P(X = 1) = 0.2$, $P(X = 2) = 0.5$, $P(X = 3) = 0.3$.

- Calcule a função de distribuição de X .
- Gere 1000 amostras de X por transformada inversa.
- Faça um gráfico de barras e compare com as probabilidades teóricas.

2. Uma moeda enviesada tem probabilidade 0.7 de cara e 0.3 de coroa.

- Defina a função de distribuição da experiência.
- Simule 500 lançamentos por transformada inversa.
- Compare as proporções de caras e coroas com as probabilidades esperadas.

3. Seja X com $P(X = 1) = 0.1$, $P(X = 2) = 0.3$, $P(X = 3) = 0.4$, $P(X = 4) = 0.2$. Gere 2000 amostras por transformada inversa e compare a distribuição empírica com a teórica.

4. Seja X uma variável geométrica com parâmetro $p = 0.2$, na convenção em que X representa o número de provas até ao primeiro sucesso. Assim,

$$P(X = k) = (1 - p)^{k-1}p, \quad k = 1, 2, \dots$$

Com `set.seed(1234)`, gere 1000 valores por transformada inversa e indique, com 4 casas decimais, a proporção de valores superiores à soma da média com o desvio padrão amostrais.

22.3 Variáveis aleatórias contínuas

Para variáveis contínuas, a receita é ainda mais direta, porque a função de distribuição F é, em geral, uma função contínua e estritamente crescente no suporte — logo, invertível no sentido habitual.

Proposição. Seja X uma variável aleatória contínua com função de distribuição F invertível, de inversa F^{-1} , e seja $U \sim U(0, 1)$. Então $X = F^{-1}(U)$ tem função de distribuição F .

A demonstração é imediata a partir da equivalência $F^{-1}(u) \leq x \iff u \leq F(x)$:

$$P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x),$$

usando, na última igualdade, que $U \sim U(0, 1)$ e $0 \leq F(x) \leq 1$. Basta, então, seguir os três passos da receita: determinar F , invertê-la e aplicar a inversa a um número uniforme.

Exemplo 1 (densidade $f(x) = 2x$). Seja X com densidade $f(x) = 2x$ para $0 < x < 1$ (e 0 fora). A sua função de distribuição é

$$F(x) = \int_0^x 2t \, dt = x^2, \quad 0 < x < 1.$$

Como $F(x) = x^2$ é invertível em $(0, 1)$, com $F^{-1}(u) = \sqrt{u}$, a receita diz que $X = \sqrt{U}$ tem a distribuição pretendida:

```
set.seed(123)
n <- 10000
simlist <- sqrt(runif(n))

hist(simlist, prob = TRUE, main = "Densidade f(x) = 2x", xlab = "x", col = "lightblue")
curve(2 * x, 0, 1, add = TRUE, col = "red", lwd = 2)
```

Exemplo 2 (uniforme $U(a, b)$). A função de distribuição da uniforme é $F(x) = \frac{x-a}{b-a}$, cuja inversa é $F^{-1}(u) = a + (b - a)u$. Logo, $X = a + (b - a)U$:

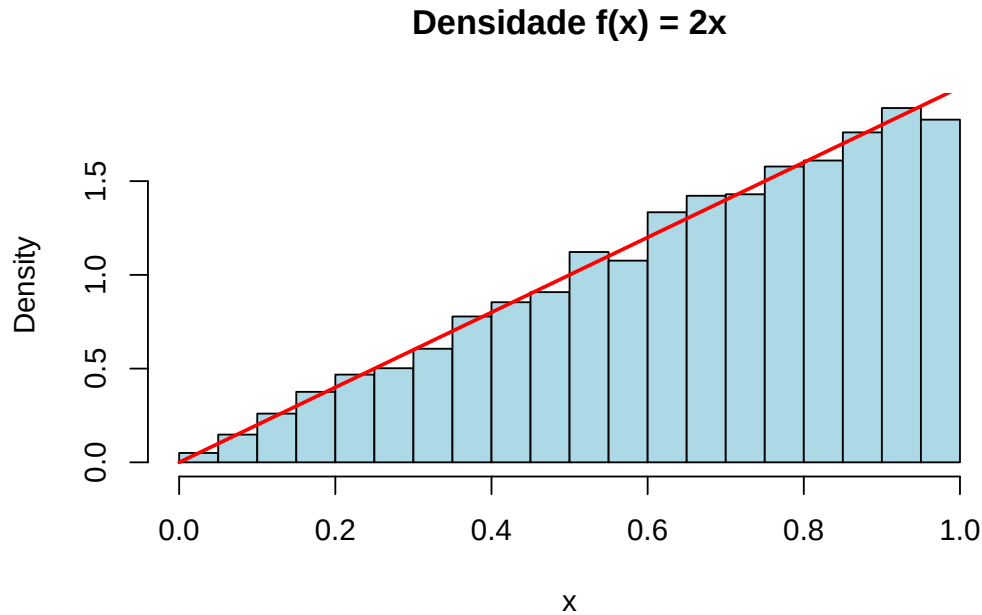


Figura 22.5: Amostra da densidade $f(x) = 2x$ gerada por $X = \sqrt{U}$.

```
a <- -2; b <- 2; n <- 10000
set.seed(123)
simlist <- a + (b - a) * runif(n)

hist(simlist, prob = TRUE, main = "Uniforme(-2, 2)", xlab = "x", col = "lightblue")
curve(dunif(x, a, b), col = "red", add = TRUE, lwd = 2)
```

Exemplo 3 (exponencial). Seja X exponencial de taxa 1, com função de distribuição

$$F(x) = 1 - e^{-x}, \quad x > 0.$$

Fazendo $u = F(x) = 1 - e^{-x}$ e resolvendo em ordem a x :

$$1 - u = e^{-x} \implies x = -\ln(1 - u).$$

Assim, $X = -\ln(1 - U)$ gera uma exponencial de taxa 1. Uma pequena simplificação: como $1 - U$ também é uniforme em $(0, 1)$, podemos usar $-\ln(U)$, com a mesma distribuição. Além disso, se X é exponencial de taxa 1, então $\frac{1}{\lambda}X$ é exponencial de taxa λ ; logo, uma exponencial de taxa λ gera-se por

$$X = -\frac{1}{\lambda} \ln(U).$$

```
lambda <- 3; n <- 10000
set.seed(123)
```

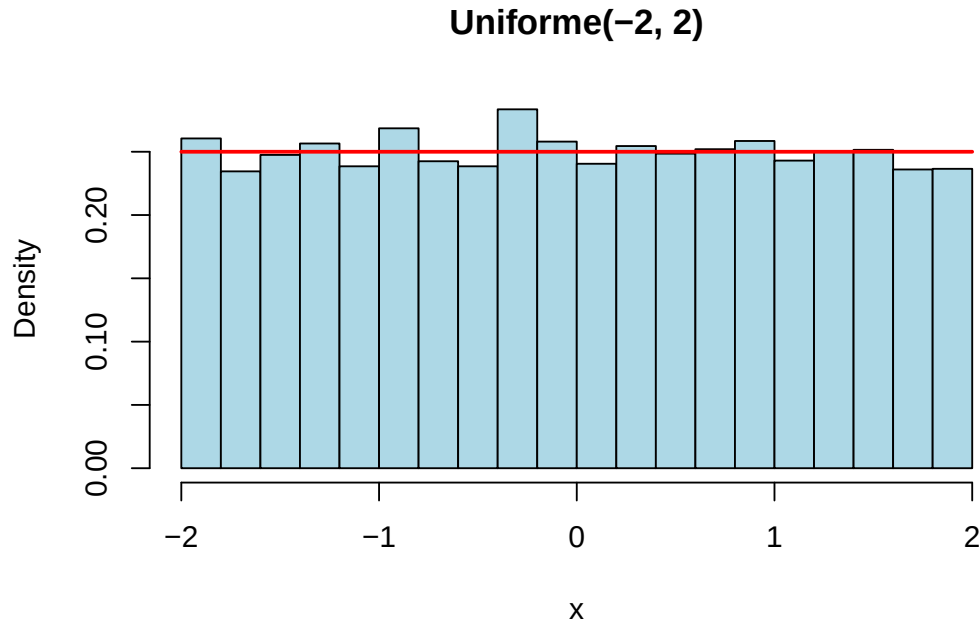


Figura 22.6: Amostra de uma Uniforme(-2, 2) por transformada inversa.

```
simlist <- -log(runif(n)) / lambda

hist(simlist, probability = TRUE, main = "Exponencial simulada vs teórica",
     xlab = "x", ylab = "Densidade", col = "lightblue", border = "black")
curve(dexp(x, rate = lambda), add = TRUE, col = "red", lwd = 2)
legend("topright", legend = c("Simulação", "Teórica"),
     col = c("lightblue", "red"), lwd = 2)
```

A recíproca do teorema — $F(X) \sim U(0, 1)$ para X contínua — é igualmente útil na prática. É ela que justifica, por exemplo, que os valores- p de um teste de hipóteses tenham distribuição uniforme quando a hipótese nula é verdadeira, e está na base de vários testes de ajustamento de distribuições. Gerar e verificar são, afinal, as duas faces da mesma transformação.

22.3.1 Exercícios

1. Para $X \sim \text{Exponencial}(\lambda = 2)$, com $F_X(x) = 1 - e^{-\lambda x}$, mostre que $X = -\frac{\ln(1 - U)}{\lambda}$. Gere 1000 valores por transformada inversa e visualize a distribuição num histograma.
2. Use a transformada inversa para gerar 500 valores de $X \sim U(2, 5)$. A função de distribuição de uma $U(a, b)$ é $F_X(x) = \frac{x - a}{b - a}$; mostre que $X = a + (b - a)U$. Visualize num histograma e compare com a densidade teórica.
3. Para a distribuição de **Cauchy** de parâmetros 0 e 1, com $F_X(x) = \frac{1}{\pi} \arctan\left(\frac{x - x_0}{\gamma}\right) + \frac{1}{2}$,

Exponencial simulada vs teórica

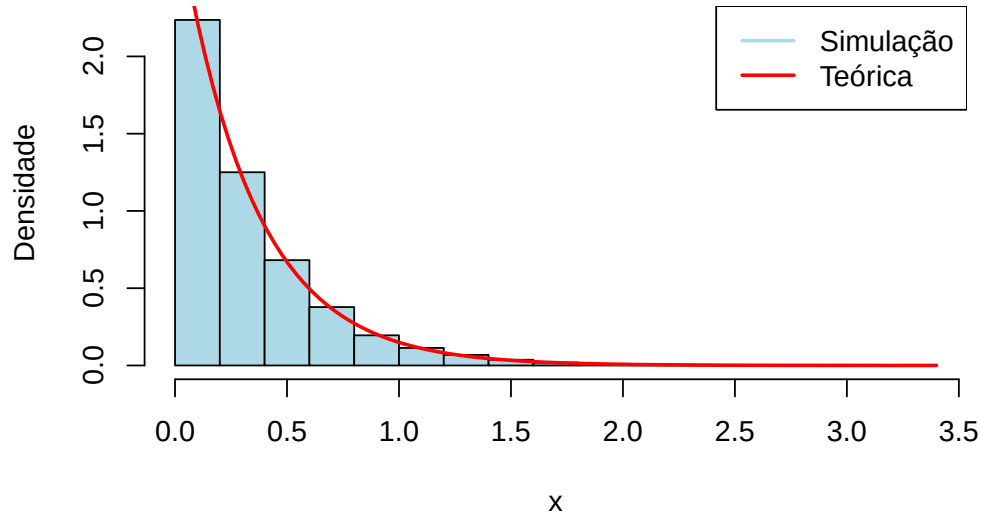


Figura 22.7: Amostra de uma Exponencial(3) gerada por $X = -\ln(U)/\text{lambda}$.

mostre que $X = x_0 + \gamma \tan[\pi(U - 0,5)]$. Gere 1000 valores, visualize num histograma e compare com a densidade teórica (`dcauchy()`).

4. Para a distribuição de **Weibull** $W(\alpha, \beta)$, com densidade $f(x) = \alpha\beta^{-\alpha}x^{\alpha-1}e^{-(x/\beta)^\alpha}$ ($x > 0$) e função de distribuição $F(x) = 1 - e^{-(x/\beta)^\alpha}$, mostre que $X = \beta[-\ln(U)]^{1/\alpha}$. Gere 10 000 valores de uma $W(2, 3)$, visualize num histograma e compare com a densidade teórica (`dweibull()`).

5. Obtenha o gerador de números aleatórios das distribuições seguintes pelo método da transformada inversa:

- (a) $F(x) = 1 - (x - 1)^2$, $0 < x < 1$;
- (b) $F(x) = x^\theta$, $0 < x < 1$, $\theta > 1$;
- (c) $F(x) = 1 - e^{-x^2/(2\tau^2)}$, $x > 0$, $\tau > 0$;
- (d) $f(x) = \frac{1}{2}\sin(x)$, $0 < x < \pi$;
- (e) $f(x) = \frac{\sec^2(x)}{\sqrt{3}}$, $0 < x < \pi/3$.

Soluções.

- (a) $F^{-1}(u) = 1 - \sqrt{1 - u}$ (ou, por simetria, $1 - \sqrt{u}$).
- (b) $F^{-1}(u) = u^{1/\theta}$.
- (c) $F^{-1}(u) = \tau\sqrt{-2\ln(1 - u)}$.
- (d) $F(x) = \frac{1 - \cos(x)}{2}$ e $F^{-1}(u) = \arccos(1 - 2u)$.
- (e) $F(x) = \frac{\tan(x)}{\sqrt{3}}$ e $F^{-1}(u) = \arctan(u\sqrt{3})$.

22.4 Quando usar o método?

Sempre que for possível, o método da transformada inversa é a primeira escolha: é exato, simples e usa um único número uniforme por observação. A sua aplicabilidade depende, porém, de conseguirmos a inversa F^{-1} :

- funciona diretamente quando $F^{-1}(u)$ tem forma fechada (foi o caso de todos os exemplos deste capítulo);
- em muitas distribuições, F existe mas **não é invertível analiticamente** (a normal é o exemplo mais conhecido: não há fórmula fechada para F^{-1});
- noutras, nem sequer dispomos de F em forma fechada, apenas da densidade f .

Nesses casos, há alternativas: aproximar numericamente F e F^{-1} , ou recorrer a métodos que dispensam a inversa e trabalham diretamente com a densidade. É a um desses métodos — o de **aceitação-rejeição** — que dedicamos o próximo capítulo.

Capítulo 23

Método da aceitação-rejeição

No capítulo anterior vimos que o método da transformação inversa nos permite gerar observações de uma variável aleatória sempre que conhecemos — e sabemos inverter — a sua função de distribuição. Essa é uma condição forte. Para muitas distribuições de grande importância prática, a função de distribuição não tem forma fechada (basta pensar na normal) ou, tendo-a, não é invertível de forma explícita. Precisamos, por isso, de uma segunda ferramenta, mais geral, que funcione mesmo quando só conhecemos a **função densidade** f e não a sua primitiva. Essa ferramenta é o **método da aceitação-rejeição**.

A ideia é engenhosamente simples: se não sabemos amostrar diretamente de f , amostramos de uma distribuição *auxiliar* que sabemos gerar com facilidade e, em seguida, “filtramos” essas observações, aceitando umas e rejeitando outras, de modo que as sobreviventes se distribuam exatamente segundo f . É como esculpir a distribuição desejada a partir de um bloco de material mais fácil de obter.

23.1 Contexto histórico

O método da aceitação-rejeição deve-se a **John von Neumann**, que o descreveu em 1951 no artigo *Various techniques used in connection with random digits*, publicado no âmbito dos primeiros trabalhos sobre o método de Monte Carlo. Von Neumann integrava, juntamente com Stanislaw Ulam e Nicholas Metropolis, o grupo de Los Alamos que, no final da década de 1940, desenvolveu a simulação estocástica como instrumento de cálculo científico — precisamente na altura em que surgiam os primeiros computadores eletrônicos capazes de produzir e manipular grandes quantidades de números pseudoaleatórios.

O que torna o método notável é a sua generalidade e a elegância do argumento que o sustenta. Não exige o conhecimento da função de distribuição: basta saber avaliar a densidade alvo, ou uma função proporcional a ela, ponto a ponto, e dispor de uma distribuição auxiliar cómoda.

Mais à frente veremos que esta possibilidade é particularmente útil quando a constante de normalização da distribuição alvo é desconhecida.

Neste capítulo aprendemos a gerar observações de uma distribuição cuja densidade alvo f é conhecida, mas cuja função de distribuição acumulada pode ser difícil de obter ou de inverter.

O método da aceitação-rejeição constrói amostras dessa distribuição a partir de uma densidade auxiliar g , da qual sabemos simular, desde que exista uma constante $c > 0$ tal que

$$f(x) \leq c g(x)$$

para todos os valores x no suporte da distribuição alvo.

A ideia central é gerar candidatos a partir de g e aceitá-los com uma probabilidade cuidadosamente escolhida, de modo que os valores aceites tenham exactamente a distribuição pretendida.

23.2 A ideia geométrica

Antes da formalização, vale a pena fixar a intuição. Suponhamos que queremos gerar pontos sob o gráfico de uma densidade f . Se conseguíssemos distribuir pontos **uniformemente** na região delimitada pelo eixo das abcissas e pela curva $y = f(x)$, então a abcissa X de um ponto escolhido ao acaso nessa região teria exactamente densidade f . Este é o princípio fundamental: gerar uniformidade numa região a duas dimensões e ler a primeira coordenada.

O problema é que raramente sabemos amostrar uniformemente sob uma curva arbitrária. A solução de von Neumann é cobrir essa região com outra, maior e mais simples — a região sob uma curva $c g(x)$, onde g é uma densidade fácil de simular e c é uma constante que garante que $c g$ fica sempre *acima* de f . Amostramos uniformemente sob a curva mais fácil e ficamos apenas com os pontos que caem também sob f . Os restantes são descartados.

23.3 Formalização do método

Sejam f a densidade da qual pretendemos amostrar (a densidade *alvo*) e g uma densidade auxiliar, dita **densidade de proposta** ou *instrumental*, que sabemos simular. A única exigência é que exista uma constante $c \geq 1$ tal que

$$f(x) \leq c g(x), \quad \text{para todo o } x \text{ com } f(x) > 0.$$

Equivalentemente, exige-se que o rácio $f(x)/g(x)$ seja limitado e que

$$c = \sup_{x: f(x) > 0} \frac{f(x)}{g(x)} < \infty.$$

Repare-se numa condição implícita mas essencial: o **suporte** de g tem de conter o suporte de f . Se existir algum ponto onde $f(x) > 0$ mas $g(x) = 0$, o rácio explode e o método não é aplicável — nunca poderíamos gerar observações nessa zona porque a proposta lá não coloca massa.

A desigualdade $f(x) \leq c g(x)$ tem de valer em **todo** o suporte de f , não apenas na região onde a maioria da massa se concentra. Se escolhermos um c demasiado pequeno, a envolvente $c g$ deixa de cobrir f nalgum ponto e as observações produzidas seguem uma distribuição *errada*, sem que qualquer mensagem de erro nos alerte. É um erro silencioso e, por isso, particularmente perigoso.

23.3.1 O algoritmo

O procedimento de aceitação-rejeição consiste em repetir os seguintes passos:

1. Gerar uma observação Y da densidade de proposta g .
2. Gerar $U \sim \mathcal{U}(0, 1)$, independente de Y .
3. Se

$$U \leq \frac{f(Y)}{c g(Y)},$$

aceitar e devolver $X = Y$. Caso contrário, **rejeitar** Y e voltar ao passo 1.

O quociente $f(Y)/[c g(Y)]$ está sempre entre 0 e 1, pelo que a comparação com um uniforme faz sentido. Intuitivamente, um valor Y é aceite com uma probabilidade tanto maior quanto mais “alta” a densidade alvo estiver nesse ponto, relativamente à envolvente. Onde f e $c g$ quase coincidem, quase todos os Y são aceites; onde f é muito menor que $c g$, a maioria é descartada.

23.3.2 Demonstração de que o método é correto

Vejamos por que razão as observações aceites têm efetivamente densidade f . Um valor devolvido pelo algoritmo é um valor de Y *condicionado* ao acontecimento “aceitar”. Calculemos primeiro a probabilidade de aceitação num único passo, para um dado $Y = y$:

$$P\left(U \leq \frac{f(Y)}{c g(Y)} \mid Y = y\right) = \frac{f(y)}{c g(y)}.$$

Integrando sobre a distribuição de Y , a probabilidade *incondicional* de aceitação em cada passo é

$$p = P(\text{aceitar}) = \int \frac{f(y)}{c g(y)} g(y) dy = \frac{1}{c} \int f(y) dy = \frac{1}{c}.$$

A identidade $P(\text{aceitar}) = 1/c$ pressupõe que f é uma densidade normalizada, isto é,

$$\int f(y) dy = 1.$$

Se, em vez de uma densidade normalizada, trabalharmos com uma função $\tilde{f}$ apenas proporcional à densidade alvo, então a interpretação da taxa de aceitação muda. Nesse caso, existe uma constante $Z > 0$ tal que

$$f(y) = \frac{\tilde{f}(y)}{Z},$$

onde f é a densidade normalizada e $Z = \int \tilde{f}(y) dy$ é a constante de normalização. Se usarmos uma constante $\tilde{c}$ tal que

$$\tilde{f}(y) \leq \tilde{c} g(y),$$

a probabilidade média de aceitação passa a ser

$$P(\text{aceitar}) = \frac{Z}{\tilde{c}},$$

e não necessariamente $1/\tilde{c}$.

Assim, quando a função alvo usada no algoritmo já está normalizada como densidade, a taxa teórica de aceitação é $1/c$. Quando se trabalha com uma função proporcional à densidade, a constante de normalização altera a interpretação directa dessa taxa.

Agora, a função de distribuição do valor devolvido é a de Y condicionada à aceitação. Para qualquer x ,

$$P(X \leq x) = P(Y \leq x \mid \text{aceitar}) = \frac{P(Y \leq x, \text{aceitar})}{P(\text{aceitar})}.$$

O numerador é

$$P(Y \leq x, \text{aceitar}) = \int_{-\infty}^x \frac{f(y)}{c g(y)} g(y) dy = \frac{1}{c} \int_{-\infty}^x f(y) dy = \frac{F(x)}{c},$$

onde F é a função de distribuição associada a f . Dividindo pelo denominador $p = 1/c$, obtém-se

$$P(X \leq x) = \frac{F(x)/c}{1/c} = F(x).$$

Ou seja, o valor aceite tem exactamente a função de distribuição F , logo a densidade f . É este o resultado que legitima o método. ■

23.3.3 Eficiência: o papel da constante c

A demonstração revelou um facto de enorme importância prática: a probabilidade de aceitar em cada tentativa é $p = 1/c$. O número N de tentativas necessárias até obter a primeira aceitação segue, portanto, uma distribuição **geométrica** de parâmetro $1/c$, e o seu valor esperado é

$$E[N] = \frac{1}{p} = c.$$

Por outras palavras, em média são precisas c tentativas para produzir uma única observação aceite. A constante c é assim, simultaneamente, o *fator de sobre-cobertura* geométrico e o *custo médio* do algoritmo.

Como $c \geq 1$ sempre (porque f e g são densidades e integram 1), o melhor cenário possível é $c = 1$, que só ocorre quando $f = g$. Na prática, quanto mais a proposta g se “assemelhar” à alvo f , menor será c e mais eficiente o método. A arte de aplicar a aceitação-rejeição está, em boa medida, na escolha de uma boa proposta.

Este é o principal critério para comparar propostas: entre duas densidades auxiliares admissíveis, prefere-se a que conduz ao menor c . Uma proposta cómoda de simular mas que force um c muito grande pode ser, no cômputo geral, pior do que uma proposta um pouco mais trabalhosa mas com c próximo de 1.

23.4 Implementação genérica em R

Começemos por traduzir o algoritmo numa função reutilizável. A função recebe o número de observações desejadas, a densidade alvo f , uma função `g_amostra` que gera observações da proposta, a densidade da proposta `g_dens` e a constante c . Para além da amostra, devolve a taxa de aceitação observada, que nos permite verificar empiricamente se ela se aproxima de $1/c$.

```
aceita_rejeita <- function(n, f, g_amostra, g_dens, c) {
  amostra    <- numeric(n)
  aceites    <- 0
  tentativas <- 0
  while (aceites < n) {
    y <- g_amostra(1)           # observação da proposta
    u <- runif(1)               # uniforme para o teste
    tentativas <- tentativas + 1
    if (u <= f(y) / (c * g_dens(y))) {
      aceites <- aceites + 1
      amostra[aceites] <- y
    }
  }
}
```

```

}
list(amostra = amostra,
      taxa_aceitacao = aceites / tentativas)
}

```

A versão acima gera uma observação de cada vez, o que é claro do ponto de vista didático mas relativamente lento em R, uma linguagem em que os ciclos explícitos têm um custo apreciável. Numa aplicação em que a eficiência importe, é preferível **vetorizar**: gerar um grande lote de candidatos e respetivos uniformes de uma só vez, aceitar em bloco os que passam no teste, e repetir apenas se ainda faltarem observações. A lógica é idêntica; muda só a forma de a executar.

23.5 Exemplo 1 — uma densidade limitada com proposta uniforme

Consideremos a densidade da distribuição Beta(2, 2), dada por

$$f(x) = 6x(1-x), \quad 0 < x < 1,$$

e nula fora de $(0, 1)$. Esta densidade tem suporte limitado, pelo que uma escolha natural para a proposta é a uniforme em $(0, 1)$, ou seja, $g(x) = 1$ nesse intervalo. Como g é constante, determinar c reduz-se a encontrar o máximo de f :

$$c = \sup_{0 < x < 1} \frac{f(x)}{g(x)} = \max_{0 < x < 1} 6x(1-x).$$

Derivando e igualando a zero, o máximo ocorre em $x = 1/2$, onde $f(1/2) = 6 \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{2}$. Logo $c = 1,5$, e esperamos uma taxa de aceitação de $1/c \approx 0,667$.

```

set.seed(2024)

f_beta <- function(x) ifelse(x > 0 & x < 1, 6 * x * (1 - x), 0)
g_unif <- function(x) ifelse(x > 0 & x < 1, 1, 0)

resultado <- aceita_rejeita(
  n      = 10000,
  f      = f_beta,
  g_amostra = function(n) runif(n),
  g_dens   = g_unif,
  c       = 1.5
)

```

```
)

resultado$taxa_aceitacao      # deverá rondar 0.667
## [1] 0.6674
```

A taxa de aceitação observada está muito próxima do valor teórico $1/1,5$, o que confirma o funcionamento do algoritmo. Comparemos agora o histograma das observações geradas com a densidade teórica.

```
hist(resultado$amostra, breaks = 40, probability = TRUE,
      col = "grey85", border = "white",
      main = "", xlab = "x", ylab = "densidade")
curve(f_beta, from = 0, to = 1, add = TRUE, lwd = 2, col = "#10656d")
legend("topright", legend = "densidade Beta(2,2)",
      lwd = 2, col = "#10656d", bty = "n")
```

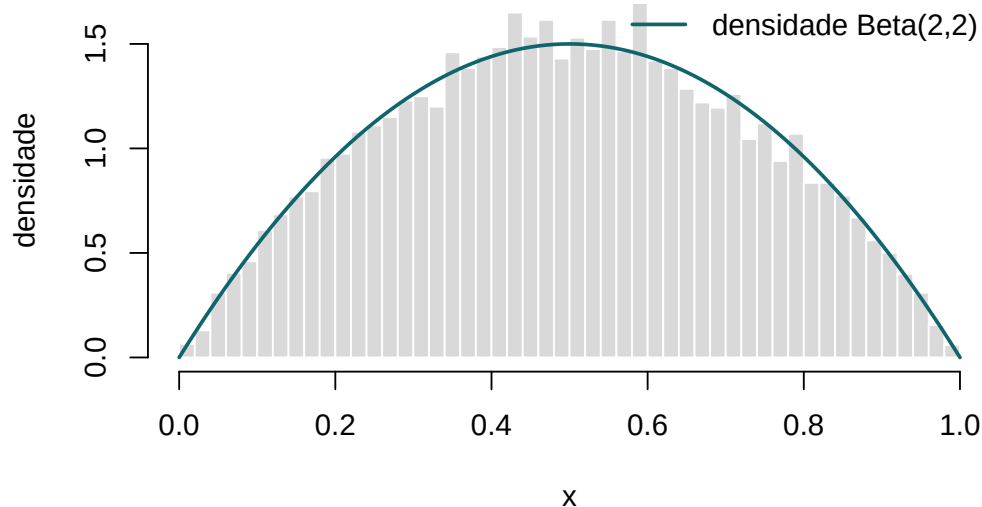


Figura 23.1: Histograma de 10 000 observações geradas pelo método da aceitação-rejeição, sobreposto à densidade teórica da $Beta(2,2)$.

O acordo entre o histograma e a curva teórica é excelente. Repare-se ainda que, tendo já implementado o método, poderíamos ter obtido a mesma amostra com `rbeta(10000, 2, 2)`; o exercício serve para *validar* a nossa implementação contra uma referência de confiança, uma boa prática sempre que se programa um gerador de raiz.

23.6 Exemplo 2 — a normal padrão a partir da exponencial

O exemplo anterior era confortável porque a densidade alvo tinha suporte limitado. Vejamos agora um caso com suporte ilimitado e uma proposta menos óbvia, que ilustra bem a subtileza do método: gerar observações da **normal padrão** $N(0, 1)$.

A estratégia clássica passa por gerar primeiro o **valor absoluto** de uma normal padrão — que segue a chamada distribuição *meia-normal*, com densidade

$$f(x) = \sqrt{\frac{2}{\pi}} e^{-x^2/2}, \quad x \geq 0,$$

e, no fim, atribuir-lhe um sinal aleatório, positivo ou negativo com igual probabilidade. Como proposta para a meia-normal usamos a exponencial de parâmetro 1, $g(x) = e^{-x}$ para $x \geq 0$, que sabemos simular facilmente pela transformação inversa.

Determinemos c . O rácio das densidades é

$$\frac{f(x)}{g(x)} = \sqrt{\frac{2}{\pi}} e^{-x^2/2+x} = \sqrt{\frac{2}{\pi}} e^{1/2} e^{-(x-1)^2/2},$$

onde completámos o quadrado no expoente. O fator $e^{-(x-1)^2/2}$ é máximo em $x = 1$, onde vale 1. Portanto

$$c = \sqrt{\frac{2}{\pi}} e^{1/2} = \sqrt{\frac{2e}{\pi}} \approx 1,3155.$$

A taxa de aceitação esperada é $1/c \approx 0,760$ — bastante boa. E a condição de aceitação simplifica-se de forma elegante:

$$U \leq \frac{f(Y)}{c g(Y)} = e^{-(Y-1)^2/2}.$$

```
set.seed(2024)

c_norm <- sqrt(2 * exp(1) / pi)
c_norm
## [1] 1.315

gerar_normal_ar <- function(n) {
  x <- numeric(n)
  aceites <- 0
  tentativas <- 0
```

```

while (aceites < n) {
  y <- rexp(1, rate = 1)           # proposta: Exp(1)
  u <- runif(1)
  tentativas <- tentativas + 1
  if (u <= exp(-(y - 1)^2 / 2)) {  # teste simplificado
    aceites <- aceites + 1
    s <- sample(c(-1, 1), 1)      # sinal aleatório
    x[aceites] <- s * y
  }
}
list(amostra = x, taxa_aceitacao = aceites / tentativas)
}

normais <- gerar_normal_ar(10000)
normais$taxa_aceitacao           # deverá rondar 0.760
## [1] 0.7596

```

```

hist(normais$amostra, breaks = 50, probability = TRUE,
     col = "grey85", border = "white",
     main = "", xlab = "x", ylab = "densidade")
curve(dnorm(x), from = -4, to = 4, add = TRUE, lwd = 2, col = "#10656d")
legend("topright", legend = "densidade N(0,1)",
     lwd = 2, col = "#10656d", bty = "n")

```

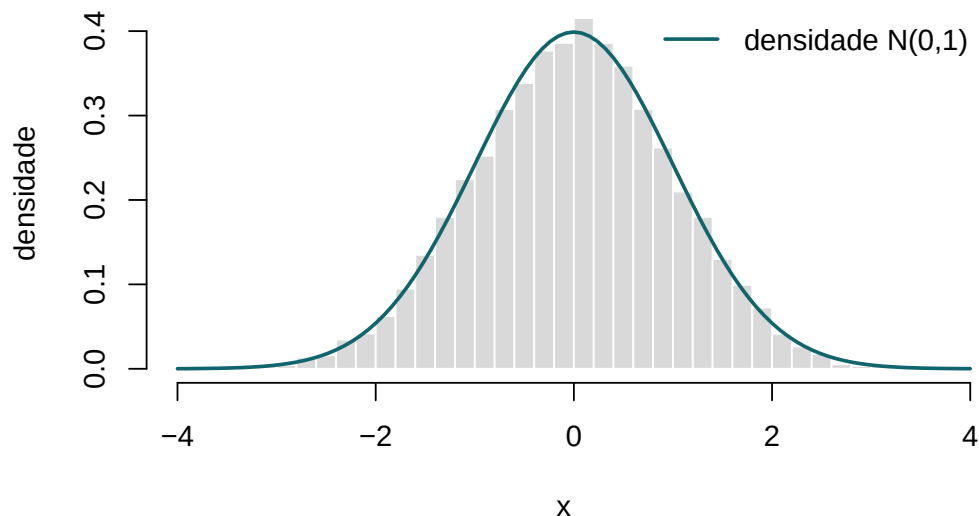


Figura 23.2: Observações da normal padrão geradas por aceitação-rejeição a partir de uma proposta exponencial, com a densidade $N(0,1)$ sobreposta.

Uma verificação numérica adicional: a média e o desvio-padrão amostrais devem aproximar-se

de 0 e 1, respetivamente.

```
c(media = mean(normais$amostra), desvio = sd(normais$amostra))
##      media      desvio
## -0.0002611  1.0000975
```

Este exemplo tem valor histórico: variantes deste esquema — combinando a exponencial como proposta com transformações inteligentes — estiveram na base dos primeiros geradores eficientes de números normais. Hoje o R usa por defeito o método da inversão (`rnorm`), mais rápido, mas o argumento da aceitação-rejeição permanece o mais transparente para compreender *porque é que* tais geradores funcionam.

23.7 O caso de funções não normalizadas

Uma virtude importante do método é que, em muitas situações, não precisamos de conhecer explicitamente a constante de normalização da distribuição alvo.

Suponhamos que queremos amostrar de uma densidade f , mas conhecemos apenas uma função $\tilde{f}$ proporcional a ela. Isto significa que

$$f(x) = \frac{\tilde{f}(x)}{Z}, \quad Z = \int \tilde{f}(x) dx,$$

onde Z é a constante de normalização. Em muitas aplicações, sobretudo em estatística bayesiana, Z pode ser difícil ou impossível de calcular analiticamente.

O método da aceitação-rejeição continua a ser aplicável se encontrarmos uma densidade de proposta g e uma constante $\tilde{c}$ tais que

$$\tilde{f}(x) \leq \tilde{c} g(x)$$

para todos os x no suporte da distribuição alvo. Nesse caso, podemos aceitar um candidato $Y \sim g$ quando

$$U \leq \frac{\tilde{f}(Y)}{\tilde{c} g(Y)}.$$

Os valores aceites têm densidade proporcional a $\tilde{f}$, logo têm a densidade alvo f . A constante de normalização Z não precisa de ser conhecida para executar o algoritmo.

Contudo, há uma diferença importante: quando usamos uma densidade normalizada f , a probabilidade média de aceitação é $1/c$. Quando usamos uma função não normalizada $\tilde{f}$, a probabilidade média de aceitação é

$$P(\text{aceitar}) = \frac{Z}{\tilde{c}}.$$

Portanto, a taxa de aceitação observada pode até fornecer informação sobre a constante de normalização Z , mas já não deve ser interpretada simplesmente como $1/\tilde{c}$.

23.8 Síntese

O método da aceitação-rejeição alarga consideravelmente o alcance da simulação: permite gerar observações de praticamente qualquer densidade, exigindo apenas que a saibamos avaliar ponto a ponto e que disponhamos de uma proposta cómoda que a domine, a menos de uma constante c . O preço a pagar é o descarte de parte das observações — em média, c tentativas por cada aceite — o que transforma a escolha da proposta no ponto crítico de toda a aplicação: uma boa proposta é a que fica “colada” à densidade alvo, mantendo c próximo de 1.

23.9 Exercícios

Nos exercícios seguintes, implemente o método da aceitação-rejeição, escolha uma constante c adequada (justificando-a) e valide sempre o resultado comparando o histograma das observações geradas com a densidade teórica. Reporte também a taxa de aceitação e confronte-a com o valor esperado $1/c$.

1. Gere variáveis aleatórias com distribuição exponencial $\text{Exp}(\lambda = 1)$ usando o método da aceitação-rejeição. Use uma função de proposta uniforme no intervalo $[0, 5]$ e visualize o histograma das observações geradas. Comente o efeito de truncar o suporte da exponencial em 5.
2. Gere variáveis aleatórias com distribuição normal $N(0, 1)$ usando uma distribuição de proposta Cauchy padrão $C(0, 1)$. Gere 1000 observações e compare-as com a densidade da normal padrão. Determine analiticamente a constante c .
3. Use o método da aceitação-rejeição para gerar observações de uma variável $\text{Beta}(2, 3)$, recorrendo a uma proposta uniforme $\mathcal{U}(0, 1)$. Calcule o valor exato de c localizando o máximo da densidade.
4. Use o método da aceitação-rejeição para gerar variáveis aleatórias $X \sim \Gamma(3, 2)$ (forma 3, taxa 2) usando uma proposta exponencial $\text{Exp}(2)$. Explique por que razão a proposta deve ter média maior ou igual à da distribuição alvo para que c seja finito.
5. Use o método da aceitação-rejeição para gerar variáveis log-normais $X \sim \text{LogNormal}(0, 1)$ usando uma proposta normal $N(0, 1)$. Discuta as dificuldades que surgem por a log-normal ter cauda mais pesada do que a normal.

6. Use o método da aceitação-rejeição para gerar variáveis de Weibull W (forma = 2, escala = 1) usando uma proposta exponencial. Determine c numericamente maximizando o rácio das densidades com `optimize()`.
7. Considere a densidade $f(x) = \frac{1}{2}(1 + \cos x)$ no intervalo $[-\pi, \pi]$ e nula fora dele. Gere observações desta distribuição com uma proposta uniforme em $[-\pi, \pi]$ e verifique o resultado graficamente.
8. Implemente uma versão **vetorizada** da função `aceita_rejeita` que gere os candidatos em lotes (por exemplo, de tamanho $2n$ de cada vez) e aceite em bloco os que passam no teste, repetindo apenas se necessário. Compare o tempo de execução com a versão baseada num ciclo `while`, usando `system.time()`, para gerar 10^5 observações da $\text{Beta}(2,2)$.
9. Estude, por simulação, como a **eficiência** do método depende da constante c . Para gerar observações da meia-normal a partir da exponencial, registre a taxa de aceitação empírica e compare-a com $1/c$. Em seguida, repita a experiência propositadamente com um c demasiado *grande* (por exemplo, o dobro do correto) e confirme que a distribuição gerada continua correta, mas que a taxa de aceitação cai. O que aconteceria com um c demasiado *pequeno*?
10. A densidade $f(x) = \frac{2}{\pi}\sqrt{1-x^2}$ em $[-1, 1]$ (a distribuição do *semicírculo*, ou lei de Wigner no intervalo unitário) surge no estudo de matrizes aleatórias. Gere 5000 observações desta distribuição por aceitação-rejeição com proposta uniforme em $[-1, 1]$, determine c analiticamente e confirme graficamente o resultado.
11. (*Densidade não normalizada.*) Pretende-se amostrar de uma densidade proporcional a $\tilde{f}(x) = e^{-x^2/2}(1 + \sin^2 3x)$ em $\mathbb{R}$, cuja constante de normalização não é conhecida em forma fechada. Usando uma proposta normal com variância suficientemente grande, mostre que consegue gerar observações de f sem calcular essa constante. Estime, a partir da taxa de aceitação, o valor da constante de normalização.

Capítulo 24

Teoremas limites

Ao longo dos capítulos anteriores, gerámos repetidamente amostras aleatórias e calculámos médias, proporções e outras estatísticas. Sempre que o fizemos, apoiámos-nos — por vezes de forma implícita — em dois resultados que constituem o alicerce de toda a inferência estatística e de toda a simulação: a **lei dos grandes números** e o **teorema limite central**. São eles que explicam por que razão a média de muitas observações se aproxima do valor esperado, e por que razão a distribuição normal surge, uma e outra vez, onde menos se espera. Este capítulo dedica-se a compreendê-los, a demonstrá-los nos casos acessíveis e, sobretudo, a *vê-los em ação* através da simulação.

Os teoremas limites clássicos da probabilidade referem-se a sequências de variáveis aleatórias independentes e identicamente distribuídas (i.i.d.). Consideremos uma tal sequência $X_1, X_2, \dots$ com média comum $E(X_i) = \mu < \infty$, e definamos a soma parcial

$$S_n = X_1 + X_2 + \dots + X_n.$$

A média amostral é $\bar{X}_n = S_n/n$. A **lei dos grandes números** afirma que $\bar{X}_n$ converge para μ quando $n \rightarrow +\infty$; o **teorema limite central** vai mais longe e descreve *como* se comporta o desvio de $\bar{X}_n$ em torno de μ , revelando que, depois de convenientemente reescalado, esse desvio é aproximadamente normal, seja qual for a distribuição de partida.

24.1 Contexto histórico

A primeira versão da lei dos grandes números deve-se a **Jacob Bernoulli**, que a demonstrou para proporções de sucessos em provas de Bernoulli na sua obra *Ars Conjectandi*, publicada postumamente em 1713. Bernoulli chamou-lhe o seu “teorema de ouro” e dedicou-lhe vinte anos de trabalho: era a prova matemática de uma convicção antiga, a de que a frequência relativa de um acontecimento tende a estabilizar em torno da sua probabilidade à medida que se acumulam

observações. O nome “lei dos grandes números” foi cunhado mais tarde, em 1837, por **Siméon Denis Poisson**.

A distinção entre as versões *fraca* e *forte* da lei — que abaixo tornaremos precisa — foi clarificada no início do século XX: **Émile Borel** estabeleceu em 1909 a lei forte para o caso binomial, e **Andrei Kolmogorov** deu-lhe, nos anos 1930, a formulação geral que hoje usamos, na sequência da sua axiomatização da teoria da probabilidade.

O teorema limite central tem uma história igualmente ilustre. **Abraham de Moivre** obteve, em 1733, a aproximação normal da distribuição binomial; **Pierre-Simon Laplace** generalizou-a substancialmente em 1810; e a versão geral, com condições rigorosas, foi estabelecida por **Aleksandr Lyapunov** (1901) e, na forma que hoje é mais citada para variáveis i.i.d., por **Lindeberg** e **Lévy**. A designação *teorema limite central* — no sentido de teorema *central* da teoria — foi popularizada por George Pólya em 1920.

24.2 Modos de convergência

Falar de convergência de variáveis aleatórias exige cuidado, porque uma sucessão de variáveis aleatórias pode convergir em sentidos diferentes. Três modos interessam-nos aqui.

Diz-se que Y_n converge **em probabilidade** para Y , e escreve-se $Y_n \xrightarrow{P} Y$, se para todo o $\varepsilon > 0$

$$\lim_{n \rightarrow \infty} P(|Y_n - Y| > \varepsilon) = 0.$$

Diz-se que Y_n converge **quase certamente** (ou com probabilidade 1) para Y , e escreve-se $Y_n \xrightarrow{\text{q.c.}} Y$, se

$$P\left(\lim_{n \rightarrow \infty} Y_n = Y\right) = 1.$$

Diz-se que Y_n converge **em distribuição** para Y , e escreve-se $Y_n \xrightarrow{d} Y$, se $F_{Y_n}(y) \rightarrow F_Y(y)$ em todos os pontos y de continuidade de F_Y .

Estes três modos não são equivalentes. A convergência quase certa é mais forte do que a convergência em probabilidade (implica-a), e esta, por sua vez, é mais forte do que a convergência em distribuição. A lei *fraca* dos grandes números diz respeito à convergência em probabilidade; a lei *forte*, à convergência quase certa; o teorema limite central, à convergência em distribuição. É esta a distinção que dá nome aos resultados.

24.3 Lei fraca dos grandes números

A lei fraca dos grandes números é também conhecida como *teorema de Bernoulli*. De acordo com a lei, a média dos resultados obtidos num grande número de provas está próxima da média da população, com probabilidade cada vez mais alta à medida que o número de provas aumenta.

Enunciar e demonstrar a lei fraca dos grandes números no caso de variância finita, compreender o seu significado e ilustrá-lo por simulação.

Sejam $X_1, \dots, X_n$ variáveis aleatórias i.i.d., cada uma com média μ e variância $\sigma^2 < \infty$. Então, para todo o $\varepsilon > 0$,

$$\lim_{n \rightarrow \infty} P(|\bar{X}_n - \mu| > \varepsilon) = 0, \quad \text{ou seja,} \quad \bar{X}_n \xrightarrow{P} \mu.$$

Demonstração. A média amostral tem valor esperado

$$E(\bar{X}_n) = \frac{1}{n} \sum_{i=1}^n E(X_i) = \mu,$$

e, pela independência das observações, variância

$$\text{Var}(\bar{X}_n) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n}.$$

Aplicando a **desigualdade de Chebyshev** à variável $\bar{X}_n$, cuja média é μ e cuja variância é σ^2/n , obtém-se

$$P(|\bar{X}_n - \mu| > \varepsilon) \leq \frac{\text{Var}(\bar{X}_n)}{\varepsilon^2} = \frac{\sigma^2}{n \varepsilon^2}.$$

Fixado $\varepsilon > 0$, o membro direito tende para zero quando $n \rightarrow \infty$, o que estabelece a convergência em probabilidade. ■

A demonstração acima usa a hipótese de variância finita porque recorre à desigualdade de Chebyshev. Existe, porém, uma versão mais geral — o *teorema de Khinchin* — que garante a lei fraca exigindo apenas que a média μ exista e seja finita, dispensando a hipótese sobre a variância. A demonstração desse resultado é mais delicada e recorre a funções características.

24.4 Lei forte dos grandes números

A lei fraca garante que, para n grande, é *improvável* que $\bar{X}_n$ se afaste de μ . A lei **forte** afirma algo qualitativamente mais poderoso: que a própria sucessão $\bar{X}_1, \bar{X}_2, \dots$ converge para μ , encarada como uma trajetória, com probabilidade 1.

Sejam $X_1, X_2, \dots$ variáveis aleatórias i.i.d. com $E|X_1| < \infty$ e média μ . Então

$$\bar{X}_n \xrightarrow{\text{q.c.}} \mu \quad \text{quando} \quad n \rightarrow \infty.$$

Este é o **teorema de Kolmogorov**. Note-se que exige apenas a existência da média — nem sequer se assume variância finita. A demonstração está para além do âmbito deste livro, mas o seu significado é fundamental e vale a pena fixá-lo.

A diferença entre as duas leis é subtil mas importante. A lei fraca diz que, para cada n suficientemente grande, a probabilidade de $\bar{X}_n$ estar longe de μ é pequena — mas não exclui que, ao longo da sucessão, ocorram ocasionalmente afastamentos grandes. A lei forte garante que, para *quase todas* as trajetórias possíveis, a sucessão acaba por entrar e permanecer arbitrariamente perto de μ : os afastamentos grandes deixam de ocorrer a partir de certa ordem. É esta a garantia que legitima, em última análise, a interpretação frequencista da probabilidade e o método de Monte Carlo.

24.4.1 Ilustração por simulação

Vejamos a lei dos grandes números em funcionamento. Simulemos o lançamento repetido de um dado equilibrado, cuja média teórica é $\mu = (1 + 2 + 3 + 4 + 5 + 6)/6 = 3,5$, e observemos a média acumulada $\bar{X}_n$ a estabilizar em torno desse valor à medida que n cresce.

```
set.seed(2024)

n <- 5000
lancamentos <- sample(1:6, size = n, replace = TRUE)
media_acumulada <- cumsum(lancamentos) / seq_along(lancamentos)

plot(media_acumulada, type = "l", col = "#10656d", lwd = 1.2,
      ylim = c(2.5, 4.5), log = "x",
      xlab = "número de lançamentos (escala logarítmica)",
      ylab = expression(bar(X)[n]))
abline(h = 3.5, lty = 2, col = "grey40")
```

O gráfico mostra com clareza o fenómeno: nos primeiros lançamentos a média acumulada oscila bastante, mas rapidamente se aproxima de 3,5 e aí permanece. A escala logarítmica no eixo horizontal ajuda a ver que a estabilização não é fruto do acaso de uma única trajetória. Para o confirmar, sobreponhamos várias trajetórias independentes.

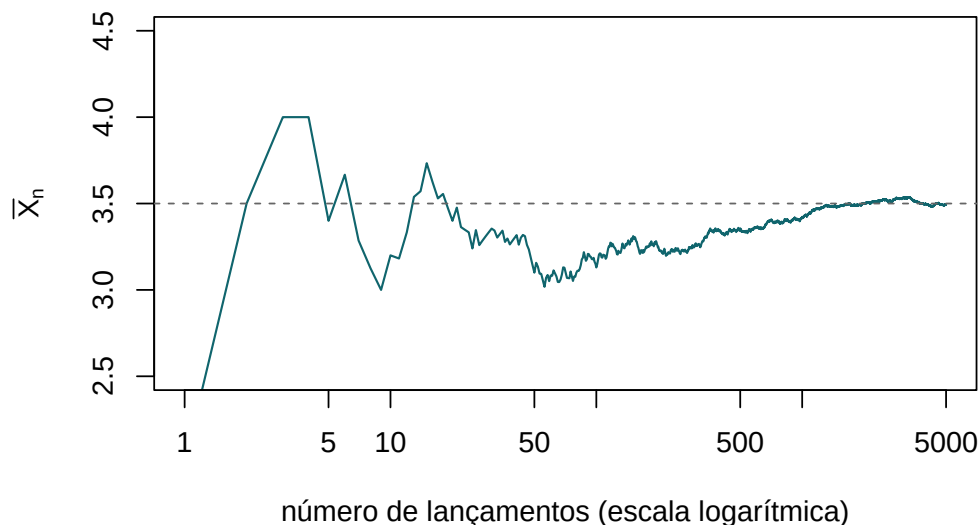


Figura 24.1: Média acumulada dos lançamentos de um dado equilibrado em função do número de lançamentos. A linha tracejada assinala a média teórica, 3,5.

```
set.seed(2024)

plot(NULL, xlim = c(1, n), ylim = c(2.5, 4.5), log = "x",
     xlab = "número de lançamentos (escala logarítmica)",
     ylab = expression(bar(X)[n]))
for (k in 1:10) {
  x <- sample(1:6, size = n, replace = TRUE)
  lines(cumsum(x) / seq_along(x), col = adjustcolor("#10656d", 0.5))
}
abline(h = 3.5, lty = 2, col = "grey30", lwd = 2)
```

Todas as trajetórias, apesar de partirem de valores iniciais dispersos, convergem para a mesma reta horizontal. É precisamente esta a mensagem da lei forte: não é uma trajetória feliz que converge, são (quase) todas.

24.5 Teorema limite central

A lei dos grandes números diz-nos *para onde* converge a média amostral, mas nada diz sobre a *rapidez* dessa convergência nem sobre a *forma* das flutuações em torno do limite. É a essa pergunta que responde o teorema limite central, um dos resultados mais surpreendentes e úteis de toda a matemática.

Sejam $X_1, X_2, \dots$ variáveis aleatórias i.i.d. com média μ e variância σ^2 , ambas finitas, com $\sigma^2 > 0$. Então a soma padronizada converge em distribuição para a normal padrão:

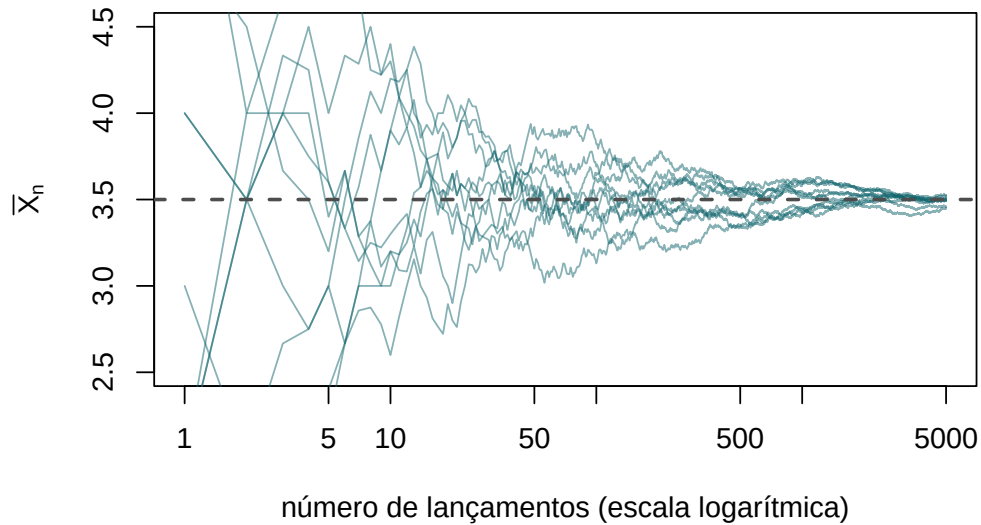


Figura 24.2: Dez trajetórias independentes da média acumulada. Todas convergem para a média teórica, ilustrando a lei forte dos grandes números.

$$Z_n = \frac{S_n - n\mu}{\sigma\sqrt{n}} = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1) \quad \text{quando} \quad n \rightarrow \infty.$$

Equivalentemente, para n grande, a média amostral é aproximadamente normal:

$$\bar{X}_n \approx N\left(\mu, \frac{\sigma^2}{n}\right).$$

O que é notável — quase paradoxal — é que esta conclusão **não depende da distribuição das X_i** . Sejam elas discretas ou contínuas, simétricas ou enviesadas, a distribuição da média padronizada converge sempre para a mesma curva em forma de sino. É esta universalidade que explica a onipresença da normal na estatística: sempre que uma quantidade resulta da soma de muitos contributos pequenos, independentes e de magnitude comparável, é natural que se comporte de forma aproximadamente normal.

O teorema garante convergência quando $n \rightarrow \infty$, mas na prática usamo-lo como *aproximação* para n finito. A qualidade dessa aproximação depende fortemente da forma da distribuição de partida: para distribuições próximas da simetria, uns poucos termos bastam; para distribuições muito enviesadas ou com caudas pesadas, pode ser preciso um n considerável. A regra prática “ $n \geq 30$ ”, frequente nos manuais, é apenas uma orientação grosseira e falha para distribuições fortemente assimétricas.

24.5.1 Ilustração por simulação

Para ver o teorema em ação, partamos de uma distribuição deliberadamente assimétrica — a exponencial de parâmetro 1, que tem média $\mu = 1$ e desvio-padrão $\sigma = 1$, e cuja densidade nada tem de “sino”. Vamos gerar muitas amostras de tamanho n , calcular a média de cada uma, e observar a distribuição dessas médias à medida que n aumenta.

```
set.seed(2024)

simular_medias <- function(n, replicas = 10000) {
  medias <- replicate(replicas, mean(rexp(n, rate = 1)))
  # padronização: (media - mu) / (sigma / sqrt(n)), com mu = sigma = 1
  (medias - 1) / (1 / sqrt(n))
}

z1 <- simular_medias(1)
z5 <- simular_medias(5)
z30 <- simular_medias(30)

op <- par(mfrow = c(1, 3), mar = c(4, 4, 2, 1))
for (par_z in list(list(z1, "n = 1"), list(z5, "n = 5"),
                    list(z30, "n = 30"))) {
  hist(par_z[[1]], breaks = 40, probability = TRUE,
       col = "grey85", border = "white",
       xlim = c(-4, 4), main = par_z[[2]], xlab = "z")
  curve(dnorm(x), add = TRUE, lwd = 2, col = "#10656d")
}
```

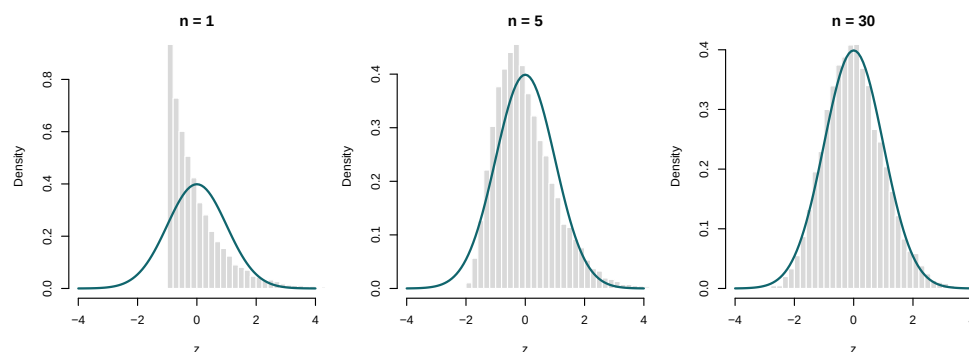


Figura 24.3: Distribuição da média amostral padronizada de observações exponenciais, para amostras de tamanho crescente. A curva a cheio é a densidade normal padrão. À medida que n aumenta, a assimetria da exponencial esbate-se e a distribuição das médias aproxima-se da normal.

```
par(op)
```

A sequência de histogramas conta toda a história. Para $n = 1$, a distribuição das médias é simplesmente a própria exponencial padronizada — visivelmente assimétrica, com uma cauda longa à direita. Para $n = 5$, a assimetria já é bem menor. Para $n = 30$, o histograma é praticamente indistinguível da curva normal sobreposta. Esta convergência, partindo de uma distribuição tão pouco parecida com um sino, é a demonstração visual da universalidade do teorema.

Podemos confirmar a normalidade de forma mais fina com um gráfico quantil-quantil, que compara os quantis empíricos com os da normal padrão. Se a aproximação for boa, os pontos alinham-se sobre a reta.

```
qqnorm(z30, main = "", col = adjustcolor("#10656d", 0.4), pch = 19, cex = 0.4)
qqline(z30, lwd = 2, col = "grey30")
```

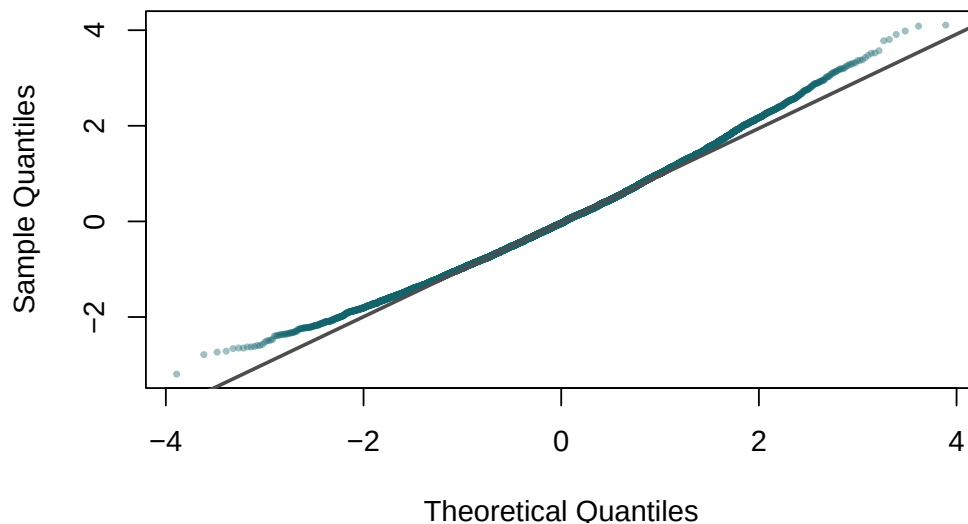


Figura 24.4: Gráfico quantil-quantil das médias padronizadas para $n = 30$. O bom alinhamento sobre a reta confirma a proximidade à normal.

24.5.2 O caso de de Moivre-Laplace

O caso histórico original — a aproximação normal da binomial — é um caso particular do teorema limite central, obtido quando as X_i são variáveis de Bernoulli. Se $X_i \sim \text{Bernoulli}(p)$, então $S_n \sim \text{Binomial}(n, p)$ e o teorema garante que, para n grande,

$$\frac{S_n - np}{\sqrt{np(1-p)}} \xrightarrow{d} N(0, 1).$$

É este resultado que justifica a clássica aproximação da binomial pela normal. Ilustremo-lo comparando a distribuição exata de uma binomial com a densidade normal correspondente.

```
set.seed(2024)

n_ensaios <- 50
p <- 0.3
x <- 0:n_ensaios
prob <- dbinom(x, n_ensaios, p)

plot(x, prob, type = "h", lwd = 2, col = "grey70",
     xlab = "número de sucessos", ylab = "probabilidade")
curve(dnorm(x, mean = n_ensaios * p,
           sd = sqrt(n_ensaios * p * (1 - p))),
      add = TRUE, lwd = 2, col = "#10656d")
```

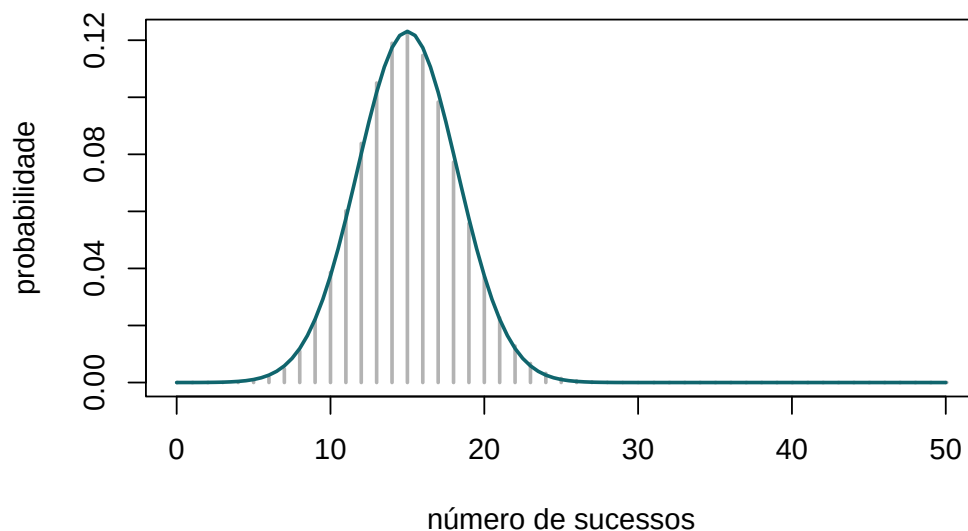


Figura 24.5: Distribuição binomial(50; 0,3) (barras) e aproximação normal (curva). Para n moderado, o acordo já é notável.

24.6 Síntese

A lei dos grandes números e o teorema limite central formam a espinha dorsal da estatística inferencial e justificam a própria legitimidade da simulação. A primeira garante que a média de muitas observações converge para o valor esperado — na versão fraca, em probabilidade; na versão forte, quase certamente. O segundo descreve as flutuações em torno desse limite, mostrando que, depois de reescaladas por $\sqrt{n}$, elas são universalmente normais, independentemente da distribuição de partida. Juntos, estes resultados dizem-nos não só *que* as estimativas

de Monte Carlo convergem, mas *a que ritmo e com que incerteza* — tema que retomaremos no próximo capítulo, ao aproximar integrais.

24.7 Exercícios

1. Simule o lançamento de uma moeda equilibrada $n = 10\,000$ vezes e represente graficamente a proporção acumulada de caras em função do número de lançamentos. Confirme visualmente a convergência para 0,5 e comente a rapidez com que a proporção se estabiliza.
2. Repita o exercício anterior com uma moeda *viciada*, em que a probabilidade de cara é 0,7. Sobreponha a reta horizontal correspondente ao valor teórico e verifique a convergência.
3. Gere $n = 2000$ observações de uma distribuição de Poisson de parâmetro $\lambda = 4$ e represente a média acumulada. Compare a trajetória com a média teórica e discuta se a lei se aplica (a Poisson tem variância finita?).
4. Ilustre a lei dos grandes números para a média amostral de uma distribuição **uniforme** $\mathcal{U}(0, 1)$. Qual o valor para o qual a média acumulada deve convergir? Confirme por simulação.
5. A distribuição de **Cauchy** padrão não tem média finita. Gere a média acumulada de $n = 5000$ observações de uma Cauchy (`rcauchy`) e represente-a graficamente. A média acumulada converge? Explique o resultado à luz das hipóteses da lei dos grandes números.
6. Verifique numericamente a lei fraca dos grandes números: para $X_i \sim \mathcal{U}(0, 1)$ (média 1/2, variância 1/12) e $\varepsilon = 0,05$, estime por simulação a probabilidade $P(|\bar{X}_n - 1/2| > \varepsilon)$ para $n = 10, 50, 100, 500$ e confirme que decresce. Compare com o majorante de Chebyshev $\sigma^2/(n\varepsilon^2)$.
7. Ilustre o teorema limite central partindo de uma distribuição **uniforme** $\mathcal{U}(0, 1)$. Gere 10 000 médias de amostras de tamanho $n = 2, 5, 12$ e sobreponha a densidade normal apropriada. A partir de que n a aproximação lhe parece satisfatória? (A soma de doze uniformes esteve, de facto, na base de geradores antigos de números normais.)
8. Repita a ilustração do teorema limite central partindo de uma distribuição fortemente enviesada, como a **log-normal** $\text{LogNormal}(0, 1)$. Compare a rapidez de convergência com a do caso exponencial estudado no texto. Qual necessita de um n maior para a aproximação normal ser aceitável?
9. Para a distribuição binomial $\text{Binomial}(n, p)$ com $p = 0,1$, compare graficamente a distribuição exata com a aproximação normal, para $n = 10, 30, 100$. Confirme que, com p pequeno, é necessário um n maior do que o habitual para que a aproximação seja razoável, e relacione isto com a assimetria da binomial.
10. (*Cobertura de um intervalo de confiança.*) Uma consequência prática do teorema limite central é a validade aproximada do intervalo de confiança $\bar{X}_n \pm z_{1-\alpha/2} s/\sqrt{n}$ para a média. Gere 5000 amostras de tamanho $n = 40$ de uma exponencial de média 1, construa em cada uma o

intervalo de confiança a 95% para a média, e estime a proporção de intervalos que efetivamente contêm o valor 1. A cobertura empírica aproxima-se de 95%?

11. (*Efeito do tamanho da amostra na cobertura.*) Repita o exercício anterior para $n = 10, 40, 200$ e discuta como a cobertura empírica se aproxima do nível nominal de 95% à medida que n cresce. Relacione o resultado com a qualidade da aproximação normal para a média de observações exponenciais.

12. Combine as duas leis num único gráfico: para observações $\mathcal{U}(0, 1)$, represente simultaneamente (a) a média acumulada de uma trajetória a convergir para $1/2$ e (b), num segundo painel, o histograma das médias de muitas amostras de tamanho $n = 30$ com a normal sobreposta. Comente como cada painel ilustra um dos dois teoremas centrais do capítulo.

Capítulo 25

Cálculo de integrais por Monte Carlo

Chegámos a um dos pontos altos deste livro, onde a simulação deixa de ser apenas uma forma de gerar dados artificiais e se torna um verdadeiro instrumento de cálculo. Muitos integrais que surgem na estatística e nas ciências não têm primitiva expressável em forma fechada — pense-se no integral da densidade normal — e mesmo os métodos numéricos determinísticos clássicos, como a regra dos trapézios ou de Simpson, tornam-se impraticáveis quando o número de dimensões cresce. É aqui que entra o **método de Monte Carlo**: usar números aleatórios para aproximar o valor de um integral. A ideia é tão simples quanto poderosa, e assenta diretamente nos teoremas limites que estudámos no capítulo anterior.

25.1 Contexto histórico

O método de Monte Carlo nasceu no final da década de 1940, no Laboratório Nacional de Los Alamos, das mãos de **Stanislaw Ulam**, **John von Neumann** e **Nicholas Metropolis**. Ulam, convalescente de uma doença e a jogar paciências, perguntou-se qual seria a probabilidade de um jogo sair bem sucedido; percebendo que o cálculo combinatório exato era inviável, ocorreu-lhe que seria mais simples jogar muitas partidas e contar. Dessa intuição — substituir o cálculo exato pela repetição aleatória — nasceu uma técnica que von Neumann rapidamente adaptou aos problemas de difusão de neutrões então em estudo. O nome, sugerido por Metropolis, é uma alusão ao famoso casino do Mónaco, evocando a natureza aleatória do procedimento.

O método tem, contudo, um precursor ilustre e bem mais antigo: a **agulha de Buffon**. Em 1777, Georges-Louis Leclerc, conde de Buffon, mostrou que a probabilidade de uma agulha lançada ao acaso cruzar uma das linhas paralelas de um soalho depende de π — o que permite *estimar* π atirando agulhas e contando cruzamentos. É, em espírito, o primeiro método de Monte Carlo da história, com quase dois séculos de antecedência sobre Los Alamos.

25.2 A ideia fundamental

Seja $g(x)$ uma função e suponhamos que se pretende calcular

$$\theta = \int_0^1 g(x) dx.$$

A chave do método está em reconhecer este integral como um **valor esperado**. Consideremos U uma variável aleatória com distribuição uniforme no intervalo $(0, 1)$, cuja densidade é 1 nesse intervalo. Por definição de valor esperado,

$$E[g(U)] = \int_0^1 g(u) \cdot 1 du = \int_0^1 g(x) dx = \theta.$$

Ou seja, o integral que queremos calcular é exatamente a média da variável aleatória $g(U)$:

$$\theta = E[g(U)].$$

Esta reformulação é decisiva, porque sabemos aproximar valores esperados: basta gerar muitas observações e calcular a média amostral. Se $U_1, \dots, U_k$ são variáveis aleatórias independentes e uniformemente distribuídas em $(0, 1)$, então $g(U_1), \dots, g(U_k)$ são i.i.d. com média θ . Pela **lei forte dos grandes números**, com probabilidade 1,

$$\hat{\theta}_k = \frac{1}{k} \sum_{i=1}^k g(U_i) \longrightarrow E[g(U)] = \theta, \quad \text{quando} \quad k \rightarrow \infty.$$

Aprender a aproximar integrais definidos — em uma ou várias dimensões, em intervalos limitados ou ilimitados — através da média de avaliações de uma função em pontos aleatórios, e saber quantificar a incerteza dessa aproximação.

Em termos práticos, para aproximar θ geramos uma grande quantidade de números aleatórios $u_1, \dots, u_k$ em $(0, 1)$ e calculamos a média dos valores $g(u_i)$:

$$\theta \approx \frac{1}{k} \sum_{i=1}^k g(u_i).$$

É esta a essência do método de Monte Carlo para o cálculo de integrais.

25.3 O erro da aproximação

Uma estimativa sem uma medida da sua incerteza tem pouco valor. Felizmente, o mesmo enquadramento probabilístico que legitima o método fornece também o erro. O estimador $\hat{\theta}_k$ é uma média de variáveis i.i.d., pelo que é **não enviesado** ($E[\hat{\theta}_k] = \theta$) e tem variância

$$\text{Var}(\hat{\theta}_k) = \frac{\sigma_g^2}{k}, \quad \text{onde} \quad \sigma_g^2 = \text{Var}(g(U)).$$

O **erro-padrão** da estimativa é, portanto, $\sigma_g/\sqrt{k}$, que decresce proporcionalmente a $1/\sqrt{k}$. Como σ_g^2 é em geral desconhecido, estimamo-lo pela variância amostral s_g^2 dos valores $g(u_i)$, obtendo o erro-padrão estimado $s_g/\sqrt{k}$. E, pelo **teorema limite central**, um intervalo de confiança aproximado a 95% para θ é

$$\hat{\theta}_k \pm 1,96 \frac{s_g}{\sqrt{k}}.$$

A convergência a uma taxa de $1/\sqrt{k}$ é lenta: para reduzir o erro para metade é preciso **quadruplicar** o número de pontos. Esta é a principal fraqueza do método de Monte Carlo em problemas de baixa dimensão, onde os métodos determinísticos são muito mais eficientes. A grande virtude do Monte Carlo revela-se noutro lado: a taxa $1/\sqrt{k}$ **não depende da dimensão** do integral. Um método determinístico baseado numa grelha vê o seu custo explodir exponencialmente com o número de dimensões (a “maldição da dimensionalidade”), ao passo que o Monte Carlo mantém a mesma taxa. É por isso que, em dimensão elevada, o Monte Carlo se torna imbatível.

25.4 Cálculo de $\int_a^b g(x) dx$

Suponhamos agora que queremos integrar g num intervalo qualquer $[a, b]$. Basta uma mudança de variável para nos reconduzir ao caso $(0, 1)$. Definamos

$$y = \frac{x - a}{b - a}, \quad \text{de modo que} \quad dy = \frac{dx}{b - a}.$$

Quando x percorre $[a, b]$, y percorre $[0, 1]$, e $x = a + (b - a)y$. O integral transforma-se em

$$\theta = \int_0^1 g(a + (b - a)y) (b - a) dy = \int_0^1 h(y) dy,$$

onde $h(y) = (b - a)g(a + (b - a)y)$. Reduzimo-lo, assim, ao caso padrão, e podemos aplicar de novo o método de Monte Carlo: geram-se números aleatórios uniformes em $(0, 1)$, avalia-se h nesses pontos e toma-se a média:

$$\theta \approx \frac{1}{k} \sum_{i=1}^k h(y_i).$$

Existe uma forma equivalente e talvez mais intuitiva de o exprimir: gerar diretamente pontos uniformes no intervalo $[a, b]$ e multiplicar a média das avaliações pela amplitude do intervalo,

$$\theta \approx (b - a) \cdot \frac{1}{k} \sum_{i=1}^k g(u_i), \quad u_i \sim \mathcal{U}(a, b).$$

25.4.1 Exemplo — um integral elementar

Ilustremos o método com um integral cujo valor exato conhecemos. Consideremos

$$\theta = \int_0^1 x^2 dx = \frac{1}{3}.$$

Geramos $k = 10\,000$ números aleatórios em $[0, 1]$, avaliamos $g(u_i) = u_i^2$ e tomamos a média, que deverá aproximar-se de $1/3$.

```
set.seed(2024)

g <- function(x) x^2
k <- 10000
u <- runif(k, min = 0, max = 1)

theta_hat <- mean(g(u))           # estimativa de Monte Carlo
theta_hat
## [1] 0.3323

integrate(g, lower = 0, upper = 1) # valor de referência
## 0.3333 with absolute error < 3.7e-15
```

Acrescentemos a estimativa do erro, que raramente deve faltar num cálculo de Monte Carlo:

```
erro_padrao <- sd(g(u)) / sqrt(k)
ic <- theta_hat + c(-1, 1) * 1.96 * erro_padrao
c(estimativa = theta_hat, erro_padrao = erro_padrao,
  ic_inf = ic[1], ic_sup = ic[2])
## estimativa erro_padrao      ic_inf      ic_sup
##      0.332306      0.002967      0.326491      0.338121
```

O valor exato $1/3 \approx 0,3333$ cai, como esperado, dentro do intervalo de confiança. Vejamos agora o integral no intervalo $[1, 2]$,

$$\int_1^2 x^2 dx = \frac{7}{3},$$

aplicando a transformação para $[0, 1]$ e, em alternativa, a forma direta com pontos em $[1, 2]$.

```
set.seed(2024)

a <- 1; b <- 2
k <- 10000

# Forma 1: pontos em (0,1) e função transformada h(y)
u <- runif(k)
mean((b - a) * g(a + (b - a) * u))
## [1] 2.332

# Forma 2: pontos diretamente em (a, b)
u2 <- runif(k, a, b)
(b - a) * mean(g(u2))
## [1] 2.322

# Valor de referência
integrate(g, lower = 1, upper = 2)
## 2.333 with absolute error < 2.6e-14
```

As duas formas coincidem, como devem, e ambas se aproximam de $7/3 \approx 2,333$.

25.4.2 A convergência em imagem

Vale a pena *ver* a estimativa a convergir à medida que acumulamos pontos. O gráfico seguinte mostra a estimativa acumulada de $\int_0^1 x^2 dx$ a estabilizar em torno de $1/3$, ladeada por uma banda de erro que se estreita ao ritmo de $1/\sqrt{k}$.

```
set.seed(2024)

k <- 5000
u <- runif(k)
valores <- u^2
estim_acum <- cumsum(valores) / seq_along(valores)
```

```

# erro-padrão acumulado
var_acum <- (cumsum(valores^2) / seq_along(valores) - estim_acum^2)
ep_acum <- sqrt(var_acum / seq_along(valores))

plot(estim_acum, type = "l", col = "#10656d", lwd = 1.2,
      ylim = c(0.25, 0.42),
      xlab = "número de pontos", ylab = expression(hat(theta)[k]))
lines(estim_acum + 2 * ep_acum, col = "grey70", lty = 3)
lines(estim_acum - 2 * ep_acum, col = "grey70", lty = 3)
abline(h = 1/3, lty = 2, col = "grey30")

```

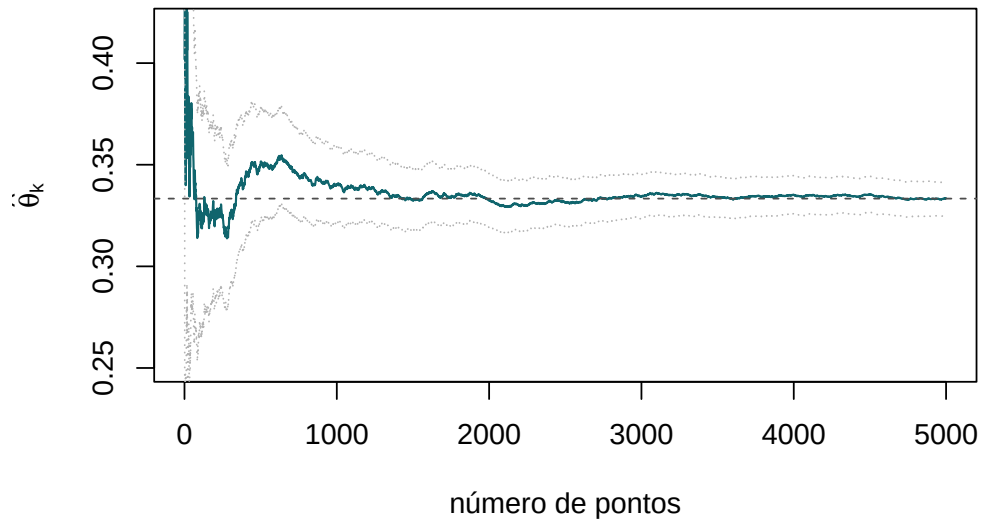


Figura 25.1: Estimativa acumulada do integral de x^2 em $[0,1]$ em função do número de pontos, com banda de dois erros-padrão. A estimativa converge para o valor exato (linha tracejada) e a incerteza estreita-se ao ritmo de $1/\sqrt{k}$.

25.5 Cálculo de $\int_0^\infty g(x) dx$

Para integrais em intervalos ilimitados, uma mudança de variável reconduz-nos igualmente ao intervalo $(0,1)$. Para $\int_0^\infty g(x) dx$, aplica-se a substituição

$$y = \frac{1}{x+1}, \quad \text{de modo que} \quad dy = -\frac{dx}{(x+1)^2} = -y^2 dx.$$

Quando x vai de 0 a $+\infty$, y vai de 1 a 0, e $x = \frac{1}{y} - 1$. Trocando os limites (o que absorve o sinal), obtém-se

$$\theta = \int_0^1 h(y) dy, \quad \text{onde} \quad h(y) = \frac{g\left(\frac{1}{y} - 1\right)}{y^2}.$$

Nem sempre esta substituição em particular é a melhor. Uma alternativa muitas vezes mais eficiente para caudas exponenciais consiste em reconhecer o integral como um valor esperado relativamente a uma densidade *não uniforme* que saibamos simular. Por exemplo, $\int_0^\infty g(x) dx = \int_0^\infty \frac{g(x)}{\lambda e^{-\lambda x}} \lambda e^{-\lambda x} dx = E\left[\frac{g(X)}{\lambda e^{-\lambda X}}\right]$ com $X \sim \text{Exp}(\lambda)$. Esta é a ideia da *amostragem por importância*, em que a escolha da densidade de amostragem pode reduzir drasticamente a variância do estimador.

25.6 O método “acerta-ou-falha” e a estimativa de π

O método que temos usado — a **média amostral** — não é a única via. Há uma variante geométrica, o método *acerta-ou-falha* (em inglês, *hit-or-miss*), que recupera diretamente a intuição de Buffon e é particularmente vívido.

A ideia: para estimar a área de uma região contida num retângulo conhecido, atiram-se pontos ao acaso sobre o retângulo e conta-se a fração que cai *dentro* da região. Essa fração, multiplicada pela área do retângulo, estima a área procurada. O exemplo canónico é a estimativa de π : um quarto de círculo de raio 1 tem área $\pi/4$ e está inscrito no quadrado unitário. Atirando pontos ao acaso no quadrado, a proporção dos que caem dentro do círculo (isto é, com $x^2 + y^2 \leq 1$) aproxima $\pi/4$.

```
set.seed(2024)

k <- 100000
x <- runif(k)
y <- runif(k)
dentro <- x^2 + y^2 <= 1

pi_hat <- 4 * mean(dentro)
pi_hat
## [1] 3.139
```

```
set.seed(2024)
n_plot <- 3000
xp <- runif(n_plot); yp <- runif(n_plot)
dentro_p <- xp^2 + yp^2 <= 1
plot(xp, yp, pch = 19, cex = 0.3, asp = 1,
     col = ifelse(dentro_p, "#10656d", "grey70"),
```

```
xlab = "x", ylab = "y")
curve(sqrt(1 - x^2), from = 0, to = 1, add = TRUE, lwd = 2, col = "grey20")
```

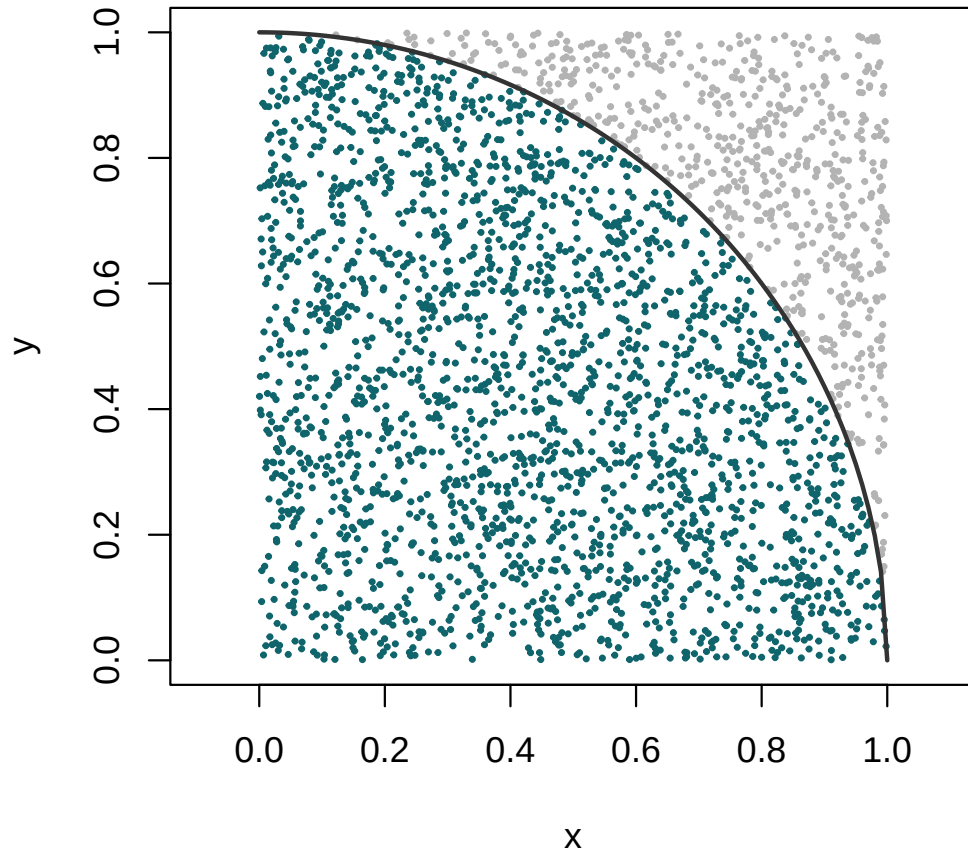


Figura 25.2: Estimativa de π pelo método acerta-ou-falha: pontos aleatórios no quadrado unitário. Os que caem dentro do quarto de círculo (a azul) representam uma fração próxima de $\pi/4$.

O método acerta-ou-falha é um caso particular do método da média amostral, aplicado à função indicadora $g(x, y) = \mathbf{1}\{x^2 + y^2 \leq 1\}$. Por essa razão, a sua variância é tipicamente *maior* do que a de estimadores baseados na média de uma função suave, pelo que, quando existe uma alternativa por média amostral, esta costuma ser preferível. O método acerta-ou-falha vale sobretudo pela transparência geométrica e pelo seu valor histórico e pedagógico.

25.7 Integrais multidimensionais

É na dimensão elevada que o método de Monte Carlo mostra todo o seu valor. Suponhamos que g é uma função de n argumentos e que se pretende calcular

$$\theta = \int_0^1 \int_0^1 \cdots \int_0^1 g(x_1, \dots, x_n) dx_1 dx_2 \cdots dx_n.$$

Tal como no caso unidimensional, este integral é um valor esperado: $\theta = E[g(U_1, \dots, U_n)]$, onde $U_1, \dots, U_n$ são uniformes independentes em $(0, 1)$. Para o estimar, geram-se k conjuntos independentes, cada um com n coordenadas uniformes independentes,

$$\begin{aligned} &U_1^1, \dots, U_n^1 \\ &U_1^2, \dots, U_n^2 \\ &\vdots \\ &U_1^k, \dots, U_n^k, \end{aligned}$$

e as variáveis $g(U_1^i, \dots, U_n^i)$, $i = 1, \dots, k$, são i.i.d. com média θ . Estima-se então

$$\theta \approx \frac{1}{k} \sum_{i=1}^k g(U_1^i, \dots, U_n^i).$$

25.7.1 Exemplo — um integral duplo

Estimemos

$$\theta = \int_0^1 \int_0^1 (x^2 + y^2) dx dy,$$

cujo valor exato é $\frac{1}{3} + \frac{1}{3} = \frac{2}{3}$.

```
set.seed(2024)

k <- 100000
x <- runif(k)
y <- runif(k)
g2 <- x^2 + y^2

theta_hat <- mean(g2)
erro_padrao <- sd(g2) / sqrt(k)

c(estimativa = theta_hat, exato = 2/3, erro_padrao = erro_padrao)
##  estimativa      exato erro_padrao
##    0.665648    0.666667    0.001332
```

A estimativa aproxima-se de $2/3$ com uma incerteza controlada — e, o que é crucial, a mesma abordagem funcionaria sem alterações significativas para um integral em 10 ou 100 dimensões, onde qualquer método de grelha seria impensável.

25.8 Síntese

O método de Monte Carlo transforma o problema de calcular um integral no problema de estimar um valor esperado, resolvido pela média de avaliações da função em pontos aleatórios. A sua justificação é a lei dos grandes números; a quantificação do seu erro, o teorema limite central. Converge lentamente, ao ritmo de $1/\sqrt{k}$, mas com a vantagem decisiva de essa taxa ser independente da dimensão — o que faz dele o método de eleição para integrais de dimensão elevada, omnipresentes na estatística moderna, na física e nas finanças. Nos casos aqui tratados usámos a distribuição uniforme como base; a generalização para outras distribuições de amostragem, com vista a reduzir a variância, abre a porta a técnicas mais avançadas, como a amostragem por importância.

25.9 Exercícios

Use simulação para aproximar os integrais seguintes. Sempre que possível, compare a sua estimativa com o valor exato e apresente também o erro-padrão e um intervalo de confiança a 95%.

1. $\int_0^1 \exp(e^x) dx.$

2. $\int_0^1 (1 - x^2)^{3/2} dx.$

3. $\int_{-2}^2 e^{x+x^2} dx.$

4. $\int_0^{\pi/2} \cos(x) dx.$ (Valor exato: 1.)

5. $\int_0^\infty x(1+x^2)^{-2} dx.$ (Valor exato: $1/2$.)

6. $\int_{-\infty}^\infty e^{-x^2} dx.$ (Valor exato: $\sqrt{\pi}$.) Sugestão: separe o integral em duas metades ou explore a simetria.

7. $\int_0^1 \int_0^1 e^{(x+y)^2} dy dx.$

8. $\int_0^1 \int_0^1 (x^2 + y^2) dx dy.$ (Valor exato: $2/3$.)

9. (*Estimativa de π e taxa de convergência.*) Estime π pelo método acerta-ou-falha e represente graficamente o valor absoluto do erro $|\hat{\pi}_k - \pi|$ em função de k , em escala logarítmica em ambos os eixos. Confirme visualmente que o erro decresce aproximadamente como $1/\sqrt{k}$ (declive $-1/2$).

10. (*A agulha de Buffon.*) Simule o lançamento de 10 000 agulhas de comprimento $\ell = 1$ sobre um soalho de linhas paralelas afastadas de $d = 2$. Sabendo que a probabilidade de uma agulha cruzar uma linha é $\frac{2\ell}{\pi d}$, use a proporção de cruzamentos para estimar π . Compare a eficiência com o método acerta-ou-falha do exercício anterior.

11. (*Erro em função de k .*) Para o integral $\int_0^1 e^x dx = e - 1$, estime θ com $k = 10^2, 10^3, 10^4, 10^5, 10^6$ e registre o erro absoluto de cada estimativa. Represente o erro em função de k numa escala log-log e confirme a taxa $1/\sqrt{k}$.

12. (*Redução de variância por variáveis antitéticas.*) Para estimar $\int_0^1 e^x dx$, compare o estimador de Monte Carlo simples com o estimador *antitético*, que usa os pares $(g(U_i), g(1 - U_i))$. Estime a variância de cada estimador por repetição e quantifique a redução obtida. Explique por que razão a técnica é eficaz quando g é monótona.

13. (*Alta dimensão.*) Estime o volume da bola unitária em dimensão n , $V_n = \int_{[-1,1]^n} \mathbf{1}\{x_1^2 + \dots + x_n^2 \leq 1\} dx_1 \dots dx_n$, pelo método acerta-ou-falha, para $n = 2, 3, 5, 10$. Compare com a fórmula exata $V_n = \pi^{n/2}/\Gamma(n/2 + 1)$ e comente como a proporção de acertos (e, portanto, a eficiência) se degrada à medida que a dimensão aumenta.

Capítulo 26

Resoluções dos exercícios

Este capítulo final reúne as resoluções dos exercícios propostos ao longo do livro. Mais do que um mero solucionário, pretende ser um capítulo de estudo: sempre que um exercício admite mais do que um caminho, apresentamos as alternativas mais instrutivas; sempre que a resposta envolve um resultado teórico, explicitamos o raciocínio antes do código; e sempre que uma solução comum contém uma armadilha subtil, assinalamo-la com uma caixa de **Atenção**.

Como ler este capítulo. O código das resoluções está, por defeito, com a opção `eval=FALSE`: é para ser executado pelo leitor, não para produzir resultados impressos aqui. Aconselhamos vivamente a que o experimente e o modifique — trocar parâmetros, aumentar o número de simulações, alterar sementes. As resoluções seguem a ordem temática do livro; dentro de cada tema, a numeração acompanha a dos enunciados. Quando existe mais do que uma forma correta de resolver um problema, indicamo-la com o comentário `# Alternativa`.

26.1 Vetores

1. Criação de vetores com `seq()`, `rep()` e o operador `:`.

```
# (a) 1, 2, ..., 20
1:20

# (b) 20, 19, ..., 1
20:1

# (c) 1, ..., 20, 19, ..., 1
c(1:20, 19:1)

# (d) 10, 20, ..., 90, 10
```

```
c(seq(10, 90, by = 10), 10)

# (e) dez 1, vinte 2, trinta 3
c(rep(1, 10), rep(2, 20), rep(3, 30))
```

2. Vetor de caracteres "nome 1", ..., "nome 20" com `paste()`.

```
paste("nome", 1:20)
```

3. e 4. Repetições com `each` e `times`. A diferença entre os dois argumentos é essencial: `each` repete cada elemento em bloco; `times` repete o vetor inteiro.

```
# 3. "A" "A" "B" "B" ... "E" "E"
rep(LETTERS[1:5], each = 2)

# 4. "a" "b" ... "e" "a" "b" ... "e"
rep(letters[1:5], times = 2)
```

5. Repetições com contagens diferentes.

```
c(rep("uva", 10), rep("maçã", 9), rep("laranja", 6), rep("banana", 1))
```

6. Nomear elementos de um vetor e aceder por nome.

```
notas <- c(7, 8, 9)
names(notas) <- c("Ana", "João", "Pedro")
notas["João"]
```

7. Ordenar e localizar extremos.

```
x <- sample(1:100, size = 15)
sort(x) # crescente
sort(x, decreasing = TRUE) # decrescente
min(x); max(x)
```

8. Estatísticas de resumo de um vetor.

```
x <- sample(1:50, size = 20)
sum(x); mean(x); sd(x); prod(x)
```

9. Indexação lógica: extrair e substituir por condição.

```
x <- sample(1:100, size = 10)
x[x > 50]      # elementos maiores que 50
x[x < 30] <- 0  # substitui in loco os menores que 30
```

10. Combinar condições com &.

```
x <- sample(1:10)
x[x > 5 & x %% 2 == 0]  # maiores que 5 E pares
```

11. Indexação por posição.

```
x <- sample(1:10)
x[c(2, 4, 6)] <- x[c(2, 4, 6)] * 2
x[length(x)] <- 100      # seq_along/length evita erros de índice
```

12. Médias na presença de valores especiais.

```
# (a) com NA
x <- c(1, 0, NA, 5, 7)
mean(x, na.rm = TRUE)

# (b) com infinitos: primeiro convertê-los em NA
y <- c(-Inf, 0, 1, NA, 2, 7, Inf)
mean(y[!is.infinite(y)], na.rm = TRUE)
```

`mean(y, na.rm = TRUE)` não remove `-Inf` nem `Inf`: `na.rm` só ignora `NA`. Com infinitos presentes, a média devolve `NaN`. É preciso filtrá-los explicitamente com `is.infinite()`, como acima.

13. Vetores construídos a partir de expressões.

```
# (a)
x <- seq(2, 6, by = 0.1)
exp(x) * sin(x)

# (b) (3, 3^2/2, 3^3/3, ..., 3^30/30)
i <- 1:30
3^i / i
```

14. Somatórios com vetorização.

```
# (a)
i <- 1:1000; sum(9 / 10^i)

# (b)
i <- 10:100; sum(i^3 + 4 * i^2)

# (c)
i <- 1:25; sum(2^i / i + 3^i / i^2)
```

15. Consumo de energia (aplicação integradora).

```
set.seed(2025)
consumo <- runif(n = 30, min = 10, max = 40) # (a)
media <- mean(consumo) # (b)
acima_da_media <- consumo > media # (c)
sum(acima_da_media) # (d) quantos dias
which(consumo > 35) # (e) posições de pico
consumo[which(consumo > 35)] # valores de pico
consumo[consumo < 15] <- NA # (f) falhas de medição
mean(consumo, na.rm = TRUE) # (g) média corrigida
consumo[seq(2, 30, by = 2)] # (h) dias pares
```

16. Produção agrícola (aplicação integradora).

```
producao <- c(80, 95, 70, 68, 105, 120, 73, 89, 110, 66, 92, 78)
media_total <- mean(producao) # (b)
which(producao < media_total) # (c) abaixo da média
producao[c(4, 10, 12)] <- producao[c(4, 10, 12)] * 1.10 # (d) correção +10%
mean(producao) # (e) nova média
alta <- producao > 100 # (f)
sum(alta); sum(alta) / length(producao) # (g) contagem e proporção
ordenado <- sort(producao) # (h)
tail(ordenado, 3) # (i) as 3 maiores
```

Aumentar em 10% escreve-se de forma mais limpa como `producao * 1.10` do que `producao + producao * 0.10`. As duas expressões dão o mesmo resultado, mas a primeira lê-se de imediato como “acrescentar dez por cento”.

26.2 Fatores

1. Criar um fator com níveis explícitos.

```
tamanhos <- c("pequeno", "médio", "grande", "pequeno", "grande")
f <- factor(tamanhos, levels = c("pequeno", "médio", "grande"))
levels(f)
```

2. Tabela de frequências de um fator.

```
cores <- factor(c("vermelho", "azul", "preto", "vermelho", "branco", "azul"))
table(cores)
```

3. Renomear níveis.

```
avaliacoes <- factor(c("bom", "médio", "ruim", "bom", "ruim"))
levels(avaliacoes) <- c("excelente", "razoável", "insatisfatório")
avaliacoes
```

4. Testar e converter para fator.

```
estado_civil <- c("solteiro", "casado", "divorciado")
is.factor(estado_civil)           # FALSE
estado_civil <- factor(estado_civil)
is.factor(estado_civil)           # TRUE
```

5. Fator ordenado e ordenação.

```
escolaridade <- factor(
  c("mestrado", "doutoramento", "licenciatura", "secundário"),
  levels = c("secundário", "licenciatura", "mestrado", "doutoramento"),
  ordered = TRUE)
sort(escolaridade)
```

6. Converter um fator em códigos numéricos.

```
pressao <- factor(c("baixo", "alto", "médio", "baixo", "alto"),
  levels = c("baixo", "médio", "alto"))
as.numeric(pressao)  # 1, 3, 2, 1, 3 - segue a ordem dos níveis
```

`as.numeric()` de um fator devolve os **códigos internos dos níveis**, não o texto. Se o fator contiver números guardados como texto (por exemplo `factor(c("10", "20"))`), `as.numeric()` devolve 1, 2 e não 10, 20. Nesse caso use `as.numeric(as.character(f))`.

7. Substituir um nível específico.

```
cores <- factor(c("verde", "amarelo", "vermelho", "verde", "amarelo"))
levels(cores)[levels(cores) == "amarelo"] <- "laranja"
cores
```

8. Inquérito de satisfação (fator ordenado).

```
respostas <- c(
  "Satisfeito", "Muito satisfeito", "Neutro", "Insatisfeito", "Satisfeito",
  "Muito insatisfeito", "Satisfeito", "Neutro", "Insatisfeito", "Satisfeito",
  "Muito satisfeito", "Neutro", "Insatisfeito", "Satisfeito", "Neutro",
  "Satisfeito", "Muito satisfeito", "Muito insatisfeito", "Neutro", "Satisfeito")

# (b) fator ordenado
rf <- factor(respostas,
  levels = c("Muito insatisfeito", "Insatisfeito", "Neutro",
    "Satisfeito", "Muito satisfeito"),
  ordered = TRUE)
str(rf)

# (c) número de respostas (Muito) insatisfeito
sum(rf == "Insatisfeito" | rf == "Muito insatisfeito")

# (d) categoria mais frequente
which.max(table(rf))

# (e) frequências relativas
table(rf) / length(rf)

# (f) renomear "Neutro" para "Indiferente"
levels(rf)[levels(rf) == "Neutro"] <- "Indiferente"
levels(rf)
```

9. Desafios associados a TDAH (fator + gráfico de barras).

```
tdah <- c("Foco", "Procrastinação", "Gestão do tempo", "Organização", "Nenhum",
  "Foco", "Foco", "Organização", "Organização", "Organização",
  "Gestão do tempo", "Gestão do tempo", "Procrastinação",
  "Procrastinação", "Procrastinação", "Procrastinação", "Foco", "Foco", "Foco")
tf <- factor(tdah)
```

```

table(tf)                                # (b) frequências absolutas
table(tf) / length(tdah)                 #   frequências relativas

# (d) renomear "Nenhum"
levels(tf)[levels(tf) == "Nenhum"] <- "Sem dificuldade declarada"

# (e) gráfico de barras
barplot(table(tf), ylab = "Frequência absoluta", main = "Desafios (TDAH)")

```

26.3 Listas

1. Criar uma lista e aceder aos elementos com \$ e [[]].

```

dados_estudante <- list(
  nome  = "João",
  idade = 21,
  notas = c(Estatistica = 85, Matematica = 90, Computacao = 95))

dados_estudante$nome
dados_estudante$idade
dados_estudante$notas["Computacao"]

```

2. Adicionar e modificar elementos.

```

dados_estudante$status <- "Aprovado"
dados_estudante$notas["Estatistica"] <- 88

```

3. e 4. Listas de listas (uma turma de estudantes).

```

estudante1 <- list(nome = "Maria", idade = 22, notas = c(78,85,90), status = "Aprovado")
estudante2 <- list(nome = "Carlos", idade = 23, notas = c(70,75,80), status = "Aprovado")
turma <- list(estudante1, estudante2)

turma[[2]]$nome           # (4a) nome do segundo estudante
mean(turma[[1]]$notas)    # (4b) média das notas do primeiro
turma[[2]]$status <- "Reprovado" # (4c) alterar o estado

```

Repare na diferença entre [] e [[]] numa lista: `turma[2]` devolve uma **sublista** (ainda uma lista, de comprimento 1), enquanto `turma[[2]]` devolve o **conteúdo** do segundo elemento (a lista do estudante). Para aceder aos campos com \$, precisa do conteúdo, logo de [[]].

5. Listas encaixadas (várias turmas).

```
estudante3 <- list(nome = "Ana", idade = 19, notas = c(70,80,96), status = "Aprovado")
estudante4 <- list(nome = "João", idade = 23, notas = c(85,80,95), status = "Aprovado")
turma1 <- list(estudante1, estudante2)
turma2 <- list(estudante3, estudante4)
cadeira <- list(turma1, turma2)

cadeira[[2]][[1]]$nome           # (a) nome do 1.º estudante da 2.ª turma
mean(cadeira[[1]][[2]]$notas)   # (b) média do 2.º estudante da 1.ª turma
```

6. Produtos de uma fábrica (aplicação com listas paralelas).

```
produtos <- list(
  nome      = c("Telemóvel", "PC", "Rato", "Fone"),
  quantidade = c(1000, 1500, 2000, 2500),
  custo     = c(266, 283, 16, 26),
  venda     = c(800, 850, 50, 80))

# margem de lucro mensal por produto
produtos$margem <- (produtos$venda - produtos$custo) * produtos$quantidade

# produto com maior lucro
produtos$nome[which.max(produtos$margem)]

# aumentar o preço de venda do 2.º produto em 5% e recalculer
produtos$venda[2] <- produtos$venda[2] * 1.05
produtos$margem[2] <- (produtos$venda[2] - produtos$custo[2]) * produtos$quantidade[2]
```

7. Registos de utilização (listas encaixadas com vetores).

```
logs <- list(
  e1 = list(nome = "Ana", mensagens = c(12, 15, 9), tempo_total = 35),
  e2 = list(nome = "Bruno", mensagens = c(7, 6, 5, 12), tempo_total = 42),
  e3 = list(nome = "Carla", mensagens = c(10, 20), tempo_total = 28))

logs$e2$mensagens[3]           # 3.ª mensagem do Bruno
sapply(logs, function(e) sum(e$mensagens)) # total de mensagens por pessoa
sapply(logs, function(e) e$tempo_total / length(e$mensagens)) # tempo médio/sessão
```

Sempre que precisar de aplicar o mesmo cálculo a todos os elementos de uma lista, prefira `sapply()/lapply()` a repetir o código manualmente para cada elemento. Além de mais curto, o código fica imune a erros de cópia e adapta-se sozinho se a lista crescer.

26.4 Matrizes e *arrays*

1. Criar uma matriz e somar uma coluna (três formas equivalentes).

```
A <- matrix(1:12, nrow = 3, ncol = 4)
sum(A[, 2])           # forma direta
colSums(A)[2]         # Alternativa
apply(A, 2, sum)[2]   # Alternativa
```

2. Operações elemento a elemento.

```
A <- matrix(sample(1:10, 6, replace = TRUE), nrow = 2)
B <- matrix(sample(1:10, 6, replace = TRUE), nrow = 2)
A + B               # soma
A * B               # produto de Hadamard (elemento a elemento)
```

3. Produto matriz-vetor.

```
M <- matrix(sample(1:9, 9, replace = TRUE), nrow = 3)
v <- c(1, 2, 3)
M %*% v             # M v (coluna)
v %*% M             # v^T M (linha)
```

* é o produto **elemento a elemento**; o produto matricial faz-se com `%*%`. Confundi-los é um dos erros mais frequentes em álgebra linear com R.

4. Transposta e somas por linha.

```
N <- matrix(1:8, nrow = 4)
Nt <- t(N)
rowSums(Nt)
```

5. Determinante e inversa.

```
P <- matrix(sample(1:9, 9, replace = TRUE), nrow = 3)
det(P)
if (det(P) != 0) solve(P) # a inversa só existe se det != 0
```

6. Extração de submatrizes.

```
Q <- matrix(sample(1:25, 25, replace = TRUE), nrow = 5)
Q[2:4, 2:4]
```

7. Somas de linha e de coluna.

```
A <- matrix(1:9, nrow = 3)
sum(A[1, ])    # 1.ª linha
sum(A[, 3])    # 3.ª coluna
```

8. Potências de matrizes e manipulação de colunas.

```
library(expm)    # necessário para o operador %^%
A <- rbind(c(1, 1, 3), c(5, 2, 6), c(-2, -1, -3))
A %*% A %*% A    # A^3
A %^% 3          # o mesmo, com expm

A[, 3] <- A[, 1] + A[, 2]    # substitui a 3.ª coluna

M <- cbind(c(20,34,99,12,10,14), c(22,55,97,16,53,21), c(23,57,71,19,24,28))
M[M %^% 2 == 0] <- 0        # anula as entradas pares
```

9. Verificar simetria.

```
M <- matrix(c(1,2,3, 2,4,5, 3,5,6), nrow = 3)
all(M == t(M))    # TRUE se simétrica
```

10. Traço de uma matriz.

```
A <- matrix(1:16, nrow = 4)
sum(diag(A))
```

11. Selecionar colunas pares.

```
M <- matrix(1:25, nrow = 5, byrow = TRUE)
M[, seq(2, 5, by = 2)]
```

12. Desempenho de alunos (matriz com nomes).

```
M <- matrix(c(12,8,5, 20,10,4, 15,9,5), nrow = 3, byrow = TRUE)
colnames(M) <- c("Perguntas", "TempoResposta", "Satisfacao")
rownames(M) <- c("AlunoA", "AlunoB", "AlunoC")

M <- rbind(M, AlunoD = c(18, 11, 3))      # (b)
M <- cbind(M, HorasUso = c(5, 7, 6, 8))  # (c)
colMeans(M)                               # (d)
rownames(M)[which.max(M[, "Perguntas"])] # (e) quem fez mais perguntas
```

13. Imagem representada por uma matriz.

```
I <- matrix(c(
  0,0,0,0,0,0,0,0,0,0,  0,9,9,9,0,0,9,9,9,0,  0,9,0,9,0,0,9,0,9,0,
  0,9,9,9,0,0,9,9,9,0,  0,9,0,0,0,0,0,0,9,0,  0,9,0,0,0,0,0,0,9,0,
  0,9,0,0,0,0,0,0,9,0,  0,9,0,0,0,0,0,0,9,0,  0,9,9,9,9,9,9,9,9,0,
  0,0,0,0,0,0,0,0,0,0), nrow = 10, byrow = TRUE)

image(I, col = gray.colors(10))          # (b) visualizar
image(9 - I, col = gray.colors(10))      # (c) inverter cores
mean(I)                                  # (d) brilho médio
I <- rbind(I, rep(0, ncol(I))); dim(I)    # (e) nova linha de zeros
```

14. Arrays a três dimensões.

```
A <- array(1:27, dim = c(3, 3, 3))
A[2, 3, 1]      # elemento
A[, , 2]        # segunda "camada"
A[3, , 2]       # 3.ª linha da 2.ª camada
```

No material original, este último acesso aparecia como `arr[3, , 2]`, mas o *array* chama-se **A** (ou `array_3d`). Referir um objeto inexistente devolve o erro `object 'arr' not found`. Verifique sempre que o nome usado é o do objeto efetivamente criado.

15. Operações com dois *arrays*.

```
set.seed(123)
a1 <- array(sample(1:10, 8, replace = TRUE), dim = c(2, 2, 2))
a2 <- array(sample(1:10, 8, replace = TRUE), dim = c(2, 2, 2))
a1 + a2                                # soma
a1 * a2                                # produto elemento a elemento
mean(a1 + a2)                          # média da soma
a1[, , 1] %*% a2[, , 2]                # produto matricial de duas camadas
```

16. Cadeia de Markov (iteração de uma matriz de transição).

```
library(expm)
M <- matrix(c(0.80, 0.20, 0.10, 0.90), nrow = 2) # colunas somam 1
v <- c(150, 150)
M %*% v          # 1.ª iteração
(M %^% 10) %*% v # 10.ª iteração
```

26.5 Data frames

1. Criar, acrescentar linha e ordenar.

```
estudantes <- data.frame(
  nome = c("Ana", "João", "Maria", "Inês", "Gonçalo"),
  idade = c(18, 19, 20, 18, 20),
  curso = c("Matemática", "Psicologia", "Geologia", "Gestão", "Física"),
  nota = c(16, 7, 14, 6, 16))
estudantes <- rbind(estudantes, list("Filipa", 20, "Matemática", 8))
estudantes[order(estudantes$nota), ]
```

Desde a versão 4.0 do R, `data.frame()` já não converte automaticamente texto em fatores (o antigo `stringsAsFactors = TRUE` deixou de ser o padrão). Por isso a coluna `curso` acima é de caracteres, e podemos atribuir-lhe novos valores sem tropeçar em “níveis” inexistentes.

2. Modificar e recodificar por condição.

```
estudantes$nota <- estudantes$nota + 0.5
estudantes$curso[estudantes$nota > 8] <- "Estatística" # Alternativa a subset
```

3. Filtrar linhas (`subset` vs. indexação).

```
subset(mtcars, cyl == 6 & hp > 100) # forma legível
mtcars[mtcars$cyl == 6 & mtcars$hp > 100, ] # Alternativa com indexação
```

4. Criar uma nova coluna.

```
mtcars$Peso_KG <- mtcars$wt * 453.592
```

5. Agregar por grupo com `aggregate()`.

```
aggregate(hp ~ cyl, data = mtcars, FUN = mean)
aggregate(hp ~ cyl, data = mtcars, FUN = min)
aggregate(hp ~ cyl, data = mtcars, FUN = max)
```

6. Aplicar uma função a todas as colunas.

```
sapply(mtcars, sd)
```

7. Converter uma coluna em fator com rótulos.

```
mtcars$am <- factor(mtcars$am, levels = c(0, 1),
                    labels = c("Automático", "Manual"))
table(mtcars$am)
```

8. Ordenar e extrair extremos.

```
ord <- mtcars[order(mtcars$mpg, decreasing = TRUE), ]
head(ord, 5)      # 5 mais econômicos
tail(ord, 5)      # 5 menos econômicos
```

9. Combinar *data frames* com `merge()`.

```
mtcars$car <- rownames(mtcars)
carros1 <- mtcars[, c("car", "mpg", "cyl")]
carros2 <- mtcars[, c("car", "hp", "wt")]
merge(carros1, carros2, by = "car")
```

10. a 14. Exploração do conjunto `airquality`.

```
data(airquality)
head(airquality); tail(airquality)           # 10
colSums(is.na(airquality))                   # 11 valores em falta por coluna
mean(airquality$Solar.R, na.rm = TRUE)       # 12
subset(airquality, Wind > 10 & Temp > 80)    # 13 filtro
airquality$Temp_Wind <- airquality$Temp + airquality$Wind # 14 nova coluna
```

15. Manipular o conjunto `iris`.

```
data(iris)
setosa <- subset(iris, Species == "setosa")
setosa$Petal.Area <- setosa$Petal.Length * setosa$Petal.Width
setosa <- setosa[setosa$Petal.Area > 0.2, ]
setosa[order(setosa$Petal.Length, decreasing = TRUE), ]
```

16. Limpeza e agregação em *airquality*.

```
limpo <- na.omit(airquality)
limpo$Temp.Celsius <- (limpo$Temp - 32) / 1.8
aggregate(Ozone ~ Month, data = limpo, FUN = mean)
plot(limpo$Temp.Celsius, limpo$Ozone, col = limpo$Month, pch = 16,
     xlab = "Temperatura (°C)", ylab = "Ozono")
```

17. Um pequeno conjunto ao estilo *Titanic*.

```
titanic <- data.frame(
  nome = c("Ana", "Bruno", "Clara", "Daniel", "Eva", "Filipe"),
  sexo = c("F", "M", "F", "M", "F", "M"),
  idade = c(22, 35, 18, 50, 28, 40),
  classe = c("1ª", "3ª", "3ª", "1ª", "2ª", "2ª"),
  sobreviveu = c(TRUE, FALSE, TRUE, TRUE, FALSE, FALSE))

sum(titanic$sexo == "F" & titanic$sobreviveu)      # (b) mulheres sobreviventes
mean(titanic$idade[titanic$sobreviveu])           # (c) idade média dos sobreviventes
sum(titanic$classe == "3ª")                       # (d) passageiros de 3.ª
mean(titanic$sobreviveu[titanic$classe == "3ª"]) # taxa de sobrevivência na 3.ª
```

A média de um vetor lógico é a **proporção de TRUE**, porque TRUE conta como 1 e FALSE como 0. Por isso `mean(titanic$sobreviveu[...])` dá diretamente a taxa de sobrevivência, sem necessidade de contar e dividir à mão.

26.6 Leitura de dados e tabelas de frequência

Este conjunto de exercícios parte de um ficheiro externo (*dados_instagram.xlsx*) e ilustra a construção de tabelas de frequência e de gráficos exploratórios. A leitura faz-se com o pacote *readxl*.

```
library(readxl)
dados <- read_excel("dados_instagram.xlsx") # ajuste o caminho ao seu ficheiro
```

Variáveis qualitativas (género, país). Para variáveis categóricas, a tabela de frequências e o gráfico de barras (ou circular) são o ponto de partida.

```
freq_abs <- table(dados$Genero)
tab <- as.data.frame(freq_abs)
names(tab) <- c("Género", "Freq_Abs")
tab$Freq_Rel <- round(tab$Freq_Abs / sum(tab$Freq_Abs), 2)

barplot(freq_abs, col = "lightblue")
pie(tab$Freq_Rel,
    labels = paste(tab$Género, round(tab$Freq_Rel * 100, 1), "%"))
```

Variáveis quantitativas (índice de influência, nº de publicações, seguidores, likes). Para variáveis contínuas, a tabela de frequências agrupa os dados em classes. Uma escolha comum para o número de classes é a **regra de Sturges**. Como o procedimento é o mesmo para todas as variáveis, vale a pena encapsulá-lo numa função — evita repetir o bloco quatro vezes.

```
tabela_frequencias <- function(x) {
  x <- x[!is.na(x)]
  k <- ceiling(1 + 3.322 * log10(length(x))) # nº de classes (Sturges)
  classes <- cut(x, breaks = seq(min(x), max(x), length.out = k + 1),
    include.lowest = TRUE)
  fa <- table(classes)
  data.frame(
    Intervalo = names(fa),
    Freq_Abs = as.integer(fa),
    Freq_Rel = round(as.integer(fa) / length(x), 3),
    Freq_Abs_Ac = cumsum(as.integer(fa)),
    Freq_Rel_Ac = round(cumsum(as.integer(fa)) / length(x), 3))
}

tabela_frequencias(dados$indice_influencia)
hist(dados$indice_influencia, breaks = "Sturges", col = "lightblue")
```

No material original, a tabela da variável `media_likes` reutilizava por distração as contagens de `num_seguidores` (`tabela_frequencia_ns_df`) no cálculo das frequências relativas e acumuladas. Encapsular o procedimento numa função, como acima, elimina de raiz este tipo de erro de copiar-colar: cada variável é processada de forma independente.

Medidas por grupo. Para comparar grupos, `tapply()` aplica uma função a uma variável quantitativa separada por uma variável qualitativa.

```
tapply(dados$indice_influencia, dados$Genero, mean)
tapply(dados$media_likes,      dados$Genero, median)
quantile(dados$num_seguidores, probs = c(0.10, 0.25, 0.75, 0.90))
```

26.7 Gráficos com o R base

1. Gráfico de barras de frequências (absolutas e relativas).

```
data(iris)
barplot(table(iris$Species), col = c("blue","green","yellow"),
        main = "Contagem de espécies", xlab = "Espécie",
        ylab = "Frequência absoluta")
barplot(table(iris$Species) / length(iris$Species),
        col = c("blue","green","yellow"), ylab = "Frequência relativa")
```

2. Barras de uma média por grupo.

```
data(airquality)
media_mes <- tapply(airquality$Temp, airquality$Month, mean, na.rm = TRUE)
barplot(media_mes, col = "lightblue",
        names.arg = c("Maio","Junho","Julho","Agosto","Setembro"),
        main = "Temperatura média por mês", ylab = "Temperatura (°F)")
```

3. e 4. Gráficos circulares.

```
fr <- table(iris$Species) / length(iris$Species)
pie(fr, labels = paste(names(fr), round(fr * 100, 1), "%"),
    col = c("blue","green","yellow"))
```

5. Histograma com curva de densidade sobreposta.

```
data(iris)
hist(iris$Sepal.Length, col = "skyblue", breaks = 15, freq = FALSE,
     main = "Comprimento da sépala", xlab = "cm")
lines(density(iris$Sepal.Length), col = "red", lwd = 2)
```

6. Dois histogramas sobrepostos (com transparência).

```
set.seed(123)
A <- rnorm(100, 150, 10); B <- rnorm(100, 160, 15)
hist(A, col = rgb(1,0,0,0.5), breaks = 10, xlim = c(100, 200),
     main = "Distribuição de alturas", xlab = "Altura (cm)")
hist(B, col = rgb(0,0,1,0.5), breaks = 10, add = TRUE)
legend("topright", c("Grupo A", "Grupo B"),
     fill = c(rgb(1,0,0,0.5), rgb(0,0,1,0.5)))
```

7. Histograma com linhas de referência (média e mediana).

```
set.seed(1234)
notas <- rnorm(200, mean = 14, sd = 4)
hist(notas, col = "lightblue", breaks = 15, main = "Distribuição de notas")
abline(v = mean(notas), col = "red", lwd = 2, lty = 2)
abline(v = median(notas), col = "blue", lwd = 2, lty = 2)
```

8. Vários gráficos numa só figura com `par(mfrow=...)`.

```
data(airquality)
par(mfrow = c(3, 2))
for (m in 5:9) {
  hist(airquality$Ozone[airquality$Month == m],
       main = paste("Ozono - mês", m), xlab = "Ozono",
       col = "lightblue", breaks = 10)
}
par(mfrow = c(1, 1))
```

`hist()` não tem argumento `na.rm`; os NA são simplesmente ignorados no traçado. Passar `na.rm = TRUE` a `hist()`, como surgia no material original, não provoca erro mas também não tem qualquer efeito — é um argumento inexistente para esta função.

9. e 10. Diagramas de caixa (*boxplots*) por grupo.

```
boxplot(Petal.Length ~ Species, data = iris,
        col = c("lightblue", "lightgreen", "lightpink"),
        main = "Comprimento da pétala por espécie")

boxplot(Ozone ~ Month, data = airquality,
        main = "Ozono por mês", xlab = "Mês", ylab = "Ozono")
abline(h = median(airquality$Ozone, na.rm = TRUE), col = "red", lwd = 2, lty = 2)
```

11. e 12. Diagramas de dispersão e séries.

```
altura <- runif(30, 150, 200)
peso <- altura * 0.4 + runif(30, -5, 5)
plot(altura, peso, xlab = "Altura (cm)", ylab = "Peso (kg)")

anos <- 2000:2019
pop <- 100000 * cumprod(1.02 + runif(20, -0.01, 0.01))
plot(anos, pop, type = "l", col = "green",
     xlab = "Ano", ylab = "População")
```

26.8 Estruturas condicionais (if ... else)

Vários destes exercícios leem números do teclado com `readline()`. Esse comando só funciona numa sessão interativa; num documento compilado deve usar-se `eval=FALSE`, como aqui.

1. a 5. Decisões simples e encadeadas.

```
# 1. sinal de um número
num <- as.numeric(readline("Introduza um número: "))
if (num > 0) print("O número é positivo")

# 2. par ou ímpar
if (num %% 2 == 0) print("Par") else print("Ímpar")

# 4. faixa etária
idade <- as.numeric(readline("Idade: "))
if (idade < 0)           print("Idade inválida") else
if (idade <= 12)         print("Criança")         else
if (idade <= 17)         print("Adolescente")     else
if (idade <= 64)         print("Adulto")           else
                        print("Idoso")

# 5. desconto por escalão de preço
preco <- as.numeric(readline("Preço: "))
if (preco < 0)           print("Preço inválido") else
if (preco < 100)         print(preco * 0.95)      else
if (preco <= 500)        print(preco * 0.90)      else
                        print(preco * 0.85)
```

6. Localização de um ponto no plano.

```
x <- as.numeric(readline("x: ")); y <- as.numeric(readline("y: "))
if (x == 0 && y == 0)      print("Origem")      else
if (x == 0)                print("Eixo Y")      else
if (y == 0)                print("Eixo X")      else
if (x > 0 && y > 0)         print("1.º quadrante") else
if (x < 0 && y > 0)         print("2.º quadrante") else
if (x < 0 && y < 0)         print("3.º quadrante") else
                           print("4.º quadrante")
```

No material original os quadrantes estavam trocados: a condição $x < 0$ & $y > 0$ aparecia rotulada como “primeiro quadrante” e $x > 0$ & $y > 0$ como “segundo”. A convenção correta é: 1.º quadrante $x > 0, y > 0$; 2.º $x < 0, y > 0$; 3.º $x < 0, y < 0$; 4.º $x > 0, y < 0$. A versão acima corrige a atribuição.

7. Ano bissexto.

```
ano <- as.numeric(readline("Ano: "))
if ((ano %% 4 == 0 && ano %% 100 != 0) || ano %% 400 == 0)
  print("Ano bissexto") else print("Não é bissexto")
```

8. Decompor um número em centenas, dezenas e unidades.

```
num <- as.numeric(readline("Número (0-999): "))
if (num >= 0 && num <= 999) {
  c <- num %/% 100; d <- (num % 100) %/% 10; u <- num % 10
  print(paste(c, "centenas", d, "dezenas", u, "unidades"))
} else print("Fora do intervalo")
```

9. Numeração romana (1 a 5).

```
num <- as.numeric(readline("Número (1-5): "))
romanos <- c("I", "II", "III", "IV", "V")
if (num >= 1 && num <= 5) print(romanos[num]) else print("Fora do intervalo")
```

Em vez de cinco ramos `if/else if` para converter 1–5 em numeração romana, um único vetor indexado — `romanos[num]` — resolve o problema numa linha. Sempre que uma cadeia de ifs apenas escolhe um de vários valores fixos consoante um inteiro, um vetor de consulta é mais limpo.

10. Menor de vários números (versão vetorizada).

```
# Alternativa concisa à leitura número a número:
nums <- as.numeric(strsplit(readline("Números separados por espaço: "), " ")[[1]])
min(nums)
```

11. a 13. ifelse() vetorizado.

```
ifelse(1:10 %% 2 == 0, "par", "ímpar") # 11

notas <- c(50, 85, 70, 40, 60, 90) # 12
ifelse(notas >= 60, "Aprovado",
       ifelse(notas >= 50, "Aprovado com restrição", "Reprovado"))

idades <- c(1, 2, 14, 20, 35, 67, 80) # 13
ifelse(idades <= 12, "Criança",
       ifelse(idades <= 18, "Adolescente", "Adulto"))
```

if avalia **uma única** condição (um valor lógico); aplicá-lo a um vetor usa apenas o primeiro elemento e, nas versões recentes do R, dá erro. Para decidir elemento a elemento ao longo de um vetor, o instrumento certo é ifelse().

26.9 Ciclos for

1. a 6. Percursos e acumuladores.

```
for (i in 0:10) print(i) # 1

for (i in 1:5) print(paste("O quadrado de", i, "é", i^2)) # 2

soma <- 0 # 3
for (num in c(2,4,6,8,10)) soma <- soma + num
soma

notas <- c(85, 90, 78, 92, 88) # 4
soma <- 0; for (n in notas) soma <- soma + n
soma / length(notas)

for (num in c(1,3,4,6,9,11,14)) # 5
  if (num %% 2 != 0) print(paste(num, "é ímpar"))
```

```
n <- 7 # 6 tabuada
for (i in 1:10) print(paste(n, "x", i, "=", n * i))
```

7. Construir um vetor dentro de um ciclo.

```
n <- 5
v <- numeric(n) # pré-alocar é melhor do que crescer com c()
for (i in seq_len(n)) v[i] <- i
v
```

Fazer crescer um vetor com `v <- c(v, i)` dentro de um ciclo obriga o R a copiar o vetor inteiro a cada passo, o que se torna lento para muitas iterações. **Pré-alocar** com `numeric(n)` e preencher por índice é substancialmente mais eficiente. Use `seq_len(n)` (ou `seq_along(x)`) em vez de `1:n`, pois estas lidam corretamente com o caso `n = 0`.

8. e 9. Números perfeitos (um número igual à soma dos seus divisores próprios).

```
# 8. testar um número
n <- 28
soma <- 0
for (i in 1:(n - 1)) if (n %% i == 0) soma <- soma + i
if (soma == n) print(paste(n, "é perfeito")) else print(paste(n, "não é perfeito"))

# 9. todos os perfeitos até 1000 (encapsulado numa função)
e_perfeito <- function(n) {
  soma <- sum((1:(n - 1))[n %% (1:(n - 1)) == 0])
  soma == n
}
for (n in 2:1000) if (e_perfeito(n)) print(n) # 6, 28, 496
```

10. e 11. Sequências e médias iterativas.

```
# 10. imprimir 0, m, 2m, ... até n
for (i in seq(0, n, by = m)) print(i)

# 11. média de n notas lidas do teclado
n <- as.numeric(readline("Quantas notas? "))
soma <- 0
for (i in seq_len(n)) soma <- soma + as.numeric(readline("Nota: "))
soma / n
```

12. Estatísticas por linha e por coluna de uma matriz.

```

A <- matrix(1:16, nrow = 4)

# média de cada coluna (com ciclo, para praticar; colMeans faria o mesmo)
media_col <- numeric(ncol(A))
for (j in seq_len(ncol(A))) media_col[j] <- mean(A[, j])

# média dos números pares de cada coluna
for (j in seq_len(ncol(A))) {
  pares <- A[, j][A[, j] %% 2 == 0]
  cat("Coluna", j, "- média dos pares:", mean(pares), "\n")
}

```

26.10 Ciclos while

1. a 5. Contadores e acumuladores controlados por condição.

```

# 1. imprimir de 0 até k
k <- 5; contador <- 0
while (contador <= k) { print(contador); contador <- contador + 1 }

# 3. somatório dos primeiros n inteiros
n <- 10; soma <- 0; i <- 1
while (i <= n) { soma <- soma + i; i <- i + 1 }
soma
# n(n+1)/2 = 55

# 5. somar números até introduzir um negativo
soma <- 0
n <- as.numeric(readline("Número (negativo para sair): "))
while (n >= 0) { soma <- soma + n; n <- as.numeric(readline("Número: ")) }
soma

```

9. Fatorial.

```

n <- 6; fat <- 1; i <- 1
while (i <= n) { fat <- fat * i; i <- i + 1 }
fat
# 720 (compare com factorial(6))

```

10. Soma dos algarismos de um número.

```
n <- 1234; soma <- 0
while (n > 0) { soma <- soma + n %% 10; n <- n %/% 10 }
soma                # 1+2+3+4 = 10
```

11. Sucessão de Fibonacci até um valor máximo (uso de `break`).

```
valor_max <- 100
f1 <- 1; f2 <- 1
print(f1); if (valor_max > 1) print(f2)
repeat {
  f <- f1 + f2
  if (f > valor_max) break
  print(f)
  f1 <- f2; f2 <- f
}
```

`while (TRUE) { ... if (cond) break }` e `repeat { ... if (cond) break }` são equivalentes; `repeat` exprime de forma mais direta a ideia de “repetir até uma condição de paragem”. Em qualquer ciclo indefinido, garanta sempre que existe um caminho que conduz ao `break` — caso contrário o ciclo nunca termina.

26.11 Funções

1. e 2. Funções elementares.

```
quadrado <- function(x) x^2
hipotenusa <- function(a, b) sqrt(a^2 + b^2)
```

3. Divisão segura (função completa — o material original omitia o cabeçalho).

```
divisao_segura <- function(numerador, denominador) {
  if (denominador == 0) return("Divisão por zero!")
  numerador / denominador
}
divisao_segura(10, 2)    # 5
divisao_segura(10, 0)    # "Divisão por zero!"
```

4. e 5. Compor funções (maior e menor de dois ou três números).


```
maior <- function(x, y) if (x > y) x else y
menor <- function(x, y) if (x < y) x else y
maior3 <- function(x, y, z) maior(maior(x, y), z)
```

6. Função que devolve várias estatísticas.

```
analise_vetor <- function(x) {
  c(soma = sum(x), media = mean(x), dp = sd(x), maximo = max(x))
}
analise_vetor(c(4, 8, 15, 16, 23, 42))
```

7. e 8. Classificação e índice de massa corporal.

```
classificar <- function(x) {
  if (x == 0) "zero" else if (x > 0) "positivo" else "negativo"
}

imc <- function(peso, altura) peso / altura^2
# a categoria pode obter-se com cut(), evitando uma longa cadeia de if/else:
categoria_imc <- function(v) {
  cut(v, breaks = c(-Inf, 18.5, 25, 30, 35, 40, Inf),
      labels = c("Baixo peso", "Normal", "Excesso de peso",
                  "Obesidade I", "Obesidade II", "Obesidade III"),
      right = FALSE)
}
categoria_imc(imc(70, 1.75))
```

9. Conversor de moedas.

```
converter <- function(euros) {
  c(dolar = euros * 1.10, libra = euros * 0.85,
     iene = euros * 130, real = euros * 5.20)
}
converter(100)
```

10. e 11. Intervalo de confiança para a média.

```
IC <- function(amostra, alpha) {
  media <- mean(amostra); n <- length(amostra)
  erro <- qnorm(1 - alpha / 2) * sd(amostra) / sqrt(n)
```

```

  c(inferior = media - erro, superior = media + erro)
}
IC(1:15, 0.05)

# 11. vários níveis de confiança de uma só vez (vetorizado)
ICs <- function(amostra, alphas) {
  media <- mean(amostra); n <- length(amostra)
  ep <- sd(amostra) / sqrt(n)
  z <- qnorm(1 - alphas / 2)
  data.frame(Alpha = alphas,
             Inferior = media - z * ep,
             Superior = media + z * ep)
}
set.seed(123)
ICs(rnorm(100, 50, 10), c(0.001, 0.01, 0.05, 0.10))

```

12. Moda (valor mais frequente).

```

calcula_moda <- function(x) {
  fr <- table(x)
  modas <- names(fr)[fr == max(fr)]
  if (is.numeric(x)) as.numeric(modas) else modas
}
calcula_moda(c(1, 2, 2, 3, 3, 4, 4))      # 2 3 4 (distribuição multimodal)

```

13. e 14. Distância euclidiana e análise de matrizes.

```

distancia <- function(x, y) sqrt(sum((x - y)^2))
distancia(c(2, 3), c(6, 9))              # 7.211

analisa_matriz <- function(A) {
  res <- list(soma = sum(A), media = mean(A))
  if (nrow(A) == ncol(A)) {
    res$traco <- sum(diag(A)); res$det <- det(A)
  }
  res
}

```

26.12 *Scripts* e a função `source()`

Estes exercícios exercitam a organização do código em ficheiros separados, carregados com `source()`. Cada *script* define funções que depois se usam. Por exemplo, um ficheiro `estatistica.R` com

```
# ficheiro estatistica.R
ordena      <- function(x) sort(x)
media       <- function(x) mean(x)
variancia   <- function(x) var(x)
desv_padrao <- function(x) sd(x)
```

é carregado e usado assim:

```
source("estatistica.R")          # ajuste o caminho ao seu ficheiro
amostra <- c(1, 2, 3, 4, 5, 6, 7)
variancia(amostra)
desv_padrao(amostra)
```

De igual modo se organizam os *scripts* `quadrado.R` (perímetro e área do quadrado), `circulo.R` (área e perímetro do círculo), `analise.R` (mínimo, máximo, amplitude), `categorias.R` (contagem de categorias), `escore_z.R` (cálculo do valor padronizado z) e `binomial.R`, este último com a função de probabilidade da binomial calculada de raiz:

```
# ficheiro binomial.R
probabilidade_binomial <- function(n, p, x) choose(n, x) * p^x * (1 - p)^(n - x)

# uso, comparando com a função interna do R
source("binomial.R")
probabilidade_binomial(5, 0.6, 3)
dbinom(x = 3, size = 5, prob = 0.6) # deve coincidir
```

Separar funções reutilizáveis em *scripts* próprios e carregá-las com `source()` mantém o código de análise limpo e as funções disponíveis para vários projetos. É o primeiro passo, informal, na direção da criação de *pacotes* de R.

26.13 A família `apply`

1. a 5. `lapply`, `sapply`, `apply` e `tapply`.

```
sapply(c("cachorro","gato","elefante","leão"), nchar)    # 2

M <- matrix(1:9, nrow = 3)
apply(M, 1, sum)      # soma por linha (MARGIN = 1)
apply(M, 2, mean)     # média por coluna (MARGIN = 2)

tapply(mtcars$mpg, mtcars$cyl, mean)    # 4 média de mpg por nº de cilindros
```

A diferença entre `lapply()` e `sapply()` é apenas o formato do resultado: `lapply()` devolve sempre uma **lista**; `sapply()` tenta **simplificar** para vetor ou matriz quando possível. Para o argumento `MARGIN` de `apply()`, a regra é 1 = linhas, 2 = colunas.

6. Aplicar uma função personalizada a cada elemento.

```
sapply(1:10, function(x) if (x %% 2 == 0) "par" else "ímpar")
```

7. e 8. Estatísticas por grupo e normalização de colunas.

```
tapply(iris$Petal.Length, iris$Species, mean)
tapply(iris$Sepal.Width, iris$Species, var)

normaliza <- function(x) (x - min(x)) / (max(x) - min(x))
dados_norm <- as.data.frame(apply(mtcars, 2, normaliza))
```

9. e 10. Aplicações a matrizes de dados simulados.

```
set.seed(123)
notas <- as.data.frame(matrix(runif(30 * 4, 0, 100), nrow = 30))
colnames(notas) <- paste0("Exame", 1:4)
apply(notas, 1, mean)      # média de cada aluno (por linha)
sapply(notas, min)        # nota mínima por exame (por coluna)
sapply(notas, max)
```

26.14 O operador *pipe*

O *pipe* (`%>%` do `magrittr`, ou `|>` nativo do R) encadeia operações, passando o resultado de cada passo como primeiro argumento do seguinte.

1. a 3. Encadeamentos numéricos.

```
library(magrittr)
1:10 %>% sum() %>% divide_by(3) %>% round(1)    # 1

set.seed(123)
rnorm(100) %>% extract(. > 0) %>% mean() %>% round(2)    # 2 (só os positivos)
```

No exercício 3, o encadeamento $2 + 2 \rightarrow$ colocar num vetor com 6 e NA $\rightarrow$ `mean(na.rm = TRUE)` $\rightarrow$ comparar com 5 devolve TRUE: a média de `c(4, 6)`, ignorando o NA, é de facto 5. O *pipe* torna esta leitura sequencial imediata, lendo-se o código da esquerda para a direita como uma frase.

4. a 7. Fluxos de manipulação de dados com o dplyr.

```
library(dplyr)
mtcars %>% select(mpg, hp, wt) %>% filter(hp > 100) %>% arrange(wt)    # 4

airquality %>% na.omit() %>% filter(Wind > 10) %>%
  summarise(ozono_medio = mean(Ozone))                                # 6

mtcars %>% mutate(hp_wt = hp / wt) %>% filter(hp_wt > 30) %>%
  select(mpg, hp_wt, gear)                                           # 7
```

26.15 Exercícios de revisão

Estes problemas integradores combinam *data frames*, funções e ciclos.

1. Satisfação de clientes: média por linha e classificação.

```
set.seed(123)
satisfacao <- data.frame(
  S1 = sample(1:10, 10, replace = TRUE),
  S2 = sample(1:10, 10, replace = TRUE),
  S3 = sample(1:10, 10, replace = TRUE))

satisfacao$media <- rowMeans(satisfacao)                                # (b)
satisfacao$grau <- ifelse(satisfacao$media >= 7,                      # (c)
  "Satisfeito", "Insatisfeito")
```

2. Metas de colaboradores: função que verifica o número de dias.

```
verifica_meta <- function(df) {
  # dias com >= 20 tarefas, por colaborador; meta = pelo menos 3 dias
  ifelse(rowSums(df >= 20) >= 3, "Atingiu Meta", "Não Atingiu Meta")
}
```

`rowSums(df >= 20)` conta, em cada linha, quantas entradas satisfazem a condição, porque `df >= 20` produz uma matriz de TRUE/FALSE (isto é, 1/0). Substitui de forma elegante um ciclo duplo sobre linhas e colunas.

3. Médias de amostras, interrompendo ao ultrapassar um limite (`while + break`).

```
medias <- rowMeans(amostras)
i <- 1
while (i <= length(medias)) {
  cat("Amostra", i, ":", medias[i], "\n")
  if (medias[i] > 75) { cat("Excedeu o limite!\n"); break }
  i <- i + 1
}
```

4. Lista `listas_transformadas` com $x_j = j^2 + i$.

```
listas_transformadas <- vector("list", 7)
for (i in 1:7) listas_transformadas[[i]] <- (1:i)^2 + i
length(listas_transformadas[[7]])           # (b) = 7

soma_elementos <- function(lst) apply(lst, sum) # (c)
soma_elementos(listas_transformadas)         # (d)
```

5. e 6. Operações vetorizadas com `sample()` e `rnorm()`.

```
set.seed(1)
x <- sample(0:999, 60, replace = TRUE)
y <- sample(0:999, 60, replace = TRUE)
y[y < 500 & x %% 3 == 0]           # (5b)
dif <- abs(x - y); c(media = mean(dif), dp = sd(dif)) # (5c-d)

x <- rnorm(80, mean = 5, sd = 2)    # 6
xbar <- mean(x)
resultado <- sqrt(abs(x^3 - xbar^2))
c(max = max(resultado), min = min(resultado), media = mean(resultado))
```

7. Temperaturas semanais de 12 cidades (função + `while`).

```
media_semanal <- rowMeans(temperaturas_cidades)
classificacao <- ifelse(media_semanal > 25, "Acima de 25°C", "Abaixo de 25°C")
temperaturas_cidades$media <- media_semanal
temperaturas_cidades$classe <- classificacao
```

8. *Bit flips* na memória (simulação com a binomial/Poisson). A modelação do número de *bits* alterados num bloco de memória por unidade de tempo faz-se naturalmente com uma binomial (muitos *bits*, cada um com probabilidade ínfima de se alterar) ou, no limite, com a Poisson. A resolução detalhada segue o mesmo padrão dos exercícios de simulação da secção seguinte: gerar, contar, comparar com a probabilidade teórica.

26.16 Amostragem com `sample()`

1. a 6. Amostragem com e sem reposição, com e sem probabilidades.

```
sample(1:20, size = 5, replace = FALSE)    # 1 sem reposição
sample(1:10, size = 10, replace = TRUE)    # 2 com reposição
sample(c("A","B","C","D","E"), size = 3,   # 3 probabilidades desiguais
       prob = c(0.1, 0.2, 0.3, 0.25, 0.15))
sample(1:10)                               # 4 permutação (size e replace por defeito)
sample(1:1000, size = 50)                  # 6 amostra simples de uma população
```

7. Soma de dois dados (10 000 lançamentos) — duas formas.

```
set.seed(1)
d1 <- sample(1:6, 10000, replace = TRUE)
d2 <- sample(1:6, 10000, replace = TRUE)
hist(d1 + d2, breaks = 0:12, col = "lightblue") # distribuição triangular em 7
```

8. Amostragem estratificada.

```
set.seed(123)
turmas <- c(rep("A", 50), rep("B", 100), rep("C", 50))
# amostrar proporcionalmente: 5 de A, 10 de B, 5 de C
idx <- c(sample(which(turmas == "A"), 5),
         sample(which(turmas == "B"), 10),
         sample(which(turmas == "C"), 5))
turmas[idx]
```

9. e 10. Cartões de bingo e afetação a grupos.

```
replicate(5, sample(1:75, 24, replace = FALSE))    # 9 cinco cartões

set.seed(1)                                         # 10 tratamento vs. controlo
grupos <- sample(c(rep("Tratamento", 20), rep("Controlo", 10)))
```

Para dividir aleatoriamente unidades em grupos de tamanhos fixos, baralhar um vetor de rótulos com `sample()` é mais seguro do que sortear índices e usar `setdiff()`: garante automaticamente que cada unidade fica em exatamente um grupo.

26.17 A semente `set.seed()`

Estes exercícios ilustram um princípio central da simulação reprodutível.

```
set.seed(123); sample(1:50, 5)    # produz sempre a mesma amostra...
set.seed(123); sample(1:50, 5)    # ...se a semente for a mesma
set.seed(456); sample(1:50, 5)    # semente diferente -> amostra diferente
```

`set.seed()` fixa o estado do gerador de números pseudoaleatórios. Repor a mesma semente **antes** de gerar reproduz exatamente a mesma sequência — indispensável para que outra pessoa (ou o próprio, mais tarde) obtenha resultados idênticos. É por isso que quase todos os exemplos de simulação deste livro começam por `set.seed()`. Note-se que a semente deve ser reposta imediatamente antes do bloco que se quer reproduzir: chamadas intermédias ao gerador alteram o estado.

26.18 Probabilidade e variáveis aleatórias

Esta secção mistura cálculo analítico com verificação em R. Sempre que possível, deduzimos o resultado exato e confirmamo-lo com `integrate()` ou com as funções de probabilidade do R.

1. Variável discreta com $P(X = x)$ dada por (0,1, 0,2, 0,3, 0,2, 0,2) para $x = 0, 1, 2, 3, 4$.

As probabilidades somam 1, logo constituem uma função massa de probabilidade válida. Os momentos são

$$E(X) = \sum_x x P(X = x) = 2,2, \quad \text{Var}(X) = \sum_x (x - 2,2)^2 P(X = x) = 1,56.$$


```
x <- 0:4
px <- c(0.1, 0.2, 0.3, 0.2, 0.2)
sum(px)                # (a) = 1
EX <- sum(x * px)      # (b) = 2.2
VarX <- sum((x - EX)^2 * px) # = 1.56
sum(px[x > 2])         # (c) P(X > 2) = 0.4
```

No material original, a alínea (c) calculava `sum(P_X[X <= 2])`, que dá $P(X \leq 2) = 0,6$, e rotulava-a como “probabilidade de haver mais de 2 reclamações”. As duas quantidades são complementares: $P(X > 2) = 1 - P(X \leq 2) = 0,4$. Atenção à fronteira: “mais de 2” exclui o valor 2.

3. Densidade definida por troços (por exemplo, um rendimento).

```
f <- function(x) ifelse(x >= 0 & x <= 2, x/10 + 1/10,
                        ifelse(x > 2 & x <= 6, -3/40 * x + 9/20, 0))
integrate(f, 0, 6)                # verifica que integra 1
integrate(function(x) x * f(x), 0, 6) # E(X)
integrate(f, 4.5, 6)              # P(X > 4.5)
```

4. Uniforme contínua em $[10, 20]$: $f(x) = 1/10$.

Analicamente, $E(X) = 15$, $\text{Var}(X) = (20 - 10)^2/12 = 100/12 \approx 8,33$, $P(X < 15) = 0,5$ e $P(12 \leq X \leq 18) = 6/10 = 0,6$.

```
f <- function(x) ifelse(x >= 10 & x <= 20, 1/10, 0)
integrate(f, 10, 15)    # 0.5
integrate(f, 12, 18)    # 0.6
integrate(function(x) x * f(x), 10, 20)      # E(X) = 15
integrate(function(x) x^2 * f(x), 10, 20)$value -
  integrate(function(x) x * f(x), 10, 20)$value^2 # Var(X)
```

5. Densidade triangular $f(y) = 0,125y$ em $[0, 4]$.

A função de distribuição é $F(y) = \int_0^y 0,125t \, dt = 0,0625y^2$ em $[0, 4]$ (note-se que $0,0625 = 1/16$, e $F(4) = 1$). Daqui, $P(Y > 3) = 1 - F(3) = 7/16$; $P(1 < Y < 3) = F(3) - F(1) = 8/16 = 0,5$; e $P(1 < Y < 3 \mid Y > 1) = \frac{8/16}{15/16} = 8/15$.

```
f <- function(y) ifelse(y >= 0 & y <= 4, 0.125 * y, 0)
F <- function(y) ifelse(y < 0, 0, ifelse(y <= 4, 0.0625 * y^2, 1))
1 - F(3)                # P(Y > 3) = 7/16
F(3) - F(1)             # P(1 < Y < 3) = 0.5
(F(3) - F(1)) / (1 - F(1)) # P(1 < Y < 3 | Y > 1) = 8/15
```

6. Exponencial de parâmetro $\lambda = 3$: $f(x) = 3e^{-3x}$, $x \geq 0$.

A função de distribuição é $F(x) = 1 - e^{-3x}$ (note-se o sinal **negativo** no expoente). Logo $P(X > 2) = e^{-6}$, e assim por diante.

```
f <- function(x) ifelse(x >= 0, 3 * exp(-3 * x), 0)
integrate(f, 2, Inf)           # P(X > 2) = e^-6
integrate(f, 0.5, 1.2)       # P(0.5 < X < 1.2)
(integrate(f, 1, 3)$value) / (integrate(f, 1, Inf)$value) # P(1 < X < 3 | X > 1)
```

A função de distribuição da exponencial é $F(x) = 1 - e^{-\lambda x}$, com sinal **negativo** no expoente — nunca $1 - e^{\lambda x}$, que seria negativa e cresceria sem limite. Este é um erro de sinal fácil de cometer e que conduz a probabilidades absurdas.

26.19 Distribuição uniforme discreta

1. e 2. Simulação e verificação de propriedades.

```
set.seed(42)
valores <- 1:10
sim <- sample(valores, 1000, replace = TRUE)
table(sim) / 1000           # frequências próximas de 0.1 cada

# esperança e variância teóricas de uma uniforme discreta em {a,...,b}
# E(X) = (a+b)/2 ; Var(X) = ((b-a+1)^2 - 1)/12
mean(sim); var(sim)
```

Para a uniforme discreta no conjunto $\{a, a+1, \dots, b\}$, a esperança é $(a+b)/2$ e a variância é $((b-a+1)^2 - 1)/12$. Repare que a fórmula da variância **não** coincide com a da uniforme contínua: o termo -1 e o uso de $(b-a+1)$ resultam de os valores serem inteiros discretos e não um intervalo contínuo. A função `var()` do R, aplicada ao conjunto dos valores, usa o divisor $n-1$ e serve apenas como verificação aproximada via simulação.

3. a 5. Somas e transformações.

```
set.seed(42)
X <- sample(1:6, 5000, replace = TRUE)
Y <- sample(1:6, 5000, replace = TRUE)
Z <- X + Y
mean(Z); var(Z); mean(Z > 8)           # soma de dois dados
```

```
W <- sample(-5:5, 5000, replace = TRUE)      # transformação não linear
V <- 3 * W^3 - 2 * W^2 + W
mean(V); mean(V > 0)
```

26.20 Distribuição de Bernoulli

1. e 2. Frequências e momentos empíricos vs. teóricos.

```
set.seed(42)
p <- 0.7
X <- rbinom(1000, size = 1, prob = p)      # Bernoulli(p) = Binomial(1, p)
table(X) / 1000                          # ~ (1-p, p)

# momentos: E(X) = p ; Var(X) = p(1-p)
p2 <- 0.4
X2 <- rbinom(10000, 1, p2)
c(media_emp = mean(X2), media_teo = p2,
  var_emp = var(X2), var_teo = p2 * (1 - p2))
```

3. Soma de 5 Bernoulli independentes é uma Binomial(5, p).

```
set.seed(42)
X <- matrix(rbinom(5000 * 5, 1, 0.5), nrow = 5000)
S <- rowSums(X)
table(S) / 5000                          # empírico
dbinom(0:5, size = 5, prob = 0.5)        # teórico
```

26.21 Distribuição binomial

1. $X \sim \text{Binomial}(10; 0,5)$.

```
dbinom(6, 10, 0.5)                      # (a)  $P(X = 6)$ 
pbinom(4, 10, 0.5)                      # (b)  $P(X \leq 4)$ 
pbinom(7, 10, 0.5)                      # (c)  $P(X \leq 7)$ 
```

2. $X \sim \text{Binomial}(12; 1/6)$ — teoria e simulação.

```

pbinom(5, 12, 1/6)                # (a)  $P(X \leq 5)$ 
set.seed(1)
amostra <- rbinom(1000, 12, 1/6)    # (b)
barplot(table(amostra) / length(amostra), col = "lightblue") # (c)
points(0:12, dbinom(0:12, 12, 1/6), col = "magenta", pch = 19)
Fn <- ecdf(amostra); Fn(5); pbinom(5, 12, 1/6) # (d) empírica vs teórica
12 * (1/6); 12 * (1/6) * (5/6)      # (e) média e variância teóricas
mean(amostra); var(amostra)         # (f) empíricas

```

A função de distribuição empírica `ecdf(amostra)` devolve uma função: `Fn(5)` estima $P(X \leq 5)$ pela proporção de observações ≤ 5 . À medida que a dimensão da amostra cresce, aproxima-se da função de distribuição teórica `pbinom` — uma manifestação direta da lei dos grandes números.

3. Convergência da frequência relativa para a probabilidade teórica.

```

set.seed(123)
prob_teorica <- dbinom(3, 7, 0.5)
for (n in c(5, 10, 100, 1000)) {
  amostra <- rbinom(n, 7, 0.5)
  cat("n =", n, "-> P(X=3) empírica:", mean(amostra == 3),
      " (teórica:", round(prob_teorica, 3), ")\n")
}

```

4. Função massa de probabilidade e moda.

```

k <- 0:10
probs <- dbinom(k, 10, 0.25)
barplot(probs, names.arg = k)
k[which.max(probs)]      # valor mais provável (moda)

```

5. a 8. Probabilidades de intervalos, caudas e comparação empírica.

```

sum(dbinom(8:12, 15, 0.6))          # 5  $P(8 \leq X \leq 12)$ 
1 - pbinom(9, 25, 0.3)              # 7  $P(X \geq 10)$ 

set.seed(123)                        # 8 histograma + fmp + ecdf
amostra <- rbinom(700, 30, 0.4)
barplot(table(amostra) / length(amostra), col = "lightblue")
points(0:30, dbinom(0:30, 30, 0.4), col = "magenta", pch = 19)

```

26.22 Distribuição geométrica

Há duas convenções para a distribuição geométrica. Na convenção matemática mais comum, X conta o **número de provas até ao primeiro sucesso** (com suporte $\{1, 2, \dots\}$). O R, porém, usa a outra convenção: `rgeom`, `dgeom` e `pgeom` referem-se ao **número de insucessos antes do primeiro sucesso** (suporte $\{0, 1, 2, \dots\}$). Assim, se Y é a variável do R e X a da convenção matemática, tem-se $X = Y + 1$. Tenha esta relação presente ao comparar médias: a média de `rgeom` é $(1 - p)/p$, e não $1/p$.

1. e 2. Simulação, função massa de probabilidade e probabilidades acumuladas.

```
set.seed(1)
amostra <- rgeom(1000, prob = 1/6)           # n° de insucessos (convenção do R)
barplot(table(amostra) / length(amostra), col = "lightblue")
points(0:max(amostra), dgeom(0:max(amostra), 1/6), col = "magenta", pch = 19)

set.seed(123)
amostra <- rgeom(5000, prob = 0.2)
mean(amostra <= 5)                          # empírica
pgeom(5, prob = 0.2)                        # teórica P(Y <= 5)
```

3. Efeito do parâmetro p na forma da distribuição.

```
par(mfrow = c(1, 3))
for (p in c(0.1, 0.5, 0.9))
  barplot(dgeom(0:20, p), names.arg = 0:20,
          main = paste("p =", p), col = "lightblue")
par(mfrow = c(1, 1))
```

Quanto **menor** p , mais dispersa a distribuição (é preciso esperar mais pelo sucesso); quanto **maior** p , mais concentrada junto de zero.

4. a 6. Média, aplicações e comparação teórica/empírica.

```
mean(rgeom(10000, 0.3))                     # 4 ~ (1-p)/p = 0.7/0.3 ~ 2.33

set.seed(123)                               # 6
p <- 0.3
pgeom(5, p)                                 # (a) P(no máximo 5 insucessos)
1 - pgeom(8, p)                             # (b) P(mais de 8)
sim <- rgeom(500, p)
c(media_emp = mean(sim), media_teo = (1 - p) / p,
  dp_emp = sd(sim), dp_teo = sqrt((1 - p) / p^2))
```

26.23 Distribuição de Poisson

1. $X \sim \text{Poisson}(\lambda = 4)$.

```
dpois(5, 4)      # (a)  $P(X = 5)$ 
ppois(3, 4)      # (b)  $P(X \leq 3)$ 
```

2. Momentos empíricos (recorde que, na Poisson, $E(X) = \text{Var}(X) = \lambda$).

```
set.seed(1)
amostra <- rpois(30, 4)
c(media = mean(amostra), dp = sd(amostra))  #  $\sim 4$  e  $\sim 2$ 
```

3. Convergência da frequência e dos momentos com o tamanho da amostra.

```
set.seed(123)
lambda <- 20
for (n in c(5, 10, 100, 1000)) {
  sim <- rpois(n, lambda)
  cat("n =", n, "| média:", round(mean(sim), 2),
      "| variância:", round(var(sim), 2), "\n")
}
```

4. $X \sim \text{Poisson}(12)$ — probabilidades e representação.

```
ppois(10, 12)      # (a)  $P(X \leq 10)$ 
ppois(15, 12, lower.tail = FALSE)  # (b)  $P(X > 15)$ 
set.seed(1)
amostra <- rpois(500, 12)
mean(amostra <= 10)      # (c) frequência empírica
c(media = mean(amostra), dp = sd(amostra))  # (d) - nota: '<-', não '<'
barplot(table(amostra) / length(amostra), col = "lightblue")  # (e)
points(0:25, dpois(0:25, 12), col = "magenta", pch = 19)
```

No material original surgia `desv_padrao_empirico < sd(amostra)`, com um único `<`. Isto **não** é uma atribuição — é uma comparação lógica, cujo resultado é descartado, deixando a variável por criar. A atribuição correta é `<=`. É um lapso comum e difícil de detetar, porque não gera erro.

5. e 6. Função de distribuição empírica vs. teórica.

```
set.seed(123)
aves <- rpois(1000, 8)
plot(ecdf(aves), verticals = TRUE, do.points = FALSE, col = "blue",
     main = "F. distribuição empírica vs teórica")
points(0:max(aves), ppois(0:max(aves), 8), col = "red", pch = 19)
```

9. e 10. Teorema limite central aplicado à Poisson (a média de amostras é aproximadamente normal).

```
set.seed(1)
m <- 1000; n <- 30
medias <- rowMeans(matrix(rpois(m * n, 2), nrow = m))
hist(medias, probability = TRUE, col = "lightblue")
curve(dnorm(x, mean = 2, sd = sqrt(2 / n)), col = "red", lwd = 2, add = TRUE)
```

26.24 Distribuição uniforme contínua

1. Momentos: $E(X) = (a + b)/2$, $\text{Var}(X) = (b - a)^2/12$.

```
set.seed(1)
amostra <- runif(1000, 0, 1)
c(media = mean(amostra), var = var(amostra)) # ~ 0.5 e ~ 1/12
```

2. Histograma com densidade teórica.

```
set.seed(1)
amostra <- runif(500, -3, 7)
hist(amostra, probability = TRUE, col = "lightblue")
curve(dunif(x, -3, 7), col = "red", lwd = 2, add = TRUE)
```

3. a 5, 7, 8. Probabilidades, transformações lineares e comparação empírica.

```
punif(100, 85, 105) # 3  $P(X < 100)$ 
set.seed(1)
amostra <- runif(10000, 2, 8); mean(amostra > 5) # 4 vs  $\text{punif}(5, 2, 8, \text{lower.tail} = \text{FALSE})$ 
mean(runif(1000, 10, 20) %in% 12:15) # 5 (aproximação da faixa [12,15])

# 7 transformação linear  $Z = 3X + 2$ , com  $X \sim U(0,1)$ 
z <- 3 * runif(1000) + 2
```

```
c(media = mean(z), media_teo = 3 * 0.5 + 2,      # E(Z) = 3·E(X)+2 = 3.5
  var = var(z),   var_teo = 9 * (1/12))          # Var(Z) = 9·Var(X) = 9/12
```

6, 9, 10. Lei dos grandes números e teorema limite central.

```
set.seed(123)                                     # 9 média converge para 0.5
sapply(c(100, 1000, 10000, 100000),
  function(n) mean(runif(n)))

set.seed(1)                                       # 10 TLC para a soma/média
m <- 8000; n <- 100
amostras <- matrix(runif(m * n, 10, 30), nrow = m)
media <- (10 + 30) / 2; variancia <- (30 - 10)^2 / 12
hist(rowMeans(amostras), probability = TRUE, col = "lightblue")
curve(dnorm(x, media, sqrt(variancia / n)), col = "red", lwd = 2, add = TRUE)
```

26.25 Distribuição normal

1. Amostra e densidade teórica.

```
set.seed(1)
amostra <- rnorm(1000, mean = 50, sd = 5)
hist(amostra, probability = TRUE, col = "lightblue")
curve(dnorm(x, 50, 5), col = "red", lwd = 2, add = TRUE)
```

2. Soma de normais independentes é normal: se $X \sim N(\mu_1, \sigma_1^2)$ e $Y \sim N(\mu_2, \sigma_2^2)$ forem independentes, $X + Y \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$.

```
set.seed(1)
soma <- rnorm(1000, 5, 2) + rnorm(1000, 10, 3)
hist(soma, probability = TRUE, col = "lightblue")
curve(dnorm(x, mean = 15, sd = sqrt(2^2 + 3^2)), col = "red", lwd = 2, add = TRUE)
```

As variâncias somam-se (para variáveis independentes), **não** os desvios-padrão: aqui $\sigma = \sqrt{2^2 + 3^2} = \sqrt{13}$, e não $2 + 3 = 5$. Confundir as duas coisas é um erro clássico.

3. Soma de quadrados de normais padrão dá uma qui-quadrado: se $Z_1, Z_2 \sim N(0, 1)$ independentes, $Z_1^2 + Z_2^2 \sim \chi_2^2$.


```
set.seed(1)
Q <- rnorm(5000)^2 + rnorm(5000)^2
hist(Q, probability = TRUE, col = "lightblue")
curve(dchisq(x, df = 2), col = "red", lwd = 2, add = TRUE)
```

4 a 9. Distribuições amostrais fundamentais (a base da inferência): a média padronizada segue uma t de Student; $(n-1)S^2/\sigma^2$ segue uma qui-quadrado; a soma de quadrados de n normais padrão é χ_n^2 .

```
set.seed(1)
n <- 20; m <- 1000

# soma de n quadrados ~ qui-quadrado com n graus de liberdade (ex. 5)
sq <- rowSums(matrix(rnorm(m * n), nrow = m)^2)
hist(sq, probability = TRUE, col = "lightblue")
curve(dchisq(x, df = n), col = "red", lwd = 2, add = TRUE)

# estatística t: sqrt(n)·média/desvio ~ t_{n-1} (ex. 8)
n <- 10
A <- matrix(rnorm(m * n), nrow = m)
Tstat <- sqrt(n) * rowMeans(A) / apply(A, 1, sd)
hist(Tstat, probability = TRUE, col = "lightblue")
curve(dt(x, df = n - 1), col = "red", lwd = 2, add = TRUE)
```

26.26 Distribuição exponencial

1. e 2. Amostra, tempo médio e probabilidades de cauda.

```
set.seed(1)
amostra <- rexp(1000, rate = 2)
mean(amostra) # ~ 1/2 (média teórica = 1/rate)
hist(amostra, probability = TRUE, col = "lightblue")
curve(dexp(x, rate = 2), col = "red", lwd = 2, add = TRUE)

pexp(3, rate = 0.5, lower.tail = FALSE) # P(X > 3) = e^{-1.5}
```

3, 9, 10, 11. Soma de exponenciais independentes com a mesma taxa é uma Gama: $\sum_{i=1}^n X_i \sim \Gamma(n, \lambda)$.

```

set.seed(1)
n <- 5; m <- 5000
soma <- rowSums(matrix(rexp(m * n, rate = 2), nrow = m))
hist(soma, probability = TRUE, col = "lightblue")
curve(dgamma(x, shape = n, rate = 2), col = "red", lwd = 2, add = TRUE)
c(media_emp = mean(soma), media_teo = n / 2,      # E = n/
  var_emp = var(soma), var_teo = n / 2^2)        # Var = n/ 2

```

4 a 8. Somas de processos, transformações lineares e teorema limite central.

```

# 8 transformação linear Y = 2X + 1
set.seed(1)
X <- rexp(5000, rate = 0.5)
Y <- 2 * X + 1
c(media = mean(Y), media_teo = 2 * (1/0.5) + 1,  # E(Y) = 2·2 + 1 = 5
  var = var(Y), var_teo = 4 * (1/0.5^2))        # Var(Y) = 4·4 = 16

# 5 TLC: média de n exponenciais é aproximadamente normal
n <- 30
medias <- rowMeans(matrix(rexp(5000 * n, rate = 1.5), nrow = 5000))
hist(medias, probability = TRUE, col = "lightblue")
curve(dnorm(x, 1/1.5, 1/(1.5 * sqrt(n))), col = "red", lwd = 2, add = TRUE)

```

Muitos destes exercícios ilustram um mesmo padrão: **somas** de exponenciais dão Gammas exatas, enquanto **médias** de qualquer distribuição, para n grande, dão aproximadamente normais (teorema limite central). Distinguir o resultado exato do aproximado é essencial: a soma de 5 exponenciais é *exatamente* $\Gamma(5, \lambda)$, mas a sua média só é *aproximadamente* normal.

26.27 Simulação de variáveis aleatórias

Esta secção reúne exercícios que geram amostras de distribuições conhecidas, comparam frequências empíricas com probabilidades teóricas (via `ecdf`) e ilustram os teoremas limites. Como o padrão se repete, apresentamos um exemplo representativo de cada tipo.

Comparar frequências empíricas com teóricas.

```

set.seed(123)
amostra <- rbinom(5000, size = 20, prob = 0.7)
hist(amostra, probability = TRUE, col = "lightblue")
points(0:20, dbinom(0:20, 20, 0.7), col = "magenta") # fmp teórica

```

```
Fn <- ecdf(amostra)
data.frame(empirica = Fn(10), teorica = pbinom(10, 20, 0.7)) #  $P(X \leq 10)$ 
```

Convergência da média amostral (lei dos grandes números).

```
set.seed(123)
lambda <- 3
data.frame(
  n = c(100, 1000, 10000, 100000),
  media = sapply(c(100, 1000, 10000, 100000),
    function(n) mean(rpois(n, lambda))),
  teorica = lambda)
```

Teorema limite central (soma e média padronizadas). Este é o exercício mais completo do conjunto: gerar muitas amostras, calcular soma e média de cada uma, e sobrepor a curva normal apropriada.

```
set.seed(1580)
m <- 1234; n <- 50; lambda <- 0.21
media_teo <- 1 / lambda; var_teo <- 1 / lambda^2
amostras <- matrix(rexp(m * n, lambda), nrow = m)

# soma ~  $N(n \cdot, n \cdot^2)$ 
soma <- rowSums(amostras)
hist(soma, probability = TRUE, col = "lightblue")
curve(dnorm(x, n * media_teo, sqrt(var_teo * n)), col = "red", lwd = 2, add = TRUE)

# soma padronizada ~  $N(0, 1)$ 
z <- (soma - mean(soma)) / sd(soma)
hist(z, probability = TRUE, col = "lightblue")
curve(dnorm(x), col = "red", lwd = 2, add = TRUE)
```

26.28 Método da transformação inversa (caso discreto)

A ideia é dividir o intervalo $[0, 1]$ em subintervalos com comprimentos iguais às probabilidades da distribuição; um $U \sim \mathcal{U}(0, 1)$ cai num deles com a probabilidade correta.

1 a 3. Variáveis discretas a partir da função de distribuição acumulada.

```

set.seed(1)
# distribuição com P(X=1)=0.2, P(X=2)=0.5, P(X=3)=0.3 => F = (0.2, 0.7, 1.0)
gerar <- function(n, valores, prob) {
  Fx <- cumsum(prob)
  u <- runif(n)
  valores[findInterval(u, Fx) + 1] # findInterval localiza o subintervalo
}
sim <- gerar(1000, valores = 1:3, prob = c(0.2, 0.5, 0.3))
table(sim) / 1000

```

Em vez de um ciclo for com uma cadeia de if/else if para localizar o subintervalo (como no material original), findInterval(u, Fx) faz o mesmo de forma **vetorizada** e para qualquer número de categorias. Além de mais curto, é muito mais rápido para amostras grandes.

4. Geométrica pela transformação inversa. Como $F(x) = 1 - (1 - p)^x$, a inversa dá $x = \lceil \ln(1 - u) / \ln(1 - p) \rceil$.

```

set.seed(1)
p <- 0.2; n <- 1000
u <- runif(n)
x <- ceiling(log(1 - u) / log(1 - p)) # n° de provas até ao 1.º sucesso
table(x) / n

```

26.29 Método da transformação inversa (caso contínuo)

Se F é a função de distribuição e $U \sim \mathcal{U}(0, 1)$, então $X = F^{-1}(U)$ tem função de distribuição F . Basta, pois, inverter F .

1. Exponencial: $F(x) = 1 - e^{-\lambda x} \Rightarrow F^{-1}(u) = -\ln(1 - u)/\lambda$.

```

gerar_exp <- function(n, lambda) -log(1 - runif(n)) / lambda
set.seed(123)
x <- gerar_exp(1000, 2)
hist(x, probability = TRUE, col = "lightblue")
curve(dexp(x, 2), col = "red", lwd = 2, add = TRUE)

```

2. Uniforme $\mathcal{U}(a, b)$: $F^{-1}(u) = a + (b - a)u$.

```

gerar_unif <- function(n, a, b) a + (b - a) * runif(n)

```

3. Cauchy: $F^{-1}(u) = x_0 + \gamma \tan(\pi(u - 1/2))$.

```

gerar_cauchy <- function(n, x0 = 0, gama = 1) x0 + gama * tan(pi * (runif(n) - 0.5))
set.seed(1)
x <- gerar_cauchy(10000)
hist(x, probability = TRUE, breaks = 100, xlim = c(-10, 10), col = "lightblue")
curve(dcauchy(x), col = "red", lwd = 2, add = TRUE)

```

4. Weibull: $F(x) = 1 - e^{-(x/\beta)^\alpha} \Rightarrow F^{-1}(u) = \beta (-\ln(1 - u))^{1/\alpha}$.

```

gerar_weibull <- function(n, alfa, beta) beta * (-log(1 - runif(n)))^(1 / alfa)
set.seed(1)
x <- gerar_weibull(10000, alfa = 2, beta = 3)
hist(x, probability = TRUE, col = "lightblue")
curve(dweibull(x, 2, 3), col = "red", lwd = 2, add = TRUE)

```

26.30 Método da aceitação-rejeição

A constante c tem de ser o supremo verdadeiro de f/g . Vários dos exercícios seguintes revelam o erro mais perigoso deste método: escolher um c demasiado pequeno (ou uma proposta cujo rácio f/g é ilimitado). Nesses casos o algoritmo produz observações de uma distribuição **errada**, sem qualquer aviso. Determinamos sempre c analiticamente — ou com `optimize()` — e nunca “à vista”.

1. Exponencial $\text{Exp}(1)$ com proposta uniforme em $[0, 5]$. Como $f(x) = e^{-x}$ e $g(x) = 1/5$, o rácio $f/g = 5e^{-x}$ é máximo em $x = 0$, logo $c = 5$. (Gera-se, na verdade, uma exponencial **truncada** em 5; como $P(X > 5) = e^{-5} \approx 0,007$, a diferença é desprezável.)

```

set.seed(123)
n <- 1000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- runif(1, 0, 5); u <- runif(1)
  if (u <= exp(-y) / (5 * (1/5))) amostra <- c(amostra, y) # aceita se u <= e^{-y}
}
hist(amostra, probability = TRUE, breaks = 30, col = "lightblue")
curve(dexp(x, 1), col = "red", lwd = 2, add = TRUE)

```

2. Normal padrão com proposta Cauchy. O rácio $f/g = \text{dnorm}/\text{dcauchy}$ é máximo em $x = \pm 1$ (não em 0), onde vale

$$c = \sqrt{\frac{\pi}{2}} \cdot 2e^{-1/2} \approx 1,520.$$

```

set.seed(123)
c <- optimize(function(x) dnorm(x) / dcauchy(x), c(0, 5), maximum = TRUE)$objective
n <- 1000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- rcauchy(1); u <- runif(1)
  if (u <= dnorm(y) / (c * dcauchy(y))) amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 30, col = "lightgreen")
curve(dnorm(x), col = "blue", lwd = 2, add = TRUE)

```

O material original usava $c = 1,5$ para este exemplo. Mas o verdadeiro supremo é $\approx 1,520$: com $c = 1,5$, a envolvente cg fica *abaixo* de f na vizinhança de $x = \pm 1$, e as observações geradas seguem uma distribuição enviesada. É exatamente o erro silencioso contra o qual avisámos.

3. Beta(2,3) com proposta uniforme. O máximo de $\text{dbeta}(x; 2, 3)$ ocorre em $x = 1/3$, onde $c = \text{dbeta}(1/3; 2, 3) = 16/9 \approx 1,778$.

```

set.seed(123)
c <- dbeta(1/3, 2, 3) # = 16/9, o máximo exato (não dbeta(0.4, ...))
n <- 1000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- runif(1); u <- runif(1)
  if (u <= dbeta(y, 2, 3) / c) amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 30, col = "lightcoral")
curve(dbeta(x, 2, 3), col = "purple", lwd = 2, add = TRUE)

```

4. Gama(3,2) com proposta exponencial. Aqui há uma subtilidade importante: com $\text{Exp}(2)$, o rácio

$$\frac{\text{dgamma}(x; 3, 2)}{\text{dexp}(x; 2)} = \frac{4x^2 e^{-2x}}{2e^{-2x}} = 2x^2$$

é **ilimitado** — não existe c finito, e o método é inaplicável. A proposta tem de decair mais devagar do que a alvo, ou seja, ter taxa **inferior** a 2. A escolha ótima iguala as médias: $\text{Exp}(2/3)$, para a qual $c \approx 1,83$.

```

set.seed(123)
alfa <- 3; beta <- 2
lambda_prop <- beta / alfa # = 2/3 : proposta com a mesma média da alvo
c <- optimize(function(x) dgamma(x, alfa, rate = beta) / dexp(x, lambda_prop),
              c(1e-4, 50), maximum = TRUE)$objective
n <- 1000; amostra <- numeric(0)

```

```
while (length(amostra) < n) {
  y <- rexp(1, lambda_prop); u <- runif(1)
  if (u <= dgamma(y, alfa, rate = beta) / (c * dexp(y, lambda_prop)))
    amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 30, col = "lightblue")
curve(dgamma(x, alfa, rate = beta), col = "red", lwd = 2, add = TRUE)
```

5. Log-normal com proposta normal. Também aqui a proposta é **inadmissível**: a cauda da log-normal é mais pesada do que a da normal, pelo que o rácio f/g é ilimitado. A solução elegante não é rejeitar de todo, mas usar a definição: se $Z \sim N(0, 1)$, então $e^Z \sim \text{LogNormal}(0, 1)$. Basta exponenciar.

```
set.seed(123)
amostra <- exp(rnorm(1000))          # gera log-normal sem qualquer rejeição
hist(amostra, probability = TRUE, breaks = 40, col = "lightyellow")
curve(dlnorm(x), col = "darkred", lwd = 2, add = TRUE)
```

6. Weibull(forma = 2, escala = 1) com proposta Exp(1). O rácio $2x e^{-x^2+x}$ é máximo em $x = 1$, onde vale exatamente $c = 2$.

```
set.seed(123)
c <- 2                                # valor exato (o material usava 1.5, pequeno demais)
n <- 1000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- rexp(1, 1); u <- runif(1)
  if (u <= dweibull(y, 2, 1) / (c * dexp(y, 1))) amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 30, col = "lightpink")
curve(dweibull(x, 2, 1), col = "purple", lwd = 2, add = TRUE)
```

7. Densidade $f(x) = \frac{1}{2}(1 + \cos x)$ em $[-\pi, \pi]$, com proposta uniforme $g = 1/(2\pi)$. O rácio $f/g = \pi(1 + \cos x)$ é máximo em $x = 0$, onde $c = 2\pi$.

```
set.seed(1)
f <- function(x) 0.5 * (1 + cos(x))
c <- 2 * pi
n <- 5000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- runif(1, -pi, pi); u <- runif(1)
```

```

  if (u <= f(y) / (c * (1 / (2 * pi)))) amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 40, col = "lightblue")
curve(f, -pi, pi, col = "red", lwd = 2, add = TRUE)

```

8. Versão vetorizada e comparação de tempos.

```

ar_vetorizado <- function(n, f, c) {                                # proposta uniforme(0,1)
  out <- numeric(0)
  while (length(out) < n) {
    y <- runif(2 * n); u <- runif(2 * n)
    out <- c(out, y[u <= f(y) / c])
  }
  out[1:n]
}
f_beta <- function(x) dbeta(x, 2, 2)
system.time(ar_vetorizado(1e5, f_beta, c = 1.5))                 # compare com a versão em while

```

9. Estudo da eficiência. Com o c correto, a taxa de aceitação empírica aproxima-se de $1/c$. Com um c **maior** do que o necessário, a distribuição gerada continua correta, mas a taxa de aceitação cai (desperdício). Com um c **menor** do que o supremo, a distribuição fica **errada** — o caso a evitar.

```

taxa <- function(c) {
  aceites <- 0; tentativas <- 0
  while (aceites < 5000) {
    y <- rexp(1); u <- runif(1); tentativas <- tentativas + 1
    if (u <= sqrt(2/pi) * exp(-y^2/2) / (c * dexp(y))) aceites <- aceites + 1
  }
  aceites / tentativas
}
c_correto <- sqrt(2 * exp(1) / pi)    # ~ 1.3155
c(taxa(c_correto), 1 / c_correto,    # empírica ~ teórica
  taxa(2 * c_correto))                # c grande demais: taxa cai para metade

```

10. Distribuição do semicírculo $f(x) = \frac{2}{\pi}\sqrt{1-x^2}$ em $[-1, 1]$, com proposta uniforme $g = 1/2$. O rácio $f/g = \frac{4}{\pi}\sqrt{1-x^2}$ é máximo em $x = 0$, onde $c = 4/\pi \approx 1,273$.


```

set.seed(1)
f <- function(x) (2 / pi) * sqrt(1 - x^2)
c <- 4 / pi
n <- 5000; amostra <- numeric(0)
while (length(amostra) < n) {
  y <- runif(1, -1, 1); u <- runif(1)
  if (u <= f(y) / (c * 0.5)) amostra <- c(amostra, y)
}
hist(amostra, probability = TRUE, breaks = 40, col = "lightblue")
curve(f, -1, 1, col = "red", lwd = 2, add = TRUE)

```

11. Densidade não normalizada. Para $\tilde{f}(x) = e^{-x^2/2}(1 + \sin^2 3x)$, usamos uma proposta normal de variância grande. A taxa de aceitação estima a constante de normalização: se aceitamos com probabilidade $1/c$ relativamente a $\tilde{f}$, então $\int \tilde{f} = c \cdot p_{\text{aceite}} \cdot \sqrt{2\pi} \sigma$ (a menos da constante da proposta). Na prática, estima-se $\int \tilde{f}$ diretamente por Monte Carlo, $\int \tilde{f} = E[\tilde{f}(X)/g(X)]$.

```

set.seed(1)
f_til <- function(x) exp(-x^2/2) * (1 + sin(3*x)^2)
X <- rnorm(1e5, 0, 2) # proposta larga
Z_hat <- mean(f_til(X) / dnorm(X, 0, 2)) # estimativa de f_til
Z_hat

```

26.31 Teoremas limites

Os exercícios deste capítulo são, na sua maioria, ilustrações por simulação. Apresentamos as resoluções essenciais e o valor teórico para que serve cada uma.

1. e 2. Proporção acumulada de caras (moeda equilibrada e viciada).

```

set.seed(1)
lancamentos <- sample(c(0, 1), 10000, replace = TRUE, prob = c(0.5, 0.5))
plot(cumsum(lancamentos) / seq_along(lancamentos), type = "l",
     ylim = c(0, 1), xlab = "lançamentos", ylab = "proporção de caras")
abline(h = 0.5, col = "red", lty = 2) # converge para p = 0.5 (para 0.7 no ex. 2)

```

3. e 4. Média acumulada (Poisson e uniforme) — converge para λ e para $1/2$, respetivamente.

```

set.seed(1)
x <- rpois(2000, 4)
plot(cumsum(x) / seq_along(x), type = "l"); abline(h = 4, col = "red", lty = 2)

```

5. Cauchy: a média acumulada **não** converge, porque a Cauchy não tem média finita — as hipóteses da lei dos grandes números falham.

```
set.seed(1)
x <- rcauchy(5000)
plot(cumsum(x) / seq_along(x), type = "l",
     main = "Média acumulada de uma Cauchy (não converge)")
```

6. Lei fraca vs. majorante de Chebyshev, para $X_i \sim \mathcal{U}(0, 1)$.

```
set.seed(1)
eps <- 0.05
for (n in c(10, 50, 100, 500)) {
  p_emp <- mean(replicate(5000, abs(mean(runif(n)) - 0.5) > eps))
  cheby <- (1/12) / (n * eps^2) # 12/(n^2), com 12 = 1/12
  cat("n =", n, "| P empírica:", round(p_emp, 4),
      "| majorante Chebyshev:", round(cheby, 4), "\n")
}
```

7. e 8. Teorema limite central a partir da uniforme e da log-normal (esta converge mais devagar, por ser fortemente assimétrica).

```
set.seed(1)
par(mfrow = c(1, 3))
for (n in c(2, 5, 12)) {
  medias <- rowMeans(matrix(runif(10000 * n), nrow = 10000))
  hist(medias, probability = TRUE, main = paste("n =", n), col = "lightblue")
  curve(dnorm(x, 0.5, sqrt((1/12) / n)), col = "red", lwd = 2, add = TRUE)
}
par(mfrow = c(1, 1))
```

10. e 11. Cobertura de um intervalo de confiança. A proporção de intervalos $\bar{x} \pm 1.96 s/\sqrt{n}$ que contêm o verdadeiro valor deve aproximar-se de 95% à medida que n cresce.

```
set.seed(1)
cobertura <- function(n, reps = 5000) {
  contido <- replicate(reps, {
    x <- rexp(n, rate = 1) # média verdadeira = 1
    erro <- 1.96 * sd(x) / sqrt(n)
    abs(mean(x) - 1) <= erro
  })
}
```

```
mean(contido)
}
sapply(c(10, 40, 200), cobertura)      # aproxima-se de 0.95 quando n cresce
```

26.32 Cálculo de integrais por Monte Carlo

Para cada integral, a estimativa é a média das avaliações da função em pontos aleatórios, com erro-padrão $s/\sqrt{k}$. Indicamos o valor exato ou de referência quando é conhecido.

1. $\int_0^1 e^x dx \approx 6,317$ (sem forma fechada).

```
set.seed(1); u <- runif(1e6)
mean(exp(exp(u)))      # ~ 6.317
```

2. $\int_0^1 (1-x^2)^{3/2} dx = \frac{3\pi}{16} \approx 0,589$.

```
set.seed(1); u <- runif(1e6)
mean((1 - u^2)^(3/2))   # ~ 0.589 = 3pi/16
```

3. $\int_{-2}^2 e^{x+x^2} dx \approx 93,16$. Usa-se a forma para intervalo $[a, b]$: multiplicar a média por $(b-a) = 4$.

```
set.seed(1); u <- runif(1e6, -2, 2)
4 * mean(exp(u + u^2))  # ~ 93.16
```

4. $\int_0^{\pi/2} \cos x dx = 1$.

```
set.seed(1); u <- runif(1e6, 0, pi/2)
(pi/2) * mean(cos(u))   # ~ 1
```

5. $\int_0^\infty x(1+x^2)^{-2} dx = \frac{1}{2}$. Substituindo $y = 1/(x+1)$, o integral torna-se $\int_0^1 h(y) dy$ com $h(y) = g(1/y - 1)/y^2$.

```
set.seed(1)
g <- function(x) x * (1 + x^2)^(-2)
u <- runif(1e6)
h <- g(1/u - 1) / u^2
mean(h)      # ~ 0.5
```

6. $\int_{-\infty}^\infty e^{-x^2} dx = \sqrt{\pi} \approx 1,772$. Uma via simples: reconhecer o integrando como proporcional à densidade de uma $N(0, 1/2)$ e usar amostragem por importância.

```
set.seed(1)
X <- rnorm(1e6)                # proposta N(0,1)
mean(exp(-X^2) / dnorm(X))     # ~ sqrt(pi) = 1.7725
```

7. $\int_0^1 \int_0^1 e^{(x+y)^2} dy dx \approx 4,899$ (integral duplo).

```
set.seed(1); k <- 1e6
x <- runif(k); y <- runif(k)
mean(exp((x + y)^2))           # ~ 4.899
```

8. $\int_0^1 \int_0^1 (x^2 + y^2) dx dy = \frac{2}{3}$.

```
set.seed(1); k <- 1e6
mean(runif(k)^2 + runif(k)^2) # ~ 0.667
```

9. e 10. Estimativa de π (acerta-ou-falha e agulha de Buffon).

```
set.seed(1); k <- 1e6
# acerta-ou-falha: fração de pontos dentro do quarto de círculo
x <- runif(k); y <- runif(k)
4 * mean(x^2 + y^2 <= 1)           # ~ pi

# agulha de Buffon: l = 1, d = 2 => P(cruzar) = 2l/(pi d) => pi ~ 2l/(dp)
centro <- runif(k, 0, 1)           # distância do centro à linha mais próxima (d/2 = 1)
angulo <- runif(k, 0, pi/2)
cruza <- centro <= 0.5 * sin(angulo)
2 * 1 / (2 * mean(cruza))          # ~ pi
```

11. Taxa de convergência $1/\sqrt{k}$. Para $\int_0^1 e^x dx = e - 1$, o erro absoluto decresce, em média, como $k^{-1/2}$ (declive $-1/2$ em escala log-log).

```
set.seed(1)
ks <- 10^(2:6)
erros <- sapply(ks, function(k) abs(mean(exp(runif(k))) - (exp(1) - 1)))
plot(log10(ks), log10(erros), type = "b",
     xlab = "log10(k)", ylab = "log10(erro)") # declive ~ -0.5
```

12. Redução de variância por variáveis antitéticas, em $\int_0^1 e^x dx$. O estimador antitético usa os pares $(g(U), g(1 - U))$; como $g(x) = e^x$ é monótona, $g(U)$ e $g(1 - U)$ são negativamente correlacionadas, o que reduz a variância da média.

```

set.seed(1)
simples    <- replicate(1000, mean(exp(runif(1000))))
antitetico <- replicate(1000, { u <- runif(500)
                                mean(c(exp(u), exp(1 - u))) })
c(var_simples = var(simples), var_antitetico = var(antitetico)) # menor no 2.º

```

13. Volume da bola unitária em dimensão n , por acerta-ou-falha. A eficiência degrada-se com n : a fração de pontos do hipercubo $[-1, 1]^n$ que caem dentro da bola tende rapidamente para zero.

```

set.seed(1)
volume_bola <- function(n, k = 1e5) {
  X <- matrix(runif(k * n, -1, 1), ncol = n)
  dentro <- rowSums(X^2) <= 1
  2^n * mean(dentro) # volume do cubo (2^n) × fração dentro
}
sapply(2:5, volume_bola)
# comparar com o valor exato: pi^(n/2) / gamma(n/2 + 1)
sapply(2:5, function(n) pi^(n/2) / gamma(n/2 + 1))

```

O último exercício ilustra tanto a força como o limite do método de Monte Carlo. A força: o mesmo código de três linhas funciona em qualquer dimensão. O limite: como a fração de acertos colapsa com n , o método acerta-ou-falha torna-se ineficiente em dimensão alta — nessa altura, técnicas como a amostragem por importância passam a ser indispensáveis. Fica, assim, o mote para um estudo mais avançado da simulação estatística, para além deste primeiro livro.